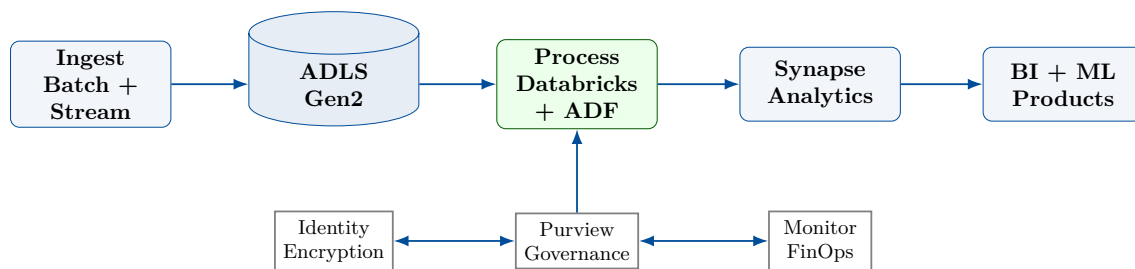

A COMPREHENSIVE, HANDS-ON ACADEMIC GUIDE

Azure Master Data Engineer



Inside this book



From secure lake storage to governed, observable, production-scale Azure data products.

PhD Aned Esquerra Arguelles

A 24-week hands-on program for Azure data engineering mastery

August 31, 2026

Preface

Welcome to *Azure Master Data Engineer*. This text is designed as an academic, hands-on, progressive program for building cloud-scale data platforms on Microsoft Azure. By working through the architecture discussions, command-line labs, and design exercises, you will move from storage foundations to governed lakehouse, warehouse, streaming, and machine-learning data products.

This book is the theory-and-architecture companion to a six-milestone Azure Data Engineering curriculum. Where the practice workspace asks you to build weekly deliverables, this book explains why each Azure service exists, what operational failure mode it addresses, and how a data engineer reasons about reliability, cost, compliance, and maintainability.

Who This Book Is For

This book is written for data engineers, analytics engineers, cloud engineers, database professionals, and technically minded analysts preparing for Azure data-platform work. It assumes basic SQL, Python, and command-line familiarity. Prior Azure experience is helpful but not required: the early chapters introduce storage accounts, identity, naming, redundancy, and life-cycle controls before the program moves into Synapse, Data Factory, Databricks, Event Hubs, Cosmos DB, Azure Machine Learning, and governance.

How This Book Is Organized

The book follows the training roadmap as **6 Parts / 6 Milestones** and **24 Chapters / 24 Weeks**, followed by appendices:

- **Part I: Core Data Infrastructure and Storage** establishes ADLS Gen2, secure access, file formats, medallion zones, and governance foundations.
- **Part II: Data Warehousing and Analytics** builds analytical serving layers with Azure Synapse Analytics, serverless SQL, dedicated SQL pools, and Power BI semantic models.
- **Part III: Batch Processing and ETL Orchestration** develops repeatable pipelines with Azure Data Factory, integration runtimes, Databricks, Spark, and Delta Lake.
- **Part IV: Streaming and Real-Time Analytics** introduces Event Hubs, IoT ingestion, Stream Analytics, real-time Spark, and operational analytics patterns with Cosmos DB.
- **Part V: Machine Learning and MLOps** connects curated data products to Azure Machine Learning, feature engineering, pipeline automation, model monitoring, and GenAI-ready data services.
- **Part VI: Security, Governance, and Exam Prep** integrates Zero Trust, observability, FinOps, compliance, DP-203-style exam review, and the enterprise capstone.

Each chapter opens with **Learning Objectives** and closes with **Further Reading**, **Key Takeaways**, and **Exercises**. Hands-on chapters include Azure CLI, JSON, SQL, or Python listings that are meant to be adapted in a real subscription.

Reading Paths

- **New to Azure data engineering:** read Part I in order, perform every hands-on storage exercise, then continue to Synapse and Data Factory.
- **Experienced SQL or BI practitioner:** skim Chapter 1 for storage vocabulary, then read Parts II and III while returning to governance notes as needed.
- **Spark or Databricks practitioner:** read Chapters 1–4 for Azure storage and governance assumptions, then focus on Databricks, Delta Lake, streaming, and MLOps.
- **Platform architect:** prioritize architecture diagrams, pitfall boxes, Well-Architected trade-offs, and the security/governance chapters.
- **Certification candidate:** complete the conceptual exercises, review the command-line examples, and use Part VI to consolidate DP-203 objectives.

Conventions

Code identifiers, resource names, and file paths appear in **monospace**. Azure CLI snippets use the shared listing style and assume PowerShell on Windows unless noted. Callout boxes flag **objectives**, **key terms**, **definitions**, **notes**, **tips**, and **pitfalls**. Citations refer to the bibliography at the back of the book. Service names use their current Microsoft branding, while design guidance emphasizes transferable principles: least privilege, explicit recovery objectives, lifecycle economics, reproducible infrastructure, observability, and governed data contracts.

Contents

0	Getting Started: Your Azure Free Account and Environment	1
0.1	Why Chapter 0 Exists	1
0.2	Creating an Azure Free Account	2
0.2.1	What the Free Account Includes	2
0.2.2	Sign-up Walkthrough	3
0.2.3	The Azure Resource Hierarchy	3
0.3	Securing Identity Before Deploying Resources	4
0.3.1	Microsoft Entra ID and Administrative Separation	4
0.3.2	Azure RBAC Roles for Labs	4
0.3.3	Create a Least-Privileged User or Service Principal	5
0.3.4	Enable MFA	5
0.4	Installing and Authenticating Tooling on Windows	5
0.4.1	Azure CLI on Windows	5
0.4.2	Azure PowerShell as an Alternative	6
0.4.3	Azure Cloud Shell	6
0.5	Cost Guardrails	7
0.5.1	Spending Limit, Budgets, and Alerts	7
0.5.2	Create a Budget from PowerShell	7
0.5.3	Cost Analysis Workflow	8
0.6	Regions, Naming, and Tagging Used by This Book	8
0.6.1	Default Region	8
0.6.2	Naming Conventions	8
0.7	Verify the Setup	9
0.7.1	Account and Subscription Check	9
0.7.2	Create a Resource Group and Storage Account	9
0.7.3	Delete the Verification Resource Group	10
0.8	Cross-Cloud Setup Equivalence	11
0.9	Environment Checklist Before Chapter 1	11
0.10	Further Reading	12
0.11	Exercises	12
0.12	Curated Community Resources	13
I	Core Data Infrastructure and Storage	16
1	Azure Data Lake Storage Gen2 and Storage Foundations	17
1.1	Week 1 Overview: Storage as the Root of the Azure Data Platform	18
1.2	Monday: Azure Storage Account Architecture	19
1.2.1	The Storage Account as a Control Boundary	19
1.2.2	Blob Storage vs. ADLS Gen2	20

1.2.3	Redundancy Options: LRS, ZRS, GRS, GZRS, and RA Variants	20
1.2.4	Monday Design Checklist	21
1.3	Tuesday: Access Tiers, Lifecycle Management, and Recoverability	21
1.3.1	Access Tiers: Hot, Cool, Cold, and Archive	21
1.3.2	Lifecycle Management Policies	22
1.3.3	Soft Delete and Blob Versioning	23
1.3.4	Tuesday Design Checklist	23
1.4	Wednesday: Migration Tools and Trade-offs	23
1.4.1	Online Migration with AzCopy	23
1.4.2	Azure Storage Explorer	23
1.4.3	Offline Migration with Azure Data Box	24
1.5	Thursday Hands-on Project: Deploy ADLS Gen2 with Lifecycle Management . .	24
1.5.1	Validation Steps	27
1.6	Friday Use Case and Architecture: Healthcare X-ray Lake at 10 PB	27
1.6.1	Scenario	27
1.6.2	Target Architecture	27
1.6.3	Compliance and Security Design	28
1.6.4	Lifecycle and Cost Model for 10 PB	28
1.6.5	Disaster Recovery Runbook	29
1.7	Architectural Decision Record Template	29
1.8	Operational Deep Dive: Naming, Zones, and Evidence	29
1.9	Troubleshooting Patterns	30
1.10	Week 1 Lab Rubric	30
1.11	Interview Q&A	30
1.11.1	Concepts and Trade-offs	30
1.12	Further Reading	31
1.13	Exercises	31
1.14	Capstone Project: A Compliant Medical Imaging Archive on ADLS Gen2	32
2	Relational Databases: Azure SQL and PostgreSQL	39
2.1	Week 2 Overview: Relational Engines in a Data Engineering Platform	40
2.2	Monday: Azure SQL Database Purchasing Models	40
2.2.1	DTU and vCore Models	40
2.2.2	Serverless for Variable Demand	41
2.2.3	Hyperscale Architecture	42
2.3	Tuesday: Azure Database for PostgreSQL Flexible Server and High Availability .	42
2.3.1	Flexible Server as the Default PostgreSQL Model	42
2.3.2	High Availability and Backups	43
2.3.3	Designing PostgreSQL for Data Engineering	43
2.4	Wednesday: Active Geo-Replication and Auto-Failover Groups	43
2.4.1	Active Geo-Replication	43
2.4.2	Auto-Failover Groups	44
2.4.3	Testing Failover Without Fooling Yourself	44
2.5	Thursday Hands-on Project: Serverless Azure SQL with Geo-Replica and Failover	45
2.6	Friday Use Case: Zero-Downtime Migration to Azure SQL Managed Instance . .	47
2.6.1	Scenario	47
2.6.2	Migration Architecture	48
2.6.3	Cutover Runbook	48
2.7	Architectural Decision Record Template	48
2.8	Operational Deep Dive: Performance, Identity, and Observability	49
2.9	Troubleshooting Patterns	49

2.10	Week 2 Lab Rubric	49
2.11	Interview Q&A	50
2.11.1	Concepts and Trade-offs	50
2.12	Further Reading	50
2.13	Exercises	51
2.14	Capstone Project: HA Geo-Replicated Azure SQL Platform for Managed Instance Cutover	52
3	High-Scale NoSQL: Azure Cosmos DB	57
3.1	Week 3 Overview: Why Cosmos DB Matters to Data Engineers	58
3.2	Monday: Core Concepts — RUs, Partitioning, and Global Distribution	59
3.2.1	Request Units as the Throughput Currency	59
3.2.2	Partition Keys and Logical Partitions	59
3.2.3	Physical Partitions and Hot Keys	60
3.2.4	Global Distribution	60
3.3	Tuesday: The Five Consistency Levels	60
3.3.1	Strong and Bounded Staleness	60
3.3.2	Session, Consistent Prefix, and Eventual	61
3.4	Wednesday: APIs and the Change Feed	61
3.4.1	Core (NoSQL), MongoDB, and Cassandra APIs	61
3.4.2	Change Feed as the Event Bridge	62
3.5	Thursday Hands-on Project: Forum Container with Python Writes and Multi-Region Writes	62
3.5.1	Single-Container Forum Schema	62
3.5.2	Inserting Forum Items with Python	63
3.6	Friday Use Case: Global Gaming Leaderboard at 1,000,000 Reads/Writes per Second	65
3.6.1	Scenario	65
3.6.2	Partitioning and Serving Strategy	65
3.6.3	Correctness and Cost Controls	65
3.7	Architectural Decision Record Template	66
3.8	Operational Deep Dive: Performance, Identity, and Observability	66
3.9	Troubleshooting Patterns	66
3.10	Week 3 Lab Rubric	67
3.11	Interview Q&A	67
3.11.1	Concepts and Trade-offs	67
3.12	Further Reading	68
3.13	Exercises	68
3.14	Capstone Project: Global Gaming Leaderboard on Cosmos DB	69
4	Hybrid Ingestion and Caching	74
4.1	Week 4 Overview: Why Caching and Hybrid Ingestion Close Part I	75
4.2	Monday: Azure Cache for Redis for Database Acceleration	76
4.2.1	What Redis Adds to a Data Platform	76
4.2.2	Cache-Aside Read Path	76
4.2.3	Keys, TTLs, and Invalidation	77
4.2.4	Availability and Failure Modes	77
4.3	Tuesday: Cosmos DB Integrated Cache	78
4.3.1	Why an Integrated Cache Exists	78
4.3.2	When Integrated Cache Fits	78
4.3.3	Consistency and Freshness Trade-offs	78
4.4	Wednesday: Azure API Management for Data Ingestion	79

4.4.1	APIM as the Data API Front Door	79
4.4.2	Policies for Data Ingestion	79
4.4.3	Synchronous and Asynchronous Ingestion	79
4.5	Thursday Hands-on Project: Redis Cache-Aside for Azure SQL Query Results	80
4.5.1	Deploying Azure Cache for Redis	80
4.5.2	Caching Azure SQL Results with Python	80
4.5.3	Measuring the Load Reduction	81
4.6	Friday Use Case: Mobile Sports API with Extreme Traffic Spikes	82
4.6.1	Scenario	82
4.6.2	Serving Architecture	82
4.6.3	Design Decisions	82
4.7	Architectural Decision Record Template	83
4.8	Operational Deep Dive: Stampedes, Security, and Observability	83
4.9	Troubleshooting Patterns	83
4.10	Week 4 Lab Rubric	83
4.11	Interview Q&A	84
4.11.1	Concepts and Trade-offs	84
4.12	Further Reading	84
4.13	Exercises	85
4.14	Milestone 1 Capstone: E-Commerce Multi-Tier Data Backend	85
II	Data Warehousing and Analytics	91
5	Azure Synapse Analytics: Dedicated SQL Pools	92
5.1	Week 5 Overview: The Provisioned Warehouse in a Lakehouse World	93
5.2	Monday: MPP Architecture and Dimensional Modeling	94
5.2.1	Control Node, Compute Nodes, and the Data Movement Service	94
5.2.2	Declaring the Grain	94
5.2.3	Fact Table Types	94
5.2.4	Conformed Dimensions and Slowly Changing Dimensions	95
5.3	Tuesday: Distribution Types	95
5.3.1	Hash Distribution	95
5.3.2	Round-Robin Distribution	96
5.3.3	Replicated Distribution	96
5.4	Wednesday: Ingestion with COPY and PolyBase	96
5.4.1	COPY: The Modern Bulk Loader	96
5.4.2	PolyBase: The Legacy Path	97
5.4.3	The Load-to-Serve Pattern	97
5.5	Thursday Hands-on Project: Ten Million Rows into a Star Schema	97
5.5.1	Provision the Pool	97
5.5.2	Generate and Stage Synthetic Data	98
5.5.3	Model the Star and Apply SCD2	98
5.5.4	Verify and Pause	98
5.6	Friday Use Case: End-of-Month BI on a Budget	98
5.6.1	Cost Model for Dedicated	98
5.6.2	Cost Model for Serverless	99
5.6.3	Recommendation	99
5.7	Architectural Decision Record Template	99
5.8	Operational Deep Dive: Concurrency, Skew, and Rowgroup Health	99
5.9	Troubleshooting Patterns	100
5.10	Week 5 Lab Rubric	100

5.11	Interview Q&A	100
5.11.1	Concepts and Trade-offs	100
5.12	Further Reading	101
5.13	Exercises	101
5.14	Capstone Project: Star Schema Warehouse with Managed SCD2 History	102
6	Query Optimization and Cost Control in Synapse	105
6.1	Week 6 Overview: Making the Warehouse Fast and the Bill Predictable	106
6.2	Monday: Indexing—Columnstore, Heap, and Clustered	106
6.2.1	Clustered Columnstore: The Default	106
6.2.2	Heap and Clustered Rowstore	106
6.3	Tuesday: Partitioning Strategies	107
6.3.1	Partitioning Is Pruning and Switching, Not Performance Magic	107
6.3.2	The Too-Many-Partitions Trap	107
6.4	Wednesday: Workload Management and Materialized Views	107
6.4.1	Workload Groups and Classifiers	107
6.4.2	Materialized Views and Result-Set Caching	108
6.5	Thursday Hands-on Project: Benchmarking Distribution Types	108
6.5.1	Build the Three Tables	108
6.5.2	Benchmark Query	108
6.5.3	Extend: Rowgroup and Partition Effects	109
6.6	Friday Use Case: The Multi-Petabyte Table That Melts the Pool	109
6.6.1	Diagnosis	109
6.6.2	Re-Architecture	109
6.7	Architectural Decision Record Template	109
6.8	Operational Deep Dive: The Monthly Optimization Review	109
6.9	Troubleshooting Patterns	110
6.10	Week 6 Lab Rubric	110
6.11	Interview Q&A	110
6.11.1	Concepts and Trade-offs	110
6.12	Further Reading	110
6.13	Exercises	111
6.14	Capstone Project: Warehouse Performance Rescue	112
7	Synapse Serverless SQL and the Data Lakehouse	114
7.1	Week 7 Overview: Warehousing Without a Warehouse	115
7.2	Monday: Querying the Lake with OPENROWSET	116
7.2.1	How Serverless SQL Executes	116
7.2.2	OPENROWSET Patterns	116
7.3	Tuesday: The Logical Data Warehouse Pattern	117
7.3.1	Views and External Tables as the Serving Layer	117
7.3.2	When the Logical Pattern Breaks Down	117
7.4	Wednesday: Delta Lake—ACID on the Data Lake	117
7.4.1	The Problem with Plain Files	117
7.4.2	What the Log Buys	117
7.4.3	File Hygiene: OPTIMIZE and VACUUM	118
7.5	Thursday Hands-on Project: CSV In, Parquet Out with CETAS	118
7.5.1	Create the Scoped Credential and Data Sources	118
7.5.2	Transform and Materialize	118
7.5.3	Verify the Round Trip	119
7.6	Friday Use Case: A 50 PB Log Lakehouse for a Media Company	119
7.6.1	Target Architecture	119

7.6.2	Cost Reasoning	120
7.7	Architectural Decision Record Template	120
7.8	Operational Deep Dive: Scan Budgets and View Contracts	120
7.9	Troubleshooting Patterns	120
7.10	Week 7 Lab Rubric	121
7.11	Interview Q&A	121
7.11.1	Concepts and Trade-offs	121
7.12	Further Reading	121
7.13	Exercises	122
7.14	Capstone Project: Logical Lakehouse Serving Layer	122
8	Data Governance with Microsoft Purview	125
8.1	Week 8 Overview: Governance Before Serving	126
8.2	Monday: The Purview Data Map and Catalog	126
8.2.1	Register, Organize, Scan	126
8.2.2	The Catalog as the Front Door	127
8.3	Tuesday: Classification and Lineage	127
8.3.1	Automated Classification	127
8.3.2	Lineage for Impact Analysis	127
8.4	Wednesday: Azure Data Share	127
8.4.1	Sharing Without Copying	127
8.4.2	Sharing Discipline	128
8.5	Thursday Hands-on Project: Scan, Classify, Trace	128
8.5.1	Provision and Register	128
8.5.2	Scan and Classify	128
8.5.3	Trace Lineage	128
8.6	Friday Use Case: A Data Clean Room for Ad-Tech	128
8.6.1	Architecture	129
8.7	Architectural Decision Record Template	129
8.8	Operational Deep Dive: Scan Cadence and Coverage Metrics	129
8.9	Troubleshooting Patterns	129
8.10	Week 8 Lab Rubric	130
8.11	Interview Q&A	130
8.11.1	Concepts and Trade-offs	130
8.12	Further Reading	130
8.13	Exercises	131
8.14	Milestone 2 Capstone: Enterprise Data Lakehouse Build	131
III	Batch Processing and ETL Orchestration	134
9	Azure Databricks and Apache Spark	135
9.1	Week 9 Overview: The Compute Engine of the Lakehouse	136
9.2	Monday: Workspace, Clusters, and DBFS	136
9.2.1	Clusters: The Cost Center You Configure	136
9.2.2	DBFS Is Not Your Data Lake	136
9.2.3	Unity Catalog	137
9.3	Tuesday: Spark Internals—Partitions, Stages, Shuffles	137
9.3.1	The Execution Model	137
9.3.2	Skew, Spill, and Small Files	137
9.4	Wednesday: Delta Live Tables	137
9.4.1	Declarative Pipelines	137

9.5	Thursday Hands-on Project: Distributed Aggregation to Delta	138
9.5.1	Create the Workspace and Cluster	138
9.5.2	The Notebook	138
9.5.3	Observe, Then Break It	139
9.6	Friday Use Case: Migrating a 100-Node Hadoop Cluster	139
9.6.1	Assessment and Strangler Pattern	139
9.6.2	The Cost Case	139
9.7	Architectural Decision Record Template	140
9.8	Operational Deep Dive: Job Reliability and Photon	140
9.9	Troubleshooting Patterns	140
9.10	Week 9 Lab Rubric	140
9.11	Interview Q&A	140
9.11.1	Concepts and Trade-offs	140
9.12	Further Reading	141
9.13	Exercises	141
9.14	Capstone Project: Production-Shaped Spark Aggregation Platform	142
10	Azure Data Factory Foundations: Visual ETL at Scale	144
10.1	Week 10 Overview: Orchestration as a First-Class Discipline	145
10.2	Monday: The ADF Object Model	145
10.2.1	Linked Services and Datasets	145
10.2.2	Pipelines, Activities, and Dependencies	146
10.2.3	Triggers	146
10.3	Tuesday: Mapping Data Flows	146
10.3.1	Visual Transformations on Managed Spark	146
10.3.2	The Expression Language	146
10.3.3	When Visual Beats Code—and When It Doesn't	146
10.4	Wednesday: Integration Runtimes and Pipelines as Artifacts	147
10.4.1	The Three Integration Runtimes	147
10.4.2	Azure IR	147
10.4.3	Self-Hosted IR	147
10.4.4	Azure-SSIS IR	147
10.4.5	Everything Is an ARM-Deployable Artifact	147
10.4.6	Parameterization for Environment Promotion	148
10.5	Thursday Hands-on Project: Join, Clean, and Load with Mapping Data Flows	148
10.5.1	Create the Factory and Connections	148
10.5.2	Author the Data Flow	149
10.5.3	Wrap, Trigger, and Break It	149
10.6	Friday Use Case: Mapping Data Flows vs. Databricks Notebooks for Business Analysts	149
10.6.1	Evaluation	149
10.6.2	Recommendation	150
10.7	Architectural Decision Record Template	150
10.8	Operational Deep Dive: Idempotent Reruns and Audit Tables	150
10.9	Troubleshooting Patterns	150
10.10	Week 10 Lab Rubric	150
10.11	Interview Q&A	151
10.11.1	Concepts and Trade-offs	151
10.12	Further Reading	151
10.13	Exercises	151
10.14	Capstone Project: Parameterized Visual ETL with Failure Discipline	152

11 Change Data Capture, Migration, and Data Contracts	154
11.1 Week 11 Overview: Reaching Data and Trusting It	155
11.2 Monday: Azure Database Migration Service	155
11.2.1 DMS Architecture	155
11.2.2 Reaching the Source	155
11.3 Tuesday: Change Data Capture Mechanisms	155
11.3.1 Platform CDC Mechanisms	155
11.3.2 Replication Lag as an SLO	156
11.4 Wednesday: Event-Driven Ingestion and Data Contracts	156
11.4.1 Event Grid as the Notification Fabric	156
11.4.2 Data Contracts at the Boundary	156
11.5 Thursday Hands-on Project: DMS CDC with a Contract Gate	157
11.5.1 Stand Up Source and Migration Service	157
11.5.2 Add the Contract Gate	157
11.5.3 Wire the Lag Alert	157
11.6 Friday Use Case: Oracle to Synapse in Near Real Time	158
11.6.1 Target Architecture	158
11.7 Architectural Decision Record Template	158
11.8 Operational Deep Dive: SHIR Fleet Hygiene	158
11.9 Troubleshooting Patterns	158
11.10 Week 11 Lab Rubric	159
11.11 Interview Q&A	159
11.11.1 Concepts and Trade-offs	159
11.12 Further Reading	159
11.13 Exercises	160
11.14 Capstone Project: Hybrid CDC Pipeline with Contract Enforcement	160
12 Delta Lake Medallion Architecture and Pipeline Orchestration	162
12.1 Week 12 Overview: From Moving Data to Operating Data Movement	163
12.2 Monday: Medallion Zones as Quality Contracts	163
12.2.1 Bronze: Fidelity Above All	163
12.2.2 Silver: Validated and Conformed	163
12.2.3 Gold: Business-Modeled	164
12.3 Tuesday: Advanced ADF Control Flow	164
12.3.1 The Control-Flow Toolkit	164
12.3.2 Event Grid Triggers	164
12.4 Wednesday: Managed Airflow and Logic Apps	165
12.4.1 Managed Airflow in ADF	165
12.4.2 Logic Apps: The Notification and Process Layer	165
12.5 Thursday Hands-on Project: Event-Driven Databricks Orchestration with Failure Notification	165
12.5.1 Wire the Trigger	165
12.5.2 Run the Notebook	165
12.5.3 Close the Failure Loop	166
12.6 Friday Use Case: Self-Healing Orchestration for Banking Reconciliation	166
12.6.1 The Self-Healing Design	166
12.6.2 What Self-Healing Is Not	166
12.7 Architectural Decision Record Template	166
12.8 Operational Deep Dive: Monitoring the Orchestrator	167
12.9 Troubleshooting Patterns	167
12.10 Week 12 Lab Rubric	167

12.11	Interview Q&A	167
12.11.1	Concepts and Trade-offs	167
12.12	Further Reading	168
12.13	Exercises	168
12.14	Milestone 3 Capstone: Automated Batch ELT Pipeline	169
IV	Streaming and Real-Time Analytics	171
13	Event Ingestion at Scale: Event Hubs and IoT Hub	172
13.1	Week 13 Overview: Unbounded Data Arrives	173
13.2	Monday: Decoupling with Service Bus	173
13.2.1	Queues and Topics	173
13.2.2	Service Bus vs. Event Hubs	173
13.3	Tuesday: Event Hubs—Partitions, Consumer Groups, Throughput	174
13.3.1	Partitions: Parallelism and Ordering	174
13.3.2	Throughput Units and Consumer Groups	174
13.3.3	At-Least-Once and Idempotency	174
13.4	Wednesday: Capture and the Kafka Endpoint	174
13.4.1	Capture: The Stream-to-Lake Bridge	174
13.4.2	Kafka Compatibility	175
13.5	Thursday Hands-on Project: IoT Telemetry Producer with Capture	175
13.5.1	Create the Namespace, Hub, and Capture	175
13.5.2	The Producer	175
13.5.3	Verify	176
13.6	Friday Use Case: Telemetry for Five Million Mobile Games	176
13.6.1	Design	176
13.6.2	The Numbers That Matter	176
13.7	Architectural Decision Record Template	177
13.8	Operational Deep Dive: IoT Hub and Device Identity	177
13.9	Troubleshooting Patterns	177
13.10	Week 13 Lab Rubric	177
13.11	Interview Q&A	178
13.11.1	Concepts and Trade-offs	178
13.12	Further Reading	178
13.13	Exercises	178
13.14	Capstone Project: Partitioned Telemetry Ingestion with Capture	179
14	Real-Time Processing with Azure Stream Analytics	181
14.1	Week 14 Overview: SQL Against the Firehose	182
14.2	Monday: ASA Architecture and Query Language	182
14.2.1	Inputs, Query, Outputs	182
14.2.2	A First Query	182
14.3	Tuesday: The Four Windows	183
14.3.1	Tumbling and Hopping	183
14.3.2	Sliding and Session	183
14.4	Wednesday: Late and Out-of-Order Events	183
14.4.1	The Two Disorder Knobs	183
14.4.2	The Silent-Stall Problem	184
14.5	Thursday Hands-on Project: Five-Minute Click Aggregates to Azure SQL	184
14.5.1	Create the Job	184
14.5.2	The Query	184

14.5.3	Test the Time Policies	184
14.6	Friday Use Case: The Offline-Mobile Dashboard	185
14.6.1	Diagnosis	185
14.6.2	The Fix	185
14.7	Architectural Decision Record Template	185
14.8	Operational Deep Dive: Sizing and SU Utilization	185
14.9	Troubleshooting Patterns	185
14.10	Week 14 Lab Rubric	186
14.11	Interview Q&A	186
14.11.1	Concepts and Trade-offs	186
14.12	Further Reading	186
14.13	Exercises	187
14.14	Capstone Project: Windowed Engagement Aggregates with Explicit Time Policies	187
15	Advanced Streaming with Spark Structured Streaming on Databricks	189
15.1	Week 15 Overview: When SQL-Over-Streams Is Not Enough	190
15.2	Monday: State Management—Watermarks, Stores, and Checkpoints	190
15.2.1	Why Unbounded State Kills Jobs	190
15.2.2	State-Store Providers	190
15.2.3	Checkpoints Bind Everything	190
15.3	Tuesday: Micro-Batch, Continuous, and Exactly-Once	191
15.3.1	Execution Modes	191
15.3.2	End-to-End Exactly-Once	191
15.4	Wednesday: Event Hubs via Kafka and Stream-Stream Joins	191
15.4.1	Consuming the Hub	191
15.4.2	Stream-Stream Joins with Bounded State	192
15.5	Thursday Hands-on Project: Event Hubs to Delta with Windowed Aggregation	192
15.5.1	Setup	193
15.5.2	Run and Observe	193
15.5.3	Break It Deliberately	193
15.6	Friday Use Case: Real-Time HL7 Anonymization	193
15.7	Architectural Decision Record Template	194
15.8	Operational Deep Dive: Lag, Restarts, and Backfills	194
15.9	Troubleshooting Patterns	194
15.10	Week 15 Lab Rubric	194
15.11	Interview Q&A	194
15.11.1	Concepts and Trade-offs	194
15.12	Further Reading	195
15.13	Exercises	195
15.14	Capstone Project: Exactly-Once Windowed Aggregation from Event Hubs	196
16	Business Intelligence, Power BI, and Reverse ETL	198
16.1	Week 16 Overview: The Last Mile of the Data Platform	199
16.2	Monday: Power BI Architecture	199
16.2.1	Desktop, Service, Semantic Models, Dataflows	199
16.2.2	The Serving Contract	199
16.3	Tuesday: Import, DirectQuery, and Dual	200
16.3.1	Import Mode	200
16.3.2	DirectQuery Mode	200
16.3.3	Dual Mode and Aggregation Tables	200
16.4	Wednesday: RLS, DAX, and Reverse ETL	201
16.4.1	Row-Level Security	201

16.4.2	DAX Fundamentals	201
16.4.3	Reverse ETL: Warehouse Truth Back to Operations	201
16.4.4	The Pattern	201
16.4.5	Identity Resolution Before Sync	201
16.4.6	Sync Frequency as a Trade-Off	202
16.5	Thursday Hands-on Project: DirectQuery Dashboard with RLS	202
16.5.1	Connect and Model	202
16.5.2	Build and Secure the Dashboard	202
16.5.3	Performance Check	203
16.6	Friday Use Case: 500 Analysts, One Governed Model	203
16.6.1	Target Design	203
16.6.2	Operating Model	203
16.7	Architectural Decision Record Template	203
16.8	Operational Deep Dive: Refresh, Endorsement, and Tenant Settings	203
16.9	Troubleshooting Patterns	204
16.10	Week 16 Lab Rubric	204
16.11	Interview Q&A	204
16.11.1	Concepts and Trade-offs	204
16.12	Further Reading	204
16.13	Exercises	205
16.14	Milestone 4 Capstone: Real-Time Fraud Detection	206
V	Machine Learning and MLOps	209
17	Feature Engineering and In-Database Machine Learning	210
17.1	Week 17 Overview: Models Where the Data Lives	211
17.2	Monday: Feature Engineering as Platform Discipline	211
17.2.1	Features Are Contracts	211
17.2.2	Point-in-Time Correctness	211
17.3	Tuesday: T-SQL PREDICT—Scoring Without a Server	212
17.3.1	The Mechanism	212
17.3.2	When In-Database Scoring Wins and Loses	212
17.4	Wednesday: Training with Synapse Spark and Exporting ONNX	212
17.4.1	The Training Path	212
17.4.2	The Schema Alignment Contract	213
17.5	Thursday Hands-on Project: Churn Model from Spark to SQL	213
17.5.1	Steps	213
17.6	Friday Use Case: SQL-Only Inventory Forecasting for 1,000 Stores	214
17.6.1	Architecture	214
17.7	Architectural Decision Record Template	214
17.8	Operational Deep Dive: Scoring SLAs and Model Inventory	214
17.9	Troubleshooting Patterns	215
17.10	Week 17 Lab Rubric	215
17.11	Interview Q&A	215
17.11.1	Concepts and Trade-offs	215
17.12	Further Reading	215
17.13	Exercises	216
17.14	Capstone Project: In-Database Churn Scoring with a Schema Contract	216
18	Machine Learning Pipelines with Azure ML and Databricks	219
18.1	Week 18 Overview: The ML Platform Layer	220

18.2 Monday: Workspace, Compute, and Data Access	220
18.2.1 Compute Instances vs. Compute Clusters	220
18.2.2 Datastores and Managed Identity	220
18.3 Tuesday: AutoML, Designer, or Custom Code	220
18.3.1 The Three Authoring Modes	220
18.3.2 The Thursday Preview: AutoML to Endpoint in One Afternoon	221
18.4 Wednesday: AML Pipelines—Components and Environments	221
18.4.1 Pipelines from Versioned Parts	221
18.4.2 The Managed Feature Store	221
18.4.3 Databricks in the Picture	222
18.5 Thursday Hands-on Project: Preprocess, Train, Evaluate, Register	222
18.5.1 Provision the Workspace	222
18.5.2 Author and Run the Pipeline	222
18.5.3 Gate Registration on a Metric	222
18.6 Friday Use Case: GPU PyTorch over a Governed Lake	222
18.6.1 Design	222
18.7 Architectural Decision Record Template	223
18.8 Operational Deep Dive: Pipeline Schedules and Reuse	223
18.9 Troubleshooting Patterns	223
18.10 Week 18 Lab Rubric	223
18.11 Interview Q&A	224
18.11.1 Concepts and Trade-offs	224
18.12 Further Reading	224
18.13 Exercises	224
18.14 Capstone Project: Gated Training Pipeline over Governed Features	225
19 Model Serving, Monitoring, and the Retraining Loop	227
19.1 Week 19 Overview: The Model Is Not Done When It Trains	228
19.2 Monday: Managed Endpoints—Real-Time and Batch	228
19.2.1 Online Endpoints	228
19.2.2 Batch Endpoints	228
19.3 Tuesday: Safe Deployment—Blue-Green and Gated Promotion	229
19.3.1 Traffic Shifting	229
19.3.2 The Promotion Gate	229
19.4 Wednesday: Drift Detection and the Retraining Loop	229
19.4.1 Three Kinds of Drift	229
19.4.2 The Automated Loop	229
19.5 Thursday Hands-on Project: Deploy, Watch, Drift, Retrain	230
19.5.1 Deploy and Test	230
19.5.2 Monitor and Inject Drift	230
19.5.3 Close the Loop Manually	231
19.6 Friday Use Case: MLOps for a Production Recommendation Engine	231
19.7 Architectural Decision Record Template	231
19.8 Operational Deep Dive: Endpoint SLIs and Cost	231
19.9 Troubleshooting Patterns	231
19.10 Week 19 Lab Rubric	232
19.11 Interview Q&A	232
19.11.1 Concepts and Trade-offs	232
19.12 Further Reading	232
19.13 Exercises	233
19.14 Capstone Project: The Full MLOps Loop for Churn	233

20 Unstructured Data and Generative AI Data Products	235
20.1 Week 20 Overview: The Other 80% of Enterprise Data	236
20.2 Monday: Document Intelligence and Vision	236
20.2.1 Layout, Prebuilt, and Custom Models	236
20.2.2 Confidence Is Part of the Schema	236
20.3 Tuesday: Language, Translator, and Enrichment	237
20.3.1 Language Understanding as a Transform	237
20.3.2 Batching and Cost	237
20.4 Wednesday: Event-Driven Document Processing and Search	237
20.4.1 The Function-on-Upload Pattern	237
20.4.2 Making It Searchable	237
20.5 Thursday Hands-on Project: Invoice Extraction Function	238
20.5.1 Provision	238
20.5.2 Test the Confidence Path	238
20.6 Friday Use Case: Terabytes of User Images, Searchable	238
20.6.1 Architecture	238
20.6.2 From Search to RAG	238
20.7 Architectural Decision Record Template	239
20.8 Operational Deep Dive: Evaluation as Pipeline Testing	239
20.9 Troubleshooting Patterns	239
20.10 Week 20 Lab Rubric	239
20.11 Interview Q&A	240
20.11.1 Concepts and Trade-offs	240
20.12 Further Reading	240
20.13 Exercises	240
20.14 Milestone 5 Capstone: End-to-End MLOps Pipeline	241
 VI Security, Governance, and Exam Prep	 244
21 Zero-Trust Data Platforms: Identity, Networking, and Secrets	245
21.1 Week 21 Overview: Part VI Opens with the Perimeter	246
21.2 Monday: Entra ID, Service Principals, and Managed Identities	246
21.2.1 The Identity Spectrum	246
21.2.2 The Access Pattern	246
21.3 Tuesday: Private Networking—Endpoints, DNS, and Bastion	247
21.3.1 Private Endpoint vs. Service Endpoint	247
21.3.2 DNS: The Step Everyone Forgets	247
21.3.3 Administration Without Exposure	247
21.4 Wednesday: Key Vault, Rotation, and CMK	247
21.4.1 Secrets, Keys, Certificates	247
21.4.2 Automated Rotation	247
21.4.3 Customer-Managed Keys	248
21.5 Thursday Hands-on Project: A Private-Only Storage Account	248
21.5.1 Build the Perimeter	248
21.5.2 Prove It	249
21.6 Friday Use Case: A No-Exfiltration Perimeter for Healthcare	249
21.6.1 The Design	249
21.7 Architectural Decision Record Template	249
21.8 Operational Deep Dive: Auditing the Perimeter	249
21.9 Troubleshooting Patterns	249
21.10 Week 21 Lab Rubric	250

21.11	Interview Q&A	250
21.11.1	Concepts and Trade-offs	250
21.12	Further Reading	250
21.13	Exercises	251
21.14	Capstone Project: Private-Only Lakehouse Perimeter	251
22	Observability, FinOps, and Governance at Scale	254
22.1	Week 22 Overview: The Operating Layer	255
22.2	Monday: Purview Deep Dive—Classifications, Labels, Policies	255
22.2.1	Custom Classification Rules	255
22.2.2	Sensitivity Labels That Travel	255
22.2.3	Data Policies and Access Provisioning	255
22.3	Tuesday: Lineage and Quality Gates	256
22.3.1	Lineage for Impact Analysis	256
22.3.2	Quality Gates in the Pipeline	256
22.4	Wednesday: Observability and FinOps	256
22.4.1	The Observability Baseline	256
22.4.2	FinOps as Engineering	257
22.4.3	Data Product SLOs	257
22.5	Thursday Hands-on Project: Custom Classification and a Quality Gate	257
22.5.1	Custom Classification	257
22.5.2	The Quality Gate	257
22.6	Friday Use Case: Governed Data Mesh for a Five-Domain Conglomerate	257
22.6.1	What Is Centralized, What Is Not	258
22.7	Architectural Decision Record Template	258
22.8	Operational Deep Dive: The Monthly Platform Review	258
22.9	Troubleshooting Patterns	258
22.10	Week 22 Lab Rubric	259
22.11	Interview Q&A	259
22.11.1	Concepts and Trade-offs	259
22.12	Further Reading	259
22.13	Exercises	260
22.14	Capstone Project: The Governed, Observable, Costed Platform	260
23	Compliance, Data Privacy, and Certification Exam Preparation	262
23.1	Week 23 Overview: Privacy Engineering and the Exam	263
23.2	Monday: Dynamic Data Masking	263
23.2.1	Presentation-Layer Protection	263
23.3	Tuesday: Always Encrypted vs. TDE	263
23.3.1	Two Encryptions, Two Threat Models	263
23.4	Wednesday: Policy, Defender, and the Legal-Hold Interaction	264
23.4.1	Continuous Audit	264
23.4.2	Legal Hold Meets Erasure	264
23.5	Thursday Hands-on Project: Masking and a Crypto-Shredding Erasure Drill	264
23.5.1	Drill 1: DDM	264
23.5.2	Drill 2: Crypto-Shredding Erasure	265
23.6	Friday Use Case: Tokenized Third-Party PII Ingestion	265
23.7	Certification Preparation: DP-203 Legacy and DP-700 Reality	265
23.7.1	Domain-Weighted Study Plan	266
23.7.2	Mock-Exam Loop	266
23.8	Architectural Decision Record Template	266
23.9	Operational Deep Dive: The Privacy Operations Calendar	266

23.10	Troubleshooting Patterns	267
23.11	Week 23 Lab Rubric	267
23.12	Interview Q&A	267
23.12.1	Concepts and Trade-offs	267
23.13	Further Reading	267
23.14	Exercises	268
23.15	Capstone Project: The Erasure-Capable Privacy Architecture	269
24	Production DataOps and the Enterprise Capstone	271
24.1	Week 24 Overview: The Program Becomes a Platform	272
24.2	Monday: Terraform and Bicep for Data Platforms	272
24.2.1	Terraform with the azurerm Provider	272
24.2.2	Bicep: The Azure-Native Alternative	273
24.3	Tuesday: dbt—Transformations as Reviewed Code	273
24.3.1	Models, Tests, Documentation	273
24.3.2	Incremental Models	273
24.4	Wednesday: CI/CD and Zero-Downtime Schema Evolution	273
24.4.1	The Pipeline	273
24.4.2	Expand–Contract Migrations	274
24.5	Thursday Hands-on Project: Terraform + dbt + GitHub Actions	274
24.5.1	Build the Repository	274
24.5.2	Prove the Gates	274
24.6	Friday: Program Review and Certification Scheduling	275
24.7	Milestone 6 Capstone: The Secure, Governed, Multi-Subscription Data Mesh	275
24.8	Key Takeaways for the Whole Program	277
24.9	Exercises	277
A	Azure CLI Reference for Data Engineers	279
A.1	Account, Identity, and Access	279
A.2	Storage and ADLS Gen2	279
A.3	Synapse Analytics	280
A.4	Data Factory, Databricks, and Event Hubs	280
A.5	Networking, Key Vault, and Security	281
A.6	Monitoring and Cost	281
A.7	Universal Flags and Query Patterns	281
A.8	Cross-References	281
B	Spark SQL and Delta Lake Quick Reference	282
B.1	Reading and Writing	282
B.2	DataFrame Operations and the Shuffle Boundary	283
B.3	Delta Lake Operations	283
B.4	Structured Streaming	283
B.5	Synapse T-SQL Essentials	284
B.6	Useful DMVs	284
B.7	Cross-References	284
C	Governance and Production-Readiness Checklists	285
C.1	New Data Product Go-Live	285
C.2	Schema Change (Expand–Contract) Gate	286
C.3	Streaming Job Production Gate	286
C.4	ML Model Promotion Gate	286
C.5	Monthly Platform Review Agenda	287

C.6	Quarterly Drills	287
D	Cross-Cloud Service Equivalence	288
D.1	Free Tiers and Environment Setup	288
D.2	Object and Data-Lake Storage	289
D.3	Managed Relational Databases	289
D.4	Elastic Compute and Analytical Warehouses	290
D.5	Data Organization and the Medallion Pattern	290
D.6	Document Data Modeling	291
D.7	Globally Distributed and NoSQL Databases	291
D.8	Managed Apache Spark	291
D.9	Query Acceleration and Materialized Views	292
D.10	In-Memory Caching and Hybrid Ingestion	292
D.11	Wide-Column and Key-Value NoSQL	293
D.12	Data Protection: Cloning, Time Travel, and Migration	293
D.13	MPP Data Warehouses	293
D.14	Warehouse Physical Design and Workload Isolation	294
D.15	Serverless SQL over the Lake	294
D.16	Data Catalogs and Governance Planes	295
D.17	Managed Visual ETL and Orchestration	295
D.18	Change Data Capture, Migration, and Hybrid Connectivity	296
D.19	Medallion Orchestration and Scheduling	296
D.20	Event Streaming Backbones	297
D.21	Managed SQL over Streams	297
D.22	Spark Structured Streaming	298
D.23	Business Intelligence and Semantic Models	298
D.24	Feature Engineering and In-Database Machine Learning	299
D.25	Managed Machine Learning Platforms	299
D.26	Model Serving, Monitoring, and Retraining	300
D.27	Unstructured Data and Generative AI	300
D.28	Zero-Trust Networking, Identity, and Secrets	301
D.29	Observability, FinOps, and Quality Gates	301
D.30	Data Privacy and Compliance Primitives	302
D.31	DataOps: Infrastructure as Code and CI/CD	302
E	Environment Setup Reference	304
E.1	Azure Subscription and Microsoft Entra ID	304
E.2	Azure CLI	304
E.3	Azure PowerShell	305
E.4	Python SDKs and Local Tooling	305
E.5	Bicep and Terraform	305
E.6	Cost Guardrails	306
E.7	Per-Chapter Tool Requirements	306
F	Lab Solution Sketches	308
F.1	Part I: Core Data Infrastructure and Storage	308
F.2	Part II: Data Warehousing and Analytics	309
F.3	Part III: Batch Processing and ETL Orchestration	310
F.4	Part IV: Streaming and Real-Time Analytics	311
F.5	Part V: Machine Learning and MLOps	312
F.6	Part VI: Security, Governance, and Exam Prep	313

G	Certification Exam Mapping: DP-203 and DP-700	315
G.1	How the Exams Differ	315
G.2	DP-203 Domain Mapping	315
G.3	DP-700 Domain Mapping	316
G.4	Objective-to-Chapter Quick Reference	316
G.5	Study Strategy	317
G.6	Self-Assessment Checklist	317
H	Interview Question Bank	320
H.1	Storage and Databases (Chapters 1–4)	320
H.2	Warehousing and Analytics (Chapters 5–7)	322
H.3	Governance and Catalogs (Chapters 8 and 22)	323
H.4	Batch Processing, ETL, and Orchestration (Chapters 9–12)	323
H.5	Streaming and Real-Time (Chapters 13–15)	324
H.6	Business Intelligence and Serving (Chapter 16)	325
H.7	Machine Learning and Generative AI (Chapters 17–20)	326
H.8	Security, Privacy, and Compliance (Chapters 21 and 23)	327
	Glossary	329

List of Figures

1	Chapter 0 setup flow from identity and subscription selection to a tagged, budgeted lab resource group.	4
1.1	ADLS Gen2 as the durable foundation for ingestion, validation, curation, serving, governance, lifecycle control, and disaster recovery.	21
1.2	Multi-region ADLS Gen2 architecture for compliant healthcare imaging with GZRS/RA-GZRS, private ingestion, governed processing, and read-access disaster recovery.	28
1.3	Capstone reference architecture for a compliant medical imaging archive on ADLS Gen2 with hierarchical namespace, customer-managed keys, lifecycle tiering, immutable retention, GZRS, and object replication.	34
2.1	Azure SQL Hyperscale separates compute from log and page storage so large databases can scale and restore differently from traditional monolithic storage. .	42
2.2	Azure SQL auto-failover groups provide listener endpoints over asynchronously replicated databases for regional resilience.	44
2.3	Zero-downtime migration pattern from an on-premises SQL Server monolith to Azure SQL Managed Instance using assessment, seeding, continuous synchronization, and controlled cutover.	48
2.4	Capstone reference architecture for a high-availability Azure SQL Managed Instance migration with continuous sync, failover group, downstream lake validation, and operational evidence.	53
3.1	Cosmos DB global distribution places replicas near users while consistency and conflict policies define correctness during replication.	60
3.2	Cosmos DB consistency levels trade read freshness and ordering for lower coordination, latency, and regional independence.	61
3.3	Global leaderboard serving architecture using Cosmos DB multi-region writes, Session consistency, Change Feed projections, and regional read paths.	65
3.4	Capstone reference architecture for a global leaderboard using regional score APIs, Cosmos DB multi-region writes, Change Feed projections, lake export, and monitoring evidence.	70
4.1	Cache-aside between an application, Azure Cache for Redis, and Azure SQL Database keeps Azure SQL authoritative while Redis absorbs hot reads.	77
4.2	Spiky-traffic mobile data API using APIM policy controls, Redis hot-path acceleration, Cosmos DB Integrated Cache, Azure SQL, and shared observability. . .	82
4.3	Milestone 1 reference architecture integrating ADLS Gen2, Azure SQL Database Serverless, Cosmos DB, Redis, APIM, and Change Feed analytics export.	87

5.1	Capstone architecture: lake-staged Parquet, COPY with managed identity, round-robin staging, hash-distributed columnstore facts, and replicated dimensions. . .	103
7.1	50 PB lakehouse: one copy of data in Delta zones; serverless views serve the hot slice; Spark handles deep-history scans.	119
8.1	Data clean room: tokenized keys match across tenants; only aggregates cross the boundary; Purview supplies classification and audit evidence.	129
8.2	Milestone 2 architecture: one Delta lakehouse, Purview-governed, served logically through serverless views to Power BI.	132
9.1	Capstone architecture: policy-governed job clusters running a DLT aggregation into cataloged Delta tables.	142
10.1	Capstone architecture: window-triggered ADF pipeline wrapping a Mapping Data Flow, with audit and notification paths.	153
11.1	Near-real-time Oracle synchronization: SHIR for reachability, DMS for continuous replication, contract gate before Synapse.	158
12.1	Self-healing reconciliation: event-driven fan-out, isolated child pipelines, quarantine routing, and a single notification router.	166
12.2	Milestone 3 architecture: event-driven ELT from lake arrival to warehouse load with quality gates and notifications.	169
13.1	Capstone architecture: partitioned hub with isolated consumer groups, real-time scoring, and lake capture.	180
15.1	Real-time HL7 anonymization: tokenize in motion, persist only de-identified data, serve through governed views.	193
16.1	Milestone 4 architecture: partitioned ingestion, sliding-window detection, idempotent alert sink, live dashboard, lake reconciliation.	207
17.1	SQL-only forecasting: Spark trains weekly, PREDICT scores nightly, and the whole serving path is operable by a SQL team.	214
18.1	Capstone architecture: governed features feed a gated pipeline; AutoML provides the recorded baseline.	226
19.1	The MLOps loop: serve, collect, monitor, alert, retrain, gate, promote. Humans own the promotion edge.	230
20.1	Milestone 5 architecture: ADF-orchestrated feature extraction, gated AML training, nightly batch scoring, drift-triggered retraining.	242
21.1	Capstone perimeter: no public path in, allowlisted egress out, every hop authenticated by identity.	252
23.1	Capstone privacy architecture: tokenize at the boundary, hold only tokens downstream, erase by destroying keys.	269
24.1	Final capstone: a three-subscription, zero-trust, code-deployed data mesh with governance as the default path.	276

List of Tables

1	The Azure free account combines time-limited credit, service-specific allowances, and ongoing free quotas.	2
2	Tags make cost allocation, cleanup, and operational ownership visible.	9
1.1	Flat Blob Storage and ADLS Gen2 share durability but differ in namespace behavior and analytics ergonomics.	20
1.2	Access tiers should be selected by data-product usage, not by a single account-wide habit.	22
1.3	Migration tools differ by scale, automation, observability, and operational risk. .	24
1.4	Week 1 rubric for storage foundation deliverables.	30
2.1	Azure SQL purchasing choices should follow workload pattern, compatibility, and operating model rather than habit.	41
2.2	PostgreSQL operational controls must be combined to meet availability and recoverability goals.	43
2.3	Week 2 rubric for relational database architecture and hands-on deliverables. . .	50
3.1	Week 3 rubric for Cosmos DB architecture, hands-on implementation, and global serving design.	67
4.1	Week 4 rubric for caching, hybrid ingestion, hands-on implementation, and operational evidence.	84
5.1	Distribution selection is the highest-leverage physical design decision in a dedicated SQL pool.	96
5.2	Azure compute cost comparison for warehousing workloads.	99
5.3	Week 5 rubric for the dedicated-pool deliverables.	100
6.1	Partitioning complements distribution; it does not replace it.	107
6.2	Week 6 rubric for the optimization deliverables.	110
7.1	Week 7 rubric for the serverless lakehouse deliverables.	121
8.1	Week 8 rubric for the governance deliverables.	130
9.1	Week 9 rubric for the Databricks deliverables.	140
10.1	Integration runtime selection.	147
10.2	Parameterization boundaries for promotable ADF artifacts.	148
10.3	Mapping Data Flows versus Databricks notebooks for analyst-built pipelines. . .	149
10.4	Week 10 rubric for the ADF foundations deliverables.	150
11.1	Week 11 rubric for the integration deliverables.	159

12.1	Week 12 rubric for the orchestration deliverables.	167
13.1	Queues distribute work; streams record history.	174
13.2	Week 13 rubric for the ingestion deliverables.	177
14.1	Window selection is a statement about the business question, not the data. . . .	183
14.2	Week 14 rubric for the Stream Analytics deliverables.	186
15.1	Week 15 rubric for the streaming deliverables.	194
16.1	Storage-mode selection for semantic models.	200
16.2	Reverse-ETL cadence trades freshness against API cost and rate limits.	202
16.3	Week 16 rubric for the Power BI deliverables.	204
17.1	Week 17 rubric for the feature-engineering deliverables.	215
18.1	Week 18 rubric for the AML pipeline deliverables.	223
19.1	Choose the endpoint family from the workload’s latency and volume, not from fashion.	228
19.2	Week 19 rubric for the serving and monitoring deliverables.	232
20.1	Week 20 rubric for the unstructured-data deliverables.	239
21.1	Week 21 rubric for the zero-trust deliverables.	250
22.1	The mesh boundary: the platform owns the rails; domains own the trains.	258
22.2	Week 22 rubric for the operating-layer deliverables.	259
23.1	TDE and Always Encrypted answer different threat models; use both when both threats are real.	264
23.2	Mapping this book to the certification skill areas.	266
23.3	Week 23 rubric for the compliance deliverables.	267

Getting Started: Your Azure Free Account and Environment

Learning Objectives

- Create an Azure free account, understand the \$200 credit window, and distinguish free credit, 12-month free services, and always-free tiers.
- Explain the Azure tenant, subscription, resource group, and resource hierarchy used throughout this book.
- Secure the lab identity with Microsoft Entra ID, MFA, Azure RBAC, least privilege, and separated administrative accounts.
- Install and authenticate Azure CLI on Windows PowerShell, and recognize Azure PowerShell and Azure Cloud Shell alternatives.
- Configure cost guardrails with Microsoft Cost Management budgets, alerts, tags, and cleanup routines before deploying labs.
- Apply the book's default region, naming, and tagging conventions to make every exercise reproducible and easy to delete.
- Verify the environment by creating, listing, and deleting a resource group and storage account from PowerShell.

Key Terms

Tenant: a Microsoft Entra ID directory that stores identities, groups, applications, and security policies. **Subscription:** an Azure billing and management boundary that contains resource groups and resources. **Resource group:** a logical lifecycle container for Azure resources that are deployed, tagged, authorized, and deleted together. **Azure RBAC:** the role-based authorization system that grants actions over Azure scopes. **Budget:** a Microsoft Cost Management rule that tracks spending and sends alerts when thresholds are crossed.

0.1 Why Chapter 0 Exists

A data engineering book should not begin with an expensive Spark cluster or a production data lake. It should begin with a controlled account, a known command-line environment, a least-privilege identity, a budget, and a cleanup habit. Most cloud accidents in early learning projects are not caused by advanced distributed-systems failures. They are caused by using the wrong subscription, leaving a resource group running, granting a broad role to make an error disappear, or forgetting that globally unique services continue to bill after the tutorial window

closes.

This chapter establishes the operating baseline for every Azure exercise in the book. The environment is intentionally Windows plus PowerShell because that is the assumed local shell for this training repository. Azure CLI commands are shown in PowerShell syntax when variables or line continuation are needed. Azure Cloud Shell remains useful when a local machine cannot install software, but the preferred workflow for repeatable labs is a local terminal, a named subscription, a standard resource group prefix, and explicit cleanup.

Definition

A **preliminary cloud environment** is the smallest secure setup that lets you deploy, inspect, measure, and delete learning resources without relying on production credentials or guesswork.

Note

Azure product names change over time. Azure Active Directory is now **Microsoft Entra ID**. Older documentation, screenshots, and exam material may still say Azure AD, but this book uses Microsoft Entra ID except when quoting legacy names.

0.2 Creating an Azure Free Account

0.2.1 What the Free Account Includes

Microsoft's Azure free account is designed for learning, evaluation, and small experiments. At sign-up time, Microsoft commonly offers three distinct benefits: a \$200 credit that can be used during the first 30 days, selected popular services free for 12 months, and always-free service quantities that continue as long as the account remains eligible [32]. These benefits are related but not identical. The \$200 credit is a short introductory balance. The 12-month services are service-specific allowances. Always-free tiers are ongoing limits for selected services, often with monthly quotas.

For this book, the free account is sufficient for many foundational labs if you create only the requested resources, choose modest SKUs, and delete resource groups after each exercise. It is not a blank check: every create command should have a matching cleanup step.

Free-account item	What it means	Engineering caution
\$200 credit	Introductory credit for eligible Azure charges during the first 30 days	Credit can be exhausted quickly by large compute, premium storage, or forgotten services
12-month free services	Popular services have limited free quantities for 12 months	Only the listed quantity and SKU are free; exceeding limits bills normally
Always-free tiers	Selected services have ongoing free monthly allowances	Quotas reset by service rules and may not cover realistic data workloads
Spending limit	Some free or credit subscriptions prevent charges beyond credit until upgraded	Upgrading or changing offer type can remove the safety net

Table 1: The Azure free account combines time-limited credit, service-specific allowances, and ongoing free quotas.

0.2.2 Sign-up Walkthrough

Begin at the Azure free account page and choose *Start free*. You need a Microsoft account, such as an Outlook account or another email address registered as a Microsoft account. Microsoft asks for contact information, phone verification, agreement to terms, and a payment card. The card is used for identity verification and fraud prevention. For a free account, Microsoft states that you are not automatically charged unless you explicitly upgrade or exceed an offer that permits billing [32].

A practical sign-up sequence is:

1. Open the Azure free account page in a private browser window so an existing work or school login does not accidentally select the wrong tenant.
2. Sign in with the Microsoft account that will own the learning environment. If you already have an organizational Azure account, keep it separate from this book's lab account unless your instructor says otherwise.
3. Complete identity verification with phone and card. Use a card you are authorized to use, and record who owns the account in your lab notebook.
4. Open the Azure Portal and confirm that a subscription exists under *Subscriptions*. Copy the subscription name and subscription ID.
5. Immediately create a budget, enable MFA, and decide a naming prefix before deploying the first technical resource.

Common Pitfall

Using the wrong tenant during sign-up. Many learners have a personal Microsoft account and a work or school account with the same email address. If the portal asks whether to use a personal or work account, pause and verify which identity should own the labs. Deploying into an employer tenant can create policy, billing, and cleanup problems.

0.2.3 The Azure Resource Hierarchy

Azure resources live in a hierarchy. At the identity layer, Microsoft Entra ID provides the tenant that contains users, groups, applications, and service principals. At the Azure management layer, management groups can organize multiple subscriptions. A subscription contains resource groups. A resource group contains resources such as storage accounts, key vaults, factories, workspaces, and virtual networks [57].

For this book, the most important scopes are subscription, resource group, and resource. The subscription is the billing and policy boundary for the labs. The resource group is the disposable unit for each chapter or week. The resource is the actual service instance. If you deploy every chapter into a clearly named resource group, cleanup is normally one command: delete that group.

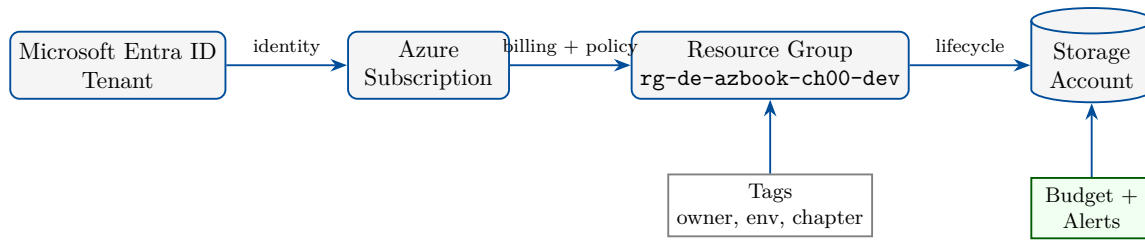


Figure 1: Chapter 0 setup flow from identity and subscription selection to a tagged, budgeted lab resource group.

Tip

Treat the resource group as the learning lab’s blast-radius boundary. A predictable resource group name lets you review all resources, export a deployment record, and delete the complete lab without hunting through the portal.

0.3 Securing Identity Before Deploying Resources

0.3.1 Microsoft Entra ID and Administrative Separation

Microsoft Entra ID is Azure’s identity platform for users, groups, enterprise applications, service principals, conditional access, and MFA [92]. In a new personal learning tenant, you may be the only user. In an organization, the tenant may already have administrators, policies, and approved groups. Either way, do not normalize daily work as a highly privileged administrator.

The **Global Administrator** role is a tenant-wide Microsoft Entra role. It can manage identity settings and many administrative functions across the tenant. It is not the right role for running day-to-day data engineering commands. Daily work should use a normal user account with only the Azure RBAC permissions needed in the lab subscription or resource group. Administrative accounts should be reserved for administrative actions, protected with MFA, and monitored.

Definition

Least privilege means granting an identity only the minimum actions, data access, and scope required to complete a task, for only as long as the task requires.

0.3.2 Azure RBAC Roles for Labs

Azure RBAC controls management-plane access at scopes such as subscription, resource group, or resource [86]. The common built-in roles are:

- **Owner:** can manage resources and assign access to others. Use sparingly because it can delegate privileges.
- **Contributor:** can create and manage resources but cannot grant access. This is often sufficient for a learner working inside a dedicated lab resource group.
- **Reader:** can view resources but cannot change them. Use for reviewers, auditors, and safe inspection.

Data services often have separate data-plane roles. For example, managing a storage account is different from reading blobs inside it. Later chapters distinguish roles such as **Storage Blob Data Reader** and **Storage Blob Data Contributor**. Chapter 0 focuses on management-plane setup so that later labs can explain data-plane permissions in context.

Common Pitfall

Solving every error with Owner. Granting Owner at the subscription level may make a command succeed, but it hides the real boundary: missing RBAC, missing data-plane role, missing ACL, wrong subscription, or blocked network path. Broad permissions are hard to audit and easy to forget.

0.3.3 Create a Least-Privileged User or Service Principal

For an individual learning account, the first identity goal is separation. Use one account for emergency administration and a different daily account for labs when your tenant license and permissions allow it. In an organization, ask for a dedicated lab subscription or resource group and a role assignment scoped to that boundary.

A service principal is an application identity used by automation. It is useful for CI/CD and non-interactive scripts, but it must be protected like any credential. For early manual labs, prefer interactive user login with MFA. Use a service principal only when a lab requires automation and store secrets in the approved secret store, not in source files or screenshots.

```

1 # Run only after creating the resource group and confirming the scope.
2 $subscriptionId = "00000000-0000-0000-0000-000000000000"
3 $rg = "rg-de-azbook-ch00-dev"
4 $scope = "/subscriptions/$subscriptionId/resourceGroups/$rg"
5
6 az ad sp create-for-rbac '
7   --name "sp-de-azbook-ch00-dev" '
8   --role Contributor '
9   --scopes $scope
10
11 # Store the output securely. Do not commit appId, password, or tenant.
```

Listing 1: Creating a resource-group scoped service principal with Azure CLI from PowerShell.

The command returns credentials once. If you lose the secret, create a new one rather than searching terminal history or files. Later chapters prefer managed identity where Azure compute supports it.

0.3.4 Enable MFA

Enable MFA before you deploy resources. MFA reduces the risk that a stolen password becomes a working cloud credential. In a personal Microsoft account, configure two-step verification or the Microsoft Authenticator app through account security settings. In a Microsoft Entra tenant, use the Entra admin center or Security Defaults when appropriate. Organizations normally enforce MFA through Conditional Access policies.

Note

MFA is not a replacement for least privilege. A phished account with MFA fatigue approval and subscription Owner rights can still create resources, assign roles, or delete evidence. Use MFA, scoped roles, budgets, and logs together.

0.4 Installing and Authenticating Tooling on Windows**0.4.1 Azure CLI on Windows**

Azure CLI is the primary command-line tool used in this book. It works across platforms and is well supported by Azure documentation [46]. On Windows, install it with the Microsoft

Installer package, with `winget`, or from Microsoft's installation page. After installation, open a new PowerShell window so the updated PATH is available.

```

1 # Option A: install with Windows Package Manager.
2 winget install --exact --id Microsoft.AzureCLI
3
4 # Confirm the CLI is available.
5 az version
6
7 # Browser-based sign-in. Complete MFA in the browser when prompted.
8 az login
9
10 # List subscriptions visible to this identity.
11 az account list --output table
12
13 # Select the subscription used for this book.
14 az account set --subscription "00000000-0000-0000-0000-000000000000"
15
16 # Confirm the selected subscription.
17 az account show --output table

```

Listing 2: Installing Azure CLI and signing in from Windows PowerShell.

If `az login` opens a browser for the wrong account, sign out of the browser or use a private browser session. If your organization requires device-code login, Azure CLI will display a code and URL. Complete the flow with the approved identity.

Tip

Pin the active subscription in every new terminal session before running destructive commands. A short `az account show --output table` is cheaper than deleting a resource group from the wrong subscription.

0.4.2 Azure PowerShell as an Alternative

Azure PowerShell is a PowerShell-native module family named `Az`. It is excellent when an organization standardizes on PowerShell scripts, Desired State Configuration, or administrative automation [45]. This book uses Azure CLI for consistency, but Azure PowerShell is a first-class alternative.

```

1 # Run PowerShell as a normal user unless your module path requires elevation.
2 Install-Module -Name Az -Scope CurrentUser -Repository PSGallery -Force
3
4 Connect-AzAccount
5
6 Set-AzContext -Subscription "00000000-0000-0000-0000-000000000000"
7
8 Get-AzContext | Select-Object Name, Subscription, Tenant

```

Listing 3: Installing the Az module and selecting a subscription.

Do not mix Azure CLI and Azure PowerShell contexts without checking both. `az account set` affects Azure CLI. `Set-AzContext` affects Azure PowerShell. They may point to different subscriptions in the same terminal.

0.4.3 Azure Cloud Shell

Azure Cloud Shell is a browser-based shell hosted by Microsoft and available from the Azure Portal [58]. It includes Azure CLI and Azure PowerShell, requires no local installation, and au-

thenticates with the portal session. It is useful for locked-down workstations, quick inspections, and demonstrations.

Cloud Shell is not the same as your local machine. It has its own file system, mounted storage, installed tools, and network path. For book labs, local PowerShell gives you clearer control over files in this repository. Use Cloud Shell when local installation is blocked, but copy important commands and outputs back into your lab notes.

0.5 Cost Guardrails

0.5.1 Spending Limit, Budgets, and Alerts

Cost control starts before deployment. Azure free and credit-based subscriptions may have a spending limit that prevents charges beyond the available credit until the subscription is upgraded or the offer changes. Do not rely on that limit as your only protection. Spending limits are offer-specific, and some resources can consume credit rapidly. Microsoft Cost Management provides cost analysis, budgets, forecasts, and alerts [68].

A budget does not usually stop resources by itself. It alerts when actual or forecasted cost crosses thresholds. That alert is valuable because it gives you time to delete, resize, pause, or investigate resources before a small mistake becomes a large bill. For learning subscriptions, create a monthly budget with early thresholds such as 25%, 50%, 75%, and 90%.

Tip

Measure it. Every chapter that creates Azure resources should end with two measurements: current resources in the lab resource group and current or forecasted cost in Cost Management. If you cannot measure it, assume it can bill.

0.5.2 Create a Budget from PowerShell

The following listing creates a lab resource group and a monthly budget at subscription scope. Replace the email address, subscription ID, and amount with your own values. The budget amount is intentionally small for a learning environment.

```

1 $subscriptionId = "00000000-0000-0000-0000-000000000000"
2 $location = "eastus"
3 $rg = "rg-de-azbook-ch00-dev"
4 $budgetName = "budget-de-azbook-monthly"
5 $alertEmail = "student@example.com"
6 $startDate = "2026-08-01"
7 $endDate = "2027-08-01"
8
9 az account set --subscription $subscriptionId
10
11 az group create '
12   --name $rg '
13   --location $location '
14   --tags project=azure-book environment=dev chapter=chapter0 owner=student
15
16 az consumption budget create '
17   --budget-name $budgetName '
18   --amount 25 '
19   --time-grain Monthly '
20   --time-period start-date=$startDate end-date=$endDate '
21   --category Cost '
22   --notifications '

```

```

23  actual25 enabled true operator GreaterThan threshold 25 contact-emails
    $alertEmail '
24  actual50 enabled true operator GreaterThan threshold 50 contact-emails
    $alertEmail '
25  forecast90 enabled true operator GreaterThan threshold 90 contact-emails
    $alertEmail

```

Listing 4: Creating a tagged resource group and subscription budget with Azure CLI.

If your Azure CLI version rejects the notification syntax, create the budget in the portal under *Cost Management + Billing* and document the same thresholds.

0.5.3 Cost Analysis Workflow

Cost Analysis in the Azure Portal lets you group spending by service, resource group, location, tag, and meter. For this book, use three views repeatedly. First, group by resource group to confirm that lab spending is isolated. Second, group by service name to see whether storage, compute, networking, or monitoring is driving cost. Third, filter by tag so you can verify that your naming and tagging strategy is working.

Common Pitfall

Assuming deletion stops every charge immediately. Some meters are delayed, some retained data bills until removed, and some services create hidden supporting resources. After deleting a lab, check Cost Analysis over the next day and verify the resource group no longer exists.

0.6 Regions, Naming, and Tagging Used by This Book

0.6.1 Default Region

Unless a chapter says otherwise, this book uses **eastus** as the default region because it is widely available and commonly supports data platform services. If **eastus** is unavailable in your subscription, choose a nearby region that supports the required service and record the substitution. Do not mix regions casually inside a lab. Cross-region data movement, private endpoints, availability zones, and service availability all affect behavior and cost.

Note

Azure service availability varies by region and subscription type. If a command fails because a SKU is unavailable, choose the nearest supported region and keep the rest of the resource names and tags unchanged.

0.6.2 Naming Conventions

Azure naming rules differ by resource type. Resource groups can contain hyphens. Storage account names must be globally unique, 3–24 characters, lowercase letters and numbers only. Key vault names, Data Factory names, Synapse workspaces, and Databricks workspaces each have their own constraints. Microsoft’s Cloud Adoption Framework recommends predictable names with components such as resource type, workload, environment, region, and instance number [37].

This book uses the following learning convention:

- **Resource group:** `rg-de-azbook-chNN-dev`, such as `rg-de-azbook-ch00-dev`.

- **Storage account:** stdeazbkchNNxxxx, where xxxx is a short unique suffix.
- **Location:** eastus unless the chapter or subscription requires another region.
- **Environment tag:** environment=dev for all book labs.
- **Chapter tag:** chapter=chapter0, chapter=chapter1, and so on.

Do not place sensitive information in names or tags. Resource names, DNS names, logs, and billing exports may be visible to operators or external systems.

Tag	Example	Purpose
project	azure-book	Groups all learning resources for reporting
environment	dev	Separates labs from production or shared sandboxes
chapter	chapter0	Enables chapter-level cost and cleanup review
owner	student	Identifies who should receive cleanup questions
delete-after	2026-08-07	Creates a human-readable cleanup expectation

Table 2: Tags make cost allocation, cleanup, and operational ownership visible.

0.7 Verify the Setup

0.7.1 Account and Subscription Check

Open a new Windows PowerShell terminal. The first verification is to prove that Azure CLI is installed, authenticated, and targeting the intended subscription.

```
1 az version
2 az login
3 az account show --query "{name:name, id:id, tenantId:tenantId, user:user.name}"
  --output table
```

Listing 5: Verifying the selected Azure subscription.

Expected output resembles the following. Your subscription name, ID, tenant ID, and user will differ.

```
1 Name                               Id                               TenantId
   User
2 -----
3 Azure subscription 1  00000000-0000-0000-0000-000000000000
   11111111-1111-1111-1111-111111111111  student@example.com
```

Listing 6: Expected shape of account verification output.

0.7.2 Create a Resource Group and Storage Account

The following lab creates a tagged resource group and a standard locally redundant storage account. The storage account name must be globally unique, so the script appends a short random suffix. The account uses inexpensive settings appropriate for a setup test. Chapter 1 creates a more complete ADLS Gen2 account.

```

1 $location = "eastus"
2 $rg = "rg-de-azbook-ch00-dev"
3 $suffix = (New-Guid).Guid.Replace("-", "").Substring(0, 6).ToLower()
4 $storage = "stdeazbkch00$suffix"
5
6 az group create '
7   --name $rg '
8   --location $location '
9   --tags project=azure-book environment=dev chapter=chapter0 owner=student
10
11 az storage account create '
12   --name $storage '
13   --resource-group $rg '
14   --location $location '
15   --sku Standard_LRS '
16   --kind StorageV2 '
17   --https-only true '
18   --min-tls-version TLS1_2 '
19   --allow-blob-public-access false '
20   --tags project=azure-book environment=dev chapter=chapter0 owner=student
21
22 az storage account list '
23   --resource-group $rg '
24   --query "[].{name:name, location:location, sku:sku.name, httpsOnly:
25     enableHttpsTrafficOnly}" '

```

Listing 7: Creating a temporary resource group and storage account.

Expected output resembles:

Name	Location	Sku	HttpsOnly
stdeazbkch00a1b2c3	eastus	Standard_LRS	True

Listing 8: Expected shape of storage account listing output.

If the command reports that the storage account name is unavailable, rerun the script or choose a different suffix. If it reports that the region or SKU is unavailable, change `eastus` to another supported region and record the difference.

0.7.3 Delete the Verification Resource Group

Cleanup is part of verification, not an optional afterthought. Delete the resource group and confirm that it is gone.

```

1 $rg = "rg-de-azbook-ch00-dev"
2
3 az group delete --name $rg --yes --no-wait
4
5 # Wait a few minutes, then verify deletion status.
6 az group exists --name $rg

```

Listing 9: Deleting the temporary verification resource group.

The final command returns `false` when deletion has completed. If it returns `true`, wait and run it again. Some services take several minutes to delete.

Tip

For every later chapter, save three pieces of evidence: creation command or deployment file, validation output, and cleanup output. This habit makes labs reproducible and protects

against surprise bills.

0.8 Cross-Cloud Setup Equivalence

Cross-Cloud Equivalence			
Setup step	AWS	Azure	GCP
Free offer	Free Tier (12 mo + always free)	Free account (\$200/30 days + always free)	Free trial (\$300/90 days + Always Free)
Sign-up guard	Card + root email	Card + Microsoft account	Card + Google account
Identity model	IAM / IAM Identity Center	Microsoft Entra ID + RBAC	Cloud IAM
Admin isolation	Never use root daily	Avoid Global Admin daily	Avoid Owner; least privilege
CLI	AWS CLI v2 (<code>aws configure</code>)	Azure CLI (<code>az login</code>)	gcloud CLI (<code>gcloud init</code>)
Browser shell	AWS CloudShell	Azure Cloud Shell	Cloud Shell
Cost guardrail	AWS Budgets + alerts	Cost Management budgets	Budgets & alerts
Console	AWS Management Console	Azure Portal	Google Cloud Console
Microsoft Fabric uses a Microsoft 365/Power BI work account plus a Fabric trial capacity (no card, org tenant); Snowflake uses a 30-day/\$400 trial (no card) guarded by Resource Monitors; MongoDB Atlas offers a forever-free M0 cluster (no card) secured by database users and an IP allowlist.			

0.9 Environment Checklist Before Chapter 1

A complete Chapter 0 environment has the following evidence:

1. Azure free account or approved organizational sandbox subscription exists.
2. Subscription ID and tenant ID are recorded in a private lab notebook.
3. MFA is enabled for the account used to access Azure.
4. Daily lab identity is not a Global Administrator account when separation is possible.
5. Azure CLI runs from Windows PowerShell and `az account show` displays the intended subscription.
6. Optional Azure PowerShell `Az` module is installed if your organization requires PowerShell-native automation.
7. Monthly budget exists with at least one alert that reaches your email address.
8. Default region is selected, normally `eastus`, or a documented substitute is chosen.
9. Naming and tagging conventions are written down before resources are deployed.
10. Verification resource group and storage account were created, listed, and deleted successfully.

0.10 Further Reading

Start with the Azure free account overview [32], then review the Azure resource organization hierarchy [57]. For identity and access, read Microsoft Entra ID fundamentals [92] and Azure RBAC documentation [86]. For tooling, use the Azure CLI installation guide [46], Azure PowerShell installation guide [45], and Azure Cloud Shell overview [58]. For guardrails, review Microsoft Cost Management budgets [68], the Azure Well-Architected Framework [52], the Cloud Adoption Framework [54], and Azure naming guidance [37].

Key Takeaways

- The Azure free account combines introductory credit, selected 12-month free services, and always-free quotas; each has limits.
- Azure resources are easiest to manage when subscriptions, resource groups, tags, budgets, and cleanup commands are planned before deployment.
- Microsoft Entra ID provides identity, while Azure RBAC grants scoped management permissions; daily work should avoid Global Administrator and broad Owner access.
- Azure CLI is the default tool for this book on Windows PowerShell; Azure PowerShell and Azure Cloud Shell are valid alternatives.
- Budgets alert; they do not replace measurement, cleanup, and conservative service selection.
- Chapter 1 should begin only after account selection, MFA, CLI authentication, budget alerts, naming conventions, and cleanup verification are complete.

0.11 Exercises

1. **(Conceptual)** Explain the difference between a Microsoft Entra tenant, an Azure subscription, a resource group, and a resource. Give one example of each from your setup.
2. **(Conceptual)** Why is Global Administrator inappropriate for daily data engineering work? Contrast it with Contributor at resource-group scope.
3. **(Hands-on)** Install Azure CLI on Windows, run `az login`, select the book subscription, and capture `az account show -output table`.
4. **(Hands-on)** Create the Chapter 0 verification resource group and storage account. Save the command output and confirm that required tags exist.
5. **(Hands-on)** Delete the verification resource group and prove that `az group exists -name rg-de-azbook-ch00-dev` returns `false`.
6. **(Cost)** Create a monthly budget with alerts. Record the amount, thresholds, contact email, and where you will check Cost Analysis after each chapter.
7. **(Security)** Enable MFA and write a short note describing how you would recover access if your primary authenticator device is lost.
8. **(Design)** Propose a naming and tagging convention for a team of five learners sharing one subscription. Include owner, chapter, environment, and delete-after tags.
9. **(Troubleshooting)** List three signs that Azure CLI is pointed at the wrong subscription. For each, state the command you would run to confirm or fix it.

10. **(Preparation checklist)** Before starting Chapter 1, verify: account created, MFA enabled, CLI authenticated, subscription selected, budget configured, default region chosen, naming convention recorded, Chapter 0 resources deleted, and Cost Analysis reviewed.

0.12 Curated Community Resources

Beyond this book and the vendor documentation, the following community-curated resources are worth your time. Do not try to consume everything at once: pick one resource per category, practice it with real data, and build something small before moving on.

Practical Case Studies (Video Walkthroughs)

Fifteen end-to-end builds that show how the tools work together:

1. SQL Data Warehouse from Scratch — SQL, ETL, data modeling, star schemas (<https://lnkd.in/gcrJ2qde>)
2. End-to-End Data Analytics Project — Python, Pandas, SQL Server, data cleaning (<https://lnkd.in/gHxTN-B2>)
3. Netflix Data Engineering Project — Python, SQL, ELT, exploratory analysis (<https://lnkd.in/gMkbpWmh>)
4. DuckDB and Python Data Engineering Project — API ingestion, pipeline development (<https://lnkd.in/gEPnubGA>)
5. Airflow Podcast Data Pipeline — Airflow, APIs, scheduling (<https://lnkd.in/gmy5rS4Z>)
6. Automated Python ETL Pipeline with Airflow — SQL Server, DAG development (<https://lnkd.in/ghrBc8BE>)
7. PySpark and dbt Data Engineering Project — incremental loading, dimensional modeling (<https://lnkd.in/gVZjvyR7>)
8. Apache Spark Data Engineering Project — PySpark, Databricks (<https://lnkd.in/gVNkqk5A>)
9. Airbnb Data Engineering Project — dbt, Snowflake, AWS, testing (<https://lnkd.in/g2f4TxPW>)
10. AWS ETL Pipeline Project — S3, Lambda, Glue, Athena, serverless ETL (<https://lnkd.in/gFC3NkNs>)
11. AWS Data Warehouse Project — PySpark, Glue, Redshift (<https://lnkd.in/gyZtHMEF>)
12. End-to-End Azure Data Engineering Project — Data Factory, Databricks, ADLS (<https://lnkd.in/gUpbRjzE>)
13. Spotify Azure Data Engineering Project — streaming, Delta Lake (<https://lnkd.in/gUGKuZ8N>)
14. Uber Real-Time Data Engineering Project — Event Hubs, structured streaming (<https://lnkd.in/gu7Tpg7D>)
15. End-to-End GCP Data Engineering Project — Cloud Storage, Dataproc, BigQuery (<https://lnkd.in/gv8Yedfu>)

Free Video Courses

Twenty videos covering the essential skill path — recommended learning order: Python, SQL, Linux, Git, Docker, data modeling, data warehousing, Spark, Kafka, Airflow, dbt, cloud, portfolio projects.

- **Foundations:** Data Engineering Course for Beginners (<https://lnkd.in/gprfkqnp>); Big Data Engineer Masterclass (<https://lnkd.in/gqPkrG63>)
- **Core tools:** Python for Data Engineers (<https://lnkd.in/gZeKJEi4>); SQL for Big Data Engineering (<https://lnkd.in/gt5DZXwQ>); PostgreSQL Full Course (<https://lnkd.in/gAU5n7gM>); Linux Command Line (<https://lnkd.in/gA-7zFaQ>); Git and GitHub Tutorial (<https://lnkd.in/gs73kHau>); Docker Tutorial (<https://lnkd.in/g59TCWPD>)
- **Warehousing and modeling:** Data Warehousing Beginner's Guide (https://lnkd.in/gS_-iJAb); Data Modeling, Star Schema and Kimball (<https://lnkd.in/g4it9vpy>)
- **Batch processing:** PySpark Tutorial for Beginners (<https://lnkd.in/gre86dH6>); PySpark Full Course (<https://lnkd.in/gZXg38hk>)
- **Streaming:** Apache Kafka Full Course (https://lnkd.in/gnnXc9_d); Kafka Crash Course with Project (<https://lnkd.in/gP6DFCem>)
- **Pipelines:** Apache Airflow Tutorial (<https://lnkd.in/gbFzzReZ>); dbt Tutorial with Netflix Project (<https://lnkd.in/gHvmdjNX>); ELT with dbt, Snowflake and Airflow (https://lnkd.in/g5ZP_xC3)
- **Cloud:** AWS for Data Engineers (<https://lnkd.in/g8vRZmkp>)
- **End-to-end projects:** Airbnb Project with Snowflake, dbt and AWS (<https://lnkd.in/g2f4TxPw>); Real-Time E-Commerce with Kafka and Snowflake (<https://lnkd.in/g58V6A3P>)

GitHub Repositories

Twenty free repositories to learn from, practice with, and build upon. Choose one roadmap, complete one project, study the code structure, then break it, fix it, and document what you learned.

- **Roadmaps and guides:** Data Engineering Zoomcamp (<https://lnkd.in/gxKmbbHc>); Data Engineer Handbook (<https://lnkd.in/gstev9z7>); Data Engineering Cookbook (<https://lnkd.in/gABzJ-vE>); Complete Data Engineer Roadmap (<https://lnkd.in/gmwQrmvm>)
- **Curated lists:** Awesome Data Engineering (<https://lnkd.in/gh4Ky273>); Awesome Big Data (https://lnkd.in/g_y-n3zm); Awesome Open-Source Data Engineering (<https://lnkd.in/ggDTw66U>)
- **Portfolio and analytics engineering:** Data Engineering Project Template (<https://lnkd.in/g-QKrMXV>); Jaffle Shop dbt Practice Project (<https://lnkd.in/gmXVhYKX>); dbt Core (<https://lnkd.in/gnCXxNbB>)
- **Ingestion and orchestration:** Apache Airflow (<https://lnkd.in/gqvG5F6N>); Airbyte (<https://lnkd.in/g2B9AVwM>); dlt (<https://lnkd.in/g3A8BMp3>)
- **Processing:** Apache Spark (<https://lnkd.in/g5CPE3hh>); Apache Kafka (<https://lnkd.in/gdH5T4xX>)

- **Storage and lakehouses:** DuckDB (<https://lnkd.in/gehQd42W>); Apache Iceberg (<https://lnkd.in/gVweVV8y>)
- **Quality, lineage and governance:** Great Expectations (<https://lnkd.in/g8TPyAy8>); OpenLineage (https://lnkd.in/gz3_MiAP); DataHub (<https://lnkd.in/gEcNMRkJ>)

Part I

Core Data Infrastructure and Storage

Azure Data Lake Storage Gen2 and Storage Foundations

Learning Objectives

- Explain the Azure Storage account abstraction and the relationship between Blob Storage and ADLS Gen2.
- Distinguish flat object namespaces from hierarchical namespaces, including effects on analytics workloads, access control, and rename operations.
- Select an appropriate redundancy option among LRS, ZRS, GRS, GZRS, RA-GRS, and RA-GZRS for a data-engineering workload.
- Apply access tiers, lifecycle management policies, soft delete, and blob versioning to manage cost and recoverability.
- Compare online and offline migration tools, including AzCopy, Azure Storage Explorer, and Azure Data Box.
- Deploy an ADLS Gen2 account with Azure CLI and validate access using the `azure-storage-file-datalake` Python SDK.
- Design a compliant, multi-region disaster-recovery storage architecture for large-scale healthcare imaging data.

Cross-Cloud Equivalence

Although this book teaches **Azure**, the Week 1 storage foundations map almost one-to-one onto the other clouds—learn one mental model and carry it everywhere.

Capability	AWS	Azure	GCP
Object storage	Amazon S3	ADLS Gen2 / Blob	Cloud Storage
Hot tier	S3 Standard	Hot	Standard
Infrequent access	S3 Standard-IA	Cool	Nearline
Archive (instant)	Glacier Instant Retrieval	Cold	Coldline
Deep archive	Glacier Deep Archive	Archive	Archive
Lifecycle rules	Lifecycle policies	Lifecycle management	Object Lifecycle Mgmt
Versioning	S3 Versioning	Blob versioning	Object Versioning
Immutability (WORM)	Object Lock	Immutable blob storage	Bucket Lock
Cross-region copy	CRR / SRR	Object replication	Dual/multi-region + Transfer
Online transfer	DataSync	AzCopy	Storage Transfer Service
Offline transfer	Snow Family	Data Box	Transfer Appliance
Design durability	11 nines	11 nines (LRS) / 16 (GRS)	11 nines
Availability SLA	99.9% (Standard)	99.9% (RA-GRS read 99.99%)	99.95% (multi-region)
Encryption at rest	SSE-S3 / SSE-KMS / SSE-C	SSE + CMK via Key Vault	Google-managed / CMEK / CSEK
Access control	IAM & bucket policies, ACLs	Azure RBAC, SAS, POSIX ACLs	IAM, ACLs, signed URLs
Pricing levers	Storage + requests + egress + retrieval	Storage + operations + egress + retrieval	Storage + operations + egress + retrieval
Consistency	Strong read-after-write	Strong read-after-write	Strong (global)

Beyond raw object storage, the same layer reappears as a managed abstraction: **Fabric OneLake** is a tenant-wide lake built on ADLS Gen2 (Delta–Parquet), **Snowflake** persists data as immutable micro-partitions on the provider’s object store, and **MongoDB’s** WiredTiger is a node-local engine rather than object storage—the contrast is itself instructive.

Key Terms

Storage account: the Azure management boundary for storage services, networking, redundancy, encryption, identity integration, and billing. **Container:** a Blob Storage namespace that becomes a file system when hierarchical namespace is enabled. **Hierarchical namespace:** ADLS Gen2 capability that provides directory semantics, atomic directory operations, and POSIX-like ACLs. **Access tier:** a cost and retrieval-latency class for blob data. **Lifecycle policy:** JSON rules that transition or delete blobs based on age, tier, prefix, and metadata.

1.1 Week 1 Overview: Storage as the Root of the Azure Data Platform

Azure data engineering begins with storage, not with Spark clusters, orchestration pipelines, dashboards, or machine-learning models. Storage is where raw evidence lands, where curated data products persist, where batch and streaming systems checkpoint state, and where audit trails prove what happened. If the storage layer is poorly named, weakly secured, under-protected, or expensive to operate, every downstream service inherits that fragility. Azure Data Lake Storage Gen2 (ADLS Gen2) is Microsoft Azure’s primary cloud object store for analytical

data lakes and lakehouses [50].

A storage foundation chapter is therefore a design chapter, not merely a catalog of buttons in the portal. A senior data engineer must connect data characteristics to platform controls: object size, arrival rate, regulatory retention, locality, recovery-point objective (RPO), recovery-time objective (RTO), analytics engine behavior, identity boundaries, and expected access pattern. The same storage account can host a quiet archive, a high-throughput ingestion zone, or a governed lakehouse serving hundreds of analysts; those workloads need different tiering, redundancy, namespace, and access decisions.

Definition

ADLS Gen2 is the combination of Azure Blob Storage durability and scale with a hierarchical namespace designed for big-data analytics. It supports file-system semantics, directory-level access control, and APIs used by Hadoop-compatible engines such as Spark, Synapse, and Databricks.

Note

Throughout this chapter, “Blob Storage” refers to Azure’s object storage service, while “ADLS Gen2” refers to Blob Storage accounts or containers with hierarchical namespace enabled. The same underlying platform is involved; the difference is the namespace and analytics-oriented semantics.

1.2 Monday: Azure Storage Account Architecture

1.2.1 The Storage Account as a Control Boundary

An Azure Storage account is both a management-plane resource and a data-plane endpoint. The management plane is controlled through Azure Resource Manager (ARM): resource group, location, tags, redundancy, networking rules, private endpoints, diagnostic settings, encryption settings, and identity-related configuration. The data plane is where applications create containers, upload blobs, list directories, acquire leases, read ranges, and set object metadata. Confusing those planes leads to common troubleshooting mistakes: a user may have Owner rights over the Azure resource but still be unable to read data-plane objects, or may have blob data permissions but be unable to alter firewall rules.

Storage accounts expose multiple service endpoints: Blob, Data Lake, Queue, Table, and File services. A data engineering lake normally depends on Blob and Data Lake endpoints. The Blob endpoint exposes flat object APIs. The Data Lake endpoint exposes file-system semantics for hierarchical namespace accounts. Many engines use ABFS or ABFSS URI forms such as `abfss://bronze@acct.dfs.core.windows.net/path/`. The `dfs.core.windows.net` endpoint is significant because it indicates the Data Lake namespace endpoint rather than the blob endpoint.

Definition

A **container** is a grouping of blobs inside a storage account. In an ADLS Gen2 account, a container is also presented as a **file system**. Data lake designs often map containers to security and lifecycle boundaries such as **raw**, **bronze**, **silver**, **gold**, or domain-specific zones.

Tip

Use separate storage accounts when you need hard isolation for networking, encryption keys, redundancy, account-level immutability, or blast-radius reduction. Use separate containers when the data shares account-level controls but needs different access, prefixes, or lifecycle rules.

1.2.2 Blob Storage vs. ADLS Gen2

Azure Blob Storage is optimized for massive, durable, low-cost object storage [49]. Objects are addressed by name, metadata, and container. Traditional object stores treat the slash in `raw/year=2026/month=07/file.parquet` as part of the object name, not as a real directory tree. Tools can simulate folders by filtering names with common prefixes, but operations such as renaming a directory become many object copy-and-delete operations.

ADLS Gen2 adds a hierarchical namespace. Directories become first-class metadata entries. Directory rename and delete operations can be atomic metadata operations instead of large object rewrites. POSIX-like access control lists can be attached at directory and file levels. This matters for analytics engines that produce many files and commit directory-level outputs. It also matters for governance: a lake can grant a data science group read access to `/curated/imaging/features/` without granting access to raw protected health information.

Capability	Blob Storage flat namespace	ADLS Gen2 hierarchical namespace
Namespace model	Object names with prefixes	File systems, directories, and files
Rename directory	Copy and delete many objects	Metadata operation for directory path
Access control	Container and blob RBAC or SAS patterns	RBAC plus POSIX-like ACLs at path level
Analytics engines	Works for object reads and writes	Designed for Hadoop, Spark, Synapse, Databricks
Best fit	General object storage, backups, media	Enterprise data lakes and lake-houses

Table 1.1: Flat Blob Storage and ADLS Gen2 share durability but differ in namespace behavior and analytics ergonomics.

Common Pitfall

Retrofitting hierarchy too late. Hierarchical namespace is a foundational account capability. Treat it as an up-front architecture decision for analytical lakes rather than a cosmetic folder setting.

1.2.3 Redundancy Options: LRS, ZRS, GRS, GZRS, and RA Variants

Azure Storage redundancy determines how many copies exist and where those copies reside [23]. It is a durability and availability design choice, not a substitute for backups, versioning, immutability, or application-level disaster recovery. The most common options are:

- **Locally redundant storage (LRS):** keeps three copies within a single data center in the primary region. It is cost-effective but vulnerable to zone or regional outages.
- **Zone-redundant storage (ZRS):** synchronously replicates across availability zones in the primary region. It improves availability for zone failures.

- **Geo-redundant storage (GRS):** replicates from the primary region to a paired secondary region. It protects against regional disaster but secondary access is controlled by failover behavior.
- **Geo-zone-redundant storage (GZRS):** combines ZRS in the primary region with asynchronous geo-replication to a secondary region.
- **Read-access geo-redundant storage (RA-GRS) and RA-GZRS:** expose read-only access to secondary replicas before failover, useful for read-only DR validation and emergency analytics.

Note

Asynchronous geo-replication means the secondary region may lag. For regulated workloads, document the expected RPO and validate whether the lag is acceptable for each data domain.

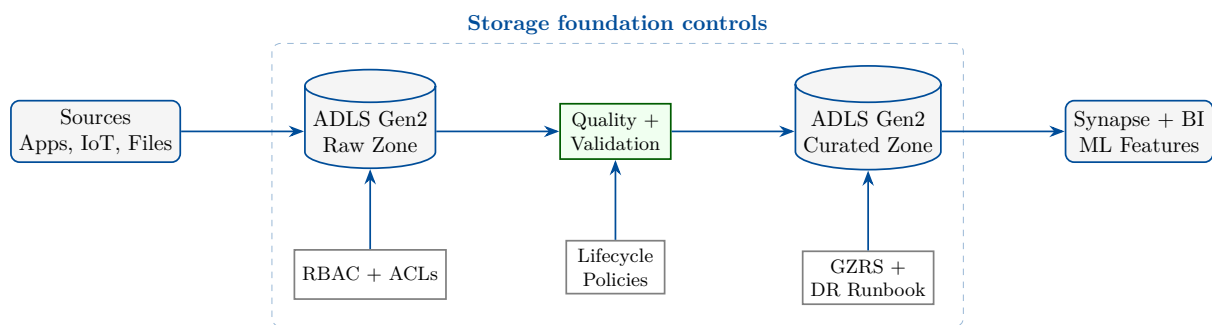


Figure 1.1: ADLS Gen2 as the durable foundation for ingestion, validation, curation, serving, governance, lifecycle control, and disaster recovery.

1.2.4 Monday Design Checklist

A storage account review should answer five questions. First, what data domains and environments will the account hold? Second, what identities need management-plane access versus data-plane access? Third, is hierarchical namespace required for analytics engines and path-level ACLs? Fourth, what regional failure mode is in scope for this workload? Fifth, what service endpoints must be private, public, or disabled? These questions are mundane, but they prevent expensive migration later.

1.3 Tuesday: Access Tiers, Lifecycle Management, and Recoverability

1.3.1 Access Tiers: Hot, Cool, Cold, and Archive

Azure Blob access tiers trade storage cost against access cost and retrieval latency [11]. The **Hot** tier is designed for frequently accessed data. **Cool** is cheaper to store but more expensive to read and generally appropriate for data accessed infrequently but still online. **Cold** extends that cost profile for very infrequent online access where supported. **Archive** is the lowest-cost offline tier, but data must be rehydrated before it can be read.

Definition

An **access tier** is a blob-level or account-default cost class that expresses how often data is expected to be read. It changes storage price, read price, minimum retention expectations, and retrieval latency.

The important design principle is that tiering should follow observed or contractually expected access patterns. Raw landing files may be hot for a few days while quality rules are fixed, cool for a month while reports are reconciled, and archived after regulatory retention begins. Curated dimensional data may stay hot because every dashboard queries it. Training snapshots may be cool or archive once a model version is approved. Tiering is therefore a data-product decision, not only an infrastructure setting.

Tier	Typical use	Access behavior	Caution
Hot	Active ingestion, current marts, daily dashboards	Frequent read/write	Higher storage cost
Cool	Recently closed periods, audit extracts, occasional backfills	Infrequent reads	Higher read and early deletion costs
Cold	Rarely accessed online data	Very infrequent reads	Verify regional and feature support
Archive	Long-term retention, legal archive, deep history	Offline until rehydrated	Rehydration delay and operational runbook required

Table 1.2: Access tiers should be selected by data-product usage, not by a single account-wide habit.

Common Pitfall

Archiving without a retrieval plan. Archive tier can save money, but an urgent investigation may wait hours for rehydration. Define who can rehydrate, where restored data lands, and which compliance approvals are required.

1.3.2 Lifecycle Management Policies

Lifecycle management is Azure’s rule engine for moving or deleting blobs based on conditions such as age, prefix, tier, and version state [56]. A lifecycle policy is JSON attached to the storage account. In data lakes, lifecycle rules should align with data classifications and medallion zones. For example, raw X-ray DICOM objects may transition to Cool after 30 days and Archive after 180 days, while curated ML feature tables remain Hot because they are part of active training pipelines.

Tip

Start lifecycle policies in report-only or narrow-prefix form when possible. Apply the first rule to a non-critical prefix, validate cost and retrieval behavior, then widen the policy. The most expensive lifecycle mistake is deleting or archiving the wrong prefix.

Lifecycle is not a governance substitute. It does not classify data, prove legal retention, or replace immutable storage. It should be one control in a broader retention design: metadata classification, legal hold requirements, data owner approval, backup or versioning strategy, observability, and exception handling.

1.3.3 Soft Delete and Blob Versioning

Soft delete protects against accidental deletion by retaining deleted blobs or containers for a configured period. Blob soft delete covers individual blobs. Container soft delete protects deleted containers. Blob versioning keeps prior versions when an overwrite occurs. Together, these controls reduce the blast radius of human error, failed jobs, and accidental overwrites.

Note

Soft delete is time-bounded recoverability, not a full backup strategy. If a malicious process overwrites sensitive data and waits until retention expires, soft delete alone is insufficient. Combine it with versioning, immutable policies where required, monitoring, and least-privilege access.

A data-engineering example clarifies the distinction. Suppose an ADF pipeline writes a corrected file over `/raw/claims/2026/07/30/batch.csv`. With blob versioning enabled, the previous object can remain available as an older version. If the pipeline deletes the file entirely, soft delete can restore it during the retention window. If the entire `raw` container is deleted, container soft delete can help. Each feature covers a different failure mode.

1.3.4 Tuesday Design Checklist

For each data zone, document freshness, read frequency, minimum retention, deletion approval, recovery expectation, legal hold requirements, and expected reprocessing cost. Then map those properties to access tier, lifecycle policy, soft delete period, versioning, immutability, and monitoring. If the table is unknown, do not guess: instrument actual access before enforcing aggressive tiering.

1.4 Wednesday: Migration Tools and Trade-offs

1.4.1 Online Migration with AzCopy

AzCopy is a command-line tool for high-performance copy operations to and from Azure Storage [40]. It supports local-to-blob, blob-to-blob, synchronization, include and exclude patterns, SAS or identity-based authentication, and job resume. It is ideal for repeatable online migrations, daily bulk loads, backfills from on-premises shares, and automation scripts.

Online migration is constrained by network bandwidth, change rate, reliability, and security. A 50 TB migration over a fast enterprise link may be practical. A 10 PB imaging migration over the public internet is usually not. The engineer must estimate transfer duration, throttling, retry behavior, validation strategy, and cutover mechanics. Hash validation, manifest files, and reconciliation queries matter as much as the copy command.

Tip

For large online migrations, split data by stable prefixes such as date, modality, business unit, or source system. Parallel jobs are easier to resume and reconcile when each job owns a deterministic prefix.

1.4.2 Azure Storage Explorer

Azure Storage Explorer is a graphical tool for inspecting and managing storage accounts. It is excellent for learning, debugging access, confirming that objects landed under the expected prefix, generating SAS tokens in controlled scenarios, and performing small manual operations.

It is not a substitute for infrastructure as code, automated migration scripts, or governed production operations.

Common Pitfall

Manual production edits through a GUI. Storage Explorer is useful, but manual deletes, uploads, and tier changes are hard to review and reproduce. Production storage changes should normally flow through scripted, logged, least-privilege paths.

1.4.3 Offline Migration with Azure Data Box

Azure Data Box is an offline transfer appliance for moving large datasets into Azure when network transfer is impractical [75]. Microsoft ships a ruggedized device, the customer copies data locally, ships it back, and Microsoft imports the encrypted data into the target storage account. This pattern is common when datasets are measured in hundreds of terabytes or petabytes, when a data center exit has a deadline, or when network egress is too slow.

Offline transfer introduces different constraints: device capacity, shipping logistics, chain of custody, encryption keys, staging procedures, copy windows, import verification, and delta synchronization. It rarely eliminates the need for online tools. A common pattern is to seed historical data with Data Box and then use AzCopy, ADF, or custom ingestion to synchronize changed data until cutover.

Tool	Best use	Trade-off
AzCopy	Scripted online copy, recurring loads, prefix-level sync	Depends on bandwidth and identity/SAS hygiene
Storage Explorer	Inspection, troubleshooting, small manual operations	Not ideal for reproducible production change
Azure Data Box	Large offline seeding and data-center migration	Requires logistics, chain of custody, and later delta sync

Table 1.3: Migration tools differ by scale, automation, observability, and operational risk.

1.5 Thursday Hands-on Project: Deploy ADLS Gen2 with Lifecycle Management

This lab deploys a storage account with hierarchical namespace enabled, creates a file system, enables recoverability controls, and applies a lifecycle policy that transitions matching blobs to Archive after 30 days. The commands assume Azure CLI is installed and that you are authenticated with sufficient permissions in a non-production subscription.

Note

The examples use globally unique storage account names. Replace `stdeadlweek1demo` with a compliant name that is unique across Azure: 3–24 lowercase letters and numbers, no hyphens.

```

1 az login
2 az account set --subscription "00000000-0000-0000-0000-000000000000"
3
4 $rg = "rg-de-azure-book-week1"
5 $location = "eastus"
6 $account = "stdeweek1adlsdemo01"
7
8 az group create '

```

```

9  --name $rg '
10 --location $location
11
12 az storage account create '
13 --name $account '
14 --resource-group $rg '
15 --location $location '
16 --sku Standard_GZRS '
17 --kind StorageV2 '
18 --hierarchical-namespace true '
19 --min-tls-version TLS1_2 '
20 --https-only true '
21 --allow-blob-public-access false '
22 --default-action Deny
23
24 az storage account blob-service-properties update '
25 --account-name $account '
26 --resource-group $rg '
27 --enable-delete-retention true '
28 --delete-retention-days 30 '
29 --enable-container-delete-retention true '
30 --container-delete-retention-days 30 '
31 --enable-versioning true
32
33 az storage fs create '
34 --account-name $account '
35 --name raw '
36 --auth-mode login
37
38 az storage fs create '
39 --account-name $account '
40 --name curated '
41 --auth-mode login

```

Listing 1.1: Deploying an ADLS Gen2 account and basic lake file systems with Azure CLI.

The account uses `Standard_GZRS`, hierarchical namespace, HTTPS-only access, TLS 1.2 minimum, and disabled public blob access. The example also sets the account firewall default action to deny. In a real lab you would add a trusted virtual network, private endpoint, or temporary client IP rule before attempting data-plane operations. If you cannot create file systems after setting `-default-action Deny`, confirm that your network path is allowed.

```

1  {
2    "rules": [
3      {
4        "enabled": true,
5        "name": "raw-to-archive-after-30-days",
6        "type": "Lifecycle",
7        "definition": {
8          "filters": {
9            "blobTypes": ["blockBlob"],
10           "prefixMatch": ["raw/landing/"]
11         },
12         "actions": {
13           "baseBlob": {
14             "tierToArchive": {
15               "daysAfterModificationGreaterThan": 30
16             }
17           }
18         }
19       }
20     ]
21   }

```

22 }

Listing 1.2: Lifecycle policy that archives raw objects after 30 days.

Save the JSON as `lifecycle-policy.json` in the book folder or pipeline workspace, then apply it with Azure CLI:

```
1 az storage account management-policy create '
2   --account-name $account '
3   --resource-group $rg '
4   --policy .\lifecycle-policy.json
5
6 az storage account management-policy show '
7   --account-name $account '
8   --resource-group $rg
```

Listing 1.3: Applying the lifecycle management policy.

Common Pitfall

Prefix mismatch in lifecycle policies. In ADLS Gen2, the policy prefix is evaluated against the blob path from the container boundary. If the intended container is `raw` and the intended directory is `landing`, validate the exact prefix format in a non-production account before rollout.

The Python SDK snippet below writes a small file through the Data Lake endpoint. In production, prefer managed identity from Azure compute. For a local lab, `DefaultAzureCredential` may use Azure CLI authentication.

```
1 from azure.identity import DefaultAzureCredential
2 from azure.storage.filedatalake import DataLakeServiceClient
3
4 account_name = "stdeweek1adlsdemo01"
5 account_url = f"https://{account_name}.dfs.core.windows.net"
6 credential = DefaultAzureCredential()
7
8 service_client = DataLakeServiceClient(
9     account_url=account_url,
10    credential=credential,
11 )
12
13 file_system = service_client.get_file_system_client("raw")
14 directory = file_system.get_directory_client("landing/week1")
15 directory.create_directory()
16
17 file_client = directory.create_file("sample-observation.txt")
18 payload = b"patient_id,event_time,source\n1001,2026-07-30T21:33:37Z,xray\n"
19 file_client.append_data(data=payload, offset=0, length=len(payload))
20 file_client.flush_data(len(payload))
21
22 read_back = file_client.download_file().readall().decode("utf-8")
23 print(read_back)
```

Listing 1.4: Writing and reading a file with `azure-storage-file-datalake`.

Tip

After a lab, delete the resource group if the account is not needed. GZRS storage and retained versions can create continuing charges, especially if you upload test data and lifecycle policies move it to tiers with early-deletion considerations.

1.5.1 Validation Steps

A complete lab validation includes the following checks: the storage account shows hierarchical namespace enabled; the account redundancy is GZRS; public blob access is disabled; soft delete and versioning are enabled; the **raw** and **curated** file systems exist; the lifecycle policy is visible; and the Python snippet can write and read a file through the DFS endpoint. Capture command output or screenshots for your engineering notebook.

1.6 Friday Use Case and Architecture: Healthcare X-ray Lake at 10 PB

1.6.1 Scenario

A healthcare provider processes 10 PB of X-ray images across regional hospitals. The platform must ingest new images daily, retain raw images for compliance and research, support downstream AI model development, serve governed analytics, and survive regional outages. The organization is subject to HIPAA and HITRUST-aligned controls, expects encryption at rest and in transit, requires auditability, and must control storage growth.

The first architectural decision is to separate data zones and duties. Raw images are immutable evidence and should be protected from accidental overwrite. Curated derivatives such as thumbnails, de-identified metadata, and feature vectors have different access and lifecycle needs. Research sandboxes may use derived, de-identified data and should not have broad access to protected health information. Operations teams need monitoring and recovery runbooks but not unrestricted patient data access.

Definition

A **compliant disaster-recovery storage architecture** defines not only where replicated bytes reside, but also who can read them, how failover is authorized, what data loss is acceptable, how recovery is tested, and how audit evidence is retained.

1.6.2 Target Architecture

The proposed design uses ADLS Gen2 storage accounts in a primary Azure region with GZRS or RA-GZRS where available and appropriate. Private endpoints keep data traffic on private networks. Microsoft Entra ID, Azure RBAC, POSIX-like ACLs, and managed identities enforce least privilege. Customer-managed keys may be used when the compliance program requires key ownership and rotation evidence. Diagnostic logs flow to a security workspace. Lifecycle policies move older raw images to Cool, Cold, or Archive based on clinical and research retrieval requirements. Immutable storage policies or legal holds are considered for records that must not be altered.

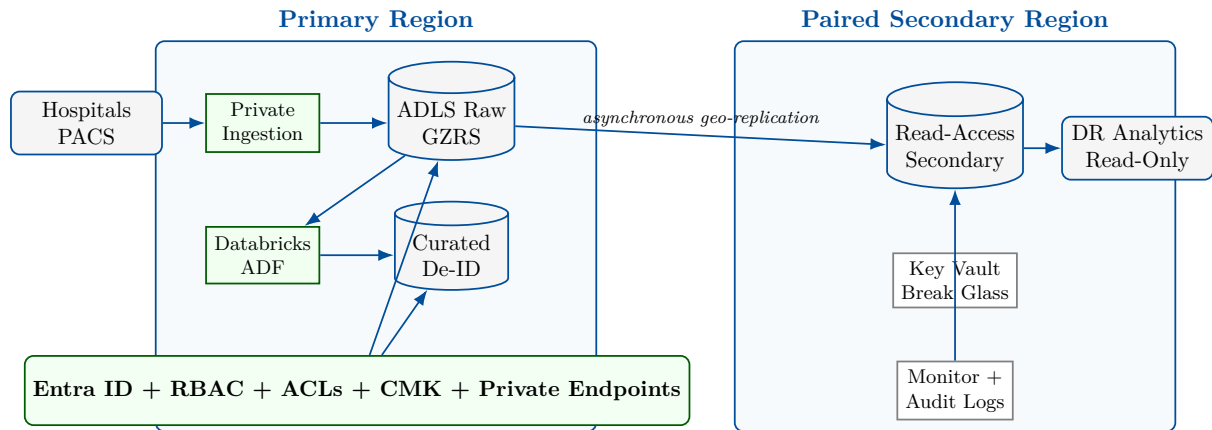


Figure 1.2: Multi-region ADLS Gen2 architecture for compliant healthcare imaging with GZRS/RA-GZRS, private ingestion, governed processing, and read-access disaster recovery.

1.6.3 Compliance and Security Design

HIPAA and HITRUST programs require administrative, technical, and physical controls; Azure services provide building blocks, but the customer owns configuration, access governance, monitoring, and operating procedures [42]. For storage, key technical controls include encryption at rest, TLS in transit, private networking, disabled public access, managed identities, least-privilege RBAC, ACL inheritance, diagnostic logging, retention controls, and incident-response evidence. Key administrative controls include data classification, role approval, periodic access reviews, backup and restore tests, and documented RPO/RTO acceptance.

Note

Compliance is not achieved by choosing a single service. It is achieved by configuring services, operating them consistently, collecting evidence, and proving that controls work under failure and audit conditions.

1.6.4 Lifecycle and Cost Model for 10 PB

At 10 PB, lifecycle mistakes are material. Even small per-GB price differences become large monthly numbers. The architecture should classify images by age, clinical status, research value, and legal hold. Recent images may remain Hot for active care and model validation. Images older than 30 or 90 days may move to Cool or Cold if clinical retrieval is rare. Deep history may move to Archive if rehydration times meet policy. De-identified derivatives used for model training may remain online because recomputing them from raw archive could be slower and more expensive than storing them.

Cost control also requires object sizing and file-format decisions. Medical images are naturally large binary objects, but metadata and derived features should be stored in analytics-friendly formats such as Parquet and Delta Lake. The lakehouse literature emphasizes unifying low-cost object storage with reliable table abstractions and analytics engines [2, 1]. For the X-ray platform, raw DICOM objects and curated Delta tables serve different purposes and deserve different lifecycle policies.

1.6.5 Disaster Recovery Runbook

A DR design is incomplete without a runbook. The runbook should specify monitoring signals for regional outage, the decision authority for failover, how to assess replication lag, which applications can operate read-only from the secondary, when to pause writes, how to communicate expected data loss, and how to return to the primary region. For RA-GZRS, read-only secondary access can support emergency analytics and validation before account failover. For write recovery, applications must be designed to avoid split-brain writes and to reconcile any lag after failback.

Common Pitfall

Assuming geo-replication equals application recovery. Storage replication preserves data copies, but applications, identities, private endpoints, keys, DNS, orchestration jobs, and user access paths must also be recoverable.

1.7 Architectural Decision Record Template

Use a short architectural decision record (ADR) for storage account decisions. A useful ADR includes: context, decision, alternatives considered, consequences, security impact, cost impact, operational runbook, and validation evidence. For Chapter 1, write an ADR that states why hierarchical namespace is enabled, why GZRS or RA-GZRS is selected, what lifecycle rules apply to each prefix, and what recovery tests must pass before production.

1.8 Operational Deep Dive: Naming, Zones, and Evidence

A production lake is easier to operate when naming, zones, and evidence are standardized from the first week. Storage account names are globally unique and should encode organization, environment, region, workload, and purpose without exposing sensitive details. Container names should be stable because they appear in URIs, ACL rules, pipeline parameters, and documentation. Directory names should support partition pruning and operational ownership. A common pattern is `/source-system/entity/year=YYYY/month=MM/day=DD/`, but event-time and processing-time partitions must not be confused.

Tip

Prefer boring names that operations staff can predict. A clever naming convention that requires a lookup table will fail during an incident.

Data zones should express quality and access expectations. The raw zone preserves source fidelity. The bronze zone may add ingestion metadata and schema capture. The silver zone standardizes, deduplicates, and validates. The gold zone serves business-ready products. Not every organization uses these exact labels, but every organization needs an explicit vocabulary for trust level. Without it, users will query whatever path is easiest to find.

Evidence completes the operating model. For every storage account, retain deployment records, policy JSON, access reviews, diagnostic settings, lifecycle rules, and DR test results. These artifacts are useful during audits, but they are also useful during ordinary engineering work. When a pipeline fails after a storage firewall change, a recorded change history is faster than guessing.

1.9 Troubleshooting Patterns

The most common ADLS Gen2 failures appear as authorization errors, network errors, path errors, or lifecycle surprises. Authorization errors require checking both RBAC and ACLs. A managed identity may have **Storage Blob Data Contributor** at the account level but still lack execute permission on an intermediate directory. Network errors require checking public network access, firewall rules, private endpoint DNS, and whether the client is using the blob or DFS endpoint. Path errors often come from case sensitivity, unexpected URL encoding, or prefix assumptions inherited from flat object storage.

Lifecycle surprises are harder because they may appear days later. A report fails because a file was archived; a backfill becomes slow because historical data moved to Cool; an overwrite creates many versions and increases cost. The remedy is observability: storage metrics, inventory reports, policy reviews, and cost analysis grouped by account, container, prefix, tier, and tag.

Common Pitfall

Granting broad permissions to solve a narrow error. If a pipeline cannot read one path, do not immediately grant account-wide contributor rights. Identify whether the missing permission is RBAC, ACL execute, ACL read, firewall, private DNS, or an incorrect endpoint.

1.10 Week 1 Lab Rubric

A complete Week 1 submission should include: a diagram of the proposed storage account and containers; Azure CLI output or screenshots proving the account settings; lifecycle policy JSON; a short Python SDK run log; an ADR explaining redundancy and tiering; and a cleanup note showing whether the resource group was deleted. Grade the work on correctness, reproducibility, security posture, and operational reasoning. A beautiful diagram without commands is incomplete; commands without architecture reasoning are also incomplete.

Criterion	Meets expectation	Needs improvement
Reproducibility	CLI commands create the described account and controls	Manual portal-only steps or missing parameters
Security	Public access disabled, TLS enforced, identity-based access preferred	Shared keys or broad permissions used without reason
Lifecycle	Policy targets a specific prefix and retention logic is explained	Policy is account-wide or prefix is unvalidated
Recovery	Soft delete, versioning, redundancy, and DR assumptions documented	Replication is assumed to solve every recovery scenario
Cost	Tiering and cleanup are discussed	GZRS and versions left running without cost awareness

Table 1.4: Week 1 rubric for storage foundation deliverables.

1.11 Interview Q&A

1.11.1 Concepts and Trade-offs

1. **Q: Why is ADLS Gen2 preferred for analytics lakes over a flat Blob Storage namespace?**

A: It adds hierarchical namespace, atomic directory operations, and path-level ACLs while retaining Blob Storage scale and durability.

2. **Q: When would you choose ZRS instead of GZRS?**

A: ZRS is appropriate when zone failure is in scope but cross-region replication cost or data-residency constraints make geo-replication unsuitable.

3. **Q: Why might Archive tier be wrong for data that is rarely read?**

A: Rare reads may still be urgent. Archive requires rehydration, so RTO and operational approval can make Cool or Cold a better fit.

4. **Q: What does soft delete protect that versioning does not?**

A: Soft delete protects deleted blobs or containers during a retention window; versioning primarily protects prior object states after overwrite.

1.12 Further Reading

Begin with Microsoft Learn’s introduction to ADLS Gen2 [50] and Blob Storage [49]. Review Azure Storage redundancy for regional design [23], access tiers [11], and lifecycle management [56]. For migration planning, read the Azure Data Box overview [75] and AzCopy documentation [40]. Use the Azure Well-Architected Framework [52] and Cloud Adoption Framework [54] to connect technical choices to reliability, security, operations, and cost governance. For lakehouse table-storage context, see Delta Lake and lakehouse architecture papers [1, 2].

Key Takeaways

- Azure Storage accounts are management, security, networking, redundancy, and billing boundaries for data workloads.
- ADLS Gen2 combines Blob Storage durability with hierarchical namespace and path-level controls designed for analytics.
- Redundancy choices express failure assumptions and RPO/RTO trade-offs; they do not replace backups or DR runbooks.
- Access tiers and lifecycle policies should follow data-product usage, retention, and retrieval requirements.
- Soft delete, container soft delete, and blob versioning address different accidental-deletion and overwrite scenarios.
- AzCopy, Storage Explorer, and Azure Data Box solve different migration problems; large programs often combine offline seeding with online delta sync.
- Healthcare-scale storage architecture requires private networking, least privilege, encryption, auditability, lifecycle economics, and tested recovery procedures.

1.13 Exercises

1. **(Conceptual)** Explain the difference between the management plane and data plane for an Azure Storage account. Give one permission from each category.
2. **(Conceptual)** Compare LRS, ZRS, GRS, and GZRS for a lake that supports daily regulatory reports. State which outage each option can tolerate.

3. **(Conceptual)** A dashboard reads a curated Delta table every 15 minutes. Should that table be Hot, Cool, Cold, or Archive? Justify your answer using latency and cost.
4. **(Hands-on)** Run the Azure CLI lab in a sandbox subscription. Capture evidence that hierarchical namespace, soft delete, versioning, and the lifecycle policy are enabled.
5. **(Hands-on)** Modify the lifecycle policy so that objects under `raw/landing/` move to Cool after 7 days and Archive after 90 days. Explain how you would test the policy safely.
6. **(Hands-on)** Use the Python SDK to create directories for `bronze`, `silver`, and `gold`. Upload a small file to each and list paths recursively.
7. **(Design)** Choose a migration tool for 500 TB of historical files plus 2 TB of daily changes. State whether you would use AzCopy, Data Box, both, or another service, and explain cutover.
8. **(Design)** Write an ADR for the healthcare X-ray architecture. Include redundancy, lifecycle, encryption, access model, monitoring, and DR validation.
9. **(Design)** Identify three ways a lifecycle policy could harm production data and specify a guardrail for each.
10. **(Interview)** Prepare a two-minute explanation of why geo-redundant storage is necessary but insufficient for disaster recovery.

1.14 Capstone Project: A Compliant Medical Imaging Archive on ADLS Gen2

Capstone Project

Mission: Design and prototype the storage foundation for a compliant medical imaging archive on Azure Data Lake Storage Gen2. Your solution must show how a healthcare provider can ingest, protect, tier, replicate, and audit X-ray images while meeting security, disaster-recovery, and cost objectives.

Estimated time: 4–6 hours.

What you will practice:

- Choosing ADLS Gen2 over a flat Blob Storage namespace when analytics, directory operations, and path-level controls are required.
- Enabling hierarchical namespace, GZRS redundancy, blob versioning, blob and container soft delete, lifecycle tiering, and immutable storage.
- Designing a Hot-to-Cool-to-Archive lifecycle for a 10 PB healthcare archive.
- Planning object replication to a secondary region and documenting RPO, RTO, and failover assumptions.
- Applying encryption with customer-managed keys, least-privilege Azure RBAC, and operational evidence for HIPAA and HITRUST-aligned controls.

Scenario

A regional healthcare provider is consolidating 10 PB of X-ray images from hospital picture archiving and communication systems into Azure. The images are clinically important, legally sensitive, expensive to store, and valuable for downstream analytics. Radiologists and care teams need recent images online. Compliance officers require retention evidence and tamper resistance. Data scientists need governed access to de-identified derivatives for model devel-

opment. Operations leaders require a disaster-recovery design that survives a regional outage without pretending that storage replication alone is a complete application recovery plan.

The archive must support HIPAA and HITRUST-aligned operating practices. Azure supplies storage, identity, encryption, logging, and replication capabilities, but the provider remains responsible for configuration, access governance, incident response, and audit evidence [42]. The design must also be cost-aware: at 10 PB, an incorrect tiering rule, overly broad version retention, or accidental hot-tier default can become a large recurring expense. Treat this capstone as an architecture exercise and a small implementation prototype, not as a full production rollout.

Requirements

Your target design must satisfy the following requirements.

- **Storage foundation:** Use ADLS Gen2 with hierarchical namespace enabled so file systems, directories, atomic directory operations, and path-level ACLs support analytics workloads [50].
- **Redundancy:** Select `Standard_GZRS` for the primary storage account so data is zone-redundant in the primary region and geo-replicated to a paired secondary region [23].
- **Lifecycle:** Move raw images from Hot to Cool after 30 days and to Archive after 180 days, with a separate rule for previous versions so retained versions do not grow without review [11, 56].
- **Recoverability:** Enable blob versioning, blob soft delete, and container soft delete. Document restore procedures for accidental overwrite, accidental blob deletion, and accidental container deletion.
- **WORM retention:** Configure immutable storage with a time-based retention policy and legal hold capability for records that must be preserved in write-once, read-many form.
- **Replication:** Configure object replication from the primary account to a secondary-region account or document the exact policy that would be applied in production.
- **Encryption:** Use encryption at rest with customer-managed keys stored in Azure Key Vault. Document key rotation ownership, break-glass access, and monitoring.
- **Access control:** Enforce least privilege with Microsoft Entra ID, Azure RBAC, managed identities, and ADLS Gen2 ACLs. Avoid shared keys and account-wide contributor assignments unless a documented exception exists [86].

Reference Architecture

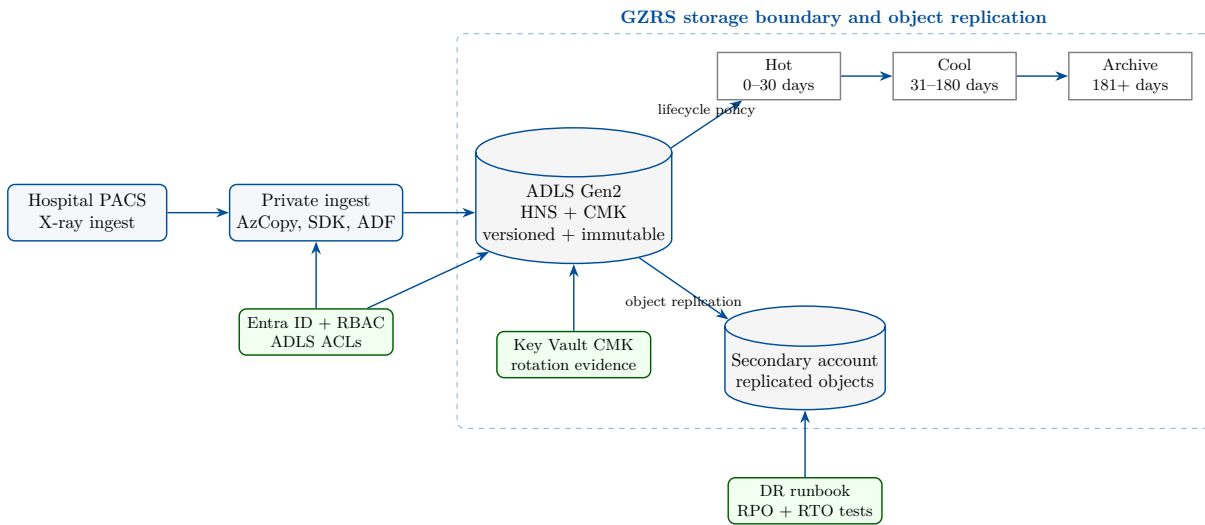


Figure 1.3: Capstone reference architecture for a compliant medical imaging archive on ADLS Gen2 with hierarchical namespace, customer-managed keys, lifecycle tiering, immutable retention, GZRS, and object replication.

Implementation Steps

Use a sandbox subscription and unique names. The commands below are written for Windows PowerShell and Azure CLI. Replace placeholder subscription, tenant, object IDs, locations, and account names before running.

1. **Create the secured ADLS Gen2 accounts.** Deploy a primary GZRS account with hierarchical namespace enabled, a secondary account in another region, and baseline recoverability controls.

```

1 az login
2 az account set --subscription "00000000-0000-0000-0000-000000000000"
3
4 $rg = "rg-meding-capstone"
5 $primaryLocation = "eastus"
6 $secondaryLocation = "westus"
7 $primary = "stmedimgprimary001"
8 $secondary = "stmedimgsecondary001"
9 $keyVault = "kv-meding-capstone-001"
10 $keyName = "cmk-meding-archive"
11 $container = "xray-raw"
12
13 az group create '
14   --name $rg '
15   --location $primaryLocation
16
17 az keyvault create '
18   --name $keyVault '
19   --resource-group $rg '
20   --location $primaryLocation '
21   --enable-rbac-authorization true
22
23 az keyvault key create '
24   --vault-name $keyVault '
25   --name $keyName '
  
```

```

26  --key RSA '
27  --size 3072
28
29 az storage account create '
30  --name $primary '
31  --resource-group $rg '
32  --location $primaryLocation '
33  --sku Standard_GZRS '
34  --kind StorageV2 '
35  --hierarchical-namespace true '
36  --min-tls-version TLS1_2 '
37  --https-only true '
38  --allow-blob-public-access false '
39  --default-action Deny '
40  --assign-identity
41
42 az storage account create '
43  --name $secondary '
44  --resource-group $rg '
45  --location $secondaryLocation '
46  --sku Standard_LRS '
47  --kind StorageV2 '
48  --hierarchical-namespace true '
49  --min-tls-version TLS1_2 '
50  --https-only true '
51  --allow-blob-public-access false '
52  --default-action Deny '
53  --assign-identity
54
55 az storage account blob-service-properties update '
56  --account-name $primary '
57  --resource-group $rg '
58  --enable-versioning true '
59  --enable-delete-retention true '
60  --delete-retention-days 30 '
61  --enable-container-delete-retention true '
62  --container-delete-retention-days 30
63
64 az storage fs create '
65  --account-name $primary '
66  --name $container '
67  --auth-mode login
68
69 az storage fs create '
70  --account-name $secondary '
71  --name $container '
72  --auth-mode login

```

Listing 1.5: PowerShell plus Azure CLI for the primary ADLS Gen2 account.

2. **Attach customer-managed keys and least-privilege roles.** Grant the storage account managed identity access to the Key Vault key, set the account encryption key source to Microsoft Key Vault, and assign data-plane roles only to ingestion and operations identities. In production, use groups or managed identities rather than individual users.
3. **Apply lifecycle management.** Save the following as `capstone-lifecycle.json`. Review the prefix carefully before applying it to production.

```

1 {
2   "rules": [
3     {
4       "enabled": true,
5       "name": "xray-hot-cool-archive",

```

```

6     "type": "Lifecycle",
7     "definition": {
8         "filters": {
9             "blobTypes": ["blockBlob"],
10            "prefixMatch": ["xray-raw/dicom/"]
11        },
12        "actions": {
13            "baseBlob": {
14                "tierToCool": {
15                    "daysAfterModificationGreaterThan": 30
16                },
17                "tierToArchive": {
18                    "daysAfterModificationGreaterThan": 180
19                }
20            },
21            "version": {
22                "tierToCool": {
23                    "daysAfterCreationGreaterThan": 30
24                },
25                "delete": {
26                    "daysAfterCreationGreaterThan": 365
27                }
28            }
29        }
30    }
31 }
32 ]
33 }

```

Listing 1.6: Lifecycle policy for raw X-ray images and older versions.

```

1 az storage account management-policy create '
2   --account-name $primary '
3   --resource-group $rg '
4   --policy .\capstone-lifecycle.json
5
6 az storage container immutability-policy create '
7   --account-name $primary '
8   --container-name $container '
9   --period 2555 '
10  --allow-protected-append-writes true '
11  --auth-mode login
12
13 az storage container legal-hold set '
14   --account-name $primary '
15   --container-name $container '
16   --tags "regulatory-review" '
17   --auth-mode login
18
19 az storage account or-policy create '
20   --resource-group $rg '
21   --account-name $secondary '
22   --source-account $primary '
23   --destination-account $secondary '
24   --rules .\object-replication-rules.json

```

Listing 1.7: Applying lifecycle, immutable retention, legal hold, and object replication.

4. **Ingest and verify a sample image manifest.** Use a non-production sample file, never real protected health information. The snippet writes one DICOM placeholder and verifies length and metadata through the ADLS Gen2 endpoint.

```

1 from azure.identity import DefaultAzureCredential
2 from azure.storage.filedatalake import DataLakeServiceClient

```

```

3
4 account_name = "stmedimgprimary001"
5 file_system_name = "xray-raw"
6 path = "dicom/facility=denver/year=2026/month=07/image-0001.dcm"
7 payload = b"DICOM-SAMPLE-PLACEHOLDER\n"
8
9 credential = DefaultAzureCredential()
10 service = DataLakeServiceClient(
11     account_url=f"https://{account_name}.dfs.core.windows.net",
12     credential=credential,
13 )
14
15 file_system = service.get_file_system_client(file_system_name)
16 directory_name, file_name = path.rsplit("/", 1)
17 directory = file_system.get_directory_client(directory_name)
18 directory.create_directory()
19
20 file_client = directory.create_file(file_name)
21 file_client.append_data(payload, offset=0, length=len(payload))
22 file_client.flush_data(len(payload))
23 file_client.set_metadata({
24     "classification": "phi",
25     "modality": "xray",
26     "retention": "worm",
27 })
28
29 properties = file_client.get_file_properties()
30 assert properties.size == len(payload)
31 assert properties.metadata["classification"] == "phi"
32 print(f"verified {path} with {properties.size} bytes")

```

Listing 1.8: Ingesting and verifying a sample object with azure-storage-file-datalake.

5. **Record validation evidence.** Capture account configuration, lifecycle policy, versioning, soft delete settings, immutability policy state, role assignments, and a DR test note. Evidence is a required deliverable, not optional paperwork.

Deliverables

Submit the following artifacts.

- A one-page architecture diagram showing primary region, secondary region, GZRS, ADLS Gen2, Key Vault, lifecycle tiers, and object replication.
- Azure CLI or infrastructure-as-code snippets that create the storage accounts, enable hierarchical namespace, configure soft delete and versioning, and apply lifecycle management.
- A short ADR explaining why ADLS Gen2, GZRS, immutable storage, customer-managed keys, and object replication were selected.
- A sample ingestion script or AzCopy command plus verification output.
- A retention and restore runbook covering overwrite recovery, blob deletion recovery, container deletion recovery, legal hold release, archive rehydration, and regional outage response.
- A cost note estimating how much of the 10 PB archive remains Hot, Cool, and Archive during normal operations.

Acceptance Criteria

- The account design uses ADLS Gen2 with hierarchical namespace enabled and does not rely on a flat namespace for analytics paths.
- Redundancy is explicitly set to GZRS or the ADR documents why a stricter data-residency rule prevents GZRS.
- Lifecycle policy transitions the intended X-ray prefix from Hot to Cool to Archive and has a separate rule for previous versions.
- Blob versioning, blob soft delete, and container soft delete are enabled and each restore scenario is explained.
- Immutable storage includes time-based retention and legal hold procedures for WORM requirements.
- Object replication policy or production-ready policy draft maps the primary X-ray container to a secondary-region account.
- Encryption uses customer-managed keys in Key Vault and includes ownership for rotation, monitoring, and emergency access.
- RBAC and ACL assignments are least-privilege and avoid shared keys for normal ingestion.
- Evidence is reproducible: another engineer can run the commands in a sandbox after changing only names and IDs.
- The DR runbook distinguishes storage replication from full application recovery and states RPO, RTO, and failover authority.

Stretch Goals

- Build a lifecycle cost model for 10 PB that compares all-Hot storage with a Hot, Cool, and Archive distribution over three years.
- Add Microsoft Purview scanning and classification for the raw and curated containers, including evidence that protected health information is tagged.
- Add private endpoints, private DNS validation, storage firewall rules, and a network diagram showing how ingestion avoids public internet exposure.
- Use Azure Policy to audit storage accounts that lack hierarchical namespace, CMK encryption, soft delete, or public access restrictions.
- Add monitoring alerts for lifecycle failures, replication lag, key access failures, denied data-plane operations, and sudden version-count growth.
- Prototype archive rehydration into a controlled restore prefix and measure the operational steps required before clinicians or auditors can read restored images.

Relational Databases: Azure SQL and PostgreSQL

Learning Objectives

- Compare Azure SQL Database purchasing models, including DTU, vCore, Serverless, and Hyperscale.
- Explain how Azure Database for PostgreSQL Flexible Server provides managed PostgreSQL operations and high availability.
- Design active geo-replication and auto-failover groups for relational workloads with clear RPO and RTO expectations.
- Deploy an Azure SQL Database Serverless tier, create a geo-replica, and simulate failover using Azure CLI and Python.
- Plan a zero-downtime migration from an on-premises SQL Server monolith to Azure SQL Managed Instance.
- Evaluate managed relational database patterns across AWS, Azure, and GCP for portability and interview readiness.
- Build a high-availability, geo-replicated Azure SQL platform that supports a SQL Server to Managed Instance cutover.

Cross-Cloud Equivalence

Managed relational databases use the same operating pattern across clouds: the provider runs backups, patching, storage durability, and high-availability plumbing while engineers retain responsibility for schema design, workload isolation, identity, network access, observability, and cost.

Capability	AWS	Azure	GCP
Managed relational	RDS / Aurora	Azure SQL / Azure DB for PostgreSQL	Cloud SQL / AlloyDB
Scale-out relational	Aurora	Azure SQL Hyperscale	AlloyDB
High availability	Multi-AZ	Zone-redundant HA / auto-failover groups	Regional HA
Read replicas	RDS replicas / Aurora replicas	Azure SQL replicas / PostgreSQL read replicas	Cloud SQL replicas / AlloyDB read pools
Global / geo	Aurora Database Global	Active geo-replication / geo-replicas	Cross-region replicas
Migration	DMS + SCT	Azure Database Migration Service	GCP DMS

MongoDB Atlas, Microsoft Fabric warehouses, and Snowflake databases echo the same managed-service lesson: teams still design data contracts, security, failover behavior, and cost controls even when the provider abstracts servers.

Key Terms

DTU: a blended Azure SQL purchasing unit representing CPU, memory, reads, and writes. **vCore**: a purchasing model that exposes compute cores, hardware family, and storage choices. **Serverless**: an Azure SQL compute tier that auto-scales and can auto-pause for intermittent workloads. **Hyperscale**: an Azure SQL architecture that separates compute from a distributed storage layer for very large databases. **Flexible Server**: the current managed PostgreSQL deployment model with zone-aware HA options.

2.1 Week 2 Overview: Relational Engines in a Data Engineering Platform

Relational databases remain central to data engineering. Operational systems publish transactions from SQL Server, PostgreSQL, Oracle, and MySQL. Analytical pipelines land change data capture events into lakes. Dimension models, reference data, master data, orchestration metadata, and serving APIs frequently depend on relational semantics. A data engineer who only knows object storage and Spark will struggle when source systems require transactional consistency, query tuning, migration windows, stored procedures, and disaster-recovery runbooks.

Azure provides several managed relational choices. **Azure SQL Database** is a platform-as-a-service relational database based on the SQL Server engine. **Azure SQL Managed Instance** provides broader SQL Server compatibility for instance-level features. **Azure Database for PostgreSQL Flexible Server** provides managed open-source PostgreSQL with Azure identity, networking, backups, and high availability [87, 78]. The engineer's job is not to memorize product names, but to map workload requirements to service boundaries.

Definition

A **managed relational database** is a database service where the cloud provider automates infrastructure operations such as host provisioning, storage durability, backup orchestration, patching, and high-availability components, while the customer owns schema, permissions, queries, workload design, data protection requirements, and application behavior.

Note

Relational service selection is often a compatibility decision before it is a performance decision. If the application requires SQL Server Agent jobs, cross-database queries, CLR, Service Broker, or near instance-level parity, evaluate Azure SQL Managed Instance before assuming single-database Azure SQL Database is enough.

2.2 Monday: Azure SQL Database Purchasing Models

2.2.1 DTU and vCore Models

Azure SQL Database offers two primary purchasing models: DTU and vCore [22]. The DTU model bundles CPU, memory, reads, and writes into a single unit. It is convenient for smaller workloads, early prototypes, and teams that want a simple knob. Its weakness is diagnostic

opacity: when a workload is constrained, a DTU percentage does not immediately say whether CPU, log throughput, memory grants, or IO latency is the root cause.

The vCore model exposes compute cores, hardware family, service tier, and storage configuration more directly. It aligns better with enterprise sizing, reserved capacity, Azure Hybrid Benefit, and conversations with database administrators who already reason in cores, memory, and IO. General Purpose separates compute from remote storage for balanced cost. Business Critical uses local SSD replicas for low-latency IO and higher availability. Hyperscale uses a different distributed storage architecture.

Model	Best fit	Trade-off
DTU	Simple workloads, early applications, predictable small databases	Blended metric can hide which resource is saturated
vCore	Enterprise sizing, reserved capacity, hybrid licensing, clearer resource reasoning	Requires more explicit capacity planning
Serverless	Intermittent applications, development, unpredictable low duty cycle workloads	Cold starts and auto-pause settings must match application expectations
Hyperscale	Very large databases, rapid scale, fast restore, read scale-out	Different operational limits and feature behavior require validation

Table 2.1: Azure SQL purchasing choices should follow workload pattern, compatibility, and operating model rather than habit.

Tip

When migrating a production SQL Server workload, begin with observed baselines: peak CPU, memory pressure, data file size, log generation rate, tempdb use, longest queries, wait statistics, maintenance windows, and business RPO/RTO. Do not size only by database size.

2.2.2 Serverless for Variable Demand

Azure SQL Database Serverless is a compute tier in the vCore model that automatically scales compute within configured limits and can auto-pause after an inactivity delay [65]. Billing is based on compute used per second and storage allocated. It is useful for development databases, departmental applications, bursty APIs, training labs, and infrequently used operational stores.

Serverless is not a universal cost saver. If an application is always active, provisioned compute may be cheaper and more predictable. If a database auto-pauses, the first connection after inactivity must resume compute. Applications with strict latency service-level objectives may need a minimum vCore setting or disabled auto-pause. Workloads with background health checks may never pause.

Common Pitfall

Treating auto-pause as free savings. Auto-pause changes user experience. Test connection retry logic, login timeouts, monitoring probes, and application pools before using Serverless for anything user-facing.

2.2.3 Hyperscale Architecture

Hyperscale supports very large Azure SQL databases by separating compute nodes from a distributed storage subsystem with page servers and log services [44]. Compute replicas read pages from the storage layer and cache hot pages locally. The log service sequences writes and enables fast restore because data pages already live in distributed storage. Read scale-out replicas can serve reporting traffic without overloading the primary compute.

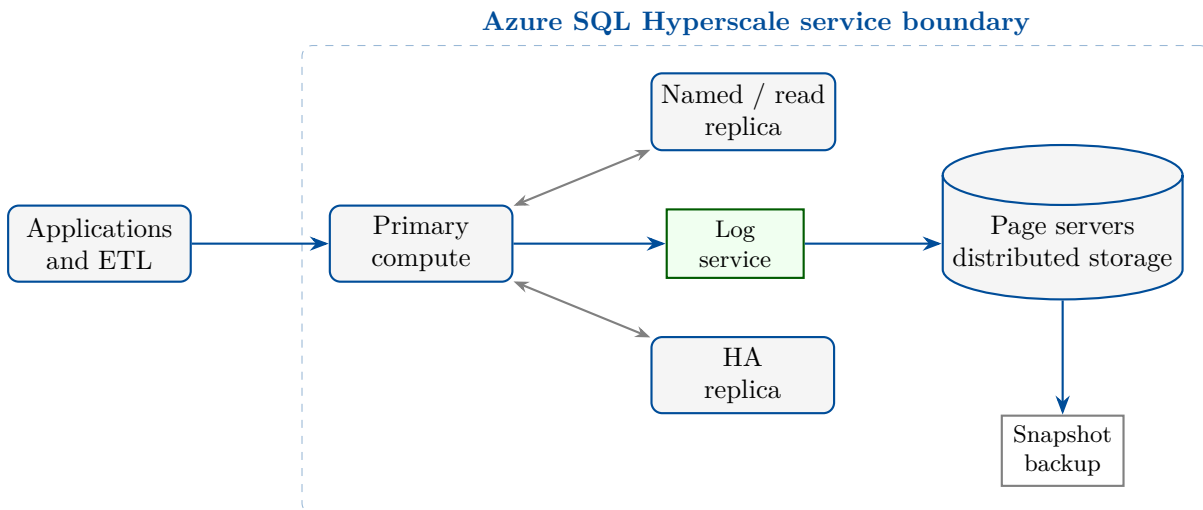


Figure 2.1: Azure SQL Hyperscale separates compute from log and page storage so large databases can scale and restore differently from traditional monolithic storage.

2.3 Tuesday: Azure Database for PostgreSQL Flexible Server and High Availability

2.3.1 Flexible Server as the Default PostgreSQL Model

Azure Database for PostgreSQL Flexible Server is the managed PostgreSQL deployment option used for most new Azure PostgreSQL workloads [78]. It provides configurable compute and storage, automatic backups, point-in-time restore, maintenance windows, private networking, read replicas, and high-availability options. Because it runs community PostgreSQL, application portability is usually stronger than with proprietary engines, but extensions, version support, parameter settings, and operational limits must still be checked.

Flexible Server separates concerns that on-premises teams often blend together. Azure owns host replacement, storage durability, backups, and platform patching. The application team owns schema migrations, connection pooling, query plans, vacuum behavior, bloat management, extension choice, and transaction design. PostgreSQL performance failures in the cloud are still often ordinary PostgreSQL failures: missing indexes, unbounded transactions, chatty connections, overloaded temp files, or poor statistics.

Definition

Flexible Server high availability provisions a standby server that can take over when the primary becomes unavailable. Zone-redundant HA places standby capacity in a different availability zone when the region supports zones; same-zone HA focuses on local resilience with lower network latency.

2.3.2 High Availability and Backups

High availability and backups solve different problems. HA reduces outage duration when a node or zone fails. Backups restore data to a previous point in time after corruption, accidental delete, bad deployments, or application mistakes. A standby faithfully replicates bad data changes; backups allow recovery before those changes. Data engineers must include both in the architecture.

Note

PostgreSQL HA failover changes the server endpoint target, but applications still need connection retry behavior. Use connection pooling carefully: stale connections can make a healthy failover look like an application outage.

Capability		What it provides	What it does not provide
Zone-redundant HA		Standby in another availability zone and automated failover	Protection from bad SQL, accidental drops, or cross-region disaster
Point-in-time restore	re-	Recovery to a prior timestamp within backup retention	Zero data loss after a primary-region outage
Read replica		Read scale-out and reporting isolation	Synchronous HA or automatic application cutover by itself
Maintenance window	win-	Predictable platform patching window	Application release governance or schema migration safety

Table 2.2: PostgreSQL operational controls must be combined to meet availability and recoverability goals.

2.3.3 Designing PostgreSQL for Data Engineering

PostgreSQL often appears in data platforms as a metadata store, operational data mart, feature-serving database, orchestration catalog, or domain service database. The design should choose indexes for access patterns, keep transactions short, use connection pooling, monitor vacuum, and isolate reporting from write-heavy workloads. Read replicas are valuable for BI or extraction workloads, but replica lag must be measured before using them for near-real-time pipelines.

Tip

For ingestion-heavy PostgreSQL workloads, track write-ahead-log generation, autovacuum lag, dead tuples, lock waits, checkpoint behavior, and replica lag. These signals are often more useful than a generic CPU chart.

2.4 Wednesday: Active Geo-Replication and Auto-Failover Groups

2.4.1 Active Geo-Replication

Active geo-replication creates up to four readable secondary databases in different regions for Azure SQL Database [12]. Replication is asynchronous. The secondary can support read-only workloads and can be promoted during disaster recovery. Because replication is asynchronous, the platform can lose the most recent committed transactions during a regional disaster. The business must decide whether that RPO is acceptable.

Active geo-replication is database-scoped. It is excellent when one critical database needs a readable copy in another region. However, many applications use several databases together. If

those databases must fail over as a unit, an auto-failover group is usually a better control-plane abstraction.

Common Pitfall

Ignoring consistency across databases. If an application writes to five databases, failing over only one database can create broken business state. Group related databases and test the full application, not only the database endpoint.

2.4.2 Auto-Failover Groups

Auto-failover groups provide a listener endpoint and group-level failover for one or more databases on Azure SQL Database logical servers or SQL Managed Instances [14]. Applications connect to a read-write listener and optionally a read-only listener. During failover, the listener redirects to the new primary. This lowers DNS and connection-string complexity, but it does not remove the need for retries, idempotent writes, and operational decision authority.

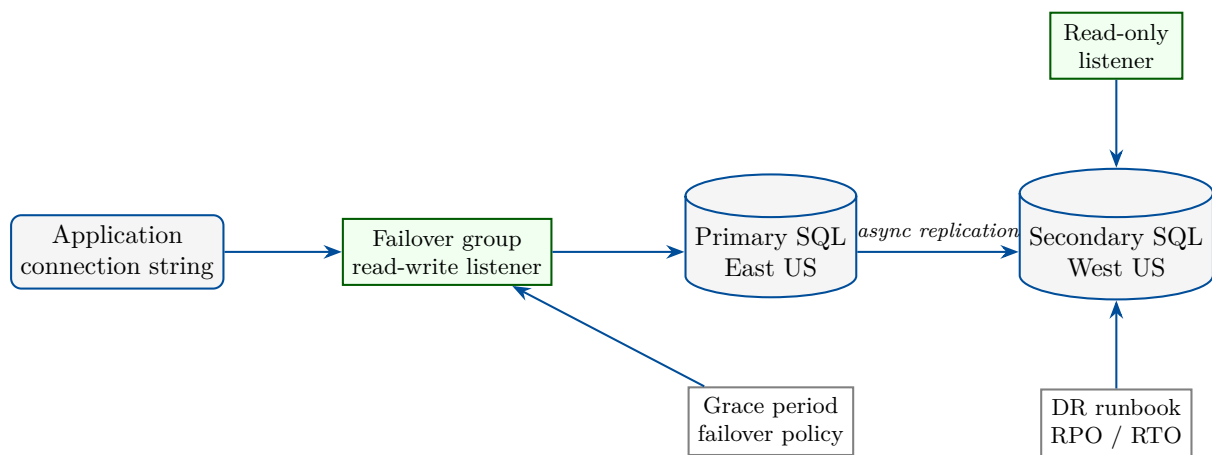


Figure 2.2: Azure SQL auto-failover groups provide listener endpoints over asynchronously replicated databases for regional resilience.

2.4.3 Testing Failover Without Fooling Yourself

A failover test should include more than pressing a button. Capture baseline lag, pause nonessential writes, run synthetic transactions, fail over, reconnect through the listener, validate data, observe application retries, and fail back only after leadership approves. If the database feeds ETL, test downstream idempotency: a pipeline may reread rows or miss rows if watermark logic is not designed for failover.

Note

Disaster recovery is a socio-technical process. Azure can provide replicas and listeners, but people still decide when to fail over, how much data loss is acceptable, who communicates impact, and how systems return to normal.

2.5 Thursday Hands-on Project: Serverless Azure SQL with Geo-Replica and Failover

This lab deploys an Azure SQL logical server, a Serverless database, a secondary server in another region, and a geo-replica. It then uses Python to write and validate a small workload before a manual failover simulation. Run it only in a sandbox subscription and delete the resource group when finished.

Note

The examples are written for Windows PowerShell with Azure CLI. Replace server names with globally unique names and use a strong administrator password stored outside source control.

```

1 az login
2 az account set --subscription "00000000-0000-0000-0000-000000000000"
3
4 $rg = "rg-de-azure-book-week2"
5 $primaryLocation = "eastus"
6 $secondaryLocation = "westus"
7 $primaryServer = "sql-week2-primary-001"
8 $secondaryServer = "sql-week2-secondary-001"
9 $db = "sqldb-orders-serverless"
10 $admin = "sqladminuser"
11 $password = Read-Host "SQL admin password" -AsSecureString
12 $plainPassword = [Runtime.InteropServices.Marshal]::PtrToStringAuto(
13     [Runtime.InteropServices.Marshal]::SecureStringToBSTR($password))
14
15 az group create '
16     --name $rg '
17     --location $primaryLocation
18
19 az sql server create '
20     --name $primaryServer '
21     --resource-group $rg '
22     --location $primaryLocation '
23     --admin-user $admin '
24     --admin-password $plainPassword
25
26 az sql server create '
27     --name $secondaryServer '
28     --resource-group $rg '
29     --location $secondaryLocation '
30     --admin-user $admin '
31     --admin-password $plainPassword
32
33 az sql db create '
34     --resource-group $rg '
35     --server $primaryServer '
36     --name $db '
37     --edition GeneralPurpose '
38     --compute-model Serverless '
39     --family Gen5 '
40     --capacity 2 '
41     --min-capacity 0.5 '
42     --auto-pause-delay 60 '
43     --zone-redundant false
44
45 az sql db replica create '
46     --resource-group $rg '
47     --server $primaryServer '

```

```

48 --name $db '
49 --partner-resource-group $rg '
50 --partner-server $secondaryServer

```

Listing 2.1: Deploying Azure SQL Database Serverless and creating a geo-replica with Azure CLI.

Open firewall access only for a controlled client IP or private network. The simple rule below is a lab convenience and should not be copied into production without review.

```

1 $clientIp = (Invoke-RestMethod -Uri "https://api.ipify.org")
2
3 az sql server firewall-rule create '
4   --resource-group $rg '
5   --server $primaryServer '
6   --name "client-ip" '
7   --start-ip-address $clientIp '
8   --end-ip-address $clientIp
9
10 az sql server firewall-rule create '
11   --resource-group $rg '
12   --server $secondaryServer '
13   --name "client-ip" '
14   --start-ip-address $clientIp '
15   --end-ip-address $clientIp
16
17 az sql db replica list-links '
18   --resource-group $rg '
19   --server $primaryServer '
20   --name $db '
21   --output table

```

Listing 2.2: Adding a narrow client firewall rule and checking replica state.

The Python snippet creates a table, inserts a row, and reads it back. Install pyodbc only in a local lab environment with the Microsoft ODBC Driver for SQL Server available.

```

1 import os
2 import pyodbc
3
4 server = os.environ["AZURE_SQL_SERVER"]
5 database = os.environ.get("AZURE_SQL_DATABASE", "sqlldb-orders-serverless")
6 username = os.environ["AZURE_SQL_USER"]
7 password = os.environ["AZURE_SQL_PASSWORD"]
8
9 connection_string = (
10     "Driver={ODBC Driver 18 for SQL Server};"
11     f"Server=tcp:{server}.database.windows.net,1433;"
12     f"Database={database};Uid={username};Pwd={password};"
13     "Encrypt=yes;TrustServerCertificate=no;Connection Timeout=30;"
14 )
15
16 with pyodbc.connect(connection_string) as conn:
17     cursor = conn.cursor()
18     cursor.execute("""
19         IF OBJECT_ID('dbo.orders') IS NULL
20         CREATE TABLE dbo.orders (
21             order_id int NOT NULL PRIMARY KEY,
22             status nvarchar(40) NOT NULL,
23             updated_at datetime2 NOT NULL DEFAULT sysutcdatetime()
24         )
25     """)
26     cursor.execute("""

```



```

27         MERGE dbo.orders AS target
28         USING (SELECT 1001 AS order_id, N'paid' AS status) AS source
29         ON target.order_id = source.order_id
30         WHEN MATCHED THEN UPDATE SET status = source.status,
31             updated_at = sysutcdatetime()
32         WHEN NOT MATCHED THEN INSERT (order_id, status)
33             VALUES (source.order_id, source.status);
34     """
35     conn.commit()
36     row = cursor.execute(
37         "SELECT order_id, status FROM dbo.orders WHERE order_id = 1001"
38     ).fetchone()
39     print(row.order_id, row.status)

```

Listing 2.3: Validating a write before failover with Python and pyodbc.

After validating the primary, simulate regional failover by promoting the replica. Update the Python environment variable to the secondary server and rerun the read query.

```

1 az sql db replica set --primary '
2   --resource-group $rg '
3   --server $secondaryServer '
4   --name $db
5
6 $env:AZURE_SQL_SERVER = $secondaryServer
7 $env:AZURE_SQL_DATABASE = $db
8 python .\validate-orders.py

```

Listing 2.4: Simulating regional failover by promoting the geo-replica.

Common Pitfall

Failover without client testing. A database can promote successfully while applications still fail because connection strings, DNS, firewall rules, secret stores, or retry policies still point to the old region.

2.6 Friday Use Case: Zero-Downtime Migration to Azure SQL Managed Instance

2.6.1 Scenario

A retailer runs a monolithic on-premises SQL Server that supports orders, inventory, billing, and customer service. The database is 6 TB, uses SQL Server Agent jobs, linked-server dependencies, cross-database queries, and several vendor stored procedures. The business wants to move to Azure with almost no downtime. Azure SQL Managed Instance is selected because it provides high SQL Server compatibility while reducing infrastructure operations [88].

Zero downtime is an aspiration, not a magic switch. The practical target is a short controlled cutover window with continuous replication before the switch. The migration team must assess compatibility, remediate blockers, establish private connectivity, provision Managed Instance, migrate logins and jobs, seed data, replicate changes, rehearse cutover, freeze risky releases, and validate application behavior after DNS or connection-string changes.

Definition

Azure SQL Managed Instance is a managed SQL Server-compatible platform-as-a-service deployment option that preserves many instance-level capabilities while Azure operates the underlying infrastructure, patching, backups, and availability components.

2.6.2 Migration Architecture

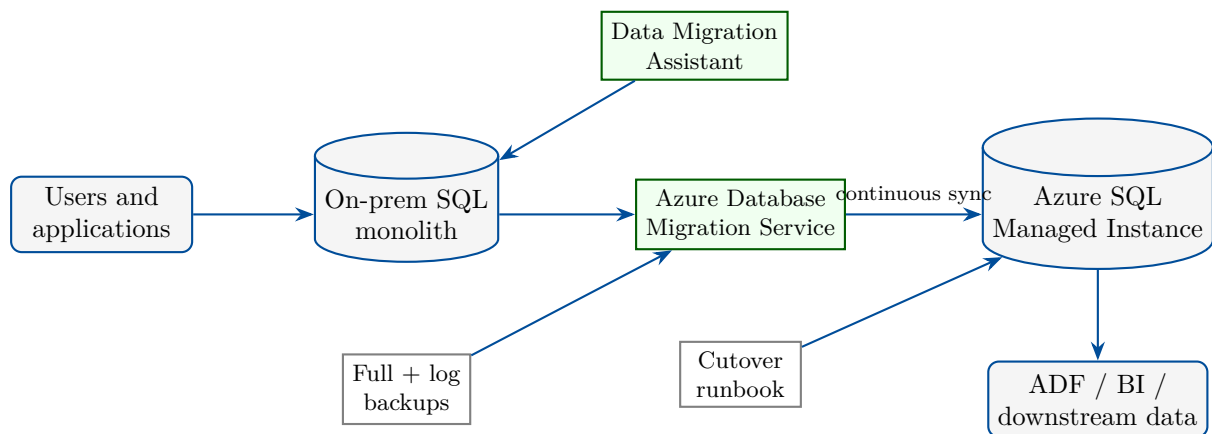


Figure 2.3: Zero-downtime migration pattern from an on-premises SQL Server monolith to Azure SQL Managed Instance using assessment, seeding, continuous synchronization, and controlled cutover.

2.6.3 Cutover Runbook

The runbook begins weeks before migration night. Data Migration Assistant or Azure Migrate assessments identify unsupported features, deprecated syntax, collation issues, SQL Agent job dependencies, and performance risks. Network teams validate ExpressRoute or VPN paths. Security teams create Microsoft Entra groups, managed identities, private DNS zones, and Key Vault entries. DBAs rehearse restore and synchronization. Data engineers validate downstream extract jobs, watermark logic, and reporting latency.

During the final cutover, the team freezes writes or places the application in maintenance mode, lets replication catch up, validates row counts and checksums, promotes Managed Instance, updates connection endpoints, starts smoke tests, watches error rates, and confirms downstream pipelines. Rollback must be explicit: either point applications back to on-premises before new writes occur in Azure, or accept Azure as the system of record and remediate forward.

Tip

For near-zero downtime, rehearse the exact cutover multiple times with production-like data and timing. The first successful migration should never be the production migration.

2.7 Architectural Decision Record Template

A Week 2 ADR should capture engine choice, compatibility constraints, purchasing model, HA pattern, backup retention, geo-replication, failover authority, private networking, identity model, migration approach, and cost controls. Include alternatives: Azure SQL Database, SQL Managed Instance, PostgreSQL Flexible Server, self-managed virtual machines, and cross-cloud options. The ADR is strongest when it records rejected options and why they were rejected.

2.8 Operational Deep Dive: Performance, Identity, and Observability

Relational platforms fail through ordinary engineering gaps. Queries miss indexes, statistics go stale, connection pools saturate, transactions hold locks, deployments change query plans, and extract jobs create accidental denial of service. Azure adds tools such as Query Store, automatic tuning, metrics, diagnostic logs, alerts, Microsoft Defender for SQL, and Azure Monitor, but the team still needs ownership.

Identity should prefer Microsoft Entra ID, managed identities, group-based RBAC, and contained database users where supported. Avoid embedding SQL administrator passwords in pipelines. Network access should use private endpoints or private link patterns for production. Observability should include CPU, storage, log IO, deadlocks, failed connections, long-running queries, replica lag, failover events, backup state, and cost by environment.

Common Pitfall

Letting ETL act like an unbounded user. A nightly extraction that scans every table can starve the operational workload. Use read replicas, change tracking, CDC, batching, and workload windows rather than brute-force reads.

2.9 Troubleshooting Patterns

Common relational incidents fall into five groups. Connectivity failures require checking firewall rules, private DNS, TLS settings, credentials, and listener names. Performance failures require Query Store, wait statistics, indexes, plans, and workload changes. HA failures require examining failover events and client retry logic. Replication surprises require checking lag, unsupported operations, and region availability. Migration failures require compatibility reports, login mappings, SQL Agent jobs, linked servers, and collation differences.

A disciplined response avoids broad fixes. Do not scale from two vCores to thirty-two vCores before checking whether a missing index caused the outage. Do not disable firewall rules because one pipeline cannot connect. Do not force failover until you know replication lag and business impact. As with storage, small precise changes preserve trust.

2.10 Week 2 Lab Rubric

A complete Week 2 submission includes Azure CLI deployment evidence, a diagram of primary and secondary relational regions, Python validation output before and after failover, an ADR explaining purchasing and HA choices, and a migration runbook for the Managed Instance scenario.

Criterion	Meets expectation	Needs improvement
Model selection	DTU, vCore, Serverless, and Hyperscale trade-offs are explained	Service tier chosen without workload evidence
HA / DR	Geo-replication, failover groups, RPO, and RTO are documented	Replica assumed to solve all application recovery
Hands-on	CLI creates database and replica; Python validates data before and after failover	Commands are portal-only or validation is missing
Security	Private access, Entra ID, least privilege, and secrets hygiene are addressed	Broad firewall or admin password habits remain
Migration	Managed Instance cutover has assessment, sync, validation, rollback, and downstream checks	Migration is described as a one-step restore

Table 2.3: Week 2 rubric for relational database architecture and hands-on deliverables.

2.11 Interview Q&A

2.11.1 Concepts and Trade-offs

- Q: When would you choose vCore over DTU for Azure SQL Database?**
 A: Choose vCore when you need explicit control over cores, hardware family, storage, reserved capacity, hybrid licensing, or enterprise capacity planning.
- Q: Why is Serverless not always cheaper?**
 A: Always-active workloads may pay continuously, auto-pause may never trigger, and cold-start latency can force a minimum compute setting.
- Q: What makes Hyperscale different from Business Critical?**
 A: Hyperscale separates compute from distributed page and log storage for very large databases and fast restore; Business Critical focuses on local SSD replicas and low-latency IO.
- Q: What does PostgreSQL Flexible Server HA protect against?**
 A: It reduces outage from server or zone failures, but it does not protect against bad application writes; backups and PITR are still required.
- Q: Why use an auto-failover group instead of only active geo-replication?**
 A: A failover group provides listener endpoints and group-level failover for related databases, simplifying application connection management.
- Q: What is the largest risk in a zero-downtime SQL Server migration?**
 A: Unrehearsed cutover dependencies: compatibility gaps, logins, jobs, connection strings, downstream ETL, and rollback rules can break despite successful data copy.

2.12 Further Reading

Start with Azure SQL Database documentation [87], purchasing model guidance [22], Serverless [65], and Hyperscale [44]. Review Azure Database for PostgreSQL Flexible Server [78] and its high-availability concepts [43]. For regional resilience, read active geo-replication [12] and auto-failover groups [14]. For migration, study Azure SQL Managed Instance [88], Azure Database Migration Service [79], and the Data Migration Assistant [34]. Use the Azure Well-Architected Framework [52] to evaluate reliability, security, operational excellence, performance, and cost.

Key Takeaways

- Azure SQL Database purchasing models differ by abstraction: DTU is blended, vCore is explicit, Serverless is elastic for intermittent demand, and Hyperscale changes storage architecture.
- PostgreSQL Flexible Server provides managed operations, but PostgreSQL fundamentals such as indexes, vacuum, transactions, and connection pooling still matter.
- HA, backups, read replicas, active geo-replication, and failover groups solve different reliability problems.
- Asynchronous geo-replication requires explicit RPO acceptance and application-level retry, idempotency, and validation.
- Zero-downtime migration depends on assessment, rehearsal, continuous sync, cutover governance, rollback rules, and downstream data validation.
- Managed database services reduce infrastructure toil; they do not remove schema, security, performance, observability, or DR ownership.

2.13 Exercises

1. **(Conceptual)** Compare DTU and vCore for a 200 GB order-processing database. Which model gives clearer capacity planning and why?
2. **(Conceptual)** Explain when Azure SQL Serverless is appropriate and list two application behaviors that can prevent auto-pause savings.
3. **(Conceptual)** Draw the difference between Azure SQL active geo-replication and an auto-failover group.
4. **(Hands-on)** Run the Serverless Azure SQL deployment in a sandbox and capture evidence of the database compute model and replica link.
5. **(Hands-on)** Modify the Python validation script to insert ten orders, fail over, and confirm all ten rows exist on the promoted replica.
6. **(Hands-on)** Create a PostgreSQL Flexible Server design checklist that includes HA, backups, private access, parameter changes, and read replicas.
7. **(Design)** Choose between Azure SQL Database, Azure SQL Managed Instance, and PostgreSQL Flexible Server for a vendor application with SQL Agent jobs and cross-database queries.
8. **(Design)** Write a zero-downtime migration runbook for the retailer scenario, including rollback criteria and downstream ETL validation.

2.14 Capstone Project: HA Geo-Replicated Azure SQL Platform for Managed Instance Cutover

Capstone Project

Mission: Design and prototype a high-availability, geo-replicated Azure SQL platform that supports a controlled SQL Server to Azure SQL Managed Instance cutover. Your solution must combine assessment, private networking, continuous migration, failover design, validation, and operational evidence.

Estimated time: 5–7 hours.

What you will practice:

- Selecting Azure SQL Managed Instance when instance-level SQL Server compatibility matters.
- Designing regional resilience with failover groups, backups, PITR, monitoring, and application retry behavior.
- Preparing a near-zero-downtime migration using assessment, seeding, continuous synchronization, and cutover rehearsal.
- Validating data integrity, downstream ETL behavior, and rollback criteria.
- Producing evidence suitable for an architecture review and migration go/no-go meeting.

Scenario

A financial services company must move its 4 TB on-premises SQL Server monolith to Azure SQL Managed Instance. The application supports payments, customer service, finance reconciliation, and regulatory reporting. Planned downtime is limited to 30 minutes. The platform must be resilient to zone failure and prepared for regional failover. Data engineers own downstream extracts into ADLS Gen2 and must prove that the cutover will not duplicate or lose data in the lake.

Requirements

- **Compatibility:** Use Azure SQL Managed Instance because the workload depends on SQL Server Agent jobs, cross-database queries, and high SQL Server surface-area compatibility.
- **High availability:** Use a Business Critical or General Purpose configuration with documented zone redundancy where supported and clear maintenance windows.
- **Geo-resilience:** Design an auto-failover group between primary and secondary Managed Instances and state asynchronous replication RPO assumptions.
- **Migration:** Use Azure Database Migration Service or a documented backup and log-shipping style approach for initial load plus continuous synchronization.
- **Security:** Use private endpoints or delegated subnet integration, Microsoft Entra groups, Key Vault for secrets, TLS, auditing, and least-privilege roles.
- **Validation:** Reconcile row counts, checksums for critical tables, SQL Agent job behavior, application smoke tests, and downstream ETL watermarks.
- **Operations:** Provide monitoring alerts, failover authority, rollback criteria, communication plan, and cleanup steps for obsolete on-premises dependencies.

Reference Architecture

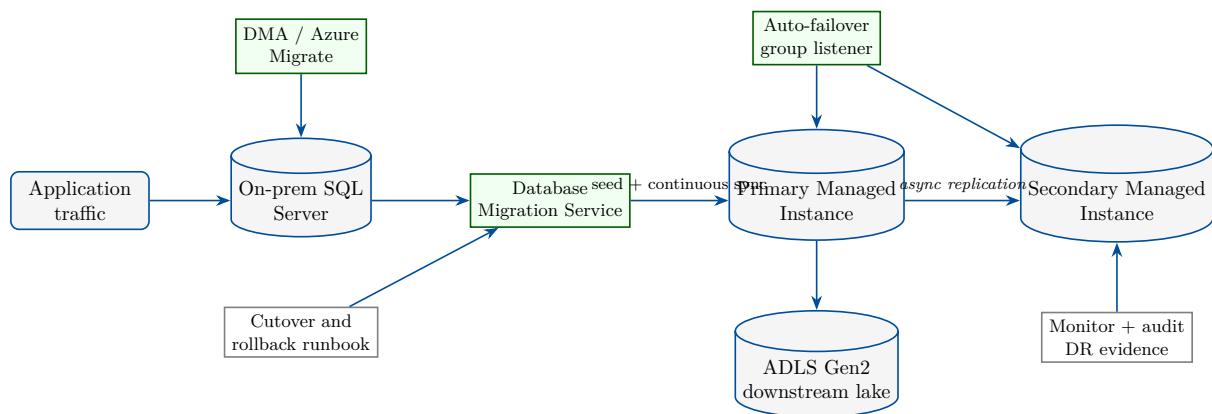


Figure 2.4: Capstone reference architecture for a high-availability Azure SQL Managed Instance migration with continuous sync, failover group, downstream lake validation, and operational evidence.

Implementation Steps

Use a sandbox subscription for the prototype. Managed Instance deployments can take significant time and incur cost, so adapt names and sizes to your lab constraints. The snippets below show the structure rather than a production sizing recommendation.

1. **Create resource group and plan the Managed Instance network.** Managed Instance requires a dedicated subnet with appropriate delegation and network controls.

```

1 az login
2 az account set --subscription "00000000-0000-0000-0000-000000000000"
3
4 $rg = "rg-sqlmi-cutover-capstone"
5 $location = "eastus"
6 $vnet = "vnet-sqlmi-cutover"
7 $subnet = "snet-managed-instance"
8 $nsg = "nsg-sqlmi-cutover"
9 $routeTable = "rt-sqlmi-cutover"
10
11 az group create '
12   --name $rg '
13   --location $location
14
15 az network vnet create '
16   --resource-group $rg '
17   --location $location '
18   --name $vnet '
19   --address-prefixes 10.40.0.0/16 '
20   --subnet-name $subnet '
21   --subnet-prefixes 10.40.1.0/24
22
23 az network nsg create '
24   --resource-group $rg '
25   --location $location '
26   --name $nsg
27
28 az network route-table create '
29   --resource-group $rg '
30   --location $location '

```

```

31  --name $routeTable
32
33  az network vnet subnet update '
34  --resource-group $rg '
35  --vnet-name $vnet '
36  --name $subnet '
37  --network-security-group $nsg '
38  --route-table $routeTable '
39  --delegations Microsoft.Sql/managedInstances

```

Listing 2.5: PowerShell plus Azure CLI skeleton for Managed Instance network preparation.

2. **Prototype the primary Managed Instance and capture configuration.** Use Azure Portal, Bicep, Terraform, or Azure CLI according to organizational standards. The key deliverable is reproducible evidence: SKU, storage, collation, time zone, subnet, administrator model, backup retention, auditing, and maintenance window.

```

1  $mi = "mi-sql-cutover-primary-001"
2  $admin = "sqlmiadmin"
3  $password = Read-Host "Managed Instance admin password" -AsSecureString
4  $plainPassword = [Runtime.InteropServices.Marshal]::PtrToStringAuto(
5  [Runtime.InteropServices.Marshal]::SecureStringToBSTR($password))
6  $subnetId = az network vnet subnet show '
7  --resource-group $rg '
8  --vnet-name $vnet '
9  --name $subnet '
10 --query id '
11 --output tsv
12
13 az sql mi create '
14 --resource-group $rg '
15 --name $mi '
16 --location $location '
17 --subnet $subnetId '
18 --admin-user $admin '
19 --admin-password $plainPassword '
20 --license-type BasePrice '
21 --storage 256GB '
22 --capacity 4 '
23 --tier GeneralPurpose '
24 --family Gen5
25
26 az sql mi show '
27 --resource-group $rg '
28 --name $mi '
29 --query "{name:name,state:state,sku:sku.name,storage:storageSizeInGB,
30 subnet:subnetId}" '
31 --output table

```

Listing 2.6: Managed Instance creation skeleton and configuration evidence commands.

3. **Plan the geo-replicated cutover.** Create a secondary Managed Instance in the paired region, configure an auto-failover group, and require application teams to connect through the listener. Document RPO expectations because replication is asynchronous.
4. **Run assessment and migration rehearsal.** Execute Data Migration Assistant reports, remediate blockers, seed a copy with Azure Database Migration Service, start continuous sync, and measure lag during normal and peak workloads.
5. **Validate data and downstream pipelines.** Before cutover, compare critical table row counts, checksums, SQL Agent jobs, permissions, and ETL watermarks. After cutover, run application smoke tests and the downstream ADLS Gen2 extraction validation.

Deliverables

- Architecture diagram showing on-premises SQL Server, DMS or migration tooling, primary Managed Instance, secondary Managed Instance, failover group listener, private network, Key Vault, monitoring, and ADLS Gen2 downstream consumers.
- ADR explaining why Azure SQL Managed Instance was selected and why Azure SQL Database, PostgreSQL Flexible Server, and SQL Server on virtual machines were rejected.
- Reproducible deployment snippets or infrastructure-as-code for network, Managed Instance, auditing, backup retention, and failover group configuration.
- Migration runbook covering assessment, remediation, seed load, continuous sync, pre-cutover checks, cutover, validation, rollback, and communication.
- Evidence pack with assessment results, row-count reconciliation, checksum queries, failover test output, application smoke-test results, and downstream ETL validation.

Acceptance Criteria

- Managed Instance is justified by documented SQL Server compatibility requirements.
- Network design uses a dedicated subnet and avoids public administrative habits for production.
- HA, backup retention, PITR, failover group, RPO, and RTO are explicitly documented.
- Migration process includes initial load, continuous synchronization, measured lag, and at least one rehearsal.
- Cutover plan includes freeze window, final sync validation, listener or connection-string switch, smoke tests, and rollback criteria.
- Downstream data engineering jobs have validated watermarks so cutover does not duplicate or skip records.
- Security design uses least privilege, Key Vault, auditing, and group-based access instead of shared administrator credentials.
- Evidence is complete enough for a go/no-go meeting and for another engineer to reproduce the lab with changed names.

Stretch Goals

- Add a Bicep or Terraform implementation for the full Managed Instance network and primary instance.
- Build a failover drill script that records listener endpoint, replication role, application smoke-test status, and elapsed recovery time.
- Add Query Store baseline capture before and after migration to compare top queries and regressions.
- Use Microsoft Defender for SQL, vulnerability assessment, and audit logs to build a security evidence appendix.
- Add a downstream CDC pipeline that writes to ADLS Gen2 and proves idempotent replay across cutover.

- Estimate three-year cost for General Purpose versus Business Critical, reserved capacity, Azure Hybrid Benefit, and secondary-region DR.

High-Scale NoSQL: Azure Cosmos DB

Learning Objectives

- Explain Azure Cosmos DB core concepts: Request Units, logical partitions, physical partitions, replicas, and global distribution.
- Compare the five Cosmos DB consistency levels and select the weakest level that still satisfies business correctness.
- Choose between the Core (NoSQL), MongoDB, and Cassandra APIs based on application model, tooling, and portability requirements.
- Use the Change Feed to connect operational NoSQL writes to streaming, lakehouse, search, and serving pipelines.
- Create a Cosmos DB for NoSQL account, design a single-container forum schema, insert items with Python, and configure multi-region writes.
- Design a globally distributed gaming leaderboard capable of high read and write rates with Session consistency.
- Build a capstone architecture for a multi-region leaderboard with partitioning, observability, cost controls, and operational evidence.

Cross-Cloud Equivalence

NoSQL and document databases look different across vendors, but data engineers ask the same questions everywhere: What is the partition key? How is throughput billed? Which consistency model is exposed? How do changes flow downstream? What happens during regional failure?

Concept	Azure DB	Cosmos	AWS DynamoDB	GCP Firestore / Bigtable	MongoDB
Primary partitioning	Container partition key routes items to logical partitions	Partition key, optional sort key	Firestore document paths; Bigtable row key	Shard key	
Throughput unit	Request Units per second (RU/s) or serverless RUs	Read / write capacity (RCU/WCU) or on-demand	Firestore operations; Bigtable nodes and storage	Cluster tier, IOPS, and workload capacity	
Secondary indexes	Automatic indexing policy with included / excluded paths and composites	Global and local secondary indexes	Firestore composite indexes; Bigtable schema by row key	Secondary and compound indexes	
Change stream	Change Feed and Change Feed Processor	DynamoDB Streams	Firestore triggers / exports; Bigtable change streams	Change streams	
TTL	Item or container time-to-live	Item TTL attribute	Firestore TTL policies; Bigtable garbage collection	TTL indexes	
Multi-region writes	Multi-region account with write regions and conflict policy	Global tables	Multi-region Firestore; Bigtable replication	Global clusters / sharded clusters	
Consistency	Five tunable levels from Strong to Eventual	Strong or eventual reads, transactional options	Strong regional semantics vary by service	Read / write concern and causal consistency	

A portable NoSQL design names its access patterns, partition strategy, indexing choices, retry behavior, and conflict rules instead of assuming one product's defaults will transfer unchanged to another cloud.

Key Terms

Request Unit (RU): a normalized charge for CPU, memory, and IO consumed by an operation. **Logical partition:** all items in a container with the same partition-key value. **Physical partition:** service-managed storage and compute that hosts one or more logical partitions. **Consistency level:** a contract that controls read freshness and ordering. **Change Feed:** an ordered feed of item changes used for event-driven processing.

3.1 Week 3 Overview: Why Cosmos DB Matters to Data Engineers

Azure Cosmos DB is a globally distributed, multi-model NoSQL database service designed for low-latency reads and writes, elastic throughput, automatic indexing, and configurable consistency [69]. It appears in data platforms as an operational serving layer, user-profile store, catalog store, IoT metadata database, event-sourced projection, feature-serving database, and high-scale API backend.

Relational databases optimize joins, transactions, and normalized constraints. Cosmos DB optimizes predictable item access by partition key, low-latency point reads, horizontal scale, regional distribution, and flexible JSON documents. The change is not merely syntax. A Cosmos DB engineer must design containers around access patterns, choose partition keys that distribute load, estimate RU consumption, define indexing policies, and accept the consistency contract needed by the product.

Definition

Azure Cosmos DB for NoSQL stores JSON items in containers. Every item has an `id`, belongs to one logical partition selected by a partition key, is automatically indexed by default, and consumes Request Units for reads, writes, queries, and maintenance operations.

Note

Cosmos DB is not a replacement for every relational database. If the workload depends on multi-row joins, large ad hoc aggregations, cross-container transactions, or strict relational integrity, use Cosmos DB as a serving projection or event store rather than as the only system of record.

3.2 Monday: Core Concepts — RUs, Partitioning, and Global Distribution

3.2.1 Request Units as the Throughput Currency

Every Cosmos DB operation consumes RUs. A point read of a 1 KB item is often close to 1 RU, while larger documents, complex queries, cross-partition scans, writes, indexing, and consistency choices consume more [63]. Provisioned throughput allocates RU/s to a database or container. Autoscale raises and lowers throughput within a configured maximum. Serverless charges per consumed RU and suits intermittent or development workloads.

RUs make capacity planning explicit. Instead of asking only how many vCores a database needs, estimate operations per second multiplied by RU per operation. For example, 20,000 point reads per second at 1 RU each require roughly 20,000 RU/s before retries, spikes, consistency overhead, and regional replication are considered. Queries that scan many partitions can consume orders of magnitude more.

Tip

Measure RU charge from real SDK responses. Theoretical estimates are useful for sizing, but query shape, item size, index policy, consistency level, and partition fan-out determine actual cost.

3.2.2 Partition Keys and Logical Partitions

A container's partition key is one of the most important design decisions. Cosmos DB uses the partition-key value to group items into logical partitions and to route requests. Good partition keys have high cardinality, even distribution, and alignment with common point-read and query filters. Poor partition keys create hot partitions, cross-partition queries, throttling, and expensive migrations.

For a forum container, `/forumId` may group all posts and comments for a small forum, but it can become hot if one public forum dominates traffic. `/tenantId` may work for business SaaS tenants with balanced activity. A synthetic key such as `/leaderboardBucket` or `/gameIdSeasonShard` may be necessary for gaming workloads where one game or region can receive massive writes.

Common Pitfall

Choosing a partition key because it is easy to explain. Choose it because it distributes writes, supports dominant reads, fits transaction boundaries, and avoids unbounded growth in a single logical partition.

3.2.3 Physical Partitions and Hot Keys

Cosmos DB manages physical partitions behind the scenes. As storage and throughput grow, the service splits data across physical partitions. Logical partition keys are still the engineer’s responsibility. If one key receives most writes, the service cannot magically distribute that single logical partition across many keys. Hot partitions show up as 429 throttling, uneven RU consumption, high latency for specific key values, and queries that require fan-out.

3.2.4 Global Distribution

Cosmos DB accounts can replicate data to multiple Azure regions. A single-write-region account sends writes to one region and replicates reads elsewhere. A multi-write account allows writes in more than one region and uses conflict resolution when concurrent updates target the same item [41]. Multi-region design reduces user latency and improves resilience, but it increases operational complexity and cost.

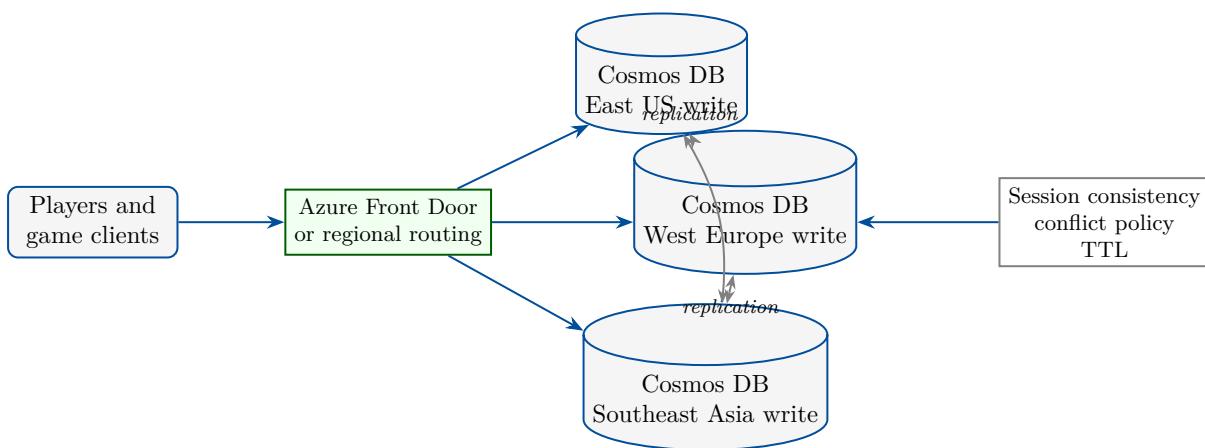


Figure 3.1: Cosmos DB global distribution places replicas near users while consistency and conflict policies define correctness during replication.

3.3 Tuesday: The Five Consistency Levels

Cosmos DB exposes five consistency levels: Strong, Bounded Staleness, Session, Consistent Prefix, and Eventual [30]. They form a practical spectrum between freshness, availability, latency, and cost. Strong consistency gives linearizable reads within supported regional configurations. Eventual consistency gives the weakest freshness guarantee but the lowest coordination burden.

Definition

A **consistency level** is the read contract an application receives after writes. It controls whether reads must observe the latest write, a bounded lag, a user’s own writes, a prefix-preserving order, or eventual convergence.

3.3.1 Strong and Bounded Staleness

Strong consistency returns the most recent committed version of an item, but it limits some global distribution choices because replicas must coordinate before reads complete. Use Strong when correctness requires immediate global visibility, such as inventory reservation for scarce resources. Do not choose it reflexively for personalization, telemetry, or leaderboards where slightly stale reads are acceptable.

Bounded Staleness allows reads to lag behind writes by a configured number of versions or time interval. It is useful when users can tolerate a known delay but the business wants a hard upper bound. It costs more coordination than Session or Eventual and should be justified by a measurable requirement.

3.3.2 Session, Consistent Prefix, and Eventual

Session consistency is the default for many application workloads because it provides read-your-writes within a client session while allowing global distribution. A player who submits a score should see that score immediately in their session even if other players in far regions see it after replication catches up. The SDK carries a session token to preserve that guarantee.

Consistent Prefix guarantees that reads never see writes out of order. If updates were A, B, and C, a client may see A or A,B, but not A,C without B. Eventual consistency only guarantees convergence when writes stop. It can be appropriate for derived counters, telemetry summaries, caches, and recommendation signals where temporary staleness is acceptable.



Figure 3.2: Cosmos DB consistency levels trade read freshness and ordering for lower coordination, latency, and regional independence.

Tip

Choose the weakest consistency level that satisfies the user story. For global applications, Session often gives excellent user experience because users see their own writes without forcing every region to agree before every read.

3.4 Wednesday: APIs and the Change Feed

3.4.1 Core (NoSQL), MongoDB, and Cassandra APIs

Cosmos DB offers several APIs over a managed distributed database service. The Core API, commonly called API for NoSQL, is the native JSON document model with SQL-like querying, automatic indexing, stored procedures, transactional batches within a partition key, SDK support, and Change Feed integration. It is the default choice when building new Azure-native document applications.

The MongoDB API supports applications and tools that speak MongoDB wire protocol. It is useful for migrations or teams with MongoDB models and drivers, but compatibility level, feature support, indexing behavior, and operational differences must be tested. The Cassandra API supports Cassandra Query Language patterns and wide-column workloads. It can help teams move selected Cassandra workloads to a managed Azure service, but partitioning and query design remain mandatory.

Note

API compatibility does not eliminate data-model review. A MongoDB or Cassandra application moved to Cosmos DB still needs throughput estimation, partition-key validation,

index review, latency testing, and failure-mode testing.

3.4.2 Change Feed as the Event Bridge

The Change Feed emits inserts and updates in logical partition order and lets processors resume from checkpoints [28]. It is a foundation for event-driven data engineering: write operational items once, then project them to Azure Functions, Event Hubs, ADLS Gen2, Synapse, Databricks, search indexes, caches, and materialized views. The Change Feed Processor library manages leases across workers so processing can scale horizontally.

Change Feed is not a universal audit log. Depending on mode and configuration, deletes may require soft-delete patterns or TTL awareness. Consumers should be idempotent because retries are normal. Downstream systems should store high-water marks or item versions so a processor can safely replay changes after failure.

Common Pitfall

Treating Change Feed consumers as exactly-once. Design them as at-least-once processors with idempotent writes, checkpoints, poison-message handling, and observability.

3.5 Thursday Hands-on Project: Forum Container with Python Writes and Multi-Region Writes

This lab creates a Cosmos DB for NoSQL account, a database, and a single container for a forum application. The schema stores posts, comments, reactions, and moderator events in one container so common forum-page reads can use the same partition key. Run it only in a sandbox subscription and delete the resource group when finished.

Note

The examples are written for Windows PowerShell with Azure CLI. Use globally unique account names, avoid committing keys, and prefer managed identity for production applications.

3.5.1 Single-Container Forum Schema

A forum page usually needs the forum metadata, recent threads, thread starter post, comments, reaction counts, and moderation state. A single-container design can store multiple item types with a shared partition key such as `/forumId`. Item IDs encode type and business key: `thread_1001`, `post_1001_0001`, `reaction_1001_user42`. The `type` field supports filtered queries and indexing. For very large public forums, introduce a synthetic partition key such as `forumId#bucket`.

```
1 az login
2 az account set --subscription "00000000-0000-0000-0000-000000000000"
3
4 $rg = "rg-de-azure-book-week3"
5 $primaryLocation = "eastus"
6 $secondaryLocation = "westus"
7 $account = "cosmos-week3-forum-001"
8 $database = "forumdb"
9 $container = "forumItems"
10 $partitionKey = "/forumId"
11
12 az group create '

```



```

13 --name $rg '
14 --location $primaryLocation
15
16 az cosmosdb create '
17 --name $account '
18 --resource-group $rg '
19 --locations regionName=$primaryLocation failoverPriority=0 isZoneRedundant=
    False '
20 --default-consistency-level Session '
21 --enable-automatic-failover true
22
23 az cosmosdb sql database create '
24 --account-name $account '
25 --resource-group $rg '
26 --name $database
27
28 az cosmosdb sql container create '
29 --account-name $account '
30 --resource-group $rg '
31 --database-name $database '
32 --name $container '
33 --partition-key-path $partitionKey '
34 --throughput 1000 '
35 --ttl 2592000

```

Listing 3.1: Creating a Cosmos DB for NoSQL account, database, and forum container.

Add a second write region only after the basic account is validated. Multi-region writes affect conflict handling, testing, and cost, so document why the application needs active-active writes instead of single-write failover.

```

1 az cosmosdb update '
2 --name $account '
3 --resource-group $rg '
4 --locations regionName=$primaryLocation failoverPriority=0 isZoneRedundant=
    False '
5         regionName=$secondaryLocation failoverPriority=1 isZoneRedundant=
    False
6
7 az cosmosdb update '
8 --name $account '
9 --resource-group $rg '
10 --enable-multiple-write-locations true
11
12 az cosmosdb show '
13 --name $account '
14 --resource-group $rg '
15 --query "{name:name,consistency:consistencyPolicy.defaultConsistencyLevel,
    writeLocations:writeLocations[].locationName,enableMultiWrite:
    enableMultipleWriteLocations}" '
16 --output table

```

Listing 3.2: Adding a second region and enabling multi-region writes for the lab account.

3.5.2 Inserting Forum Items with Python

Install `azure-cosmos` in a local virtual environment for the lab. Store the endpoint and key in environment variables or use managed identity in production. The script below writes several item types and prints RU charges so students can see how writes consume throughput.

```

1 import os
2 from datetime import datetime, timezone
3 from azure.cosmos import CosmosClient, PartitionKey

```

```

4
5 endpoint = os.environ["COSMOS_ENDPOINT"]
6 key = os.environ["COSMOS_KEY"]
7 database_name = os.environ.get("COSMOS_DATABASE", "forumdb")
8 container_name = os.environ.get("COSMOS_CONTAINER", "forumItems")
9
10 client = CosmosClient(endpoint, credential=key)
11 database = client.create_database_if_not_exists(database_name)
12 container = database.create_container_if_not_exists(
13     id=container_name,
14     partition_key=PartitionKey(path="/forumId"),
15     offer_throughput=1000,
16 )
17
18 now = datetime.now(timezone.utc).isoformat()
19 items = [
20     {
21         "id": "thread_1001",
22         "forumId": "azure-data-engineering",
23         "type": "thread",
24         "title": "How do RUs work?",
25         "authorId": "user_7",
26         "createdAt": now,
27         "lastActivityAt": now,
28     },
29     {
30         "id": "post_1001_0001",
31         "forumId": "azure-data-engineering",
32         "threadId": "thread_1001",
33         "type": "post",
34         "authorId": "user_7",
35         "body": "A point read is cheap; fan-out queries are not.",
36         "createdAt": now,
37     },
38     {
39         "id": "reaction_1001_user_42",
40         "forumId": "azure-data-engineering",
41         "threadId": "thread_1001",
42         "postId": "post_1001_0001",
43         "type": "reaction",
44         "userId": "user_42",
45         "reaction": "upvote",
46         "createdAt": now,
47     },
48 ]
49
50 for item in items:
51     response = container.upsert_item(item)
52     charge = container.client_connection.last_response_headers.get(
53         "x-ms-request-charge"
54     )
55     print(response["id"], "RU charge", charge)

```

Listing 3.3: Writing forum items with the azure-cosmos SDK and recording RU charges.

Query by partition key for predictable RU usage. Avoid broad scans in production APIs; use Change Feed or analytical replicas for downstream processing.

```

1 query = """
2 SELECT c.id, c.type, c.title, c.body, c.createdAt
3 FROM c
4 WHERE c.forumId = @forumId AND c.threadId = @threadId
5 ORDER BY c.createdAt
6 """

```

```

7 parameters = [
8     {"name": "@forumId", "value": "azure-data-engineering"},
9     {"name": "@threadId", "value": "thread_1001"},
10 ]
11
12 for item in container.query_items(
13     query=query,
14     parameters=parameters,
15     partition_key="azure-data-engineering",
16 ):
17     print(item)

```

Listing 3.4: Reading a forum thread inside one logical partition.

3.6 Friday Use Case: Global Gaming Leaderboard at 1,000,000 Reads/Writes per Second

3.6.1 Scenario

A global game studio runs a competitive leaderboard for tens of millions of players. During tournaments, the service must handle 1,000,000 combined reads and writes per second globally. Players submit scores from North America, Europe, and Asia. They expect to see their own score immediately after submission, but a leaderboard visible to all players may be seconds behind. The design uses Cosmos DB with multi-region writes and Session consistency.

3.6.2 Partitioning and Serving Strategy

The main container stores score submissions and current leaderboard entries. The partition key uses a synthetic value such as `/boardShard: gameId#seasonId#region#bucket`. Buckets distribute writes for a hot tournament. A Change Feed processor maintains materialized top-N documents per game, season, region, and bucket. API reads point to precomputed leaderboard documents instead of running expensive global aggregations on every request.

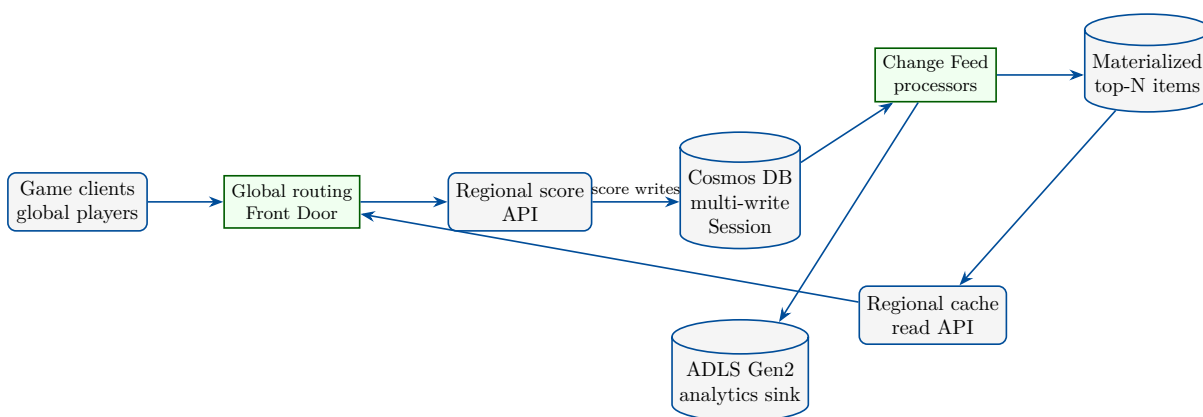


Figure 3.3: Global leaderboard serving architecture using Cosmos DB multi-region writes, Session consistency, Change Feed projections, and regional read paths.

3.6.3 Correctness and Cost Controls

Session consistency ensures a player sees their own submitted score when the SDK preserves the session token. Global top-N displays can be eventually updated through Change Feed

projections. Conflict resolution should be deterministic: for example, keep the highest score for a player and use earliest timestamp as a tie breaker. The API should make score submissions idempotent by using item IDs derived from player, match, and attempt.

Cost control starts with point reads and materialized views. The system should avoid cross-partition aggregation in the hot read path. Autoscale or provisioned throughput must be modeled per region and per container. Observability should track RU consumption, 429 throttling, p99 latency, conflict counts, Change Feed lag, item size, index utilization, and regional availability.

3.7 Architectural Decision Record Template

A Week 3 ADR should capture the access patterns, container strategy, partition key, consistency level, API choice, indexing policy, throughput model, multi-region write decision, conflict policy, TTL, Change Feed consumers, retry rules, and cost guardrails. It should also list rejected alternatives such as Azure SQL, Redis-only storage, Event Hubs plus lake-only storage, DynamoDB, Firestore, MongoDB Atlas, and self-managed Cassandra.

3.8 Operational Deep Dive: Performance, Identity, and Observability

Cosmos DB performance incidents usually come from predictable causes: hot partition keys, cross-partition scans, large items, unbounded arrays, excessive indexing, missing composite indexes, overly strong consistency, retry storms, or Change Feed processors falling behind. Scaling RU/s can help, but it will not fix a partition key that sends all writes to one logical partition.

Identity should prefer Microsoft Entra ID and managed identities where supported by the chosen SDK and API. Production accounts should use private endpoints, role-based access, diagnostic settings, customer-managed keys where required, and alerts on throttling and regional failover. Keys should be rotated and kept out of notebooks, source code, logs, and screenshots.

Common Pitfall

Using Cosmos DB as a reporting warehouse. Large scans and ad hoc aggregations are expensive in an operational NoSQL serving store. Use Change Feed, analytical store, Synapse Link, or lakehouse patterns for broad analytics.

3.9 Troubleshooting Patterns

A Cosmos DB troubleshooting workflow starts with the failing access pattern. For point reads, confirm item ID, partition key, region, consistency, and retry behavior. For writes, check item size, index policy, partition hot spots, conflict rates, and 429 throttling. For queries, inspect whether the query is scoped to one partition, whether it uses an index, whether it requires a composite index, and how many RUs it consumes. For Change Feed, verify leases, checkpoint progress, poison records, and downstream idempotency.

Do not hide 429 responses by adding unbounded retries. Cosmos DB SDKs already include retry policies. Applications should apply backpressure, monitor request charge, and isolate noisy tenants or hot boards before raising throughput blindly.

3.10 Week 3 Lab Rubric

A complete Week 3 submission includes Azure CLI deployment evidence, a documented forum single-container schema, Python insert and query output with RU observations, evidence of multi-region write configuration, and an ADR explaining partitioning, consistency, and Change Feed decisions.

Criterion	Meets expectation	Needs improvement
Partitioning	Partition key matches access patterns and distributes load	Key is low-cardinality or creates hot partitions
Consistency	Five levels are compared and Session is justified for user experience	Strong or Eventual is chosen without correctness analysis
Hands-on	CLI creates account, database, container, and multi-region write setting; Python writes items	Portal-only steps or no RU evidence
Change Feed	Consumers are idempotent and checkpointed with lag monitoring	Feed is treated as exactly-once or audit-complete by default
Global design	Regional writes, conflict policy, routing, and failover behavior are documented	Multi-region writes are enabled without conflict or cost analysis

Table 3.1: Week 3 rubric for Cosmos DB architecture, hands-on implementation, and global serving design.

3.11 Interview Q&A

3.11.1 Concepts and Trade-offs

1. **Q: What is a Request Unit?**

A: A Request Unit is Cosmos DB's normalized measure of CPU, memory, and IO used by reads, writes, queries, and maintenance operations.

2. **Q: What makes a good partition key?**

A: High cardinality, even distribution, alignment with common point reads and queries, and support for transaction boundaries within a logical partition.

3. **Q: Why is Session consistency common for global apps?**

A: It provides read-your-writes for a user session while allowing lower latency and more regional independence than Strong consistency.

4. **Q: When would you avoid multi-region writes?**

A: Avoid them when the workload cannot tolerate conflict resolution complexity or when single-write failover meets latency and availability goals.

5. **Q: Why use Change Feed for data engineering?**

A: It turns operational item changes into scalable downstream projections for functions, streams, lakes, caches, search, and materialized views.

6. **Q: What is the biggest risk in a high-scale leaderboard?**

A: A hot partition or global aggregation path can throttle the system even if the total account RU/s appears high.

3.12 Further Reading

Start with the Azure Cosmos DB overview [69], Request Unit guidance [63], partitioning guidance [59], consistency levels [30], global distribution [41], and multi-region write concepts [29]. Review the Change Feed documentation [28], API for NoSQL quickstart [62], Python SDK documentation [17], and the Azure Well-Architected Framework [52] for reliability, security, performance, operations, and cost review.

Key Takeaways

- Cosmos DB scales by partitioned item access, not by hiding poor data modeling behind more throughput.
- RUs make performance and cost visible; every access pattern should be measured by request charge and latency.
- Partition-key choice determines distribution, transaction scope, query efficiency, and hot-key risk.
- The five consistency levels are business correctness tools, not trivia; Session is often the best global user-experience compromise.
- Change Feed is the bridge from operational NoSQL writes to analytics, search, cache, and materialized-serving projections.
- Multi-region writes require deterministic conflict rules, idempotent clients, observability, and explicit cost justification.

3.13 Exercises

1. **(Conceptual)** Explain RUs to a product manager and show why a cross-partition query can cost more than a point read.
2. **(Conceptual)** Compare Strong, Bounded Staleness, Session, Consistent Prefix, and Eventual consistency for a shopping-cart application.
3. **(Conceptual)** Choose a partition key for a forum with thousands of small private communities and one massive public community.
4. **(Hands-on)** Run the Cosmos DB account and container creation commands in a sandbox and capture the consistency and region output.
5. **(Hands-on)** Modify the Python script to insert comments, reactions, and moderation events, then report RU charges for each item type.
6. **(Hands-on)** Add a query that reads one thread by partition key and compare its RU charge with a cross-partition query.
7. **(Design)** Design a Change Feed consumer that writes forum events to ADLS Gen2 with idempotent replay and checkpoint evidence.
8. **(Design)** Write an ADR for the gaming leaderboard, including partition key, consistency, conflict policy, multi-region writes, and cost controls.

3.14 Capstone Project: Global Gaming Leaderboard on Cosmos DB

Capstone Project

Mission: Design and prototype a global gaming leaderboard on Azure Cosmos DB that uses Session consistency, multi-region writes, synthetic partition keys, Change Feed projections, and operational evidence suitable for a launch review.

Estimated time: 5–7 hours.

What you will practice:

- Modeling a high-scale NoSQL serving workload around point reads, writes, and materialized top-N views.
- Selecting Session consistency and explaining the user-experience guarantee it provides.
- Designing partition keys and buckets to avoid hot logical partitions during tournaments.
- Enabling multi-region writes with deterministic conflict handling and idempotent score submission.
- Using Change Feed to maintain leaderboard projections and analytics exports.

Scenario

A game studio is launching a worldwide ranked tournament. Players submit match scores from the Americas, Europe, and Asia. The public leaderboard must update quickly, but it may lag by a few seconds. Each player must immediately see their own submitted score. The platform must support regional write paths, survive a regional outage, and export score events to the analytics lake.

Requirements

- **Database:** Use Azure Cosmos DB for NoSQL with Session consistency and multi-region writes enabled.
- **Partitioning:** Use a synthetic partition key such as `/boardShard` combining game, season, region, and bucket to distribute writes.
- **Data model:** Store score submissions, current player-best records, leaderboard projection records, and processor checkpoints with clear item types.
- **Conflict handling:** Keep the highest score per player and use earliest timestamp as a deterministic tie breaker.
- **Serving:** Read public leaderboards from materialized top-N records rather than scanning raw score submissions.
- **Change Feed:** Maintain projections and export immutable score events to ADLS Gen2 or a lakehouse landing zone.
- **Operations:** Monitor RU consumption, 429 throttling, p95 and p99 latency, conflict count, regional health, Change Feed lag, and cost.

Reference Architecture

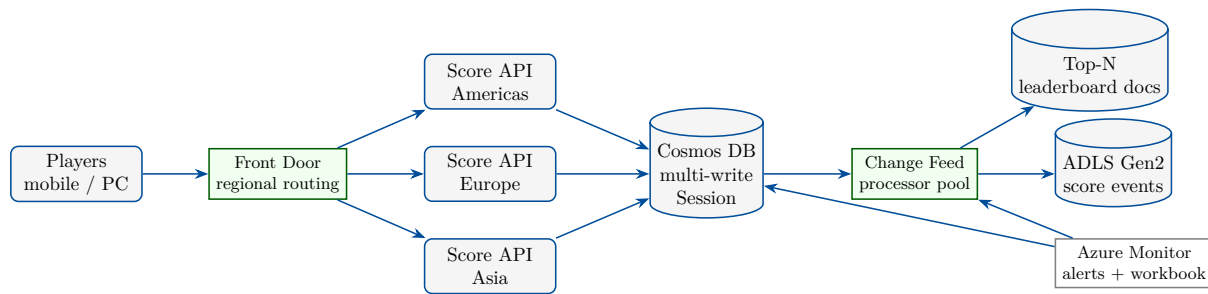


Figure 3.4: Capstone reference architecture for a global leaderboard using regional score APIs, Cosmos DB multi-region writes, Change Feed projections, lake export, and monitoring evidence.

Implementation Steps

Use a sandbox subscription. The snippets below demonstrate the structure; production systems should use infrastructure-as-code, private endpoints, managed identity, and formal capacity tests.

1. **Create a multi-region Cosmos DB account and containers.** Use one container for score items and one container for Change Feed leases if you build a processor.

```

1 $rg = "rg-cosmos-leaderboard-capstone"
2 $primaryLocation = "eastus"
3 $secondaryLocation = "westeurope"
4 $thirdLocation = "southeastasia"
5 $account = "cosmos-leaderboard-capstone-001"
6 $database = "leaderboarddb"
7 $scores = "scores"
8 $leases = "leases"
9
10 az group create --name $rg --location $primaryLocation
11
12 az cosmosdb create '
13   --name $account '
14   --resource-group $rg '
15   --locations regionName=$primaryLocation failoverPriority=0
16     isZoneRedundant=False '
17     regionName=$secondaryLocation failoverPriority=1
18     isZoneRedundant=False '
19     regionName=$thirdLocation failoverPriority=2 isZoneRedundant=
20     False '
21   --default-consistency-level Session '
22   --enable-multiple-write-locations true '
23   --enable-automatic-failover true
24
25 az cosmosdb sql database create '
26   --account-name $account '
27   --resource-group $rg '
28   --name $database
29
30 az cosmosdb sql container create '
31   --account-name $account '
32   --resource-group $rg '
33   --database-name $database '
34   --name $scores '
35   --partition-key-path "/boardShard" '

```



```

33  --throughput 4000
34
35  az cosmosdb sql container create '
36  --account-name $account '
37  --resource-group $rg '
38  --database-name $database '
39  --name $leases '
40  --partition-key-path "/id" '
41  --throughput 400

```

Listing 3.5: PowerShell plus Azure CLI skeleton for the capstone Cosmos DB account.

2. **Write idempotent score submissions.** The item ID combines player, match, and attempt so retries do not create duplicate submissions.

```

1  import os
2  from datetime import datetime, timezone
3  from azure.cosmos import CosmosClient, PartitionKey
4
5  client = CosmosClient(os.environ["COSMOS_ENDPOINT"], os.environ["COSMOS_KEY"]
6  db = client.create_database_if_not_exists("leaderboarddb")
7  scores = db.create_container_if_not_exists(
8      id="scores",
9      partition_key=PartitionKey(path="/boardShard"),
10     offer_throughput=4000,
11 )
12
13 def board_shard(game_id, season_id, region, player_id, buckets=64):
14     bucket = abs(hash(player_id)) % buckets
15     return f"{game_id}#{season_id}#{region}#{bucket:02d}"
16
17 def submit_score(game_id, season_id, region, player_id, match_id, attempt,
18 value):
19     timestamp = datetime.now(timezone.utc).isoformat()
20     item = {
21         "id": f"score_{player_id}_{match_id}_{attempt}",
22         "type": "scoreSubmission",
23         "gameId": game_id,
24         "seasonId": season_id,
25         "region": region,
26         "playerId": player_id,
27         "matchId": match_id,
28         "attempt": attempt,
29         "score": value,
30         "submittedAt": timestamp,
31         "boardShard": board_shard(game_id, season_id, region, player_id),
32     }
33     return scores.upsert_item(item)
34
35 for i in range(10):
36     saved = submit_score("racer", "2026-summer", "americas", f"player_{i}",
37         "m77", 1, 1000 + i)
38     print(saved["id"], saved["boardShard"])

```

Listing 3.6: Submitting leaderboard scores with deterministic IDs and synthetic partition keys.

3. **Build the projection worker.** Read changes, compute player-best and bucket top-N records, and write projection items with idempotent upserts. Store checkpoints through the Change Feed Processor or an equivalent lease mechanism.
4. **Validate consistency and conflict behavior.** Submit a score, read it back with the same SDK session, then test concurrent submissions for the same player and confirm the

deterministic winner.

5. **Create operational evidence.** Capture RU charge per operation, 429 count, p99 latency, regional write status, Change Feed lag, conflict count, and a cost estimate for normal and tournament traffic.

Deliverables

- Architecture diagram showing clients, routing, regional APIs, Cosmos DB regions, Change Feed processors, top-N projection items, lake export, and monitoring.
- ADR explaining why Cosmos DB for NoSQL, Session consistency, multi-region writes, and the synthetic partition key were selected.
- Reproducible Azure CLI or infrastructure-as-code snippets for account, database, containers, regions, consistency, throughput, and diagnostic settings.
- Python scripts or pseudocode for idempotent score submission, partition-key generation, and projection updates.
- Evidence pack with RU measurements, load-test assumptions, throttling behavior, conflict tests, Change Feed lag, and failover observations.

Acceptance Criteria

- The design explains why Session consistency satisfies player read-your-writes while allowing global leaderboard projections to lag briefly.
- Partition keys distribute tournament writes across buckets and avoid a single hot logical partition.
- Public leaderboard reads use materialized top-N items rather than cross-partition scans of raw score submissions.
- Score submissions are idempotent and conflict resolution is deterministic.
- Multi-region write behavior, failover assumptions, RPO, and regional routing are documented.
- Change Feed processing is idempotent, checkpointed, monitored for lag, and safe to replay.
- Security design avoids committed keys and addresses private access, managed identity or least-privilege roles, and diagnostic logging.
- Operational evidence is complete enough for another engineer to reproduce the lab with different names and regions.

Stretch Goals

- Add a synthetic load script that estimates RU/s for 1,000,000 combined reads and writes per second across regions.
- Implement custom conflict resolution logic or an application-level merge function for player-best records.
- Add Azure Functions or a containerized worker that processes Change Feed leases and writes top-N projection documents.
- Export immutable score submissions to ADLS Gen2 in partitioned folders for downstream analytics.

- Create an Azure Monitor workbook that displays RU charge, 429s, latency percentiles, region health, conflict counts, and Change Feed lag.
- Estimate cost for provisioned throughput, autoscale, and serverless variants under normal day and tournament traffic.

Hybrid Ingestion and Caching

Learning Objectives

- Explain why low-latency data products need caching, admission control, TTLs, and clear invalidation rules.
- Use Azure Cache for Redis to accelerate Azure SQL Database reads with a cache-aside pattern.
- Compare Redis-based caching with Cosmos DB Integrated Cache for read-heavy document workloads.
- Position Azure API Management as an ingestion gateway for data APIs, mobile applications, partners, and event producers.
- Deploy an Azure Cache for Redis instance and write a Python cache-aside script over Azure SQL Database query results.
- Design a high-throughput, low-latency mobile data API that survives extreme traffic spikes during live sports events.
- Complete the Part I milestone by integrating Azure SQL Database Serverless, Cosmos DB, ADLS Gen2, Redis, APIM, and Change Feed.

Cross-Cloud Equivalence

Caching is a portable architecture pattern even when product names differ. The engineer must define what is cached, where the cache lives, how entries expire, how writes update or invalidate entries, and what happens when the cache is unavailable.

Concept	Azure	AWS	GCP	Other platforms
Managed Redis	Azure Cache for Redis with Enterprise tiers for advanced Redis features	Amazon ElastiCache for Redis or MemoryDB for Redis	Memorystore for Redis Cluster or Memorystore for Redis	Redis Enterprise Cloud, self-managed Redis, KeyDB
Database-integrated cache	Cosmos DB Integrated Cache on dedicated gateway	DynamoDB Accelerator (DAX) for DynamoDB	No exact single-service equivalent; combine Memorystore with Firestore or Spanner patterns	MongoDB Atlas tiered storage plus application caches; Cassandra row cache
Cache-aside	Application reads Redis first, then Azure SQL, Cosmos DB, or API source on miss	Application reads ElastiCache first, then RDS, DynamoDB, or service source	Application reads Memorystore first, then Cloud SQL, Firestore, or Spanner	Common pattern across Redis, Memcached, Hazelcast, and application memory
Write-through / write-behind	Application or service updates cache with the write path; less common for Azure PaaS databases than cache-aside	ElastiCache libraries or custom write-through paths	Custom service layer with Memorystore and database writes	Product-specific data grids and stream processors
TTL and eviction	Redis TTL, maxmemory policies, and Cosmos DB Integrated Cache staleness window	Redis TTL, eviction policies, DAX item cache TTL	Memorystore TTL and eviction policies	Memcached expiration, Redis-compatible TTL, local LRU caches

A cloud-neutral cache design documents freshness, ownership, invalidation, fallback, warm-up, and observability. Product choice matters, but these contracts matter more.

Key Terms

Cache hit: a request served from cache without contacting the origin database. **Cache miss:** a request that falls through to the origin and may populate the cache. **TTL:** time-to-live, the maximum age of a cached entry. **Eviction:** removal of cached entries due to memory pressure or policy. **Ingestion gateway:** an API boundary that validates, throttles, authenticates, and routes incoming data.

4.1 Week 4 Overview: Why Caching and Hybrid Ingestion Close Part I

Part I has introduced storage foundations, relational databases, and document-scale NoSQL. Week 4 adds the serving patterns that make those systems usable under real product traffic: cache hot reads, shield databases from spikes, and place a governed API gateway in front of ingestion paths. Azure Cache for Redis provides managed in-memory acceleration for databases and services [74]. Cosmos DB Integrated Cache reduces RU consumption for read-heavy API for NoSQL workloads that use the dedicated gateway [47]. Azure API Management provides the policy and gateway layer for data APIs [73].

Caching is not only a performance trick. It is an architectural contract. A cache introduces a second copy of data, so the team must explain freshness, invalidation, fallback, and observability. Done well, caching reduces database load and user-visible latency. Done poorly, it hides stale data, creates retry storms, and makes incidents harder to reason about.

Definition

Hybrid ingestion combines synchronous API ingestion, operational databases, caches, and asynchronous downstream flows. It is common when mobile apps, partner systems, and web backends need low-latency writes while analytics systems need durable event history.

Note

This chapter closes Milestone 1. The final capstone combines the storage, relational, NoSQL, and caching concepts from Weeks 1–4 into an e-commerce data backend.

4.2 Monday: Azure Cache for Redis for Database Acceleration

4.2.1 What Redis Adds to a Data Platform

Azure Cache for Redis is a managed, in-memory key-value store based on Redis. Data engineers use it for hot database query results, session state, rate-limit counters, distributed locks, pub/sub notifications, feature flags, and small materialized views. For database acceleration, the usual goal is simple: keep the most frequently requested objects close to the application so Azure SQL Database or another origin can focus on writes, uncommon reads, and durable transactions.

Redis is fast because it keeps data in memory and uses simple data structures. That speed does not remove the need for durable storage. Treat Redis as an acceleration layer unless the tier and architecture explicitly provide the persistence and availability characteristics your workload requires.

Definition

Cache-aside is a pattern where the application looks in cache first. On a miss, it reads from the origin database, stores the result in cache with a TTL, and returns the value to the caller.

4.2.2 Cache-Aside Read Path

The cache-aside pattern is the default starting point for Azure SQL acceleration because it keeps the origin database authoritative. The application owns cache keys and serialization. The cache can be flushed without data loss because Azure SQL remains the system of record.

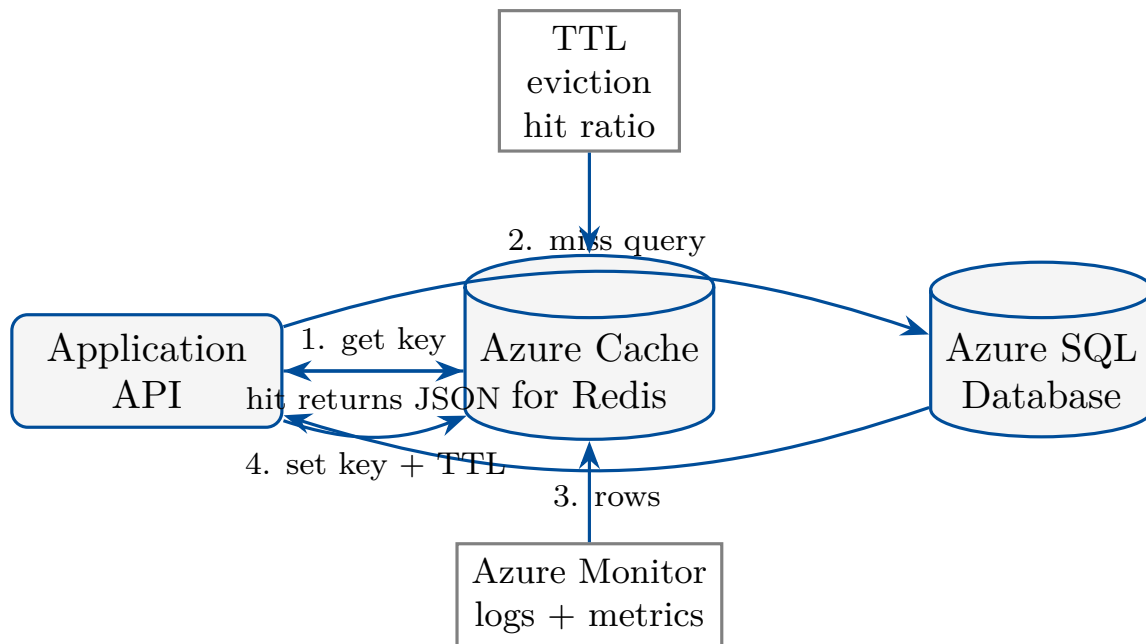


Figure 4.1: Cache-aside between an application, Azure Cache for Redis, and Azure SQL Database keeps Azure SQL authoritative while Redis absorbs hot reads.

4.2.3 Keys, TTLs, and Invalidation

Good cache keys are stable, specific, and versionable. A profile query might use `profile:v1:user:42`. A product-card query might use `product-card:v3:sku:ABC123`. Version prefixes let teams invalidate a family of entries by changing the key format during deployment. TTLs bound staleness and prevent old entries from living forever.

Invalidation is the difficult part. For read-mostly reference data, a five-minute TTL may be enough. For product prices, billing status, inventory reservations, and permissions, the write path should delete or update affected cache keys immediately. If the application cannot name every affected key, use short TTLs and design user experiences that tolerate brief staleness.

Common Pitfall

Caching data without a freshness contract. A cache entry with no TTL and no invalidation rule is a future incident. Every cached object needs an owner, expiration, and fallback behavior.

4.2.4 Availability and Failure Modes

A database acceleration cache must fail safely. If Redis is unavailable, the application should usually fall back to Azure SQL for critical reads, apply backpressure to avoid overwhelming the database, and record degraded-mode metrics. Do not allow thousands of application instances to stampede the database on the same cache miss. Use request coalescing, jittered TTLs, and sensible retries.

Tip

Track cache hit ratio, server load, memory pressure, evicted keys, connected clients, command latency, errors, and database query rate together. A high hit ratio that serves stale or wrong data is not success.

4.3 Tuesday: Cosmos DB Integrated Cache

4.3.1 Why an Integrated Cache Exists

Cosmos DB already provides low-latency reads, but every read consumes RUs unless served by the Integrated Cache through the dedicated gateway. Integrated Cache stores point reads and query results in a gateway-side cache. Repeated reads can return without consuming RUs, subject to the configured maximum integrated cache staleness [47].

This is different from Redis cache-aside. The application does not manually set keys for Integrated Cache. Instead, it sends supported requests through the dedicated gateway and accepts a staleness window. The operational model is closer to a database-integrated read cache than to an application-owned key-value cache.

Definition

Maximum integrated cache staleness defines how old a cached Cosmos DB response may be before the gateway must refresh it from the backend partitions.

4.3.2 When Integrated Cache Fits

Integrated Cache fits read-heavy document workloads where the same point reads or queries repeat often and slight staleness is acceptable. Examples include catalog pages, product recommendations, configuration documents, reference metadata, and public leaderboard projections. It is less useful for write-heavy workloads, unique one-off queries, cross-partition scans with low repetition, or correctness paths that require the newest committed item.

The cache can reduce RU cost and latency, but it does not fix bad data modeling. A query that fans out across partitions may still be an expensive origin operation when the cache misses. Partition keys, item size, index policy, and materialized views remain central to design.

Note

Use Redis when the application needs explicit keys, counters, sorted sets, rate limiting, pub/sub, or cross-origin caching. Use Cosmos DB Integrated Cache when repeated Cosmos DB reads can accept bounded staleness and the dedicated gateway model fits the application.

4.3.3 Consistency and Freshness Trade-offs

Integrated Cache is a deliberate freshness trade. If a user updates a shopping cart and immediately reads through a stale cache path, the application might show the previous state until the staleness window expires. Some workloads can tolerate this; others cannot. Critical user-owned writes should be read with session-aware direct reads or cache bypass where needed.

Common Pitfall

Assuming Integrated Cache has the same semantics as Session consistency. The cache has a staleness setting. Cosmos DB consistency still matters, but the cached response may intentionally be older than the newest backend value.

4.4 Wednesday: Azure API Management for Data Ingestion

4.4.1 APIM as the Data API Front Door

Azure API Management is a managed gateway for publishing, securing, transforming, throttling, and observing APIs. In data engineering, APIM often sits in front of ingestion endpoints used by mobile apps, partner systems, internal services, IoT gateways, and batch jobs. It provides a controlled boundary before traffic reaches Azure Functions, App Service, Container Apps, Event Hubs, Azure SQL, Cosmos DB, or custom services.

The gateway should not become a hidden monolith. Keep APIM responsible for boundary concerns: authentication, authorization, subscription keys, quotas, rate limits, schema validation, request transformation, routing, response shaping, and logging. Keep business validation and durable writes in backend services where they can be tested and versioned.

Definition

Ingestion gateway is the controlled API boundary that receives external or cross-team data, enforces policy, and routes validated requests to backend ingestion services.

4.4.2 Policies for Data Ingestion

APIM policies can validate JWTs, check subscription keys, limit call rates, reject oversized payloads, require headers, transform JSON, route versions, cache selected responses, and emit diagnostics. For ingestion, the most important policies are often admission-control policies. A backend cannot maintain data quality if a gateway accepts unlimited malformed requests during a traffic spike.

Typical APIM controls include per-consumer quotas, burst rate limits, request size limits, schema validation, correlation IDs, and backend timeout policies. For write APIs, avoid response caching unless the semantics are explicitly idempotent. For read APIs, APIM response caching may complement Redis or Integrated Cache, but it must follow the same freshness contract.

Tip

Use APIM to protect the backend before scaling the backend. Rejecting invalid or abusive traffic at the edge is cheaper and safer than letting it consume database RUs, DTUs, vCores, or queue capacity.

4.4.3 Synchronous and Asynchronous Ingestion

Hybrid ingestion often uses APIM for synchronous request acceptance and Event Hubs, queues, or Change Feed for asynchronous processing. A mobile app may POST a clickstream event through APIM, receive a quick accepted response, and let a backend write to Cosmos DB or Event Hubs. A partner may submit billing adjustments that require synchronous validation against Azure SQL. The correct pattern depends on user feedback, correctness, ordering, and replay requirements.

Note

APIM is not a message broker. If the system needs durable buffering, replay, or consumer groups, route accepted requests to Event Hubs, Service Bus, Storage queues, Cosmos DB, or another durable ingestion store.

4.5 Thursday Hands-on Project: Redis Cache-Aside for Azure SQL Query Results

This lab deploys Azure Cache for Redis and uses Python to cache Azure SQL Database query results. Run it only in a sandbox subscription. Delete the resource group when finished, and never commit database passwords, connection strings, or Redis access keys.

Note

The examples are written for Windows PowerShell with Azure CLI. Production systems should use managed identity for Azure SQL where possible, private endpoints, Key Vault, and infrastructure-as-code.

4.5.1 Deploying Azure Cache for Redis

The snippet below creates a resource group and a Basic C0 Redis instance for learning. Basic is not appropriate for production availability, but it keeps the lab small. Use Standard, Premium, or Enterprise tiers for real workloads that require replication, clustering, persistence, private networking, or stronger availability characteristics.

```

1 az login
2 az account set --subscription "00000000-0000-0000-0000-000000000000"
3 $rg = "rg-de-azure-book-week4"
4 $location = "eastus"
5 $redis = "redis-week4-sqlcache-001"
6
7 az group create --name $rg --location $location
8
9 az redis create '
10 --name $redis --resource-group $rg --location $location '
11 --sku Basic --vm-size c0 '
12 --enable-non-ssl-port false
13
14 az redis show '
15 --name $redis --resource-group $rg '
16 --query "{name:name,hostName:hostName,sku:sku.name,sslPort:sslPort}" '
17 --output table
18
19 az redis list-keys '
20 --name $redis --resource-group $rg --query primaryKey --output tsv

```

Listing 4.1: Creating an Azure Cache for Redis instance for the Week 4 lab.

4.5.2 Caching Azure SQL Results with Python

Install `redis` and `pyodbc` in a local virtual environment. The script reads a user profile from Azure SQL Database, serializes the result as JSON, stores it in Redis with a TTL, and prints whether the response was a cache hit or miss.

```

1 import json
2 import os
3 import time
4
5 import pyodbc
6 import redis
7
8 CACHE_TTL_SECONDS = int(os.environ.get("CACHE_TTL_SECONDS", "300"))
9
10 cache = redis.Redis(
11     host=os.environ["REDIS_HOST"],

```

```

12     port=6380,
13     password=os.environ["REDIS_KEY"],
14     ssl=True,
15     decode_responses=True,
16 )
17
18 def cache_key(user_id: int) -> str:
19     return f"profile:v1:user:{user_id}"
20
21 def fetch_profile_from_sql(user_id: int) -> dict | None:
22     query = """
23     SELECT UserId, DisplayName, LoyaltyTier, UpdatedAt
24     FROM dbo.UserProfiles
25     WHERE UserId = ?
26     """
27     with pyodbc.connect(os.environ["SQL_CONNECTION_STRING"], timeout=5) as
connection:
28         row = connection.execute(query, user_id).fetchone()
29         if row is None:
30             return None
31         return {
32             "userId": row.UserId,
33             "displayName": row.DisplayName,
34             "loyaltyTier": row.LoyaltyTier,
35             "updatedAt": row.UpdatedAt.isoformat(),
36         }
37
38 def get_profile(user_id: int) -> dict | None:
39     key = cache_key(user_id)
40     cached = cache.get(key)
41     if cached:
42         print("cache hit", key)
43         return json.loads(cached)
44
45     print("cache miss", key)
46     profile = fetch_profile_from_sql(user_id)
47     if profile is not None:
48         cache.setex(key, CACHE_TTL_SECONDS, json.dumps(profile))
49     return profile
50
51 if __name__ == "__main__":
52     start = time.perf_counter()
53     result = get_profile(int(os.environ.get("USER_ID", "42")))
54     elapsed_ms = (time.perf_counter() - start) * 1000
55     print(json.dumps(result, indent=2))
56     print(f"elapsed_ms={elapsed_ms:.1f}")

```

Listing 4.2: Cache-aside Python script using redis-py and Azure SQL query results.

4.5.3 Measuring the Load Reduction

Set REDIS_HOST, REDIS_KEY, SQL_CONNECTION_STRING, and USER_ID in PowerShell, install redis and pyodbc, then run the script twice. The first run should miss the cache and query Azure SQL. The second run should hit Redis until the TTL expires. Capture Azure SQL query duration, Redis latency, cache hit ratio, and database CPU before and after repeated reads.

4.6 Friday Use Case: Mobile Sports API with Extreme Traffic Spikes

4.6.1 Scenario

A sports media company runs a mobile app for live games. Traffic is normal during the week, then spikes by 50 times when a championship game starts, when a star player scores, or when fantasy-league results update. Users expect low-latency reads for scores, rosters, schedules, profile settings, and personalized offers. The backend must protect Azure SQL and Cosmos DB from sudden read amplification.

4.6.2 Serving Architecture

APIM sits at the public API boundary. Redis caches hot scoreboards, team pages, user profile summaries, and rate-limit counters. Cosmos DB stores high-volume event and personalization documents, with Integrated Cache for repeated public document reads where staleness is acceptable. Azure SQL stores billing, subscriptions, and relational user-profile records. The write path invalidates or refreshes affected cache entries after authoritative updates.

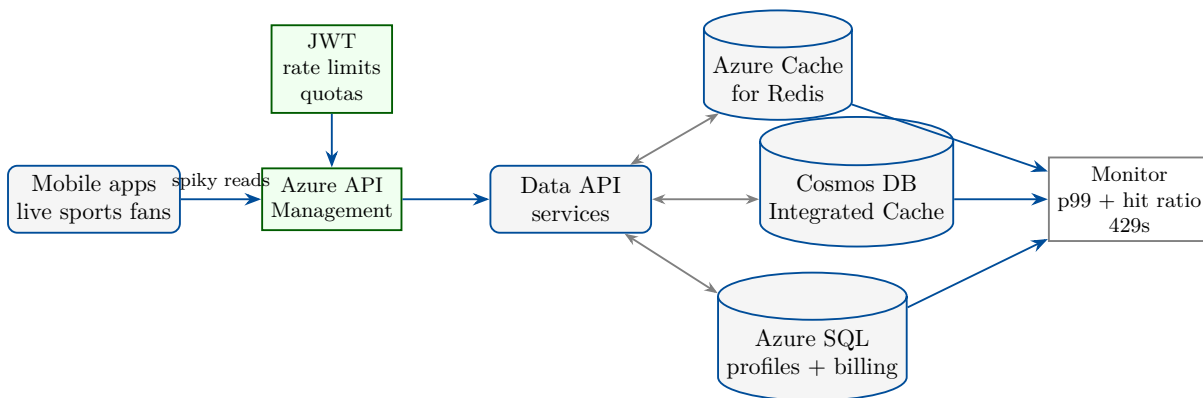


Figure 4.2: Spiky-traffic mobile data API using APIM policy controls, Redis hot-path acceleration, Cosmos DB Integrated Cache, Azure SQL, and shared observability.

4.6.3 Design Decisions

The live scoreboard read path should be cache-first with very short TTLs, often one to five seconds, because users value low latency and tolerate tiny delays. User billing status should have stricter invalidation because access control and payments require correctness. Personalized offers may use Redis or Cosmos DB Integrated Cache with a longer TTL if stale recommendations are acceptable.

APIM rate limits should distinguish anonymous public reads, authenticated users, partners, and internal services. The backend should degrade gracefully: serve slightly stale public scoreboards, disable nonessential personalization, and preserve billing and identity correctness. During the spike, observability should show p95 and p99 latency, cache hit ratio, Redis server load, Cosmos DB RU consumption, Azure SQL CPU, APIM rejected requests, and backend error rates.

4.7 Architectural Decision Record Template

A Week 4 ADR should capture each cached object, source of truth, key format, TTL, invalidation trigger, stale-read tolerance, fallback behavior, ownership, and metrics. It should also document APIM policies, consumer tiers, rate limits, authentication, backend routing, payload limits, idempotency, and rejected alternatives such as database-only reads, CDN-only caching, direct partner database access, or queue-only ingestion.

4.8 Operational Deep Dive: Stampedes, Security, and Observability

Cache incidents often come from stampedes, hot keys, overlarge values, missing TTLs, network isolation gaps, secret leakage, and retry storms. A stampede occurs when many callers miss the same key and all query the origin. Prevent it with jittered TTLs, request coalescing, background refresh, small stale-while-revalidate windows where acceptable, and backend concurrency limits.

Security follows the same principles as earlier chapters. Do not commit Redis keys, SQL passwords, connection strings, or APIM subscription keys. Prefer managed identity for backend services, Key Vault for secrets, private endpoints for production, TLS-only connections, least-privilege access, and diagnostic settings that do not log sensitive payloads.

Common Pitfall

Letting cache success hide origin weakness. If the origin database cannot survive a cold start, regional failover, or cache flush, the architecture needs backpressure and capacity planning before production launch.

4.9 Troubleshooting Patterns

Start with the user symptom: stale data, high latency, wrong data, database overload, or gateway rejection. For stale data, inspect TTL, invalidation, key version, and whether the request is using Redis, Integrated Cache, APIM cache, or a direct database path. For latency, separate APIM gateway time, backend service time, Redis command latency, Cosmos DB RU and gateway metrics, and Azure SQL query duration.

For wrong data, confirm cache keys include all dimensions that affect the result: tenant, user, locale, permissions, API version, and product version. For overload, check whether cache misses are correlated, whether APIM limits are too loose, whether database retries are multiplying traffic, and whether a single hot key is saturating Redis.

4.10 Week 4 Lab Rubric

A complete Week 4 submission includes Azure CLI deployment evidence for Redis, a working Python cache-aside script, documented SQL source query, before-and-after latency observations, cache hit and miss evidence, and an ADR explaining TTLs, invalidation, fallback, and APIM ingestion controls.

Criterion	Meets expectation	Needs improvement
Cache design	Keys, TTLs, invalidation, fallback, and owner are documented	Cache is added without freshness or ownership rules
Hands-on	Azure CLI deploys Redis; Python shows miss then hit over Azure SQL	Portal-only deployment or no measurable cache-hit evidence
Correctness	Sensitive or transactional data has stricter invalidation or bypass rules	Billing, permissions, or inventory rely on stale cache entries
APIM ingestion	Authentication, rate limits, quotas, payload limits, and diagnostics are specified	Gateway only forwards traffic without admission controls
Operations	Hit ratio, latency, memory, evictions, errors, SQL load, and RU usage are monitored together	Only average latency or a single service metric is reported

Table 4.1: Week 4 rubric for caching, hybrid ingestion, hands-on implementation, and operational evidence.

4.11 Interview Q&A

4.11.1 Concepts and Trade-offs

1. **Q: What is the cache-aside pattern?**

A: The application checks cache first, reads the origin on a miss, stores the result with a TTL, and returns the value.

2. **Q: Why is Redis not usually the system of record?**

A: It is optimized for in-memory speed. Durable databases should own authoritative state unless the Redis tier and design explicitly meet durability requirements.

3. **Q: When should you prefer Cosmos DB Integrated Cache over Redis?**

A: When repeated Cosmos DB reads can use the dedicated gateway and accept a bounded staleness window without application-managed cache keys.

4. **Q: What causes a cache stampede?**

A: Many callers miss the same key at once and all query the origin, often after a popular key expires or the cache is flushed.

5. **Q: What belongs in APIM for ingestion?**

A: Authentication, authorization, quotas, rate limits, validation, transformations, routing, timeout policy, and diagnostics at the API boundary.

6. **Q: What metric is more useful than cache hit ratio alone?**

A: Hit ratio combined with p95 and p99 latency, origin load, stale-read incidents, errors, evictions, and business correctness metrics.

4.12 Further Reading

Start with Azure Cache for Redis concepts and planning guidance [74], Redis best practices [25], Cosmos DB Integrated Cache [47], dedicated gateway guidance [36], and Azure API Management overview and policy documentation [73, 15]. Review Azure SQL Database [87], Cosmos DB Change Feed [28], ADLS Gen2 [50], and the Azure Well-Architected Framework [52] for the milestone architecture review.

Key Takeaways

- Caches improve latency and reduce origin load only when freshness, invalidation, and fallback are explicit.
- Azure Cache for Redis is the general-purpose in-memory acceleration layer for hot keys, counters, sessions, and query results.
- Cosmos DB Integrated Cache reduces repeated-read RU cost when bounded staleness and the dedicated gateway fit the workload.
- APIM protects ingestion backends with authentication, quotas, rate limits, validation, transformations, routing, and diagnostics.
- Cache stampedes, hot keys, secret leakage, stale permission data, and retry storms are common production risks.
- Milestone 1 combines durable storage, relational integrity, document scale, hot-path caching, and event-driven analytics export.

4.13 Exercises

1. **(Conceptual)** Explain cache-aside to a product manager and describe why Azure SQL remains the source of truth.
2. **(Conceptual)** Compare Redis and Cosmos DB Integrated Cache for a product catalog API with repeated public reads.
3. **(Conceptual)** Choose TTLs for live scores, product prices, user permissions, and marketing recommendations.
4. **(Hands-on)** Deploy Azure Cache for Redis in a sandbox and capture hostname, SKU, TLS port, and deletion plan.
5. **(Hands-on)** Run the Python cache-aside script twice and record cache miss latency, cache hit latency, and SQL query count.
6. **(Hands-on)** Add key invalidation after an Azure SQL profile update and prove that the next read refreshes Redis.
7. **(Design)** Write an APIM policy plan for a partner ingestion API, including authentication, quotas, payload limits, and correlation IDs.
8. **(Design)** Create an ADR for the live sports mobile API, including cached objects, TTLs, invalidation, fallback, and observability.

4.14 Milestone 1 Capstone: E-Commerce Multi-Tier Data Backend

Capstone Project

Mission: Design and prototype an e-commerce multi-tier data backend that integrates Weeks 1–4: Azure SQL Database Serverless for user profiles and billing, Cosmos DB for active shopping carts and clickstream tracking, ADLS Gen2 for product images with lifecycle and encryption controls, Azure Cache for Redis for hot-path acceleration, and Cosmos DB Change Feed to push events to storage for analytics.

Estimated time: 8–10 hours.

What you will practice:

- Combining relational, document, object-storage, cache, API, and event-driven patterns in one reference architecture.
- Separating systems of record from acceleration layers and analytics sinks.
- Designing hot-path reads, cart writes, image storage, billing correctness, and downstream clickstream analytics.
- Producing implementation evidence, operational metrics, and an architecture decision record suitable for a milestone review.

Scenario

An e-commerce company is preparing for a seasonal launch. Customers browse product images, sign in, update profiles, add items to carts, receive personalized recommendations, and complete billing workflows. Marketing expects unpredictable spikes after social media announcements. The platform must keep user-facing APIs fast while preserving billing correctness and exporting behavioral events for analytics.

Requirements

- **User profiles and billing:** Use Azure SQL Database Serverless for relational user, address, payment-status, invoice, and billing-reference data.
- **Carts and clickstream:** Use Cosmos DB for active shopping carts, cart item updates, clickstream events, and short-lived personalization state.
- **Images:** Store product images in ADLS Gen2 or Blob Storage with hierarchical namespaces where needed, lifecycle policies, encryption, and least-privilege access.
- **Caching:** Use Azure Cache for Redis for product cards, profile summaries, feature flags, and hot campaign landing-page data.
- **Analytics export:** Use Cosmos DB Change Feed to write cart and clickstream events to ADLS Gen2 in partitioned folders.
- **Gateway:** Use APIM to protect public and partner APIs with authentication, rate limits, quotas, request validation, and diagnostics.
- **Operations:** Monitor p95 and p99 latency, Redis hit ratio, SQL CPU and auto-pause behavior, Cosmos DB RU consumption, Change Feed lag, storage lifecycle transitions, and API rejections.

Reference Architecture

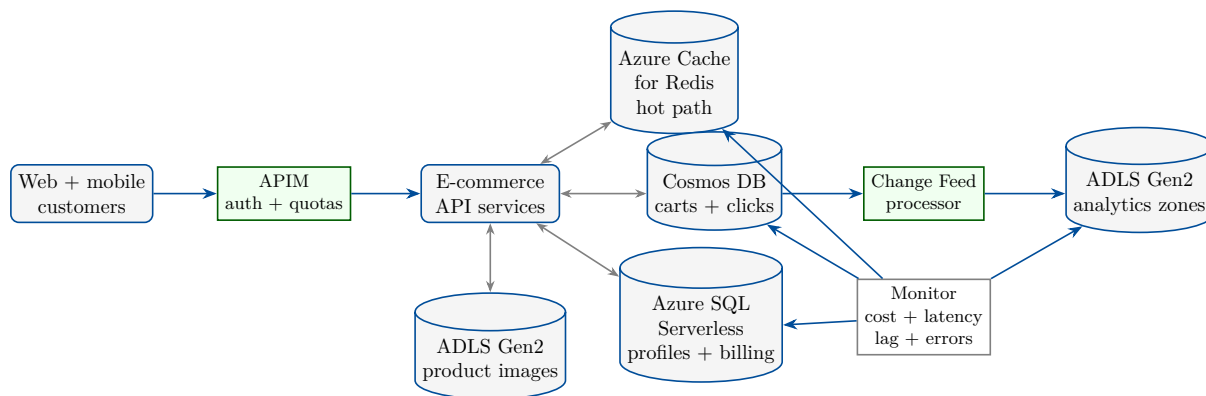


Figure 4.3: Milestone 1 reference architecture integrating ADLS Gen2, Azure SQL Database Serverless, Cosmos DB, Redis, APIM, and Change Feed analytics export.

Implementation Steps

Use a sandbox subscription and globally unique names. The snippets show a reproducible skeleton; production deployments should use Bicep, Terraform, or another reviewed infrastructure-as-code workflow.

1. **Create core resources.** Deploy one resource group, a storage account with hierarchical namespace, Azure SQL logical server and serverless database, Cosmos DB database and containers, and Azure Cache for Redis.

```

1 $rg = "rg-ecommerce-milestone1"
2 $location = "eastus"
3 $storage = "stecommilestone1001"
4 $sqlServer = "sql-ecomm-milestone-001"
5 $sqlDb = "profilesbilling"
6 $cosmos = "cosmos-ecomm-milestone-001"
7 $cosmosDb = "ecommerce"
8 $cartContainer = "carts"
9 $clickContainer = "clickstream"
10 $redis = "redis-ecomm-milestone-001"
11 $adminUser = "sqladminuser"
12 $adminPassword = "<use-Key-Vault-or-secure-prompt>"
13
14 az group create --name $rg --location $location
15
16 az storage account create '
17   --name $storage --resource-group $rg --location $location '
18   --sku Standard_LRS --kind StorageV2 --hierarchical-namespace true '
19   --min-tls-version TLS1_2 '
20   --allow-blob-public-access false
21
22 az sql server create '
23   --name $sqlServer --resource-group $rg --location $location '
24   --admin-user $adminUser --admin-password $adminPassword
25
26 az sql db create '
27   --resource-group $rg --server $sqlServer --name $sqlDb '
28   --compute-model Serverless --edition GeneralPurpose '
29   --family Gen5 --capacity 2
30
31 az cosmosdb create '

```

```

32 --name $cosmos --resource-group $rg '
33 --locations regionName=$location failoverPriority=0 isZoneRedundant=False
34 --default-consistency-level Session
35
36 az cosmosdb sql database create '
37 --account-name $cosmos --resource-group $rg --name $cosmosDb
38
39 az cosmosdb sql container create '
40 --account-name $cosmos --resource-group $rg '
41 --database-name $cosmosDb --name $cartContainer '
42 --partition-key-path "/userId" --throughput 1000 --ttl 604800
43
44 az cosmosdb sql container create '
45 --account-name $cosmos --resource-group $rg '
46 --database-name $cosmosDb --name $clickContainer '
47 --partition-key-path "/eventShard" --throughput 1000 --ttl 2592000
48
49 az redis create '
50 --name $redis --resource-group $rg --location $location '
51 --sku Basic --vm-size c0 '
52 --enable-non-ssl-port false

```

Listing 4.3: Azure CLI skeleton for the Milestone 1 e-commerce backend.

2. **Prototype hot-path reads and event writes.** Use Redis for product-card reads and Cosmos DB for cart and clickstream writes. The sketch below shows the boundary between source-of-truth stores and acceleration.

```

1 import os
2 from datetime import datetime, timezone
3
4 import redis
5 from azure.cosmos import CosmosClient, PartitionKey
6
7 cache = redis.Redis(
8     host=os.environ["REDIS_HOST"],
9     port=6380,
10    password=os.environ["REDIS_KEY"],
11    ssl=True,
12    decode_responses=True,
13 )
14
15 cosmos = CosmosClient(os.environ["COSMOS_ENDPOINT"], os.environ["COSMOS_KEY"]
16                        ])
17 db = cosmos.create_database_if_not_exists("ecommerce")
18 carts = db.create_container_if_not_exists(
19     id="carts",
20     partition_key=PartitionKey(path="/userId"),
21     offer_throughput=1000,
22 )
23 clicks = db.create_container_if_not_exists(
24     id="clickstream",
25     partition_key=PartitionKey(path="/eventShard"),
26     offer_throughput=1000,
27 )
28
29 def product_key(sku: str) -> str:
30     return f"product-card:v1:sku:{sku}"
31
32 def get_product_card(sku: str, load_from_sql_or_catalog):
33     key = product_key(sku)
34     cached = cache.get(key)

```

```

34     if cached:
35         return cached
36     product = load_from_sql_or_catalog(sku)
37     cache.setex(key, 300, product)
38     return product
39
40 def add_to_cart(user_id: str, sku: str, quantity: int):
41     now = datetime.now(timezone.utc).isoformat()
42     return carts.upsert_item({
43         "id": f"cart_{user_id}_{sku}",
44         "type": "cartItem",
45         "userId": user_id,
46         "sku": sku,
47         "quantity": quantity,
48         "updatedAt": now,
49     })
50
51 def record_click(user_id: str, sku: str, action: str):
52     now = datetime.now(timezone.utc).isoformat()
53     shard = f"{now[:10]}#{abs(hash(user_id)) % 32:02d}"
54     return clicks.create_item({
55         "id": f"click_{user_id}_{sku}_{now}",
56         "type": "clickstream",
57         "userId": user_id,
58         "sku": sku,
59         "action": action,
60         "eventTime": now,
61         "eventShard": shard,
62     })

```

Listing 4.4: Python sketch for product-card caching and clickstream writes.

3. **Configure image governance.** Create containers or file systems for product images, define lifecycle movement to cool or archive tiers, document encryption settings, and assign least-privilege roles to upload and serving identities.
4. **Implement Change Feed export.** Build or describe a Change Feed processor that reads cart and clickstream changes, writes JSON or Parquet files to ADLS Gen2 by date and event type, and stores checkpoints or leases.
5. **Add APIM policy evidence.** Document authentication, rate limits, quotas, request size limits, correlation IDs, backend timeouts, version routing, and diagnostic settings.
6. **Validate the hot path.** Demonstrate Redis miss then hit for product-card reads, point reads and writes for carts, SQL correctness for billing data, and Change Feed output arriving in storage.

Deliverables

- Architecture diagram showing APIM, API services, Azure SQL Database Serverless, Cosmos DB, Redis, ADLS Gen2 images, Change Feed processing, analytics storage, and monitoring.
- ADR explaining system-of-record boundaries, partition keys, TTLs, invalidation, APIM policies, storage lifecycle, and analytics export.
- Reproducible Azure CLI or infrastructure-as-code snippets for the core resources.
- Python scripts or pseudocode for cache-aside product reads, cart writes, clickstream event writes, and Change Feed export.

- Evidence pack with cache hit and miss output, SQL serverless configuration, Cosmos DB RU observations, storage lifecycle policy, and monitoring screenshots or query outputs.

Acceptance Criteria

- Azure SQL Database Serverless owns user profile and billing records; cache entries never become authoritative for billing correctness.
- Cosmos DB containers use partition keys that support cart point reads and distributed clickstream writes.
- ADLS Gen2 image storage includes lifecycle, encryption, access-control, and deletion-retention considerations.
- Redis keys, TTLs, invalidation triggers, fallback behavior, and monitored metrics are documented.
- Cosmos DB Change Feed export is idempotent, checkpointed, and writes analytics-ready events to storage partitions.
- APIM policies protect public and partner APIs from malformed, unauthenticated, or excessive traffic.
- The design explains how it degrades during traffic spikes without corrupting billing, carts, or analytics history.
- Another engineer can reproduce the prototype with different globally unique names and produce equivalent evidence.

Stretch Goals

- Replace password-based SQL access with managed identity and Microsoft Entra authentication.
- Add private endpoints for SQL, Redis, Cosmos DB, and storage, then document DNS implications.
- Implement an APIM product with separate quotas for anonymous users, authenticated customers, partners, and internal services.
- Add a lifecycle management policy that moves old product images to cool storage and deletes expired campaign images.
- Build a Change Feed processor with poison-record handling and a replay test.
- Create a small load test that demonstrates Redis protecting Azure SQL during repeated product-card reads.
- Estimate monthly cost for normal traffic, launch-day traffic, and a cache-cold recovery event.

Part II

Data Warehousing and Analytics

Azure Synapse Analytics: Dedicated SQL Pools

Learning Objectives

- Explain the massively parallel processing (MPP) architecture of Azure Synapse Analytics dedicated SQL pools, including the control node, compute nodes, and the Data Movement Service.
- Relate Data Warehouse Units (DWUs and cDWUs) to performance, concurrency, and hourly cost, and explain pause, resume, and scale operations.
- Apply Kimball dimensional modeling: declare the grain, choose among transaction, periodic snapshot, and accumulating snapshot fact tables, and design conformed dimensions.
- Implement slowly changing dimensions with SCD Type 1 and SCD Type 2 patterns, including a MERGE-based Type 2 upsert.
- Select hash, round-robin, or replicated distribution for a table based on join behavior and data volume.
- Bulk-load Parquet data from ADLS Gen2 with the COPY statement and managed identity authentication.
- Estimate the monthly cost of a dedicated pool versus a serverless SQL pool for an end-of-month BI workload.

Cross-Cloud Equivalence

The MPP data warehouse is one of the most portable ideas in analytics engineering. Synapse dedicated SQL pools distribute tables across 60 physical distributions and execute queries with a control node coordinating many compute nodes; the same mental model applies to Amazon Redshift's ra3 clusters and Google BigQuery's slot-based execution.

Capability	AWS	Azure	GCP
MPP warehouse	Amazon Redshift	Synapse dedicated SQL pool	BigQuery (slot-based)
Sizing unit	ra3 nodes / RPUs	cDWU (pauseable)	Slots / autoscaling editions
Distribution	DISTKEY / DIST-STYLE	Hash / round-robin / replicated	Automatic (no user control)
Bulk load from lake	COPY from S3	COPY INTO from ADLS Gen2	LOAD DATA from Cloud Storage
Serverless SQL over lake	Athena / Redshift Spectrum	Synapse serverless SQL pool	BigQuery external tables
Logical warehouse	Redshift views + Spectrum	Serverless views + external tables	BigQuery views
Fabric warehouse: T-SQL MPP on OneLake; Snowflake : virtual warehouses; MongoDB : Atlas SQL / federated queries (not supported)			
The Kimball discipline—grain declarations, conformed dimensions, and slowly changing dimensions—is engine-neutral. A star schema designed well in this chapter would port to Redshift, BigQuery, Snowflake, or a Fabric warehouse with only dialect changes.			

Key Terms

Dedicated SQL pool: the provisioned MPP data warehouse inside Azure Synapse Analytics, billed per hour while running. **DWU / cDWU:** Data Warehouse Unit, the bundled compute-memory-IO sizing metric of a dedicated pool. **Distribution:** the placement strategy that decides which of the 60 distributions holds each row. **Grain:** the precise business meaning of one row in a fact table. **SCD Type 2:** a dimension design that preserves history by closing old rows and inserting new versions.

5.1 Week 5 Overview: The Provisioned Warehouse in a Lakehouse World

Month 2 shifts from storing data to serving it at warehouse scale. Azure Synapse Analytics is Microsoft’s umbrella analytics service; inside it, the **dedicated SQL pool** is the provisioned, MPP data warehouse formerly known as Azure SQL Data Warehouse [89]. The pool distributes a table’s rows across 60 distributions mapped to compute nodes, and a control node parses queries, builds a distributed execution plan, and coordinates the compute nodes. This architecture is what lets a single query scan hundreds of terabytes—and it is also why placement decisions made at table-creation time dominate query performance later.

Week 5 therefore pairs two disciplines that are often taught separately: the platform mechanics of Synapse dedicated pools (cDWUs, distributions, COPY) and the modeling discipline of Kimball dimensional design (grain, fact types, conformed dimensions, SCD Type 2). Neither is sufficient alone. A perfectly tuned pool running a grain-confused fact table produces fast wrong answers; a beautiful star schema on round-robin distributions produces slow right answers.

Definition

Massively parallel processing (MPP) is an architecture in which a query is decomposed into parallel steps that execute across many compute nodes, each holding a slice of the data, with a control node assembling partial results into a final answer.

Note

A dedicated pool bills by the hour whenever it is running, even when idle. Pause or scale down outside working hours, and treat an Azure Monitor cost alert as a mandatory guardrail: an idle DW1000c pool can cost thousands of dollars per month.

5.2 Monday: MPP Architecture and Dimensional Modeling

5.2.1 Control Node, Compute Nodes, and the Data Movement Service

A dedicated SQL pool has one control node and one or more compute nodes. The control node accepts the T-SQL connection, optimizes the query into distributed operations, and coordinates execution. Compute nodes store distributions of user tables in Azure Storage-backed structures and execute the parallel steps. When a query needs rows that live on different compute nodes—for example, a join between two tables distributed on different keys—the **Data Movement Service** (DMS) shuffles rows between nodes. Data movement is the single most expensive operation in an MPP system; the craft of warehouse design is mostly the craft of avoiding it.

Definition

A **Data Warehouse Unit** bundles CPU, memory, and IO into a single capacity knob. The current generation is expressed as cDWUs (for example DW100c through DW30000c). Scaling up adds compute nodes and raises concurrency limits; pausing stops compute billing while storage continues to bill.

Sizing decisions should be driven by measured workload behavior, not by the size of the data alone. A 10 TB warehouse serving three analysts differs from a 1 TB warehouse serving three hundred concurrent dashboards. Scale up for load windows, scale down after, and pause when the pool is unused for hours.

5.2.2 Declaring the Grain

Kimball dimensional modeling begins with a contract: *one row in this fact table means exactly one thing* [8]. Declaring the grain—“one row per order line,” “one row per account per day,” “one row per claim status change”—decides what joins are safe, what aggregations are legal, and which dimensions can attach. Skipping the declaration is the most common cause of double-counted revenue in production warehouses, because two fact rows at different grains joined together silently fan out.

5.2.3 Fact Table Types

- **Transaction fact:** one row per business event (a sale, a click, a payment). Highest detail, largest volume.
- **Periodic snapshot fact:** one row per entity per time period (inventory per SKU per day). Answers “what was the level at point T” questions.
- **Accumulating snapshot fact:** one row per process instance, updated as milestones complete (order placed, shipped, delivered). Models pipelines with measurable stage durations.

Tip

When a business question is “how long does it take from order to delivery,” reach for an accumulating snapshot. When it is “what was on hand on March 31,” reach for a periodic snapshot. Forcing either question onto a transaction fact produces convoluted SQL and wrong answers.

5.2.4 Conformed Dimensions and Slowly Changing Dimensions

Conformed dimensions are dimensions shared, with identical keys and meanings, across multiple fact tables and data marts. A single `dim_date`, `dim_customer`, and `dim_store` allow a revenue mart and an inventory mart to be combined in one query without reconciliation meetings. Conformance is an organizational achievement as much as a technical one: it requires one team to own the dimension and every other team to consume it.

Slowly changing dimensions handle the fact that descriptive attributes change. **SCD Type 1** overwrites the old value—correct for typo fixes. **SCD Type 2** closes the current row (set `valid_to`, set `is_current` = 0) and inserts a new version, preserving history—required when the business asks “what did we believe about this customer when the sale happened?”

```

1  -- SCD Type 2 upsert on a Synapse Dedicated SQL Pool
2  MERGE INTO dim_customer AS tgt
3  USING stg_customer AS src
4      ON tgt.customer_id = src.customer_id AND tgt.is_current = 1
5  WHEN MATCHED AND (tgt.email <> src.email OR tgt.city <> src.city) THEN
6      UPDATE SET tgt.valid_to = CAST(GETDATE() AS date), tgt.is_current = 0
7  WHEN NOT MATCHED THEN
8      INSERT (customer_id, email, city, valid_from, valid_to, is_current)
9      VALUES (src.customer_id, src.email, src.city,
10             CAST(GETDATE() AS date), '9999-12-31', 1);
11
12 -- insert new versions for the rows just closed by the MERGE
13 INSERT INTO dim_customer (customer_id, email, city, valid_from, valid_to,
14                          is_current)
15     SELECT src.customer_id, src.email, src.city,
16            CAST(GETDATE() AS date), '9999-12-31', 1
17 FROM stg_customer AS src
18 JOIN dim_customer AS tgt
19     ON tgt.customer_id = src.customer_id AND tgt.is_current = 1
20 WHERE tgt.email <> src.email OR tgt.city <> src.city;

```

Listing 5.1: SCD Type 2 upsert on a Synapse dedicated SQL pool: close changed rows, then insert the new versions.

Common Pitfall

Forgetting the second insert. A MERGE alone closes changed rows but does not insert their new versions. The canonical Synapse SCD2 pattern pairs the MERGE with the follow-up INSERT shown in Listing 5.1. Omitting it silently deletes history.

5.3 Tuesday: Distribution Types

5.3.1 Hash Distribution

Hash distribution applies a hash function to one distribution column and assigns each row to one of the 60 distributions. Rows with the same hash value land together, so a join on the distribution key between two hash-distributed tables is *co-located*: each compute node joins its

local slices with no data movement. Choose as the distribution key a high-cardinality column that participates in the workload’s dominant large joins—`customer_id` for a customer-centric warehouse, `order_id` for order-line facts. Avoid columns with heavy skew (one value holding 30% of rows creates one hot distribution that straggles every query) and avoid nullable columns, because all nulls hash to the same distribution.

5.3.2 Round-Robin Distribution

Round-robin assigns rows to distributions in rotation, giving perfectly even storage but no co-location guarantee. Every large join on a round-robin table requires data movement. Use it for staging tables (where the goal is fast load, not fast join) and for small temporary tables where no good distribution key exists.

5.3.3 Replicated Distribution

A replicated table stores a full copy on every compute node. Reads against it never move data, so it is ideal for small dimensions—date, region, status codes—that join to large facts. The cost is paid at write time: every insert, update, and rebuild must write every copy, so replication is wrong for large or frequently updated tables.

Distribution	Mechanism	Best for	Watch-outs
Hash	Hash of one column assigns the row	Large facts and large dimensions with a dominant join key	Skew and nulls concentrate rows
Round-robin	Rows rotate across distributions	Staging, loads, no good key	Every large join shuffles data
Replicated	Full copy on each compute node	Small conformed dimensions	Writes multiply; rebuilds are heavy

Table 5.1: Distribution selection is the highest-leverage physical design decision in a dedicated SQL pool.

Common Pitfall

Round-robin production tables. Leaving a production fact table on the default round-robin distribution makes every large join pay a full shuffle through the Data Movement Service. Benchmark, then hash-distribute on the dominant join key.

5.4 Wednesday: Ingestion with COPY and PolyBase

5.4.1 COPY: The Modern Bulk Loader

The `COPY INTO` statement is the recommended bulk-load path for dedicated SQL pools [31]. It reads CSV, Parquet, and ORC directly from ADLS Gen2 or Blob Storage, supports wildcards over folder hierarchies, and loads in parallel across distributions. Crucially, it authenticates with **managed identity**: grant the pool’s system-assigned identity the `Storage Blob Data Reader` role on the lake container and the load needs no keys, no SAS tokens, and no secrets in code.

```

1 COPY INTO stg_sales_raw
2 FROM 'https://stadlsdemo.dfs.core.windows.net/raw/sales/year=*/month=*/.*
   parquet'
3 WITH (
4     FILE_TYPE = 'PARQUET',

```

```

5 CREDENTIAL = (IDENTITY = 'Managed Identity'),
6 MAXERRORS = 100,
7 ERRORFILE = 'https://stadlsdemo.dfs.core.windows.net/rejects/'
8 );

```

Listing 5.2: Bulk-loading Parquet from ADLS Gen2 with COPY and managed identity.

5.4.2 PolyBase: The Legacy Path

PolyBase predates COPY and loads through external tables: an external data source, an external file format, and an external table definition, then `INSERT INTO ... SELECT` from the external table. PolyBase remains useful when a load needs external-table semantics—for example, repeatedly querying a lake folder as a table from the dedicated pool—but for pure bulk loading, COPY is simpler, faster to authenticate, and the direction Microsoft documentation recommends [26].

Note

Load into a round-robin staging table, then `INSERT ... SELECT` or `CREATE TABLE AS SELECT` (CTAS) into the final hash-distributed, clustered-columnstore table. CTAS is fully parallel and minimally logged; row-by-row inserts into a columnstore produce tiny row-groups and poor compression.

5.4.3 The Load-to-Serve Pattern

A disciplined dedicated-pool pipeline has three stages: land raw files in ADLS Gen2, COPY into round-robin staging tables, then CTAS into hash-distributed, columnstore production tables with the SCD2 MERGE applied for dimensions. This pattern keeps loads fast, keeps joins co-located, and gives a clean point to reject malformed files via `MAXERRORS` and `ERRORFILE`.

5.5 Thursday Hands-on Project: Ten Million Rows into a Star Schema

This lab provisions a dedicated SQL pool, loads 10 million synthetic Parquet rows from ADLS Gen2 with COPY, models them as a star schema, and applies the SCD Type 2 MERGE. Work in a sandbox subscription; pause or delete the pool when finished.

5.5.1 Provision the Pool

```

1 az login
2 az account set --subscription "00000000-0000-0000-0000-000000000000"
3 $rg = "rg-de-azure-book-week5"
4 $loc = "eastus"
5 $ws = "synapse-week5-001"
6 $pool = "sqlpoolwk5"
7 $adls = "stadlsweek5001"
8 $sqladmin = "sqladminuser"
9 $sqlpwd = Read-Host "SQL admin key" -AsSecureString
10
11 az group create --name $rg --location $loc
12
13 az storage account create --name $adls --resource-group $rg '
14   --location $loc --sku Standard_LRS --kind StorageV2 --hns true
15 '
16 az synapse workspace create --name $ws --resource-group $rg '
17   --storage-account $adls --file-system synapsefs '

```

```

18 --sql-admin-login-user $sqladmin '
19 --sql-admin-login-password (ConvertFrom-SecureString $sqlpwd -AsPlainText) '
20 --location $loc
21
22 az synapse sql pool create --name $pool --workspace-name $ws '
23 --resource-group $rg --performance-level DW100c
24
25 az synapse workspace firewall-rule create --name allowClient '
26 --workspace-name $ws --resource-group $rg '
27 --start-ip-address "<your-ip>" --end-ip-address "<your-ip>"

```

Listing 5.3: Provisioning a Synapse workspace and a small dedicated SQL pool with Azure CLI.

Grant the workspace’s system-assigned managed identity **Storage Blob Data Contributor** on the storage account so COPY can read the lake without secrets.

5.5.2 Generate and Stage Synthetic Data

Use the Python **faker** and **pyarrow** packages to write 10 million order rows as partitioned Parquet files (`year=/month=/`), then upload with `az storage fs file upload` or **AzCopy**. Ten million rows is large enough that distribution and columnstore choices are observable but small enough to load on DW100c in minutes.

5.5.3 Model the Star and Apply SCD2

Declare the grain first: *one row per order line*. Create `fact_sales` hash-distributed on `customer_id` with a clustered columnstore index; create `dim_date` as replicated; create `dim_customer` hash-distributed on `customer_id`. Load staging with COPY (Listing 5.2), CTAS into production tables, then run the SCD2 MERGE from Listing 5.1 against a changed-customer staging batch and verify history with `SELECT ... WHERE is_current = 1`.

5.5.4 Verify and Pause

Confirm row counts match between lake files and staging, confirm the columnstore rowgroups are compressed (`sys.pdw_nodes_column_store_row_groups`), and confirm a sample customer shows two Type 2 versions. Then pause the pool:

```

1 az synapse sql pool pause --name $pool --workspace-name $ws --resource-group
  $rg

```

Listing 5.4: Pausing the pool to stop compute billing.

5.6 Friday Use Case: End-of-Month BI on a Budget

A financial services firm runs heavy BI only during the last five business days of each month: roughly 120 concurrency-hours of report load per month on 8 TB of curated data. The architecture question: dedicated pool, serverless pool, or a hybrid?

5.6.1 Cost Model for Dedicated

A dedicated pool bills per hour while running. A DW500c pool running 12 hours per day for five days is 60 hours—cheap. The same pool left running all month is roughly 720 hours—an order of magnitude more expensive with no additional value. The dedicated answer is viable only with disciplined automation: scale up at month-end, pause immediately after. Missed pauses, not the rate card, are the risk.

5.6.2 Cost Model for Serverless

The serverless SQL pool bills per TB scanned. If the month-end dashboards are well-partitioned Parquet with aggressive pruning and scan 2 TB per report cycle, the serverless bill can be a small fraction of a dedicated pool’s idle spend—and there is nothing to pause. But cost scales with bytes scanned: a dashboard that scans CSV, or that re-scans a full 8 TB folder because queries lack partition filters, inverts the comparison immediately [66].

5.6.3 Recommendation

For this workload—predictable calendar, high concurrency in bursts, curated Parquet—recommend a **hybrid**: serverless SQL views over the gold lake zone for exploration and lightweight reporting, with a DW500c dedicated pool scaled in for the month-end crunch when concurrency and latency SLAs are tight. The decision driver is not the rate card; it is the ratio of busy hours to idle hours and the discipline of the pause automation.

Option	Billing model	Best for	Watch-outs
Synapse dedicated SQL pool	cDWU per hour; pauseable	Predictable, high-concurrency warehouse workloads	Idle pools still bill; pause or resize off-hours
Synapse serverless SQL	Per TB of data scanned	Ad-hoc exploration, logical warehouse over the lake	Cost scales with bytes scanned; prune partitions, use Parquet
Fabric capacity (F SKU)	Reserved capacity units, per-second billing, pauseable	Unified lakehouse plus Power BI on one capacity	Burst-then-throttle when oversubscribed; size the SKU
Azure SQL Database	vCore/hour or DTU; serverless option	OLTP systems and small departmental marts	Not MPP; scale ceiling below Synapse for large analytics

Table 5.2: Azure compute cost comparison for warehousing workloads.

5.7 Architectural Decision Record Template

Write an ADR for every dedicated-pool deployment: the grain declaration for each fact table; the distribution key per table with measured skew evidence; the cDWU sizing and the scale/pause schedule; the COPY authentication model; and the cost ceiling with the alert threshold that enforces it. An ADR that names the rejected alternatives—round-robin for the fact table, SAS-token loads, 24/7 running—is more useful in an incident than one that only describes the chosen design.

5.8 Operational Deep Dive: Concurrency, Skew, and Rowgroup Health

Three metrics predict dedicated-pool health before users complain. First, **concurrency slot pressure**: each cDWU level grants a fixed number of concurrency slots; when queued queries grow, either scale up or reduce concurrent load rather than letting queues lengthen silently. Second, **distribution skew**: query `sys.dm_pdw_request_steps` and the distribution-level row counts; if one distribution holds multiples of the median, the hash key is wrong. Third, **row-group quality**: columnstore rowgroups should sit near one million rows; frequent small loads or updates fragment them, and `ALTER INDEX ... REORGANIZE` or a CTAS rebuild restores compression.

Tip

Schedule a weekly report of the ten largest tables with their distribution type, row count per distribution, and average rowgroup size. Skew and fragmentation are slow leaks; a dashboard makes them visible before the quarter-end query that exposes them.

5.9 Troubleshooting Patterns

Slow joins usually mean data movement: check the query plan for `ShuffleMoveOperation` and re-examine distribution keys. COPY failures with authorization errors mean the managed identity lacks a data-plane RBAC role—Owner on the resource is not enough. MAXERRORS rejections that cluster on one folder usually mean a schema drift in one source file. Sudden cost spikes almost always mean a pool that was never paused; check the pause automation before checking the rate card.

5.10 Week 5 Lab Rubric

Criterion	Meets expectation	Needs improvement
Modeling	Grain declared; fact type chosen deliberately; SCD2 verified with a two-version customer	Grain implied; SCD2 MERGE run but history never queried
Distribution	Hash keys justified with join evidence; replicated small dims	Everything left round-robin without measurement
Ingestion	COPY with managed identity; rejects captured via <code>ERRORFILE</code>	Keys or SAS tokens in code; silent load errors
Cost	Pool paused or deleted; cost estimate documented	Pool left running; no cost evidence

Table 5.3: Week 5 rubric for the dedicated-pool deliverables.

5.11 Interview Q&A

5.11.1 Concepts and Trade-offs

- Q: Why must you declare the grain before designing a fact table?**
 A: The grain defines what one row means; without it, joins fan out and aggregates double-count silently.
- Q: Which fact type models a process with milestones such as order, ship, deliver?**
 A: An accumulating snapshot—one row per process instance, updated as each milestone completes.
- Q: When is SCD Type 2 required instead of Type 1?**
 A: When the business must reconstruct what was known at a past point in time, such as the customer's segment when an order was placed.
- Q: When does a serverless SQL pool beat a dedicated pool on cost?**
 A: When queries are infrequent or bursty, data is columnar and partition-pruned, and the bytes scanned per month cost less than keeping a provisioned pool running.

5.12 Further Reading

Start with the Azure Synapse Analytics overview [89] and the dedicated SQL pool best practices [26]. Study the COPY statement reference [31] before any bulk-load design. For dimensional modeling, Kimball and Ross remain the canonical text [8]. Compare with the serverless SQL pool documentation [66] ahead of Chapter 7, and connect sizing and pause discipline to the Well-Architected cost pillar [52].

Key Takeaways

- A dedicated SQL pool is an MPP warehouse: one control node coordinates compute nodes, and the Data Movement Service shuffles rows when distributions do not align.
- cDWUs bundle compute, memory, and IO; billing is hourly while running, so pause and scale discipline dominates the cost story.
- Declare the grain before building any fact table; choose transaction, periodic snapshot, or accumulating snapshot deliberately.
- SCD Type 2 in Synapse is a MERGE to close old rows *plus* a second INSERT for the new versions.
- Hash-distribute large tables on the dominant join key, replicate small dimensions, and round-robin only staging.
- COPY with managed identity is the modern bulk loader; PolyBase external tables remain for external-table semantics.

5.13 Exercises

1. **(Conceptual)** Explain the roles of the control node, compute nodes, and the Data Movement Service during a hash-mismatched join.
2. **(Conceptual)** A fact table stores one row per account balance per night. Which fact type is this, and what questions does it answer well?
3. **(Conceptual)** Why is `customer_segment` a poor hash distribution key for `dim_customer`?
4. **(Hands-on)** Run the Thursday lab, then compare load time of COPY into a round-robin staging table versus directly into a hash-distributed table. Explain the difference.
5. **(Hands-on)** Modify the SCD2 MERGE to also track `loyalty_tier`. Demonstrate a three-version customer.
6. **(Hands-on)** Write a query against `sys.pdw_nodes_column_store_row_groups` that reports compressed versus open rowgroups per table.
7. **(Design)** A warehouse has 400 concurrent dashboard users from 08:00–18:00 weekdays only. Propose a scale and pause schedule and estimate the monthly cDWU-hours.
8. **(Design)** Given `fact_orders` (hash on `order_id`) and `fact_shipments` (hash on `shipment_id`), explain the data movement cost of joining them and propose a redesign.
9. **(Design)** Write an ADR choosing DW1000c over DW500c for a quarter-end close. Include the evidence you would collect to defend the choice.
10. **(Interview)** Prepare a two-minute explanation of why “the pool is big enough” is not an answer to a slow-query complaint.

5.14 Capstone Project: Star Schema Warehouse with Managed SCD2 History

Capstone Project

Mission: Build a small but production-shaped dedicated SQL pool warehouse: load 10 million rows from ADLS Gen2 with COPY and managed identity, model a declared-grain star schema with hash-distributed facts, replicated dimensions, and SCD Type 2 customer history, and demonstrate a measured scale/pause cost plan.

Estimated time: 4–6 hours.

What you will practice:

- Provisioning a Synapse workspace and dedicated pool with Azure CLI and firewall-scoped access.
- Bulk-loading partitioned Parquet with COPY using the workspace managed identity.
- Declaring grain, choosing fact types, and implementing conformed dimensions.
- Running the two-step SCD2 MERGE and verifying historical versions.
- Hashing on the dominant join key and replicating small dimensions, with skew evidence.

Scenario

A retail analytics team needs a warehouse over five years of order-line history. Analysts join sales to customers, dates, and stores thousands of times per day; the customer dimension changes weekly as segments and addresses are corrected. Finance requires that any historical report can be re-run as of a past date, so customer history must be preserved, not overwritten.

Requirements

- **Grain contract:** `fact_sales` is one row per order line, stated in the README and enforced by a uniqueness check.
- **Distribution:** `fact_sales` and `dim_customer` hash on `customer_id`; `dim_date` and `dim_store` replicated; staging round-robin.
- **Ingestion:** COPY with `CREDENTIAL = (IDENTITY = 'Managed Identity')`; malformed files rejected to an error folder, never silently skipped.
- **History:** SCD Type 2 on `dim_customer` with `valid_from`, `valid_to`, and `is_current`; an as-of query demonstrates reconstruction of a past state.
- **Cost:** documented scale/pause schedule; pool paused at submission time; a cost estimate with the DWU-hours assumption.

Reference Architecture

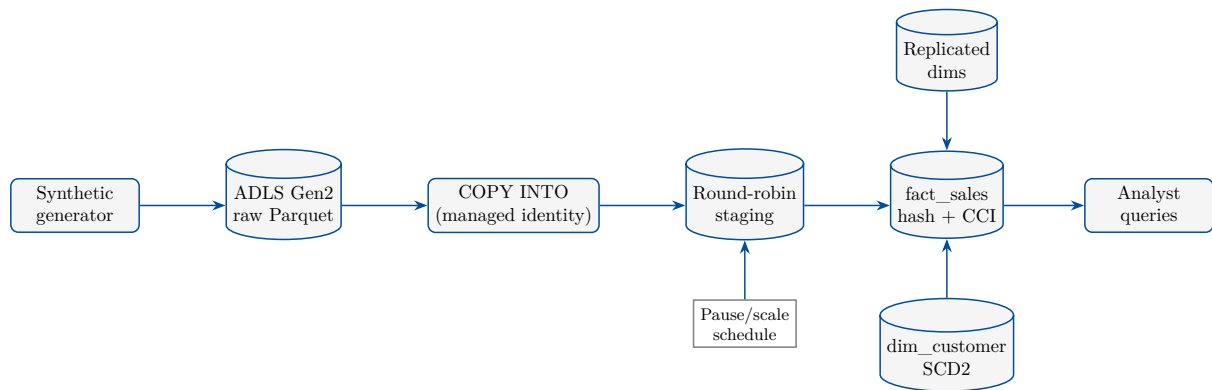


Figure 5.1: Capstone architecture: lake-staged Parquet, COPY with managed identity, round-robin staging, hash-distributed columnstore facts, and replicated dimensions.

Implementation Steps

1. Provision the workspace and pool from Listing 5.3; assign the managed identity **Storage Blob Data Contributor**.
2. Generate 10 M order-line Parquet files partitioned by `year/month`; upload to `raw/sales/`.
3. COPY into `stg_sales_raw`; confirm `MAXERRORS` rejects land in `rejects/`.
4. Create the star schema; CTAS staging into `fact_sales`; load dimensions.
5. Run the SCD2 MERGE (Listing 5.1) against a batch with changed emails; verify two versions.
6. Benchmark a star join before and after fixing distributions; record the timings.
7. Pause the pool; record the monthly cost estimate.

Deliverables

- SQL scripts for staging, star schema, COPY, CTAS, and the SCD2 MERGE.
- A one-page grain declaration and distribution rationale per table.
- Query output proving two SCD2 versions and an as-of reconstruction.
- Before/after join timings and a monthly cost estimate with pause evidence.

Acceptance Criteria

- Row counts reconcile between lake files, staging, and the fact table.
- No credentials appear in any script; all loads use managed identity.
- `is_current = 1` returns exactly one row per customer.
- The final pool state is paused or deleted.

Stretch Goals

- Add a periodic snapshot fact for daily inventory and justify the grain in the README.
- Measure distribution skew per table and document the fix if any distribution exceeds twice the median.
- Re-run the COPY benchmark at DW200c and DW400c and plot load time versus cDWU.

Query Optimization and Cost Control in Synapse

Learning Objectives

- Select the right index structure—clustered columnstore, heap, or clustered index with nonclustered indexes—for each Synapse table role.
- Apply table partitioning in dedicated pools without falling into the too-many-partitions trap.
- Configure workload groups, workload classifiers, and resource isolation so critical dashboards survive ad-hoc load.
- Decide when a materialized view or result-set cache pays off and when it just moves cost around.
- Benchmark distribution types on JOIN-heavy queries and interpret the execution plan’s data-movement steps.
- Diagnose and re-architect a multi-petabyte table that causes CPU bottlenecks during daily dashboard aggregations.
- Execute a benchmark-driven re-architecture of a slow warehouse workload, from diagnosis to a measured, cheaper steady state.

Cross-Cloud Equivalence

Columnar storage, partition pruning, and workload isolation are universal warehouse concerns; only the knobs differ. Synapse exposes physical design explicitly (you choose distribution and indexes), while BigQuery and Snowflake automate more of it—the trade-off is control versus toil.

Capability	AWS (Redshift)	Azure (Synapse)	GCP (BigQuery)
Columnar storage	Columnar by default	Clustered columnstore index	Columnar by default
Physical layout	Sort keys + dist keys	Distribution + partitioning	Clustering + partitioning
Workload isolation	WLM queues	Workload groups + classifiers	Slot reservations
Precomputed results	Materialized views	Materialized views + result-set cache	Materialized views + BI Engine
Query cache	Result cache	Result-set cache	Cached query results

Fabric: warehouse with automatic tuning; **Snowflake:** micro-partition pruning + resource monitors; **MongoDB:** indexes + c

The benchmark habit this chapter teaches—three physical designs, one query workload,

measured timings—is the antidote to cargo-cult optimization on every platform.

Key Terms

Clustered columnstore index (CCI): the default analytical storage format; column-wise compressed segments of up to 1M rows. **Partitioning:** dividing a table’s data by a boundary column within each distribution for pruning and fast switching. **Workload group:** a resource container that reserves or caps CPU, memory, and concurrency for a class of queries. **Classifier:** a rule mapping incoming requests to a workload group by user, role, label, or time. **Materialized view:** a precomputed, automatically maintained query result stored like a table.

6.1 Week 6 Overview: Making the Warehouse Fast and the Bill Predictable

Chapter 5 built the provisioned warehouse. This chapter makes it perform and proves it with measurement—before Chapter 7 asks how much of that warehouse you even need. Optimization in Synapse has a strict order of operations, and most failed tuning efforts violate it: first fix *data layout* (distribution, indexing, partitioning), then *resource governance* (workload management), and only then *precomputation* (materialized views, caching). Teams that jump to materialized views while their fact table round-robins are paying to precompute a shuffle [26].

Cost control is the same discipline viewed from the invoice side: the expensive query is usually the badly laid-out one, and the expensive month is usually the ungoverned one.

6.2 Monday: Indexing—Columnstore, Heap, and Clustered

6.2.1 Clustered Columnstore: The Default

A clustered columnstore index stores data in compressed column segments (rowgroups of up to 1,048,576 rows). Analytics queries read only the referenced columns, and compression shrinks IO by 10× or more versus row storage. CCI is the right default for every fact table and large dimension. Its enemies are **small rowgroups**: singleton inserts or trickle loads create thousands of open delta rowgroups that must be read as row storage. Bulk loads (COPY, CTAS) build compressed rowgroups directly; row-by-row writes destroy them.

6.2.2 Heap and Clustered Rowstore

A **heap** is an unindexed table—fastest to load because nothing is maintained, slowest to query selectively. Heaps are the right structure for transient staging tables whose entire contents are consumed once. A **clustered (rowstore) index with nonclustered indexes** fits small lookup dimensions and tables with frequent singleton updates, where columnstore’s batch-oriented write path works against you. The decision rule: large and scanned → CCI; transient staging → heap; small and point-looked-up → clustered rowstore.

Tip

After any load, check rowgroup health with `sys.pdw_nodes_column_store_row_groups`. A table whose average rowgroup is under 100k rows is a candidate for `ALTER INDEX ... REORGANIZE` (merge delta rowgroups) or a CTAS rebuild (full recompression).

Common Pitfall

Adding nonclustered indexes to a fact table. In an MPP warehouse, indexes are maintained per distribution; a handful of nonclustered indexes can slow loads measurably while almost never being chosen by the optimizer for scan-heavy analytics. Fix the layout, not the index list.

6.3 Tuesday: Partitioning Strategies

6.3.1 Partitioning Is Pruning and Switching, Not Performance Magic

Dedicated-pool partitioning divides each of the 60 distributions by a boundary column—typically date. Its two real wins are **partition elimination** (queries filtered on the boundary skip whole partitions) and **partition switching** (instant, metadata-only load or purge of a day/month slice). What partitioning does *not* do is parallelize scans—that is what distribution already does.

6.3.2 The Too-Many-Partitions Trap

The partition count multiplies by 60 distributions and, for CCI, each partition builds its own rowgroups. A daily-partitioned 10-year table holds $3,650 \times 60 = 219,000$ partitions; if each holds only a few thousand rows, every columnstore rowgroup is a fragment and every query pays metadata overhead. Size partitions so each holds at least a few million rows: monthly or quarterly partitioning is right for most warehouses; daily partitioning requires billions of rows per year to justify.

Design choice	Right use	Failure mode
Daily partitions	Multi-billion-row tables; daily purge by switching	Fragmented rowgroups; slow everything
Monthly/quarterly	Most fact tables	Fewer, larger switching units
No partitioning	Small dims; evenly scanned tables	Losing the switch-based load/purge pattern
Hash on join key	Large facts and joined dims	Skew concentrates load on one distribution

Table 6.1: Partitioning complements distribution; it does not replace it.

6.4 Wednesday: Workload Management and Materialized Views

6.4.1 Workload Groups and Classifiers

Without governance, one runaway ad-hoc query can starve the executive dashboard. Workload management assigns incoming requests to **workload groups** via **classifiers**—rules on login, role, query label, or time window [24]. Each group gets importance, memory percentages, and concurrency caps. The canonical split: a **dashboards** group with high importance and guaranteed resources, an **etl** group capped to leave headroom, and an **adhoc** group with low importance and tight caps.

```

1 CREATE WORKLOAD GROUP wg_dashboards
2 WITH ( MIN_PERCENTAGE_RESOURCE = 30,
3        CAP_PERCENTAGE_RESOURCE = 60,
4        IMPORTANCE = HIGH,
5        QUERY_EXECUTION_TIMEOUT_SEC = 600 );
6

```

```

7 CREATE WORKLOAD CLASSIFIER wc_finance
8 WITH ( WORKLOAD_GROUP = 'wg_dashboards',
9         MEMBERNAME = 'finreporting_role',
10        IMPORTANCE = HIGH );

```

Listing 6.1: Workload group and classifier: pin finance dashboards above ad-hoc queries.

6.4.2 Materialized Views and Result-Set Caching

A **materialized view** stores a precomputed aggregation (for example, daily revenue by region) and is maintained automatically as the base table changes; the optimizer can rewrite queries to use it transparently. It pays off when many queries share the same expensive aggregation and the base table is append-mostly. **Result-set caching** is cheaper still: repeated identical queries return the cached result instantly. Both are the *last* optimization step—they multiply the value of good layout and cannot rescue bad layout.

6.5 Thursday Hands-on Project: Benchmarking Distribution Types

This project proves, with numbers, what Chapter 5 asserted: distribution choice decides join performance. You will create three copies of a 50-million-row orders table—hash on the join key, hash on the wrong key, and round-robin—and benchmark a JOIN-heavy query against each.

6.5.1 Build the Three Tables

```

1 CREATE TABLE fact_orders_hash
2 WITH ( DISTRIBUTION = HASH(customer_id), CLUSTERED COLUMNSTORE INDEX )
3 AS SELECT * FROM stg_orders;
4
5 CREATE TABLE fact_orders_wrongkey
6 WITH ( DISTRIBUTION = HASH(order_id), CLUSTERED COLUMNSTORE INDEX )
7 AS SELECT * FROM stg_orders;
8
9 CREATE TABLE fact_orders_rr
10 WITH ( DISTRIBUTION = ROUND_ROBIN, CLUSTERED COLUMNSTORE INDEX )
11 AS SELECT * FROM stg_orders;

```

Listing 6.2: Three physical designs, identical data.

6.5.2 Benchmark Query

```

1 DBCC FREEPROCCACHE; -- clear result-set cache between runs
2 SELECT c.city, d.year, SUM(f.net_amount) AS revenue
3 FROM fact_orders_hash f -- swap per run
4 JOIN dim_customer c ON f.customer_id = c.customer_id
5 JOIN dim_date d ON f.date_key = d.date_key
6 GROUP BY c.city, d.year
7 ORDER BY revenue DESC
8 OPTION (LABEL = 'bench-hash'); -- label enables DMV lookup

```

Listing 6.3: The benchmark: a star join with aggregation.

Run the query against all three tables, then pull timings from `sys.dm_pdw_exec_requests` filtered by label. Inspect the plans: `ShuffleMoveOperation` steps in the wrong-key and round-robin runs are the Data Movement Service doing the work the hash table avoided. Record timings and row-movement counts in a table—that table *is* the deliverable.

6.5.3 Extend: Rowgroup and Partition Effects

Repeat the benchmark after trickle-loading 1% new rows one-by-one into each table (rowgroup fragmentation) and again after **REORGANIZE**. The degradation curve teaches more about CCI than any diagram.

6.6 Friday Use Case: The Multi-Petabyte Table That Melts the Pool

A retailer's 3 PB clickstream fact table causes 100% CPU every morning at 08:00 when 200 dashboards refresh their daily aggregations. Queries queue for forty minutes. The current design: round-robin distribution, CCI with fragmented rowgroups, daily partitions, no workload management, every dashboard computing `COUNT(DISTINCT session_id)` from raw rows.

6.6.1 Diagnosis

Layout first: round-robin forces a full shuffle per query; fragmented rowgroups inflate IO; `COUNT(DISTINCT)` per dashboard per morning recomputes the same aggregates 200 times; and with no workload groups, a single accidental cross-join from an analyst notebook can starve everything.

6.6.2 Re-Architecture

1. **Layout:** CTAS the fact to hash on `session_id`; rebuild rowgroups; collapse daily partitions to monthly (row counts per partition were far below the millions threshold).
2. **Precompute:** a materialized view of daily session counts per dashboard grain; dashboards retarget to the view.
3. **Governance:** `wg_dashboards` with guaranteed resources and high importance; `wg_adhoc` capped at 10%; analysts routed by classifier on role.
4. **Cost:** with the layout fixed and aggregations precomputed, the pool drops from DW3000c to DW1500c for the same SLA—the optimization pays for itself.

Common Pitfall

Scaling up to hide bad design. Doubling cDWUs halves the queue time and doubles the bill; three months later the data doubles and the incident repeats. Measure layout, governance, and precomputation before touching the size slider.

6.7 Architectural Decision Record Template

Record per major table: distribution key with skew measurement, index type with rowgroup-health baseline, partition scheme with rows-per-partition arithmetic, and the review date. Record platform-wide: workload groups and classifier rules, materialized views with the queries they absorb, and the cDWU size with the utilization evidence that justifies it.

6.8 Operational Deep Dive: The Monthly Optimization Review

Three habits keep a Synapse pool fast. **Statistics:** the optimizer needs current statistics on join and filter columns—enable auto-create and auto-update, and manually refresh stats on distribution keys after large loads. **Top-N query review:** monthly, pull the ten most expensive

queries by elapsed time and CPU from the DMVs; they are almost always a new report missing a filter or a join fan-out. **Utilization review:** if the pool sits under 20% utilization outside the morning peak, the Friday use case’s hybrid dedicated/serverless split deserves another look.

6.9 Troubleshooting Patterns

Slow query after a big load → stale statistics. Slow query that was fast last month → rowgroup fragmentation or data growth crossing a partition threshold. All queries slow at the same hour → concurrency slot exhaustion; check queue depth before blaming any query. One user’s queries slow → they are classified into the wrong workload group. Sudden wrong results after adding a materialized view → you did not add one; the optimizer did—verify the rewrite is semantically what you intended.

6.10 Week 6 Lab Rubric

Criterion	Meets expectation	Needs improvement
Benchmark rigor	Three distributions timed with cache cleared, labels, DMV evidence	Anecdotal timings, single runs, warm cache
Diagnosis	Shuffle steps identified in plans; root cause named	“It was slow, now it is fast” with no mechanism
Governance	Workload groups and classifiers configured and tested	All users share the default group
Cost	cDWU change justified with utilization evidence	Scale changed without measurement

Table 6.2: Week 6 rubric for the optimization deliverables.

6.11 Interview Q&A

6.11.1 Concepts and Trade-offs

1. **Q: Why is clustered columnstore the default for large fact tables?**
A: Columnar compression cuts IO roughly 10× and analytics queries touch few columns; CCI reads only what is referenced.
2. **Q: What problem does hash distribution solve for large JOINS?**
A: It co-locates rows with equal keys on the same distribution, so joins execute locally without Data Movement Service shuffles.
3. **Q: What do workload classifiers map incoming queries to?**
A: Workload groups—based on login, role, label, or time—so each class of work gets reserved resources, caps, and importance.
4. **Q: When does a materialized view beat adding an index?**
A: When many queries repeat the same aggregation; a materialized view precomputes it, while an index only speeds row lookup and barely helps scans.

6.12 Further Reading

The workload-management documentation [24] and the dedicated-pool best practices [26] are the primary references. The Data Engineer Handbook collects community tuning playbooks

[3]. For the physical-design theory behind distribution and columnar storage, revisit the Delta Lake and lakehouse papers [1, 2].

Key Takeaways

- Optimize in order: layout (distribution, CCI, partitioning), then governance (workload groups), then precomputation (materialized views, caching).
- CCI rowgroups near one million rows are the health metric; trickle loads fragment them and REORGANIZE or CTAS rebuilds repair them.
- Partitions exist for elimination and switching; multiply by 60 distributions and keep each partition in the millions of rows.
- Workload classifiers route users to groups with reserved resources and importance—the mechanism that protects dashboards from ad-hoc queries.
- Benchmark distribution choices with cache cleared and labels set; the ShuffleMoveOperation is the cost you are measuring.
- Scaling up hides bad design at $2\times$ the price; measure first.

6.13 Exercises

1. **(Conceptual)** Why does partitioning interact with columnstore rowgroups in a 60-distribution pool?
2. **(Conceptual)** Explain the difference between importance and resource caps in workload groups.
3. **(Conceptual)** When is a heap the correct index choice in a warehouse?
4. **(Hands-on)** Run the distribution benchmark and produce the timings table with plan screenshots.
5. **(Hands-on)** Fragment a CCI with singleton inserts, measure the regression, and repair it with REORGANIZE.
6. **(Hands-on)** Create the `wg_dashboards/wc_finance` objects from Listing 6.1 and verify routing with a test query label.
7. **(Design)** Given a 500 GB fact table growing 2 GB/day, choose partition granularity and justify with rows-per-partition math.
8. **(Design)** Write the ADR for the Friday use case re-architecture, including the rejected “just scale up” alternative.
9. **(Design)** Define a materialized view for daily revenue by region and list which of your dashboard queries it can absorb.
10. **(Interview)** Prepare a two-minute answer to “the pool is at 100% CPU every morning—what do you check first?”

6.14 Capstone Project: Warehouse Performance Rescue

Capstone Project

Mission: Take a deliberately degraded Synapse workload—round-robin fact table, fragmented columnstore, no workload governance, dashboards recomputing aggregates—and rescue it with measured, in-order optimization: layout, governance, precomputation, then a justified resize.

Estimated time: 4–6 hours.

What you will practice:

- Diagnosing with DMVs and execution plans before changing anything.
- The distribution benchmark from Listing 6.2 as decision evidence.
- Workload groups and classifiers protecting critical dashboards.
- Materialized views absorbing repeated aggregations.

Scenario

You inherit the Friday use case: a 3 PB clickstream fact table, 200 morning dashboards, forty-minute queues, and a pool at 100% CPU. Leadership's only constraint: fix the SLA before buying more capacity.

Requirements

- **Diagnosis:** DMV and plan evidence naming the top three cost drivers (data movement, rowgroup fragmentation, repeated aggregation).
- **Layout:** CTAS rebuild to hash distribution on the dominant key; rowgroup repair; partition scheme validated by rows-per-partition math.
- **Governance:** dashboard and ad-hoc workload groups with classifiers; demonstrated protection during a load test.
- **Precomputation:** one materialized view absorbing the top repeated aggregation, with the query rewrite verified.
- **Cost:** before/after timings, and a resize *down* or a justified decision to hold, with utilization evidence.

Reference Architecture

The Chapter 5 star schema re-architected: hash-distributed CCI facts, monthly partitions, a materialized-aggregate layer, and workload groups separating dashboards, ETL, and ad-hoc analysts.

Implementation Steps

1. Baseline: run the dashboard workload; capture timings, queue depth, and shuffle steps.
2. Rebuild the fact table with correct distribution and healthy rowgroups; re-measure.
3. Create the workload groups and classifiers (Listing 6.1); load-test with an ad-hoc bully query.
4. Add the materialized view; retarget dashboards; re-measure.
5. Produce the before/after table and the resize decision with evidence.

Deliverables

- All SQL scripts (diagnosis queries, rebuilds, workload objects, view).
- The before/after measurements table and plan screenshots.
- An ADR including the rejected “just scale up” alternative.

Acceptance Criteria

- Dashboard p95 latency meets the stated SLA under the ad-hoc load test.
- Every optimization is backed by a measurement, before and after.
- Final cDWU is justified by utilization evidence, not by guesswork.

Stretch Goals

- Quantify result-set caching’s contribution by re-running the benchmark with it disabled.
- Add statistics maintenance to the runbook and measure a stale-stats regression.
- Repeat the rescue on a serverless-pool equivalent and compare cost per dashboard run.

Synapse Serverless SQL and the Data Lakehouse

Learning Objectives

- Explain how the Synapse serverless SQL pool queries files in ADLS Gen2 without provisioned compute, and how per-TB billing changes cost engineering.
- Query CSV, JSON, Parquet, and Delta folders in place with OPENROWSET, controlling schema inference, file parsing, and partition elimination.
- Materialize query results back to the lake with CETAS and choose when to persist versus recompute.
- Build a logical data warehouse from external tables and views that serve BI tools without duplicating data.
- Describe what Delta Lake adds to plain files—ACID transactions, a transaction log, time travel, and schema enforcement.
- Cut serverless scan cost an order of magnitude with columnar formats, partition pruning, and right-sized files.
- Design a 50 PB log-analytics lakehouse that keeps raw data in ADLS Gen2 while querying alongside curated business data.

Cross-Cloud Equivalence

Querying the lake in place, without loading a warehouse first, exists on every major cloud. The billing model—pay per byte scanned—is also shared, which makes format and partitioning discipline a cross-cloud skill, not a Synapse trick.

Capability	AWS	Azure	GCP
Serverless lake SQL	Amazon Athena	Synapse serverless SQL pool	BigQuery external tables
Query-in-place API	S3 SELECT / Athena	OPENROWSET	External data sources
Materialize to lake	CTAS into S3	CETAS	Export / CREATE TABLE AS
Open table format	Delta Lake / Iceberg / Hudi on S3	Delta Lake on ADLS Gen2	BigLake tables / Iceberg
Logical warehouse	Athena views	Serverless views + external tables	BigQuery logical views
Billing lever	Per TB scanned	Per TB scanned	Per TB scanned (on-demand)
Fabric: SQL analytics endpoint over OneLake; Snowflake: external tables / Iceberg; MongoDB: Atlas Data Federation over			
The lakehouse thesis—warehouse-class reliability on lake storage—originated with Delta Lake [1, 2] and is now the default architecture vocabulary across all six platforms in this series.			

Key Terms

Serverless SQL pool: the per-query-billed T-SQL endpoint built into every Synapse workspace, with no provisioned compute. **OPENROWSET:** the T-SQL rowset function that reads files directly from ADLS Gen2. **CETAS:** CREATE EXTERNAL TABLE AS SELECT—persisting a query result to the lake as Parquet or Delta. **Logical data warehouse:** a serving layer of views and external tables over lake files instead of copied warehouse tables. **Transaction log:** Delta Lake’s `_delta_log` that records every commit and enables ACID behavior and time travel.

7.1 Week 7 Overview: Warehousing Without a Warehouse

Chapter 5 built a provisioned MPP warehouse. This chapter asks when you do not need one. Every Synapse workspace includes a **serverless SQL pool**: a shared, always-available T-SQL endpoint that reads files directly from ADLS Gen2 and bills only for data processed [66]. There is nothing to pause, nothing to scale, and nothing to size—and also nothing holding data close to compute. Every query re-reads the lake.

That trade defines the lakehouse serving pattern: keep one governed copy of data in ADLS Gen2 as Delta tables, and let serverless SQL—plus Power BI—query it logically through views. When it works, you delete an entire copy of the data estate and its refresh pipelines. When it fails, it fails in one of two predictable ways: queries that scan too many bytes (a cost problem) and queries that need sub-second latency or heavy concurrency (a provisioned-compute problem). Chapter 6 returned to both.

Definition

A **lakehouse** is an architecture that stores all data once in open formats on object storage and serves warehouse workloads—SQL, BI, and ML—directly from it, using a table format such as Delta Lake for ACID guarantees [2].

7.2 Monday: Querying the Lake with OPENROWSET

7.2.1 How Serverless SQL Executes

A serverless query is compiled into a distributed plan executed over stateless compute that reads the lake files at query time. Because nothing is provisioned, the unit of cost is **data processed**: the bytes the engine must read to answer the query. Three levers reduce that number: columnar formats (read only referenced columns), partition pruning (read only matching folders), and file sizing (avoid millions of tiny files that force metadata overhead).

7.2.2 OPENROWSET Patterns

```

1  -- CSV with an explicit schema (inference reads the whole file)
2  SELECT *
3  FROM OPENROWSET(
4      BULK 'https://stadls.dfs.core.windows.net/raw/logs/year=2026/month=08/*.csv',
5      FORMAT = 'CSV', PARSER_VERSION = '2.0', HEADER_ROW = TRUE
6  ) WITH (
7      device_id  VARCHAR(50),
8      event_ts   DATETIME2,
9      temp_c     FLOAT
10 ) AS rows;
11
12 -- Parquet: schema comes from the file footer; wildcards prune partitions
13 SELECT device_id, AVG(temp_c) AS avg_temp
14 FROM OPENROWSET(
15     BULK 'https://stadls.dfs.core.windows.net/silver/telemetry/year=*/month=
16     =*/*.parquet',
17     FORMAT = 'PARQUET'
18 ) AS t
19 WHERE t.filepath(1) = '2026'
20 GROUP BY device_id;
21
22 -- Delta Lake: point at the table folder, not individual files
23 SELECT *
24 FROM OPENROWSET(
25     BULK 'https://stadls.dfs.core.windows.net/gold/fact_sales/',
26     FORMAT = 'DELTA'
27 ) AS f;

```

Listing 7.1: OPENROWSET over CSV, Parquet, and Delta folders.

Tip

Use the `filepath()` function in `WHERE` clauses over wildcarded paths. Partition columns encoded in folder names (`year=*/month=`) are not real columns; `filepath(n)` exposes them so the engine can prune folders before reading data.

Common Pitfall

Schema inference on CSV. Without an explicit `WITH` clause, serverless SQL reads the data to infer types—you pay to scan the file once for inference and again for the query. Always declare schemas for CSV.

7.3 Tuesday: The Logical Data Warehouse Pattern

7.3.1 Views and External Tables as the Serving Layer

A logical data warehouse organizes lake data into a SQL-consumable layer without copying it. The building blocks are **external data sources** (named connections to lake containers with a credential), **external tables** (schema-bound references to folders), and **views** (named queries over external tables or OPENROWSET). BI tools connect to the serverless endpoint and see an ordinary database; the data never leaves the lake.

```

1 CREATE EXTERNAL DATA SOURCE ds_lake
2 WITH (LOCATION = 'https://stadls.dfs.core.windows.net/gold');
3 GO
4
5 CREATE VIEW gold.vw_sales_by_region AS
6 SELECT s.region, d.year, SUM(f.net_amount) AS revenue
7 FROM OPENROWSET(
8     BULK 'https://stadls.dfs.core.windows.net/gold/fact_sales/',
9     FORMAT = 'DELTA') AS f
10 JOIN gold.dim_store s ON f.store_key = s.store_key
11 JOIN gold.dim_date d ON f.date_key = d.date_key
12 GROUP BY s.region, d.year;
13 GO

```

Listing 7.2: Building a logical warehouse: data source, view, and a gold-zone star query.

Definition

A **logical data warehouse** exposes a warehouse-shaped SQL interface—schemas, views, security—over data that physically remains in the data lake, eliminating the copy pipeline between lake and warehouse.

7.3.2 When the Logical Pattern Breaks Down

The pattern is excellent for curated, read-mostly, moderate-concurrency serving. It degrades when queries demand interactive latency under heavy concurrency (no local caching or indexes), when tables are mutated frequently (external tables are read-only), or when scan cost exceeds provisioned cost. Those are the cases Chapter 5’s dedicated pool—or Chapter 6’s optimization toolkit—exists for.

7.4 Wednesday: Delta Lake—ACID on the Data Lake

7.4.1 The Problem with Plain Files

Object stores offer atomic single-object writes but no multi-object transactions. A Spark job writing 200 Parquet files can fail halfway, leaving a corrupt “table.” Concurrent readers see partial state. There is no schema enforcement, no rollback, and no history. Delta Lake fixes this with a **transaction log**: a folder of JSON commit files (compacted periodically into Parquet checkpoints) that records exactly which data files belong to the table at each version [1, 5].

7.4.2 What the Log Buys

- **ACID transactions:** a commit is one atomic log entry; readers always see a consistent snapshot.

- **Time travel:** `VERSION AS OF` and `TIMESTAMP AS OF` query prior states—auditing, debugging, and reproducible training sets.
- **Schema enforcement and evolution:** writes that violate the table schema fail; additive changes are made explicitly.
- **Upserts:** `MERGE` brings warehouse semantics to lake files.

7.4.3 File Hygiene: OPTIMIZE and VACUUM

Frequent small appends produce the **small-file problem**: thousands of tiny files that every query must open. `OPTIMIZE` compacts small files into 1 GB targets; `VACUUM` deletes files no longer referenced by the log beyond the retention horizon. Schedule both. A lakehouse that never compacts gets slower every week while looking healthy in every status dashboard.

Common Pitfall

Serving gold as CSV. Serverless billing is per byte scanned; CSV forces full-row reads and type parsing. The same data as Delta/Parquet with partition pruning routinely cuts scan volume—and cost—by 10× or more.

7.5 Thursday Hands-on Project: CSV In, Parquet Out with CETAS

This lab reads raw CSVs in ADLS Gen2 with the serverless pool, transforms them in SQL, and writes the result back to the lake as partitioned Parquet with CETAS—the standard “serverless ETL” pattern. Use the workspace and storage account from Chapter 5.

7.5.1 Create the Scoped Credential and Data Sources

```

1 -- in the built-in serverless pool's database
2 CREATE DATABASE SCOPED CREDENTIAL lake_identity
3 WITH IDENTITY = 'Managed Identity';
4 GO
5 CREATE EXTERNAL DATA SOURCE ds_raw
6 WITH (LOCATION = 'https://stadls.dfs.core.windows.net/raw',
7        CREDENTIAL = lake_identity);
8 CREATE EXTERNAL DATA SOURCE ds_curated
9 WITH (LOCATION = 'https://stadls.dfs.core.windows.net/curated',
10        CREDENTIAL = lake_identity);
11 GO

```

Listing 7.3: Credential and data sources for the raw and curated containers.

Grant the workspace managed identity `Storage Blob Data Contributor` on the storage account first, or both reads and the CETAS write will fail with authorization errors.

7.5.2 Transform and Materialize

```

1 CREATE EXTERNAL TABLE curated.orders_clean
2 WITH (
3     LOCATION = 'orders_clean/',
4     DATA_SOURCE = ds_curated,
5     FILE_FORMAT = parquet_snappy_ff -- defined via CREATE EXTERNAL FILE
6     FORMAT
7 ) AS
8 SELECT

```



```

8      CAST(order_id AS BIGINT)           AS order_id,
9      TRY_CAST(order_ts AS DATETIME2)    AS order_ts,
10     UPPER(TRIM(status))                AS status,
11     CAST(amount AS DECIMAL(18,2))       AS amount,
12     YEAR(TRY_CAST(order_ts AS DATETIME2)) AS order_year,
13     MONTH(TRY_CAST(order_ts AS DATETIME2)) AS order_month
14 FROM OPENROWSET(
15     BULK 'orders/year=*/month=*/*.csv',
16     DATA_SOURCE = ds_raw,
17     FORMAT = 'CSV', PARSER_VERSION = '2.0', HEADER_ROW = TRUE
18 ) WITH (
19     order_id VARCHAR(32), order_ts VARCHAR(32),
20     status VARCHAR(20), amount VARCHAR(20)
21 ) AS r
22 WHERE TRY_CAST(order_ts AS DATETIME2) IS NOT NULL;

```

Listing 7.4: CETAS: transform raw CSV into curated Parquet partitioned by year and month.

7.5.3 Verify the Round Trip

Query the new external table, then query the underlying folder with `FORMAT = 'PARQUET'` and confirm the results match. Compare `DATA_PROCESSED` from the serverless query stats between a query with and without a `filepath()` partition filter—that number is your invoice.

7.6 Friday Use Case: A 50 PB Log Lakehouse for a Media Company

A media company accumulates 50 PB of CDN, player, and application logs and wants analysts to query them alongside curated subscription and advertising data—without maintaining a 50 PB warehouse.

7.6.1 Target Architecture

Keep all 50 PB in ADLS Gen2 in Delta format across bronze, silver, and gold zones. Serve the hot analytical slice—the last 90 days of gold-zone aggregates—through serverless SQL views for dashboards, and let data science teams query deep history with Synapse Spark on demand. Purview (Chapter 8) catalogs everything; serverless views give BI a stable contract.

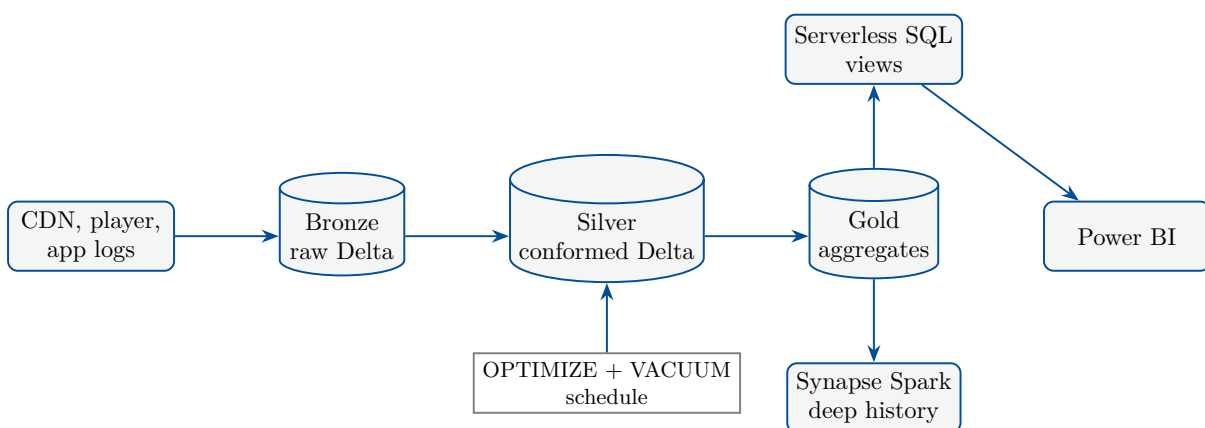


Figure 7.1: 50 PB lakehouse: one copy of data in Delta zones; serverless views serve the hot slice; Spark handles deep-history scans.

7.6.2 Cost Reasoning

At 50 PB, storing the data twice (lake plus warehouse) is untenable; the logical pattern wins by construction. The remaining cost fight is bytes scanned: partition gold tables by date, keep file sizes near 1 GB with `OPTIMIZE`, enforce partition filters in the view definitions, and monitor `data_processed` per query in Log Analytics. A full-table ad-hoc query against 50 PB of Parquet is a five-figure query; partition pruning makes the same question a two-figure one.

Common Pitfall

Treating serverless as free because there is no cluster. Per-TB billing means the most expensive query is the one nobody scoped. Publish per-view scan budgets and alert on queries that blow past them.

7.7 Architectural Decision Record Template

For each lakehouse serving decision, record: which zones exist and what formats they use; which entities are served logically (views) versus materially (dedicated pool); the partition scheme per gold table and the evidence it matches query filters; the `OPTIMIZE/VACUUM` schedule; and the monthly scan budget with the alert that enforces it.

7.8 Operational Deep Dive: Scan Budgets and View Contracts

Treat every published view as a product with a contract: columns, types, freshness, and a *scan budget* in GB per execution. Review the top ten queries by data processed monthly—they are usually a handful of views missing partition predicates. When a view must change, version it (`vw_sales_v2`) rather than breaking consumers silently; the logical warehouse has no deployment transaction, so compatibility is a discipline, not a feature.

7.9 Troubleshooting Patterns

`Cannot bulk load` errors are almost always RBAC: the managed identity lacks `Storage Blob Data Reader` (or Contributor for CETAS). Wrong results on Delta folders usually mean pointing `OPENROWSET` at a subfolder instead of the table root. Sudden cost jumps trace to one query—find it in the request DMVs or Log Analytics and look for a missing partition filter. Type errors on CSV mean inference crept back in; check that the `WITH` clause is still present after a view edit.

7.10 Week 7 Lab Rubric

Criterion	Meets expectation	Needs improvement
Querying	Explicit schemas on CSV; <code>filepath()</code> pruning used	Inference left on; full-folder scans
Materialization	CETAS writes partitioned Parquet; round trip verified	Output unpartitioned or never validated
Delta fluency	Log, time travel, OPTIMIZE/VACUUM explained with evidence	Delta treated as “just Parquet”
Cost	<code>data_processed</code> compared with and without pruning	No scan measurement in the submission

Table 7.1: Week 7 rubric for the serverless lakehouse deliverables.

7.11 Interview Q&A

7.11.1 Concepts and Trade-offs

1. **Q: How is serverless SQL billed, and how do you cut that cost?**
A: Per TB of data processed. Use Parquet/Delta, partition pruning via `filepath()`, explicit schemas, and appropriately sized files.
2. **Q: What is the logical data warehouse pattern?**
A: A serving layer of external tables and views over lake files, so BI queries a warehouse-shaped interface without copying data out of the lake.
3. **Q: What does Delta Lake add to plain files on ADLS Gen2?**
A: A transaction log that provides ACID commits, time travel, schema enforcement, and MERGE upserts over object storage.
4. **Q: What does CETAS do?**
A: `CREATE EXTERNAL TABLE AS SELECT` materializes a serverless query result back to the lake as Parquet or Delta—serverless ETL.

7.12 Further Reading

Begin with the serverless SQL pool overview [66] and the CETAS guide [27]. The Delta Lake papers explain the transaction log and the lakehouse thesis [1, 2]; the Delta Lake project site tracks the open format [5]. Chapter 6’s workload-management material and the dedicated-pool best practices [26] complete the serving-layer picture.

Key Takeaways

- The serverless SQL pool queries ADLS Gen2 in place and bills per TB processed; the cost levers are columnar formats, partition pruning, and file sizing.
- `OPENROWSET` with explicit schemas and `filepath()` predicates is the core skill; schema inference on CSV is a silent tax.
- CETAS turns a serverless query into persisted, partitioned Parquet or Delta—the serverless ETL pattern.

- A logical data warehouse serves BI through views over lake data, deleting the lake-to-warehouse copy pipeline.
- Delta Lake’s transaction log brings ACID, time travel, and upserts to object storage; OPTIMIZE and VACUUM are mandatory hygiene.
- At 50 PB scale, one governed copy in Delta zones plus logical serving beats any duplicated warehouse design on cost.

7.13 Exercises

1. **(Conceptual)** Explain why a dedicated pool and a serverless pool fail in opposite ways under heavy concurrency.
2. **(Conceptual)** Why does CSV schema inference double the scan cost of a query?
3. **(Conceptual)** What breaks if you VACUUM a Delta table with a zero-hour retention while a long-running reader holds an old snapshot?
4. **(Hands-on)** Run the CETAS lab, then measure `data_processed` for a query with and without a `filepath(1)` filter. Report both numbers.
5. **(Hands-on)** Create an external table over a Delta folder and query two historical versions. Explain what the log files contain.
6. **(Hands-on)** Append 500 small files to a Delta table, time a count query, run OPTIMIZE, and time it again.
7. **(Design)** Design the gold-zone views for the media company’s executives. Specify partition filters and a scan budget per view.
8. **(Design)** Decide for three workloads—executive dashboard, data-science exploration, regulatory audit—whether each uses serverless views, Spark, or a dedicated pool. Justify.
9. **(Design)** Write an ADR for keeping the 50 PB estate in Delta rather than loading 5 PB of hot data into a dedicated pool.
10. **(Interview)** Prepare a two-minute explanation of why “the lakehouse deleted our warehouse” is a cost statement, not a latency statement.

7.14 Capstone Project: Logical Lakehouse Serving Layer

Capstone Project

Mission: Build a governed, serverless serving layer over a Delta lakehouse: land raw CSVs, transform them with CETAS into partitioned Parquet, publish a versioned set of gold views with scan budgets, and demonstrate a 10× cost reduction from format and partition discipline.

Estimated time: 3–5 hours.

What you will practice:

- Serverless credentials, data sources, external file formats, and external tables.
- OPENROWSET with explicit schemas and `filepath()` partition pruning.
- CETAS materialization with partitioning.
- Measuring and reducing data processed per query.

Scenario

A retailer has five years of order CSVs landing nightly in **raw/**. Analysts currently export data to spreadsheets because there is no warehouse. Leadership wants governed SQL access *without* provisioning one, and finance wants proof that the per-TB bill is under control.

Requirements

- **Ingestion:** raw CSVs in **raw/orders/year=/month=**; malformed rows rejected by explicit TRY_CAST filters.
- **Curation:** CETAS writes **curated/orders_clean/** as Parquet, partitioned by year and month.
- **Serving:** a gold schema of views with documented columns and per-view scan budgets.
- **Cost evidence:** **data_processed** measurements showing the pruning/format improvement.
- **Security:** managed-identity credential only; no keys in any script.

Reference Architecture

Reuse Figure 7.1 at retail scale: bronze CSV → CETAS to curated Parquet → gold views → Power BI, with the OPTIMIZE/VACUUM schedule annotated on the curated zone.

Implementation Steps

1. Create the scoped credential and data sources (Listing 7.3).
2. Run the CETAS transformation (Listing 7.4) and verify the round trip.
3. Publish three gold views; record their scan footprints with and without partition filters.
4. Version one view (**_v2**) to demonstrate the compatibility discipline.
5. Write the one-page scan-budget report as the cost deliverable.

Deliverables

- All SQL scripts (credential, data sources, file format, CETAS, views).
- The scan-budget report with before/after measurements.
- A README stating the view contracts and the versioning policy.

Acceptance Criteria

- Curated Parquet row counts reconcile with valid raw rows.
- Every gold view carries a declared scan budget and measured footprint.
- No credentials in code; the managed identity performs all lake access.

Stretch Goals

- Convert the curated zone to Delta and add a time-travel query to the audit story.
- Add a Log Analytics alert on any single query exceeding 500 GB processed.

- Compare the same workload's cost against a DW100c dedicated pool for one month and write the recommendation.

Data Governance with Microsoft Purview

Learning Objectives

- Explain why a governed catalog is a prerequisite—not an afterthought—for a data platform that serves hundreds of users.
- Register ADLS Gen2 and Synapse sources in Microsoft Purview and run scans with managed-identity authentication.
- Interpret automated classifications, and locate every PII_Email column in a lake with a single catalog query.
- Trace lineage from raw lake files through pipelines and views to a Power BI report for impact analysis.
- Share datasets across Azure tenants with Azure Data Share without copying files into partner-controlled storage.
- Architect a data clean room where an ad agency and an advertiser match audiences without exposing raw PII.
- Complete the Part II milestone: an enterprise lakehouse with Delta storage, Purview governance, serverless serving, and measured cost.

Cross-Cloud Equivalence

Every cloud pairs its storage and compute with a governance plane: a metastore-backed catalog, automated classification, and lineage. Purview’s distinction is scope—it catalogs Azure, AWS, GCP, and on-premises sources in one data map.

Capability	AWS	Azure	GCP
Catalog / data map	Glue Data Catalog + Lake Formation	Microsoft Purview	Dataplex
PII discovery	Macie	Purview classifications	Sensitive Data Protection
Lineage	Glue / CloudTrail analysis	Purview lineage	Dataplex lineage
Cross-tenant sharing	AWS Data Exchange / S3 sharing	Azure Data Share	Analytics Hub / BigQuery sharing
Clean room	AWS Clean Rooms	Data Share + confidential compute patterns	BigQuery data clean rooms

Fabric: Purview hub + OneLake catalog; **Snowflake:** Horizon Catalog + listings; **MongoDB:** Atlas auditing + federated governance

The governance loop taught here—register, scan, classify, trace, share with policy—is identical in shape on every platform; Chapter 22 extends it with custom classifications, sensitivity

labels, and data-mesh operating models.

Key Terms

Data map: Purview’s automatically built graph of data assets, their schemas, classifications, and relationships. **Classification:** a detected or assigned data category (for example, email address) attached to columns and assets. **Lineage:** the recorded flow of data from source through transformations to reports. **Azure Data Share:** the service for sharing datasets across tenants in place, with invitations, terms, and revocation. **Data clean room:** an architecture where parties match and analyze overlapping data without either side seeing the other’s raw PII.

8.1 Week 8 Overview: Governance Before Serving

Chapters 5–7 built the warehouse, tuned it, and replaced its copy pipelines with logical serving. Week 8 closes Month 2 with the question every audit, incident, and schema migration asks: *what data do we have, where did it come from, and who can see it?* Microsoft Purview is Azure’s answer: a unified governance service that scans sources into a searchable data map, auto-classifies sensitive columns, records lineage from lake to report, and brokers cross-tenant sharing [55].

The ordering principle of this chapter is the one the Milestone 2 capstone enforces: **classify before you serve**. A gold view published before the Purview scan runs is a PII exposure with a dashboard attached.

Definition

Data governance is the operating discipline of knowing your data estate: what exists (catalog), what is sensitive (classification), how it flows (lineage), and who may use it (policies). Tools automate discovery; the discipline is deciding and enforcing.

8.2 Monday: The Purview Data Map and Catalog

8.2.1 Register, Organize, Scan

Purview adoption follows three steps. **Register** sources: ADLS Gen2 accounts, Synapse workspaces, SQL databases, Power BI tenants, even S3 buckets and on-premises SQL Server. **Organize** them into **collections**—the management hierarchy that scopes scan rules and access, typically mirroring domains or environments. **Scan:** a scan connects with the Purview account’s managed identity (or a self-hosted integration runtime for private sources—the Chapter 10 pattern applied to governance), crawls the source, extracts schemas and samples, and populates the data map.

Scans are incremental and schedulable. The operational contract: every production source is registered, scanned on a cadence that matches its rate of schema change, and owned by a named steward. An unregistered source is invisible to every downstream governance control—classification, lineage, and policy all start at registration.

Tip

Grant the Purview managed identity read-only data-plane access (for example, **Storage Blob Data Reader**) at registration time and nothing more. Governance needs to see

schemas and samples, not write data.

8.2.2 The Catalog as the Front Door

Once scanned, assets are searchable by name, schema, classification, glossary term, and owner. The cultural shift this enables: analysts *find* the certified gold dataset instead of exporting a CSV, because finding is easier than asking. Business glossary terms (“net revenue,” “active subscriber”) attach to assets so technical and business vocabularies meet in one place.

8.3 Tuesday: Classification and Lineage

8.3.1 Automated Classification

During scans, Purview applies built-in classifiers—over two hundred rules for emails, credit cards, national IDs, and more—plus any custom rules (Chapter 22) to column samples. The result is queryable: “show every column classified as email address across the estate” is a catalog search, not an audit-week archaeology dig. Classifications travel with the asset, so a PII_Email column discovered in bronze stays labeled when it flows into silver and gold.

Common Pitfall

Scanning after serving. If the first Purview scan runs after dashboards ship, PII was discoverable only after analysts could already query it. Scan and classify *before* any serving view is published—and re-scan before every new gold view goes live.

8.3.2 Lineage for Impact Analysis

Purview ingests lineage automatically from ADF pipelines, Mapping Data Flows, Synapse, Databricks, and Power BI [33]. Two questions become queries instead of meetings. *Upstream*: this report is wrong—which dataset, pipeline, and source file fed it? *Downstream*: this column is changing—which views, models, and reports read it? The downstream direction is what makes Chapter 24’s expand–contract migrations safe: the contract step drops a column only when lineage proves no consumer reads it.

Lineage completeness has a hard rule: it covers what flows through instrumented services. An analyst’s ad-hoc notebook that reads bronze and writes gold leaves no lineage edge. Standardize execution paths (Chapter 12’s orchestration, Chapter 22’s delivery templates) so that the lineage graph is the truth, not a sample of it.

8.4 Wednesday: Azure Data Share

8.4.1 Sharing Without Copying

The traditional way to give a partner data—export files to their storage—creates an ungoverned copy you cannot update, watermark, or revoke. Azure Data Share shares datasets *in place*: the provider creates a share, selects datasets (ADLS Gen2 containers or folders, SQL tables), invites the consumer, and the consumer maps the share to their own storage or SQL target with snapshot-based synchronization on a schedule [77]. The provider sees who accepted, what they received, and when snapshots ran; revocation stops future snapshots.

8.4.2 Sharing Discipline

Treat every share as a data product with a contract: schema, classification, permitted use, refresh cadence, and a named owner. Share curated gold-zone data, never raw zones. Snapshot schedules create point-in-time copies in the consumer’s tenant—your classification work means you know exactly which columns require contractual protection before they leave.

8.5 Thursday Hands-on Project: Scan, Classify, Trace

Configure Purview over the lab ADLS Gen2 account and Synapse workspace, run a scan that auto-classifies PII_Email columns, and trace lineage from lake to report.

8.5.1 Provision and Register

```

1 $rg = "rg-de-azure-book-week8"
2 az group create --name $rg --location eastus
3 az purview account create --name purview-wk8 --resource-group $rg '
4   --location eastus
5
6 # grant the Purview managed identity read access to the lake
7 az purview account show --name purview-wk8 --resource-group $rg '
8   --query identity.principalId -o tsv
9 az role assignment create --assignee-object-id "<purview-principal-id>" '
10  --assignee-principal-type ServicePrincipal '
11  --role "Storage Blob Data Reader" '
12  --scope "/subscriptions/<sub>/resourceGroups/$rg/providers/Microsoft.Storage/
    storageAccounts/<adls>"

```

Listing 8.1: Create the Purview account and register the lake.

In the Purview portal: register the storage account and the Synapse workspace, create a collection **retail**, and scope both registrations to it.

8.5.2 Scan and Classify

Create a scan over the lake’s bronze and silver containers with the default rule set (which includes email classification), scoped to the **retail** collection, and run it. Then search the catalog for the email classification and confirm the customers table’s email column appears—this is the “find every PII_Email column” query made concrete. Seed the lake first with the synthetic customer data from earlier chapters so the classifier has samples to work with.

8.5.3 Trace Lineage

Run the Chapter 10 ADF pipeline (or the Chapter 7 CETAS job) against the registered sources, then open the gold asset in Purview and walk the lineage graph upstream to the raw files and downstream to the Power BI semantic model. Screenshot the end-to-end chain: it is the impact-analysis deliverable.

8.6 Friday Use Case: A Data Clean Room for Ad-Tech

An ad agency wants to measure campaign overlap with an advertiser’s customers: “how many people who saw our ads bought from you?” Both parties hold PII; neither may see the other’s raw data.

8.6.1 Architecture

Each party keeps its CRM in its own tenant, in its own ADLS Gen2 account, classified and cataloged by Purview. The matching key (email) is tokenized with a keyed hash (HMAC) whose key is shared for the match but whose mapping tables never leave either tenant—Chapter 23’s tokenization discipline applied between organizations. The agency publishes a *tokenized, aggregate-only* audience extract through Azure Data Share; the advertiser’s Synapse workspace joins tokens against its own tokenized customers, producing overlap counts and segment statistics—never row-level records. Purview classification proves which columns could leave; the share’s snapshot history provides the audit trail; terms-of-use acceptance is recorded per consumer.

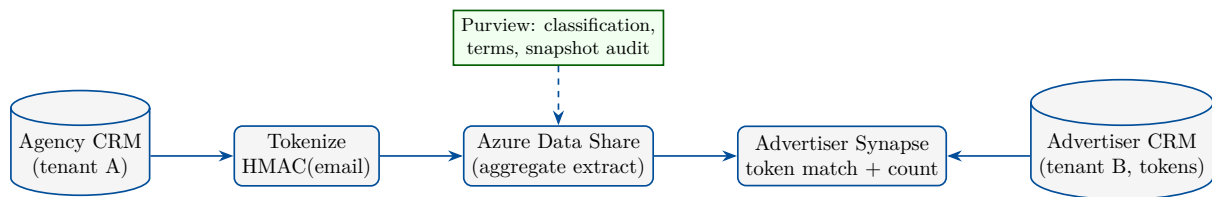


Figure 8.1: Data clean room: tokenized keys match across tenants; only aggregates cross the boundary; Purview supplies classification and audit evidence.

Common Pitfall

Sharing row-level extracts “because the query is aggregated downstream.” Once raw rows cross the tenant boundary, no downstream aggregation promise protects you. The clean room’s invariant is that only tokens and aggregates ever move—enforce it in the share definition, not in a memo.

8.7 Architectural Decision Record Template

Per governance domain record: the collection hierarchy and its owners, the scan cadence per source class, the classification rule set in use, the lineage coverage statement (which execution paths are instrumented), and the sharing policy—which zones are shareable, to whom, under what terms.

8.8 Operational Deep Dive: Scan Cadence and Coverage Metrics

Governance decays by omission: new sources appear, schemas drift, and the data map quietly diverges from reality. Track three coverage metrics monthly: percentage of production storage accounts registered, percentage with a scan run inside its cadence window, and percentage of curated-zone assets carrying at least one classification or glossary term. The third metric is the one that moves behavior—unclassified gold assets are unowned data products.

8.9 Troubleshooting Patterns

Scan finds no assets → the managed identity lacks data-plane read, or the scan scope points at the wrong container. Classifications missing → the rule set excludes the file type, or sampling was too small for the classifier’s confidence threshold. Lineage gaps after a pipeline change →

the transform ran outside an instrumented service; re-route through ADF/Synapse/Databricks. Data Share snapshots failing → the consumer’s mapping target permissions or a schema drift at the source broke the snapshot contract.

8.10 Week 8 Lab Rubric

Criterion	Meets expectation	Needs improvement
Registration	Sources registered in collections with managed-identity read access	Sources unregistered or scanned with broad keys
Classification	PII_Email columns discovered by catalog query with evidence	Classification asserted but not queried
Lineage	End-to-end chain traced from lake to report	Lineage partial or unexamined
Sharing	Share defined with terms, cadence, and gold-zone-only scope	Raw zones shared or no terms recorded

Table 8.1: Week 8 rubric for the governance deliverables.

8.11 Interview Q&A

8.11.1 Concepts and Trade-offs

1. **Q: What does a Purview scan automatically discover?**
A: Assets, schemas, and—via built-in and custom classifiers—sensitive data categories, organized into a searchable data map.
2. **Q: How does lineage help impact analysis before a schema change?**
A: The downstream query shows which views, models, and reports read a column, so you know exactly what breaks before you change it.
3. **Q: Why use Azure Data Share instead of copying files to a partner?**
A: Sharing in place keeps the data governed: invitations, terms, snapshot schedules, and revocation replace an ungoverned copy in someone else’s storage.
4. **Q: What is a data clean room, and what must it never expose?**
A: An architecture where parties match overlapping data via tokenized keys and exchange only aggregates; it must never expose row-level raw PII across the boundary.

8.12 Further Reading

The Microsoft Purview documentation hub [55] and the lineage concept guide [33] are the primary sources, with Azure Data Share covered in [77]. Chapter 22 extends this foundation with custom classifications, sensitivity labels, access policies, quality gates, and the data-mesh operating model; Chapter 23 adds the privacy and compliance layer.

Key Takeaways

- Governance starts at registration: unregistered sources are invisible to classification, lineage, and policy.
- Classify before you serve; re-scan before every new gold view goes live.

- Classification turns “where is our PII” into a catalog query.
- Lineage answers both “where did this report come from” and “what breaks if I change this column.”
- Azure Data Share replaces ungoverned copies with in-place sharing, terms, and revocation.
- A clean room moves tokens and aggregates only; the invariant is enforced in the share definition.

8.13 Exercises

1. **(Conceptual)** Why does governance tooling fail culturally when finding data is harder than asking for a CSV?
2. **(Conceptual)** Explain the difference between a collection and a classification in Purview.
3. **(Conceptual)** Why does lineage completeness depend on standardized execution paths?
4. **(Hands-on)** Complete the Thursday project: scan, classify PII_Email, and capture the lineage chain.
5. **(Hands-on)** Create a business glossary term, attach it to a gold asset, and find the asset by the term.
6. **(Hands-on)** Share a curated folder with a second subscription via Data Share and document the consumer mapping.
7. **(Design)** Design the collection hierarchy for a three-domain retail group; justify the boundaries.
8. **(Design)** Write the ADR for the clean room: tokenization key custody, share scope, and revocation procedure.
9. **(Design)** Define the three coverage metrics’ targets and the monthly report that tracks them.
10. **(Interview)** Prepare a two-minute answer to “an auditor asks where customer emails exist across the company—walk me through your answer.”

8.14 Milestone 2 Capstone: Enterprise Data Lakehouse Build

Capstone Project

Mission: Assemble Part II into one platform: ingest five years of retail data into ADLS Gen2 as Delta Lake, catalog and classify it in Microsoft Purview, and serve executive dashboards through Synapse serverless SQL views—without duplicating data into a dedicated warehouse.

Estimated time: 8–10 hours.

What you will practice:

- Bronze/silver/gold Delta zones with OPTIMIZE and VACUUM hygiene.
- Purview registration, scanning, PII classification, and lineage.
- Serverless views as the governed serving contract to Power BI.
- Cost estimation across storage, scanned TBs, and Purview.

Scenario

A retailer holds five years of order, customer, and store history in exported CSVs. Leadership wants executive dashboards refreshed daily, PII discovered and classified before any analyst touches the data, and no warehouse compute bill while concurrency is low.

Requirements

- **Storage:** ADLS Gen2 bronze/silver/gold zones in Delta format; gold modeled as the Chapter 5 star schema with declared grain and SCD2 customers.
- **Hygiene:** scheduled `OPTIMIZE` + `VACUUM`; documented retention horizon.
- **Governance:** Purview scan of the lake with `PII_Email` classification evidenced before any gold view is published.
- **Serving:** serverless SQL views over the gold Delta tables with partition filters and scan budgets (Chapter 7 contract).
- **Dashboards:** Power BI connected with DirectQuery; at least one page of executive visuals.
- **Cost:** monthly estimate (storage, TBs scanned, Purview) with two documented optimization decisions.

Reference Architecture

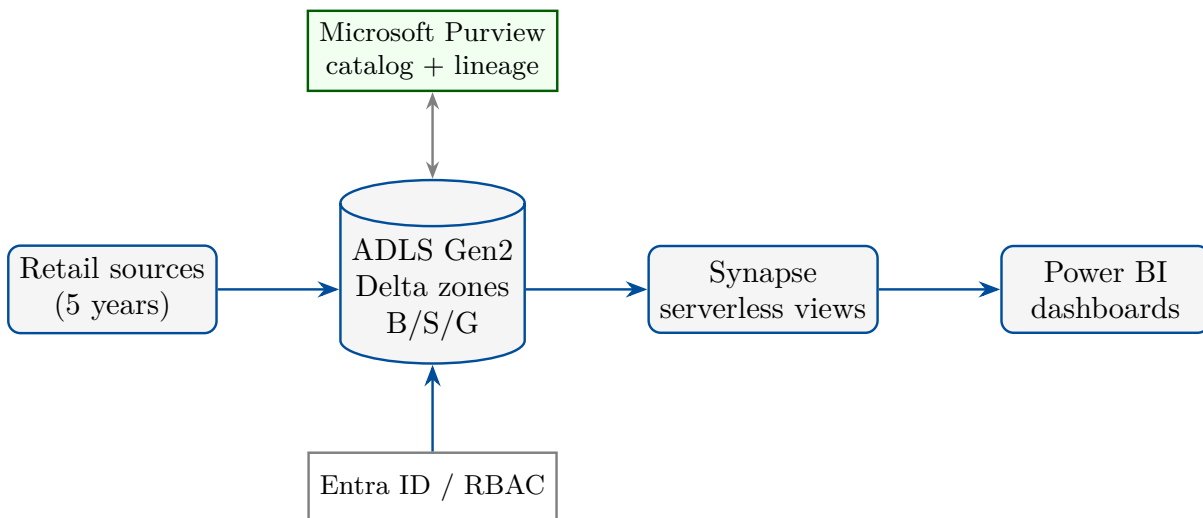


Figure 8.2: Milestone 2 architecture: one Delta lakehouse, Purview-governed, served logically through serverless views to Power BI.

Implementation Steps

1. Land raw retail files in bronze; clean and deduplicate into silver Delta tables.
2. Model gold as the star schema (grain declared first; SCD2 customers from Chapter 5).
3. Run `OPTIMIZE` and `VACUUM`; record the schedule.
4. Register and scan the lake in Purview; confirm the PII classifications.

5. Publish the gold serverless views with `filepath()` pruning; connect Power BI via DirectQuery.
6. Produce the cost estimate and the two optimization decisions.

Deliverables

- Repo with all SQL/Spark scripts and a README that runs end-to-end from a clean environment.
- Architecture diagram of the lake zones and serving layer.
- Purview evidence: classifications and a lineage screenshot from lake to report.
- Scan-budget report and monthly cost estimate.

Acceptance Criteria

- Pipeline runs end-to-end from the README on a clean workspace.
- At least three automated data-quality checks on the gold tables with a demonstrated failing case.
- PII is classified in Purview *before* any serving view exposes it.
- Monthly cost estimate includes two optimization decisions with quantified impact.

Stretch Goals

- Add the Chapter 6 workload groups to a small dedicated pool and compare executive-dashboard latency against pure serverless.
- Wire a Log Analytics alert on any serverless query scanning over 500 GB.
- Add RLS from Chapter 16 so regional executives see only their regions.

Part III

Batch Processing and ETL Orchestration

Azure Databricks and Apache Spark

Learning Objectives

- Explain the Databricks workspace model: clusters (all-purpose versus job), notebooks, DBFS, and Unity Catalog governance.
- Describe Spark’s execution model—driver, executors, partitions, stages, tasks, and shuffles—and read a physical plan.
- Author PySpark notebooks that read ADLS Gen2 via ABFS, perform distributed aggregations, and write Delta tables.
- Diagnose and mitigate skew, spill, and small-file pathologies in Spark jobs.
- Build a Delta Live Tables pipeline with expectations that quarantine bad records declaratively.
- Choose job clusters with cluster policies for production workloads and all-purpose clusters for development.
- Design a migration strategy for a 100-node on-premises Hadoop cluster to Azure Databricks.

Cross-Cloud Equivalence

Spark is the most portable compute engine in this series—the same API runs everywhere—but each cloud wraps it in a different management and billing model.

Capability	AWS	Azure	GCP
Managed Spark	EMR / Glue jobs	Azure Databricks / Synapse Spark	Dataproc / Serverless Spark
Interactive notebooks	EMR Studio / SageMaker	Databricks notebooks	Dataproc Jupyter
Declarative pipelines	Glue workflows (imperative)	Delta Live Tables	Dataflow (Beam model)
Table format	Iceberg/Hudi on S3	Delta Lake on ADLS Gen2	BigLake / Iceberg
Legacy Hadoop migration	EMR (direct lift)	Databricks / HDInsight (retiring)	Dataproc (direct lift)

Fabric: Spark pools on OneLake; **Snowflake:** Snowpark (not Spark) or external engines; **MongoDB:** Atlas Spark Connector.

The execution model taught here—partitions, shuffles, the Catalyst optimizer—is identical on EMR, Dataproc, Fabric, and self-managed Spark. Learn it once; only the billing changes.

Key Terms

Workspace: the Databricks management boundary—clusters, notebooks, jobs, repos, and catalog. **Cluster:** the Spark compute; **all-purpose** for interactive work, **job** clusters for scheduled production runs. **DBFS:** the Databricks File System abstraction over the workspace’s root storage. **Shuffle:** the exchange of rows across executors when a wide transformation (join, groupBy) requires re-partitioning. **Delta Live Tables:** declarative pipelines where you define tables and expectations; the engine manages execution.

9.1 Week 9 Overview: The Compute Engine of the Lakehouse

Part III moves from serving data to moving and transforming it at scale, and Databricks is the compute engine at the center. Some transformations outgrow visual tools: skewed joins, iterative feature engineering, ML scoring, exotic file formats. That is Spark’s territory, and Azure Databricks is Microsoft’s first-party, jointly engineered Spark service—the reference implementation of the lakehouse pattern on Azure [80].

The chapter runs on two tracks. The conceptual track teaches Spark’s execution model well enough to *predict* performance instead of discovering it: what a partition is, when a shuffle happens, why skew is fatal. The operational track teaches Databricks discipline: job clusters for production, cluster policies as guardrails, DBFS as scratch space rather than a data home, and Delta Live Tables when the pipeline shape is standard.

9.2 Monday: Workspace, Clusters, and DBFS

9.2.1 Clusters: The Cost Center You Configure

A cluster is a driver node plus worker nodes, sized by VM type and count, with optional autoscaling. The production rule is absolute: **scheduled workloads run on job clusters**, which spin up for the run and terminate on completion at 2–3× lower DBU cost than all-purpose clusters [80]. All-purpose clusters are for humans exploring. **Cluster policies** enforce the boundary: maximum size, mandatory auto-termination, allowed VM families, and tagging—the difference between a governed platform and a surprise invoice.

9.2.2 DBFS Is Not Your Data Lake

Every workspace has a DBFS root backed by a workspace storage account. It is the right place for libraries, init scripts, and scratch; it is the wrong place for production data, because it is tied to the workspace lifecycle, shares the workspace’s blast radius, and is invisible to your lake governance. Production data lives in your ADLS Gen2 accounts, accessed over ABFS with the workspace’s managed identity or Unity Catalog credential:

```

1 LAKE = "abfss://curated@stadls.dfs.core.windows.net"
2
3 orders = spark.read.parquet(f"{LAKE}/orders_clean/")
4
5 # scratch output goes to DBFS; durable output goes to the lake
6 orders.write.format("delta").mode("overwrite").save(f"{LAKE}/orders_v2/")

```

Listing 9.1: Reading the governed lake over ABFS; DBFS only for scratch.

9.2.3 Unity Catalog

Unity Catalog is Databricks' governance layer: catalogs, schemas, tables, volumes, and the permissions over them, shared across workspaces. Register lake tables once and every cluster, notebook, and job sees the same governed view—the Databricks-side complement to Purview's estate-wide catalog (Chapter 8).

9.3 Tuesday: Spark Internals—Partitions, Stages, Shuffles

9.3.1 The Execution Model

A Spark application is a driver coordinating executors. DataFrames are partitioned into slices processed in parallel as **tasks**; tasks are grouped into **stages** separated by **shuffles**. Narrow transformations (filter, select, map) stay within a partition; wide transformations (groupBy, join, distinct) require a shuffle that re-partitions data across the network. The physical plan—visible via `.explain()`—shows this: scan, narrow ops, an **Exchange** (the shuffle), then the next stage.

Definition

A **partition** is the unit of parallelism: one task processes one partition per stage. Too few partitions waste executors; too many drown the driver in task scheduling. The default shuffle partition count (200) is wrong for both a 1 MB and a 1 TB shuffle—tune `spark.sql.shuffle.partitions` or enable adaptive query execution.

9.3.2 Skew, Spill, and Small Files

The three recurring Spark pathologies:

- **Skew:** one key holds 40% of rows; one task runs for an hour while 199 finish in a minute. Fixes: salting the skewed key, broadcast joins when one side is small, or skew hints in adaptive execution.
- **Spill:** partitions exceed executor memory and overflow to disk—visible as spill metrics in the UI; fix with more memory per partition or more partitions.
- **Small files:** ten thousand 50 KB Parquet files cost more in planning and open overhead than the data is worth; fix at write time with `repartition/coalesce` and Delta's `OPTIMIZE`.

Tip

When a job is slow, open the Spark UI and ask three questions in order: which stage is slowest, does one task dominate it (skew), and is spill non-zero. That sequence resolves most Spark performance mysteries in minutes.

9.4 Wednesday: Delta Live Tables

9.4.1 Declarative Pipelines

Delta Live Tables (DLT) inverts control: you declare *what* the tables are—including data-quality **expectations**—and the engine decides how to compute, order, retry, and incrementally refresh them. A DLT pipeline is a directed graph of live tables over streaming or batch sources, with expectations that **warn**, **drop**, or **fail** on bad records.

```

1 import dlt
2 from pyspark.sql import functions as F
3
4 @dlt.table(comment="Raw order events")
5 def orders_bronze():
6     return spark.readStream.format("cloudFiles") \
7         .option("cloudFiles.format", "json") \
8         .load("abfss://raw@stadls.dfs.core.windows.net/orders/")
9
10 @dlt.table(comment="Validated orders")
11 @dlt.expect_or_drop("valid_amount", "amount >= 0")
12 @dlt.expect_or_fail("has_id", "order_id IS NOT NULL")
13 def orders_silver():
14     return (dlt.read_stream("orders_bronze")
15         .withColumn("order_ts", F.to_timestamp("order_ts")))

```

Listing 9.2: A Delta Live Tables bronze-to-silver hop with expectations.

DLT is the right tool when the pipeline is structurally standard—medallion hops with quality gates—because the engine absorbs orchestration, incrementalization, and quality reporting. Hand-written Structured Streaming (Chapter 15) is the right tool when you need control over state, triggers, and sinks that DLT does not expose.

Common Pitfall

Expectations without a quarantine view. `expect_or_drop` silently deletes bad records. Pair drops with a DLT event-log query or a quarantine table so dropped volume is visible; data quality that cannot be measured cannot be defended.

9.5 Thursday Hands-on Project: Distributed Aggregation to Delta

Deploy a Databricks workspace, mount the governed lake, run a distributed aggregation in PySpark, and write the result as a Delta table—the minimal loop every later Spark chapter extends.

9.5.1 Create the Workspace and Cluster

```

1 $rg = "rg-de-azure-book-week11"
2 az group create --name $rg --location eastus
3 az databricks workspace create --name dbx-week11 --resource-group $rg '
4     --location eastus --sku premium

```

Listing 9.3: Provisioning the workspace; cluster creation happens in the UI or via the clusters API.

Create a small cluster (for example, 2 workers, autoscaling to 4, 30-minute auto-termination) and assign the workspace identity Storage Blob Data Contributor on the lab storage account.

9.5.2 The Notebook

```

1 LAKE = "abfss://raw@stadls.dfs.core.windows.net"
2 CURATED = "abfss://curated@stadls.dfs.core.windows.net"
3
4 orders = (spark.read.parquet(f"{LAKE}/orders/")
5     .filter("amount IS NOT NULL")
6     .withColumn("order_date", F.to_date("order_ts")))

```

```

7
8 # inspect the plan BEFORE running: find the Exchange (shuffle)
9 daily = (orders.groupBy("order_date", "region")
10         .agg(F.count("*").alias("orders"),
11             F.sum("amount").alias("revenue")))
12 daily.explain(True)
13
14 (daily.write.format("delta").mode("overwrite")
15     .option("overwriteSchema", "true")
16     .save(f"{CURATED}/daily_revenue/"))
17
18 spark.sql(f"CREATE TABLE IF NOT EXISTS daily_revenue "
19          f"USING DELTA LOCATION '{CURATED}/daily_revenue/'")

```

Listing 9.4: Distributed aggregation: lake in, Delta out, plan inspected.

9.5.3 Observe, Then Break It

Find the `Exchange hashpartitioning` line in the explained plan and match it to a stage boundary in the Spark UI. Then re-run with `spark.conf.set("spark.sql.shuffle.partitions", 8)` versus 2000 and compare stage times—the tuning knob made physical.

9.6 Friday Use Case: Migrating a 100-Node Hadoop Cluster

A bank runs a 100-node on-premises Hadoop cluster: HDFS storage, Hive tables, 400 nightly MapReduce and Hive jobs. Hardware is due for refresh; leadership wants cost reduction and an exit from cluster administration.

9.6.1 Assessment and Strangler Pattern

Inventory first: classify jobs by engine (Hive SQL → Spark SQL is usually mechanical), data sensitivity, and dependency depth. Migrate in **strangler** waves: move storage to ADLS Gen2 (one-time bulk copy with Data Box, then DMS-style delta sync), point Hive jobs at the lake via Databricks' Hive-metastore-compatible tables, convert MapReduce jobs to Spark on job clusters, and decommission nodes wave by wave. The cluster shrinks; it never big-bangs.

9.6.2 The Cost Case

The on-prem cluster is sized for the month-end peak and idle 80% of the month. Job clusters invert that: 400 nightly jobs run on clusters that exist only while running. The honest cost model includes engineering time—expect 70% of Hive jobs to convert trivially, 25% to need SQL dialect work, and 5% (the MapReduce ones with side effects) to need real engineering. The migration is justified by eliminating capacity planning, patching, and the hardware refresh; not by pretending conversion is free.

Common Pitfall

Lifting HDFS habits into the lake. Small-file flushes, per-job temp tables in the system storage, and interactive all-purpose clusters running overnight are on-prem habits that become cloud bills. Cluster policies and DBFS discipline are migration deliverables, not afterthoughts.

9.7 Architectural Decision Record Template

Per Spark workload record: cluster type and policy with the cost ceiling, shuffle partition strategy, skew handling, small-file policy at write time, and the job-cluster schedule with its SLA. For migrations, additionally record the wave plan and the per-job conversion class.

9.8 Operational Deep Dive: Job Reliability and Photon

Production jobs need retries with backoff, timeout caps, and alerting on duration regression—a job that usually takes 20 minutes and now takes 60 is a data-shape change announcing itself. Enable **Photon**, Databricks’ vectorized C++ execution engine, for SQL-heavy and Parquet-heavy workloads; it accelerates the scans and joins that dominate ETL with no code change. Keep init scripts minimal: every init-script dependency is a cold-start failure mode.

9.9 Troubleshooting Patterns

Job stuck with 199/200 tasks done → skew; look at the task time distribution. `OutOfMemoryError` with spill → partitions too large; raise shuffle partitions or executor memory. Writes producing thousands of files → missing `coalesce/repartition` or a high shuffle partition count at write time. Table reads suddenly slow → the Delta small-file problem; run `OPTIMIZE` and check your append frequency.

9.10 Week 9 Lab Rubric

Criterion	Meets expectation	Needs improvement
Execution literacy	Plan inspected; shuffle identified; partition tuning measured	Job run blind; “it worked”
Discipline	Job cluster + policy; lake via ABFS; DBFS for scratch only	All-purpose cluster left running; data in DBFS
Quality	Expectations or explicit validation with visible bad-record counts	Silent drops; no quality evidence
Cost	Auto-termination set; cluster sizing justified	Max-size cluster for a toy job

Table 9.1: Week 9 rubric for the Databricks deliverables.

9.11 Interview Q&A

9.11.1 Concepts and Trade-offs

1. **Q: Why use job clusters instead of all-purpose clusters in production?**
A: They terminate on completion and bill 2–3× less per DBU; all-purpose clusters are priced for interactivity and keep billing while idle.
2. **Q: What does a Spark partition control during a shuffle?**
A: Parallelism and memory per task: each shuffle partition becomes one task; too few causes spill, too many floods the driver.
3. **Q: How do Delta Live Tables differ from plain Delta tables?**
A: DLT is a declarative pipeline framework—you define tables and expectations, the engine orchestrates computation, retries, and incremental refresh.

4. Q: What is DBFS, and why should production data not live there?

A: The workspace’s root storage abstraction. Production data belongs in governed ADLS accounts over ABFS, not in workspace-scoped storage.

9.12 Further Reading

The Azure Databricks documentation [80] covers workspaces, clusters, and Unity Catalog. The Apache Spark project [96] and the Delta Lake paper [1] explain the engine and the table format. The Data Engineer Handbook [3] collects migration playbooks relevant to the Friday use case.

Key Takeaways

- Job clusters for production, all-purpose for humans, cluster policies to enforce the boundary.
- DBFS is scratch space; production data lives in governed ADLS Gen2 over ABFS.
- Spark performance is partitions, shuffles, and skew; the UI answers “which stage, which task, how much spill.”
- The default 200 shuffle partitions is wrong at both extremes; tune it or enable adaptive execution.
- Delta Live Tables trade control for declarative pipelines with built-in quality expectations.
- Hadoop migrations succeed as strangler waves with an honest conversion-class estimate, not as big-bang rewrites.

9.13 Exercises

1. **(Conceptual)** Walk a `groupBy` aggregation through partitions, stages, and the shuffle.
2. **(Conceptual)** Why does salting fix skew, and what does it cost?
3. **(Conceptual)** When is DLT the wrong choice for a pipeline?
4. **(Hands-on)** Complete the Thursday notebook and capture the explained plan with the Exchange highlighted.
5. **(Hands-on)** Reproduce the shuffle-partitions experiment and graph stage time versus partition count.
6. **(Hands-on)** Convert the notebook’s quality checks into a DLT pipeline with `expect_or_drop` and a dropped-records report.
7. **(Design)** Write the wave plan for the 400-job Hadoop migration, classifying each job type.
8. **(Design)** Define the cluster policies for a data platform team: sizes, auto-termination, tags, job-cluster enforcement.
9. **(Design)** Write the ADR choosing Databricks over Synapse Spark for the bank’s migration.
10. **(Interview)** Prepare a two-minute answer to “your Spark job takes an hour; the data doubled. What breaks first?”

9.14 Capstone Project: Production-Shaped Spark Aggregation Platform

Capstone Project

Mission: Deliver a governed, job-cluster-based Spark aggregation pipeline: lake input over ABFS, distributed aggregation with a tuned shuffle, Delta output registered in the catalog, quality expectations with visible drop counts, and a cluster policy that would survive a real team.

Estimated time: 4–6 hours.

What you will practice:

- Workspace, cluster policy, and job-cluster discipline.
- Plan reading and shuffle tuning with measurements.
- Delta output hygiene: right-sized files, catalog registration.
- Declarative quality with DLT expectations.

Scenario

The retailer's data platform team is standardizing daily aggregations (revenue, order counts, basket sizes) as curated Delta tables. Today each analyst runs personal notebooks on shared all-purpose clusters; costs are opaque and quality checks are absent.

Requirements

- **Compute:** scheduled job cluster created by policy-compliant configuration; auto-termination enforced.
- **Storage:** all durable data in the governed lake over ABFS; DBFS used only for libraries and scratch.
- **Quality:** expectations on `order_id` presence and non-negative amounts, with dropped-record counts exported.
- **Evidence:** explained plan, shuffle tuning comparison, and post-write file-size histogram.

Reference Architecture

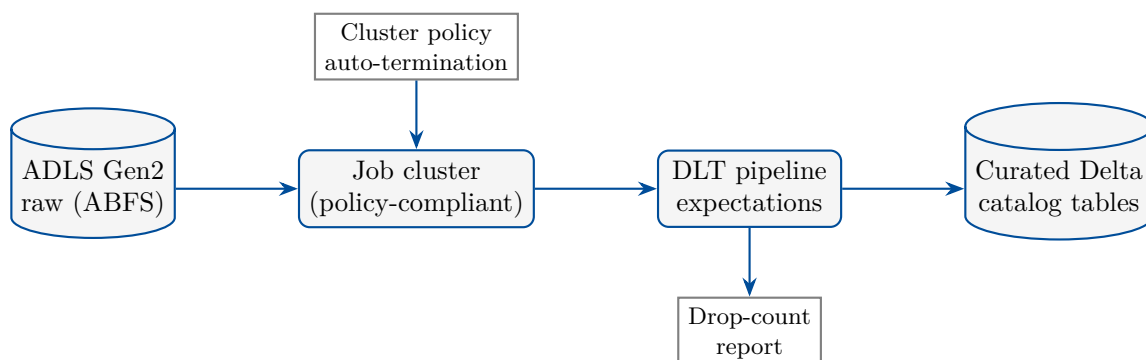


Figure 9.1: Capstone architecture: policy-governed job clusters running a DLT aggregation into cataloged Delta tables.

Implementation Steps

1. Create the workspace, cluster policy, and job cluster configuration.
2. Port the Thursday notebook to a scheduled job; verify ABFS access with the workspace identity.
3. Run the shuffle-partition experiment; set the final value with evidence.
4. Add DLT expectations; export drop counts to an audit table.
5. Register output tables; verify file sizing; document the cost per run.

Deliverables

- Notebooks, job definition JSON, and cluster policy JSON in the repo.
- The plan/tuning evidence pack and the drop-count report.
- A README that recreates workspace configuration from scratch.

Acceptance Criteria

- The job runs to completion on a fresh job cluster with no manual steps.
- Dropped records are counted and explained, not invisible.
- No durable data resides in DBFS; all tables resolve through the catalog.

Stretch Goals

- Enable Photon and measure the delta on the aggregation stage.
- Add a duration-regression alert on the daily job.
- Convert one aggregation to a DLT streaming live table with `cloudFiles` ingestion.

Azure Data Factory Foundations: Visual ETL at Scale

Learning Objectives

- Explain ADF’s object model—linked services, datasets, pipelines, activities, and triggers—and how the pieces compose into a data pipeline.
- Choose between Copy activity, Mapping Data Flows, and code-based activities for a given transformation problem.
- Author a Mapping Data Flow that joins two ADLS Gen2 datasets, filters nulls, transforms timestamps, and loads a clean result into Synapse.
- Select the right Integration Runtime—Azure, Self-Hosted, or Azure-SSIS—for network, data-residency, and legacy-package requirements.
- Parameterize pipelines and datasets so the same artifacts promote cleanly across dev, test, and prod.
- Wire success and failure dependency paths, including notifications, instead of letting failures go dark.
- Evaluate Mapping Data Flows versus Databricks notebooks for a team of business analysts.
- Read and extend the JSON definition of an ADF pipeline artifact.

Cross-Cloud Equivalence

Managed, visual ETL orchestration exists on every cloud, and ADF’s four-part object model (connection, data pointer, activity graph, schedule) has a direct analog in each.

Capability	AWS	Azure	GCP
Managed ETL service	AWS Glue	Azure Data Factory	Cloud Data Fusion
Visual transform engine	Glue Studio / DataBrew	Mapping Data Flows	Data Fusion wrangler
Orchestration	Step Functions / MWAA	ADF pipelines and triggers	Cloud Composer (Airflow)
Serverless compute for transforms	Glue jobs (Spark)	Data Flows on managed Spark	Dataflow (Beam)
Connection abstraction	Glue connections	Linked services	Data Fusion connections

Fabric: Data Factory pipelines + dataflows gen2; **Snowflake:** tasks/streams + partner tools; **MongoDB:** Atlas Triggers + C

Mapping Data Flows compile to Spark under the hood, so the execution concepts from

Chapter 9 (partitions, shuffles) apply underneath the visual canvas—the abstraction leaks upward when performance matters.

Key Terms

Linked service: a named connection (endpoint plus credential) to an external system. **Dataset:** a named, parameterized pointer to data within a linked service. **Pipeline:** a directed graph of activities with dependency edges and parameters. **Mapping Data Flow:** a visual transformation graph compiled to Spark and executed on managed compute. **Activity:** one step in a pipeline—copy, data flow, notebook call, stored procedure, control flow. **Integration Runtime (IR):** the compute ADF executes activities on—Azure-hosted, self-hosted, or SSIS.

10.1 Week 10 Overview: Orchestration as a First-Class Discipline

Part III moves from “where data lives and is served” to “how data moves.” Azure Data Factory (ADF) is the movement layer: a serverless orchestration and ETL service with ninety-plus connectors, a visual authoring canvas, and a JSON artifact model underneath everything [76]. Chapter 9 introduced Databricks as the heavy-compute engine; this week teaches ADF’s object model, its visual transformation engine, and the compute substrate—integration runtimes—underneath it; Chapters 11 and 12 add change data capture and the orchestration patterns that tie everything together.

The professional skill in ADF is not drawing pipelines—it is designing them for *promotion* and *failure*. A pipeline that hard-codes a storage account name cannot leave the dev environment. A pipeline with no failure path page-fails silently at 03:00. Parameterization and dependency design are this chapter’s through-lines.

Definition

Azure Data Factory is Azure’s managed ETL/ELT and orchestration service. It executes pipelines—graphs of activities—on integration runtimes, moving and transforming data between connected systems without managing servers.

10.2 Monday: The ADF Object Model

10.2.1 Linked Services and Datasets

A **linked service** answers “where and how do I authenticate?”—an ADLS Gen2 account via managed identity, a SQL database via Key Vault-referenced credentials, a REST endpoint with a token. A **dataset** answers “which data, in what shape?”—the `raw/orders/` folder as `DelimitedText`, the `dbo.Customers` table. Keeping the two separate is what makes pipelines reusable: ten pipelines share one linked service, and one dataset definition parameterizes across folders.

Tip

Authenticate linked services with the ADF managed identity wherever the target supports it. Every connection string embedded in a linked service is a secret that must be rotated and can leak into exported ARM templates; the managed identity eliminates both problems.

10.2.2 Pipelines, Activities, and Dependencies

A pipeline is a graph of activities with four dependency connectors: **on success**, **on failure**, **on completion**, and **on skip**. The failure connector is the one teams forget: with no failure path, a failed activity fails the pipeline run and—unless monitoring catches it—nothing else happens. The minimal professional pattern is: transform activity → on success: audit/logging activity; on failure: notification activity (Logic App, Chapter 12).

10.2.3 Triggers

Pipelines start from three trigger families: **schedule** (cron-like wall-clock), **tumbling window** (aligned, backfillable intervals with exactly-once-per-window semantics), and **event-based** (storage blob created/deleted, or custom events—Chapter 12 covers Event Grid wiring). Tumbling windows are the workhorse for periodic batch: they carry a window start/end that downstream activities can parameterize on, and they can be re-run for a historical window without touching the schedule.

10.3 Tuesday: Mapping Data Flows

10.3.1 Visual Transformations on Managed Spark

Mapping Data Flows let you compose sources, filters, joins, derived columns, aggregates, and sinks on a canvas; ADF compiles the graph to a Spark job and runs it on a managed cluster sized by the data flow's compute settings. You get Spark's scale without writing Spark code—and you inherit Spark's physics, so a join in a data flow is a shuffle and behaves like one.

10.3.2 The Expression Language

Data flow logic lives in a small expression language: `toTimestamp(order_ts, 'yyyy-MM-dd HH:mm:ss')`, `isNull(email)`, `iif(amount < 0, 'refund', 'sale')` [35]. Learn twenty functions and you can read any data flow; learn the **derived column**, **conditional split**, **join**, **aggregate**, and **alter row** transformations and you can build most of them.

10.3.3 When Visual Beats Code—and When It Doesn't

Data flows shine for analysts and for standard shapes: join, clean, conform, load. They struggle with iterative algorithms, exotic file formats, complex error handling, and anything where you need a debugger and a stack trace. The Friday use case develops this comparison; the short version: *data flows for the 80% of pipelines that are structurally boring, notebooks for the 20% that are not.*

Common Pitfall

Big compute settings on small data. Data flow debug sessions and executions spin up Spark clusters; an 8-core General cluster against 50 MB of CSVs bills cluster minutes to move kilobytes. Size compute to data, and prefer the Copy activity when no transformation is needed at all.

10.4 Wednesday: Integration Runtimes and Pipelines as Artifacts

10.4.1 The Three Integration Runtimes

Every activity executes on an **integration runtime** (IR). Choosing among the three—Azure, Self-Hosted, and Azure-SSIS—is a decision about network reach, data residency, and legacy package investment.

10.4.2 Azure IR

The default. A fully managed, serverless compute pool in a chosen Azure region (or auto-resolve). It executes Copy activities between public or Private-Linked endpoints, runs Mapping Data Flows, and dispatches control activities. Choose the region deliberately: data movement crosses regions at egress prices and, for regulated data, possibly against residency policy.

10.4.3 Self-Hosted IR

The SHIR is an agent you install on Windows machines inside the network that holds the data. It initiates *outbound-only* connections to ADF over TLS—no inbound firewall rules—and executes copy and dispatch activities locally [48]. Deploy at least two nodes for high availability; the agent load-balances and fails over between registered nodes sharing one logical IR. Patching, CPU headroom, and the credential store on SHIR nodes become your operational responsibility.

10.4.4 Azure-SSIS IR

A managed cluster that executes existing SQL Server Integration Services packages in the cloud—the lift-and-shift path for estates with hundreds of SSIS packages. You provision it into a VNet, point it at an SSIS catalog database, and run packages unchanged. It exists for migration velocity, not for new development.

Runtime	Runs where	Reaches	You manage
Azure IR	Microsoft-managed pool, chosen region	Cloud/public or Private-Link endpoints	Nothing
Self-Hosted IR	Your VMs, on-prem or cloud VNet	Private networks, outbound-only	Nodes, HA, patching
Azure-SSIS IR	Managed SSIS cluster in your VNet	Whatever the packages reach	Package compatibility

Table 10.1: Integration runtime selection.

Common Pitfall

One SHIR for everything. A single SHIR node is a single point of failure and a throughput ceiling; a single logical IR shared by dev and prod pipelines lets a dev load starve prod. Minimum: two nodes per logical IR, separate logical IRs per environment.

10.4.5 Everything Is an ARM-Deployable Artifact

Every ADF object is JSON. Reading the artifact model matters because code review, source control, and CI/CD (Chapter 24) operate on JSON, not the canvas. The pipeline below wraps a data flow execution with an audit step on success and a notification on failure:

```

1 {
2   "name": "pl_join_clean_load",
3   "properties": {
4     "activities": [
5       {
6         "name": "JoinCleanDataFlow",
7         "type": "ExecuteDataFlow",
8         "typeProperties": {
9           "dataflow": { "referenceName": "df_join_orders_customers" },
10          "compute": { "coreCount": 8, "computeType": "General" },
11          "staging": {
12            "linkedService": { "referenceName": "ls_adls_staging" },
13            "folderPath": "adf/staging"
14          }
15        },
16        "onSuccess": [{ "activity": "AuditRowCounts" }],
17        "onFailure": [{ "activity": "NotifyFailureLogicApp" }]
18      }
19    ]
20  }
21 }

```

Listing 10.1: Pipeline JSON: execute a data flow, audit on success, notify on failure.

10.4.6 Parameterization for Environment Promotion

Parameters are the promotion mechanism. A dataset whose folder path is `@dataset().folder`, a pipeline whose storage account comes from `@pipeline().parameters.env`, and global parameters per environment let one artifact set deploy to dev, test, and prod with only parameter files changing. Hard-coded values are technical debt that comes due at the first production deployment.

Element	Parameterize	Do not parameterize
Linked service	Endpoint, database, Key Vault reference	Credential type
Dataset	Folder path, file name, table name	File format structure
Pipeline	Environment name, batch size, window bounds	Activity graph shape
Trigger	Window size, start time	Trigger type

Table 10.2: Parameterization boundaries for promotable ADF artifacts.

10.5 Thursday Hands-on Project: Join, Clean, and Load with Mapping Data Flows

Build a visual pipeline that joins orders and customers from ADLS Gen2, drops nulls, normalizes timestamps, and loads the clean result into a Synapse dedicated pool.

10.5.1 Create the Factory and Connections

```

1 az login
2 $rg = "rg-de-azure-book-week9"
3 $loc = "eastus"
4 $adf = "adf-week9-001"
5
6 az group create --name $rg --location $loc

```

```

7 az datafactory create --name $adf --resource-group $rg --location $loc
8
9 # the factory's system-assigned identity needs lake and SQL access
10 az datafactory show --name $adf --resource-group $rg --query identity.
    principalId
11 az role assignment create '
12   --assignee-object-id "<principal-id>" --assignee-principal-type
    ServicePrincipal '
13   --role "Storage Blob Data Contributor" '
14   --scope "/subscriptions/<sub>/resourceGroups/$rg/providers/Microsoft.Storage/
    storageAccounts/<adls>"

```

Listing 10.2: Creating the data factory and granting its managed identity lake access.

10.5.2 Author the Data Flow

In the ADF studio: create linked services to ADLS Gen2 and Synapse (managed identity); create datasets for `raw/orders/*.csv` and `raw/customers/*.csv`; create a data flow `df_join_orders_customers` with source $\rightarrow$ filter (`not(isNull(customer_id))`) $\rightarrow$ derived column (`toTimestamp(order_ts, 'yyyy-MM-dd HH:mm:ss')`) $\rightarrow$ join on `customer_id` $\rightarrow$ sink to a Synapse staging table.

10.5.3 Wrap, Trigger, and Break It

Wrap the data flow in the pipeline from Listing 10.1; implement `AuditRowCounts` as a stored-procedure activity writing row counts to an audit table, and `NotifyFailureLogicApp` as a webhook activity. Run once on good data, then run on a file with malformed timestamps and confirm the failure path fires and the audit table records the rejection.

10.6 Friday Use Case: Mapping Data Flows vs. Databricks Notebooks for Business Analysts

A retailer's analytics team—twenty business analysts, strong SQL, weak Python—needs to build daily reporting pipelines joining POS exports with product and store reference data.

10.6.1 Evaluation

Dimension	Mapping Data Flows	Databricks notebooks
Authoring	Visual canvas; expression language	PySpark/SQL code
Skill floor	Low—SQL-adjacent analysts succeed	Requires Spark fluency
Debuggability	Data preview per transformation	Full debugger, logs, REPL
Ceiling	Standard transforms; struggles with exotic logic	Arbitrary code, ML, streaming
Governance	ARM artifacts, visual review in PRs	Notebooks in git; reviewable as code
Cost profile	Cluster warm-up per run	Job clusters; shared pools possible

Table 10.3: Mapping Data Flows versus Databricks notebooks for analyst-built pipelines.

10.6.2 Recommendation

Mapping Data Flows for this team, with two guardrails: a data engineer owns the shared linked services, datasets, and the Synapse sink contract; and a complexity tripwire—any pipeline needing custom logic beyond the expression language graduates to a Databricks notebook owned by engineering. The hybrid fails only when the tripwire is never enforced and analysts build unmaintainable visual spaghetti; the review process, not the tool, prevents that.

10.7 Architectural Decision Record Template

Per pipeline family record: linked-service authentication method, dataset parameterization boundary, trigger choice with the freshness requirement that drove it, the failure-path design, and the dev/test/prod parameter files. For data flows, additionally record the compute sizing and the measurement that justified it.

10.8 Operational Deep Dive: Idempotent Reruns and Audit Tables

Batch pipelines get re-run—after failures, for backfills, during cutovers. Design for it: sinks that *merge* or *delete-then-insert* a known slice rather than blindly append; a pipeline-run audit table (pipeline name, run id, window, rows in/out, status) written on both success and failure; and alerts keyed on the audit table so a “successful” run that moved zero rows is still caught. ADF’s tumbling window trigger plus a slice-scoped sink is the canonical idempotent pair.

10.9 Troubleshooting Patterns

Activity fails with unauthorized → the factory’s managed identity lacks the data-plane role on the target. Data flow stuck at *Starting* → cluster acquisition; check compute size and regional capacity. Pipeline succeeds but no rows land → source filter parameter mismatch; check the triggered window values. Trigger never fires → event triggers need the storage account’s Event Grid system topic and the trigger published, not just saved.

10.10 Week 10 Lab Rubric

Criterion	Meets expectation	Needs improvement
Object model	Linked services use managed identity; datasets parameterized	Connection strings embedded; hard-coded paths
Failure design	On-failure notification and on-success audit both fire in tests	Happy path only; failures silent
Promotion	Dev/test/prod run from one artifact set with parameter files	Separate factories per environment by copy-paste
Transformation	Data flow handles nulls and timestamp normalization; bad file rejected	Malformed rows silently dropped or loaded

Table 10.4: Week 10 rubric for the ADF foundations deliverables.

10.11 Interview Q&A

10.11.1 Concepts and Trade-offs

1. **Q: What is the difference between a linked service and a dataset?**

A: The linked service is the connection and credential to a system; the dataset is a parameterized pointer to specific data within it.

2. **Q: When do Mapping Data Flows beat hand-written Spark?**

A: For standard join/clean/conform transforms built by SQL-strong analysts; visual authoring and managed Spark beat code when the logic is boring.

3. **Q: What happens in ADF when no onFailure path is defined?**

A: The activity fails, the pipeline run fails, and nothing downstream runs—no notification unless external monitoring catches it.

4. **Q: How do you parameterize a pipeline for dev/test/prod promotion?**

A: Parameterize endpoints, paths, and environment names; keep one artifact set and swap parameter files per environment.

10.12 Further Reading

The ADF documentation hub [76] and the data-flow expression reference [35] are the primary sources. Integration runtimes—this chapter’s Wednesday subject—are covered in [48]. For the orchestration patterns that build on this chapter, see the pipeline execution and triggers documentation [60].

Key Takeaways

- ADF’s object model separates connection (linked service), data pointer (dataset), logic (pipeline of activities), and start condition (trigger).
- Managed identity on linked services eliminates secret rotation and template-leak risk.
- Mapping Data Flows are visual Spark: analyst-friendly authoring with Spark’s execution physics underneath.
- Every activity needs a failure path; every batch pipeline needs an idempotent sink and an audit table.
- Parameterization is the promotion mechanism; one artifact set, many parameter files.
- Choose tumbling-window triggers for backfillable batch, event triggers for arrival-driven processing.

10.13 Exercises

1. **(Conceptual)** Explain why separating linked services from datasets improves reuse and security.
2. **(Conceptual)** Compare tumbling-window and schedule triggers for a nightly load that may need backfills.
3. **(Conceptual)** Why does a Mapping Data Flow join behave like a Spark shuffle?

4. **(Hands-on)** Complete the Thursday project, including the induced failure, and capture the audit-table rows for both runs.
5. **(Hands-on)** Parameterize the lab pipeline for two environments and deploy both from one artifact set.
6. **(Hands-on)** Rebuild the same transform as a Copy activity with a pre-copy stored procedure. Which is simpler and why?
7. **(Design)** Design the audit-table schema for a 200-pipeline estate. What makes zero-row runs visible?
8. **(Design)** Write the ADR choosing Mapping Data Flows over Databricks for the analyst team, including the complexity tripwire.
9. **(Design)** Sketch the JSON for a pipeline that retries a flaky REST source three times with backoff.
10. **(Interview)** Prepare a two-minute answer to “why not just write everything in notebooks?”

10.14 Capstone Project: Parameterized Visual ETL with Failure Discipline

Capstone Project

Mission: Deliver a promotable ADF solution: a parameterized pipeline family that joins, cleans, and loads retail data from ADLS Gen2 into Synapse, with audited success paths, notifying failure paths, and one artifact set promoting across dev and prod.

Estimated time: 4–5 hours.

What you will practice:

- Managed-identity linked services and parameterized datasets.
- Mapping Data Flow authoring and validation.
- Success/failure dependency design with an audit table.
- Environment promotion through parameter files.

Scenario

The retailer’s analytics team needs a repeatable daily load: orders and customers land in the lake each night, must be joined and cleaned, and loaded into Synapse before 07:00. Failures must page someone; reruns must not duplicate rows.

Requirements

- **Connections:** all linked services authenticate with the factory’s managed identity.
- **Transform:** data flow with null filtering, timestamp normalization, and the customer join; rejects auditable.
- **Reliability:** on-success audit row with counts; on-failure notification; sink is idempotent for a rerun of the same window.
- **Promotion:** identical artifacts deploy to a second environment via parameter files.

Reference Architecture

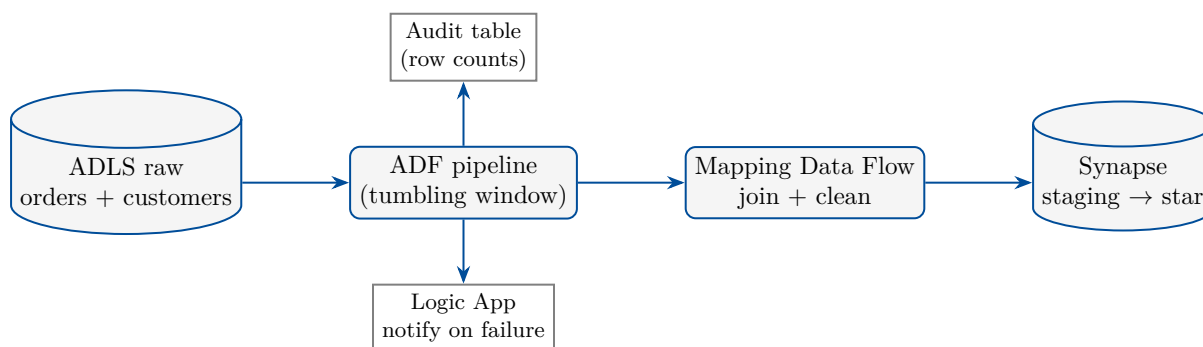


Figure 10.1: Capstone architecture: window-triggered ADF pipeline wrapping a Mapping Data Flow, with audit and notification paths.

Implementation Steps

1. Create the factory and managed-identity linked services (Listing 10.2).
2. Author the data flow and the pipeline wrapper (Listing 10.1).
3. Implement the audit and notification activities; test both paths.
4. Prove idempotency: run the same window twice; counts unchanged.
5. Deploy to the second environment from parameter files; run there.

Deliverables

- Exported ARM/JSON artifacts and both parameter files in the repo.
- Audit-table evidence of a success run, a failure run, and an idempotent rerun.
- A README that deploys the solution from a clean subscription.

Acceptance Criteria

- No credentials in any artifact; managed identity throughout.
- A bad input file produces a notification and an audit row, not a silent success.
- Re-running a window does not change the warehouse row count.

Stretch Goals

- Add a second source (product reference data) and a three-way join in the data flow.
- Emit a custom metric for rows-loaded per run and graph it in Azure Monitor.
- Convert the schedule trigger to a tumbling window and demonstrate a 7-day backfill.

Change Data Capture, Migration, and Data Contracts

Learning Objectives

- Explain Azure Database Migration Service architecture—offline versus online migration—and reach private sources through the integration runtimes from Chapter 10.
- Run a DMS migration task with ongoing replication (CDC) enabled and reconcile before cutover.
- Compare CDC mechanisms—Azure SQL CDC, Cosmos DB change feed, and DMS replication—and monitor replication lag as an SLO.
- Design event-driven ingestion with Azure Event Grid as the notification fabric.
- Enforce versioned JSON data contracts at the lake boundary, routing violations to quarantine.
- Architect near-real-time synchronization from an on-premises Oracle database to Synapse for live BI.

Cross-Cloud Equivalence

Hybrid connectivity and change-data-capture are universal integration problems. Every cloud offers an agent/relay that reaches private networks and a replication service that streams database changes.

Capability	AWS	Azure	GCP
Private data movement agent	Glue connectors / DataSync agent	Self-Hosted Integration Runtime	Cloud VPN/Interconnect + Data Fusion remote agent
Database migration	AWS DMS (+SCT)	Azure DMS (+DMA)	Database Migration Service
CDC stream	DMS ongoing replication	Azure SQL CDC / DMS CDC	Datastream
Event fabric	EventBridge	Event Grid	Eventarc
Schema enforcement	Glue Schema Registry	Contract checks in ADF/Databricks	Dataflow / Dataplex validation

Fabric: on-premises data gateway; **Snowflake:** Snowpipe + connectors; **MongoDB:** change streams (CDC-native).

Data contracts—schema, versioning, quarantine—are platform-neutral governance. The implementation differs; the obligation to reject malformed producer output at the boundary

does not.

Key Terms

CDC: change data capture—streaming inserts, updates, and deletes from a source database instead of re-extracting full tables. **Replication lag:** the delay between a source commit and its appearance downstream; a first-class SLO. **Data contract:** a versioned schema plus semantics agreed between a data producer and its consumers. **Quarantine:** a holding zone for records that fail contract validation, isolating poison data from curated zones.

11.1 Week 11 Overview: Reaching Data and Trusting It

Chapter 10’s pipelines moved data on schedules between systems the integration runtimes could reach. Real estates violate two deeper assumptions: that data is *fresh enough* when extracted nightly, and that it is *well-formed* when it arrives. The source may be an Oracle server behind a corporate firewall whose dashboards must reflect orders within minutes, and the files it emits drift schema whenever the vendor patches. Week 11 addresses both: **change data capture** solves freshness without full re-extracts, and **data contracts** solve trust at the boundary.

Definition

Change data capture (CDC) publishes a database’s insert/update/delete stream so downstream systems can apply deltas continuously. Compared with nightly full extracts, CDC cuts source load, shrinks latency from hours to minutes, and makes deletes representable.

11.2 Monday: Azure Database Migration Service

11.2.1 DMS Architecture

Azure Database Migration Service orchestrates schema and data movement between database engines [79]. An **offline** migration copies once and cuts over; an **online** migration performs the initial bulk copy, then keeps the target synchronized by streaming the source’s transaction log until cutover. Online migrations require the premium tier and a source configuration that exposes change records (for Azure SQL targets, SQL Server CDC or replication; for Oracle sources, supplemental logging).

11.2.2 Reaching the Source

DMS and ADF reach private networks through Chapter 10’s integration runtimes: a Self-Hosted IR pair installed inside the source network dials *outbound* to Azure, so no inbound firewall exposure is required. Size the SHIR nodes for the initial bulk copy’s parallelism, and remember that the same pair will carry the steady-state CDC stream afterward—plan CPU and concurrency headroom for both phases.

11.3 Tuesday: Change Data Capture Mechanisms

11.3.1 Platform CDC Mechanisms

- **Azure SQL CDC:** per-table change tables populated from the transaction log; consumers poll with `cdc.fn_cdc_get_all_changes_*`. Durable, exact, and pull-based.

- **Cosmos DB change feed:** an ordered log of inserts/updates per logical partition, consumed by push (Functions trigger) or pull; deletes require soft-delete flags or the all-versions mode (Chapter 3).
- **DMS replication:** managed streaming from heterogeneous sources (Oracle, SQL Server, MySQL) to Azure targets, hiding the per-engine log plumbing.

11.3.2 Replication Lag as an SLO

CDC pipelines fail quietly: replication stalls, dashboards go stale, and nobody is paged because nothing errored. Treat **lag**—the delay between a source commit and its lake/warehouse appearance—as a first-class SLO. Azure Monitor metrics on the DMS task expose latency; alert when it exceeds the SLA (for example, five minutes), and give the alert a runbook: check source log pressure, SHIR health, and target write throughput, in that order.

Common Pitfall

CDC without a delete story. Inserts and updates are easy; deletes are where synchronization designs diverge. Decide explicitly whether deletes propagate as hard deletes, soft-delete flags, or tombstones—and test it before go-live, not during the first data-privacy erasure request.

11.4 Wednesday: Event-Driven Ingestion and Data Contracts

11.4.1 Event Grid as the Notification Fabric

Azure Event Grid is a serverless publish/subscribe broker for resource and application events [81]. The canonical data-engineering pattern: a system topic on the storage account emits `BlobCreated`; subscriptions fan that event out to an ADF event trigger, a Function, or a Logic App. Polling disappears; latency drops from “next schedule” to seconds; and cost follows events, not empty checks. Chapter 12 composes these events into full orchestration.

11.4.2 Data Contracts at the Boundary

A **data contract** is a versioned, machine-checkable agreement: the JSON schema of the payload, the semantics of each field, the SLA, and the ownership. The producer owns the contract; consumers build against versions of it. Adding an optional field is **non-breaking**; renaming a field or changing its type is **breaking** and requires a major version with a migration window.

Enforcement happens at the lake boundary, before landing: validate each payload against its versioned schema—in a Databricks notebook with `spark.read.schema(...)` or an ADF data flow with derived-column/assert patterns—and route violations to a **quarantine** container with the failure reason attached. Quarantined data is a producer bug made visible; silently coerced data is a producer bug made permanent.

```

1 {
2   "contract": "orders",
3   "version": "2.1.0",
4   "owner": "checkout-team@example.com",
5   "schema": {
6     "type": "object",
7     "required": ["order_id", "customer_id", "order_ts", "amount"],
8     "properties": {
9       "order_id": { "type": "string", "pattern": "^ORD-[0-9]+$" },
10      "customer_id": { "type": "string" },
11      "order_ts": { "type": "string", "format": "date-time" },

```

```

12     "amount": { "type": "number", "minimum": 0 },
13     "coupon_code": { "type": ["string", "null"] }
14   },
15 },
16 "compatibility": "backward",
17 "sla": { "freshnessMinutes": 15 }
18 }

```

Listing 11.1: A versioned contract (excerpt) with a breaking-change policy.

Tip

Put the contract version *in the payload or its metadata*, not in tribal knowledge. A quarantine record that cannot say which schema version it violated cannot be reprocessed after the producer fixes the bug.

11.5 Thursday Hands-on Project: DMS CDC with a Contract Gate

Configure an Azure DMS task with CDC to stream changes from an Azure SQL Database to ADLS Gen2, then gate the landing zone with schema validation that quarantines contract-violating records.

11.5.1 Stand Up Source and Migration Service

```

1 az provider register --namespace Microsoft.DataMigration
2 $rg = "rg-de-azure-book-week10"
3 az group create --name $rg --location eastus
4
5 az network vnet create --name vnet-dms --resource-group $rg '
6   --subnet-name snet-dms --address-prefix 10.10.0.0/16 --subnet-prefix
7     10.10.1.0/24
8
9 az dms create --name dms-week10 --resource-group $rg '
10  --location eastus --subnet "/subscriptions/<sub>/resourceGroups/$rg/providers
    /Microsoft.Network/virtualNetworks/vnet-dms/subnets/snet-dms" '
    --sku-name Premium_4vCores

```

Listing 11.2: Registering the DMS resource provider and creating the service.

In the portal or via task JSON: define the source (Azure SQL Database with CDC-enabled tables), the target (ADLS Gen2 container `cdc/orders/`), and enable ongoing replication. Seed the source with the `orders` table from the Chapter 10 lab.

11.5.2 Add the Contract Gate

A Databricks notebook (or ADF data flow) reads each landed CDC batch, applies the contract from Listing 11.1, splits into `silver/orders/` (valid) and `quarantine/orders/` (invalid with `_error` reason column), and writes both with the batch's event time. Then induce three violations—missing `order_id`, negative `amount`, malformed `order_ts`—and prove each lands in quarantine with a readable reason while valid rows flow through.

11.5.3 Wire the Lag Alert

Create a Log Analytics workspace, enable diagnostic settings on the DMS task, and add an alert: *replication latency > 5 minutes for 3 consecutive evaluations*. Test it by pausing the consumer, not by hoping.

11.6 Friday Use Case: Oracle to Synapse in Near Real Time

A manufacturer runs order management on on-premises Oracle. Executives want BI dashboards reflecting orders within minutes, not the current 24-hour batch.

11.6.1 Target Architecture

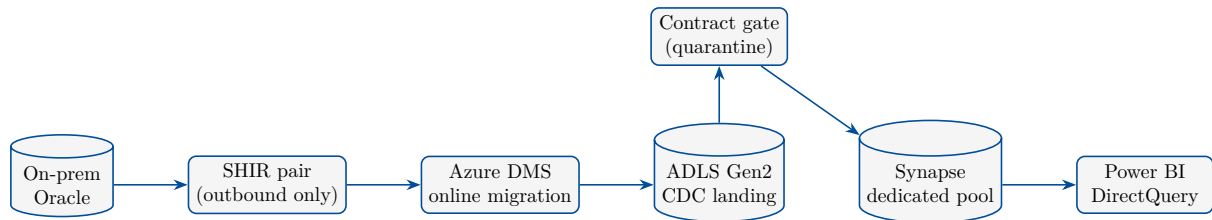


Figure 11.1: Near-real-time Oracle synchronization: SHIR for reachability, DMS for continuous replication, contract gate before Synapse.

The design combines this chapter’s pieces: a two-node SHIR reaches Oracle through outbound-only connections; DMS online migration performs the initial load and streams redo-log changes; a contract gate validates and quarantines; a merge job applies deltas (including the tested delete semantics) into the hash-distributed Synapse tables from Chapter 5; Power BI DirectQuery (Chapter 16) serves the live dashboards. The lag SLO—five minutes, alerted—is the operational contract that makes “near real time” a commitment instead of a hope.

Common Pitfall

Big-bang cutover. Run the CDC stream in parallel with the legacy batch for at least two reconciliation cycles. Cut over only when row counts and control totals match for consecutive windows; keep the batch path disabled-but-runnable for one month as the rollback.

11.7 Architectural Decision Record Template

Per integration record: IR type and node count with the network evidence; CDC mechanism and its delete semantics; lag SLO with the alert threshold; contract owner, version, and compatibility mode; quarantine location and reprocessing runbook; and the reconciliation plan used at cutover.

11.8 Operational Deep Dive: SHIR Fleet Hygiene

SHIR nodes are servers you own: patch them, monitor their CPU and queue depth, and rotate their certificates. Log Analytics can ingest the agent’s diagnostic logs; alert on node heart-beat loss, because a dead node in a two-node pair halves capacity silently. Keep the SHIR’s concurrent-jobs limit aligned with the source’s tolerance—the IR will happily open enough parallel connections to flatten an old Oracle instance.

11.9 Troubleshooting Patterns

Copy fails with *server not found* through SHIR → DNS resolution on the agent node, not in Azure. DMS task stuck at 99% → long-running open transactions pinning the source log.

Sudden lag growth → a bulk operation on the source generating change volume the sink cannot absorb. Quarantine spike after a vendor release → contract drift; diff the payloads against the previous version and page the producer, not the pipeline.

11.10 Week 11 Lab Rubric

Criterion	Meets expectation	Needs improvement
Connectivity	IR choice justified; SHIR HA with two nodes	Single node, no justification
CDC	Online migration with deletes handled explicitly	Deletes ignored or untested
Contracts	Versioned schema enforced; violations quarantined with reasons	Validation absent or silently coercive
Observability	Lag SLO defined and alert tested	No lag metric; staleness discovered by users

Table 11.1: Week 11 rubric for the integration deliverables.

11.11 Interview Q&A

11.11.1 Concepts and Trade-offs

1. **Q: Why would you need a Self-Hosted Integration Runtime?**
A: When sources sit in private networks; the SHIR dials outbound to ADF so no inbound firewall exposure is needed.
2. **Q: How does CDC differ from a nightly full extract?**
A: CDC streams deltas from the transaction log—minutes of latency, low source load, deletes representable—versus hours of latency and full table scans.
3. **Q: Is adding an optional field a breaking contract change?**
A: No—backward-compatible. Renaming a field or changing its type breaks existing consumers and requires a major version.
4. **Q: Who owns a data contract?**
A: The producer owns and versions it; consumers pin to versions, and breaking changes ship with a migration window.

11.12 Further Reading

The integration runtime concepts [48], Azure DMS overview [79], and Event Grid documentation [81] are the primary references. The Cosmos DB change feed [28] completes the CDC survey, and Chapter 12's orchestration patterns compose these events into self-healing pipelines.

Key Takeaways

- Choose the IR by network reach: Azure IR for cloud endpoints, SHIR (two nodes, per environment) for private networks, Azure-SSIS for package lift-and-shift.
- DMS online migration is bulk copy plus log streaming; cutover is a reconciliation decision, not a date.

- Replication lag is an SLO—metric, alert, runbook—because stalled CDC fails silently.
- Event Grid replaces polling with push; storage events are the canonical ingestion trigger.
- Data contracts make producer drift visible at the boundary; quarantine with reasons, never coerce silently.
- Deletes are the hardest part of synchronization; decide their semantics before go-live.

11.13 Exercises

1. **(Conceptual)** Why does the SHIR need no inbound firewall rules, and what does that cost you operationally?
2. **(Conceptual)** Compare polling-based and change-feed-based consumption of Cosmos DB changes.
3. **(Conceptual)** Why must a quarantine record carry the contract version it violated?
4. **(Hands-on)** Complete the Thursday lab: DMS CDC stream plus contract gate, with three induced violations quarantined.
5. **(Hands-on)** Configure the lag alert and capture it firing during a consumer pause.
6. **(Hands-on)** Write the reprocessing script that replays a quarantined batch after a producer fix.
7. **(Design)** Design delete propagation (hard, soft, tombstone) for the Oracle-to-Synapse pipeline and justify the choice.
8. **(Design)** Write the ADR for DMS versus a nightly ADF full-extract for the Oracle estate.
9. **(Design)** Specify the reconciliation report that gates cutover: counts, totals, and sampling strategy.
10. **(Interview)** Prepare a two-minute answer to “the dashboards are stale but nothing failed—walk me through your morning.”

11.14 Capstone Project: Hybrid CDC Pipeline with Contract Enforcement

Capstone Project

Mission: Build the Friday use case end-to-end at lab scale: an online DMS migration streaming changes through a contract gate into a queryable warehouse, with a tested lag alert and a quarantine reprocessing runbook.

Estimated time: 5–7 hours.

What you will practice:

- DMS online migration with CDC and explicit delete semantics.
- Contract validation with quarantine and replay.
- Lag as a measured, alerted SLO.
- Cutover reconciliation and rollback planning.

Scenario

The manufacturer tolerates a five-minute staleness SLO on order dashboards. The current nightly batch violates it structurally. You are replacing it with continuous replication, and compliance requires that malformed vendor data never reaches the warehouse unmarked.

Requirements

- **Replication:** DMS online migration from the source with inserts, updates, and deletes all demonstrated.
- **Contract:** versioned schema enforced before the silver zone; violations quarantined with machine-readable reasons.
- **Observability:** lag metric, five-minute alert, and a captured firing.
- **Cutover:** reconciliation report comparing batch and CDC outputs over a full window; documented rollback.

Reference Architecture

Figure 11.1 is the reference; at lab scale, Azure SQL Database stands in for Oracle and the Azure IR stands in for the SHIR pair.

Implementation Steps

1. Create DMS and the online task (Listing 11.2); verify initial load and streaming.
2. Implement the contract gate and quarantine; induce and fix a violation end-to-end.
3. Apply deltas to the warehouse with merge semantics including deletes.
4. Wire the lag alert; trigger it deliberately.
5. Produce the reconciliation report and the rollback note.

Deliverables

- Task definitions, contract files, gate code, and the merge procedure in the repo.
- Evidence pack: quarantined violations, lag alert firing, reconciliation output.
- Runbook: reprocess-from-quarantine and rollback-to-batch procedures.

Acceptance Criteria

- A source insert, update, and delete each appear correctly in the warehouse within the SLO.
- No malformed record reaches the warehouse without a quarantine record.
- The reconciliation report shows zero unexplained differences for a full window.

Stretch Goals

- Replay a 24-hour quarantine backlog through a corrected contract version.
- Replace the polling merge with an Event Grid-triggered apply.
- Add a chaos drill: kill the consumer for 30 minutes, then measure catch-up time against the SLO.

Delta Lake Medallion Architecture and Pipeline Orchestration

Learning Objectives

- Design bronze, silver, and gold medallion zones with explicit quality contracts between them.
- Apply Delta Lake operations—MERGE upserts, time travel, OPTIMIZE, VACUUM—as production pipeline primitives.
- Compose advanced ADF control flow: ForEach, If Condition, Until, and Execute Pipeline.
- Trigger pipelines from Event Grid blob events with correct folder and file parameter plumbing.
- Position Managed Airflow in ADF for teams with existing DAG investments.
- Use Azure Logic Apps for failure notification and business-process automation around pipelines.
- Design a self-healing orchestration layer for a 15-system banking reconciliation, and complete the Part III milestone capstone.

Cross-Cloud Equivalence

The medallion pattern and pipeline orchestration are cloud-agnostic disciplines. Bronze/silver/gold is the same idea whether the lake is S3, ADLS, or GCS, and every platform needs a scheduler with dependency graphs and failure semantics.

Capability	AWS	Azure	GCP
Medallion zones	S3 raw/curated + Glue	ADLS Gen2 + Delta Lake	GCS + BigLake
Managed Airflow	MWAA	Managed Airflow in ADF	Cloud Composer
Control-flow orchestration	Step Functions	ADF pipeline activities	Workflows
Event trigger	EventBridge → Lambda/Glue	Event Grid → ADF trigger	Eventarc → Workflows
Notification workflows	SNS / Step Functions	Logic Apps	Cloud Functions + Workflows

Fabric: OneLake medallion + pipelines; **Snowflake:** tasks/streams DAGs; **MongoDB:** Atlas Triggers + Charts refresh.

Airflow’s DAG model—tasks, dependencies, retries, backfills—is itself the cross-cloud skill: MWAA, Cloud Composer, and Managed Airflow in ADF all execute the same Python DAG

files [94].

Key Terms

Medallion architecture: the bronze (raw fidelity), silver (validated, conformed), gold (business-ready) zone pattern over Delta Lake. **MERGE:** Delta’s upsert primitive matching source and target rows for insert/update/delete. **Control flow:** ADF activities—ForEach, Until, If Condition, Execute Pipeline—that compose pipelines. **Managed Airflow:** Apache Airflow hosted inside ADF, running standard DAG files. **Self-healing:** orchestration that detects, retries, reroutes, and escalates failures without human intervention for known failure modes.

12.1 Week 12 Overview: From Moving Data to Operating Data Movement

Chapters 9–11 built the moving parts: ADF pipelines, integration runtimes, and Spark compute. Week 12 assembles them into an *operated system*: pipelines that trigger on events, retry known-transient failures, route poison data aside, notify humans only for novel failures, and leave an audit trail. The medallion architecture gives the system its data-quality semantics; the orchestration layer gives it its failure semantics. Together they are the difference between a demo and a platform.

Definition

The **medallion architecture** organizes a lakehouse into bronze (raw, append-only, full fidelity), silver (validated, deduplicated, conformed), and gold (aggregated, business-modeled) zones. Each hop is a contract: silver promises schema validity; gold promises business semantics.

12.2 Monday: Medallion Zones as Quality Contracts

12.2.1 Bronze: Fidelity Above All

Bronze stores what the source said, when it said it, and how it arrived: raw payload plus ingestion metadata (`_ingested_at`, `_source_file`, `_batch_id`). No joins, no cleaning, no schema changes. When a silver bug is discovered six months later, bronze is what makes reprocessing possible. Append-only Delta tables with generous time-travel retention are the implementation.

12.2.2 Silver: Validated and Conformed

Silver applies the contract: schema enforced, nulls handled, duplicates removed, timestamps normalized, keys conformed across sources. The Chapter 11 contract gate feeds this zone. Silver is where **MERGE** upserts live—idempotent by natural key, so a re-run repairs rather than duplicates:

```
1 MERGE INTO silver.orders AS tgt
2 USING staging.orders_batch AS src
3   ON tgt.order_id = src.order_id
4 WHEN MATCHED AND src.updated_at > tgt.updated_at THEN
5   UPDATE SET *
6 WHEN NOT MATCHED THEN
7   INSERT *
```

```

8 WHEN NOT MATCHED BY SOURCE AND tgt._batch_id <> src._batch_id THEN
9   DELETE; -- optional: propagate deletes explicitly, per your delete semantics

```

Listing 12.1: Idempotent silver upsert: the re-run-safe MERGE.

12.2.3 Gold: Business-Modeled

Gold is the Chapter 5 star schema and the Chapter 7 serving views: declared grain, conformed dimensions, SCD2 history, aggregation tables. Gold tables are rebuilt from silver when logic changes; silver is rebuilt from bronze when contracts change. The reprocessing direction only ever flows backward through the zones—which is why bronze fidelity is non-negotiable.

Common Pitfall

Skipping silver. “Bronze straight to gold” pipelines embed validation and conformance inside business logic, where it cannot be reused and cannot be reprocessed independently. The zone you skip is the reprocessing you cannot do.

12.3 Tuesday: Advanced ADF Control Flow

12.3.1 The Control-Flow Toolkit

ADF pipelines compose with four control activities [60]:

- **ForEach:** iterate a collection (files, tables, partitions) sequentially or in parallel batches—the fan-out for multi-entity loads.
- **If Condition:** branch on an expression—route by file size, row count, or a quality-check result.
- **Until:** loop until a predicate holds with timeout—poll for an upstream file, wait for a DMS task to reach steady state.
- **Execute Pipeline:** call child pipelines with parameters—the decomposition mechanism that keeps pipelines small, testable, and reusable.

The composition pattern for a governed batch: parent pipeline per domain → ForEach over entities → Execute Pipeline (child) per entity → child does validate → transform → audit, with If Condition routing contract failures to quarantine. Parents own fan-out; children own the unit of work; the audit table owns the truth.

12.3.2 Event Grid Triggers

The blob-event trigger starts a pipeline when a blob lands in a watched path, passing `folderPath` and `fileName` as parameters [81]:

```

1 {
2   "name": "trg_blob_created",
3   "properties": {
4     "type": "BlobEventsTrigger",
5     "typeProperties": {
6       "blobPathBeginsWith": "/raw/incoming/",
7       "ignoreEmptyBlobs": true,
8       "events": ["Microsoft.Storage.BlobCreated"]
9     }
10  }
11 }

```

Listing 12.2: Blob-event trigger: fire only on completed uploads to the incoming path.

Common Pitfall

Triggering on partial uploads. A multi-part upload can emit `BlobCreated` before the file is complete. Have producers write to a temp path and rename (rename is atomic in ADLS Gen2) so only complete files appear under `/raw/incoming/`.

12.4 Wednesday: Managed Airflow and Logic Apps

12.4.1 Managed Airflow in ADF

For teams with existing Airflow DAGs—or a deliberate multi-cloud posture—ADF hosts Managed Airflow: standard Python DAG files on Microsoft-operated Airflow infrastructure, integrated with the factory’s identity and monitoring [94]. The decision is portfolio-shaped: pure-Azure greenfield pipelines favor ADF’s visual model; estates with hundreds of existing DAGs, or teams hired for Airflow skills, favor Managed Airflow. The two coexist; what must not coexist is *two orchestrators scheduling the same table*.

12.4.2 Logic Apps: The Notification and Process Layer

Azure Logic Apps is the low-code workflow service with hundreds of connectors—Teams, Outlook, ServiceNow, SAP [82]. In a data platform it plays one role supremely: the **human-facing edge of pipeline failures**. A pipeline’s on-failure path calls a Logic App webhook; the Logic App posts an adaptive Teams card with the run id, the failed activity, and buttons linking to the run and the runbook. Richer than an email, cheaper than an incident tool, and extensible into ticket creation later.

Tip

Route *all* pipeline notifications through one Logic App “router” workflow that enriches and dispatches. When the on-call tool changes, you edit one workflow, not two hundred pipelines.

12.5 Thursday Hands-on Project: Event-Driven Databricks Orchestration with Failure Notification

Build the canonical Part III pipeline: an Event Grid trigger fires on blob arrival, ADF runs a Databricks notebook, and a Logic App messages Teams on failure.

12.5.1 Wire the Trigger

Enable the Event Grid system topic on the storage account; create the ADF trigger from Listing 12.2; map `@triggerBody().folderPath` and `@triggerBody().fileName` into pipeline parameters. Verify with a test upload that the pipeline fires with the file’s identity in its parameters.

12.5.2 Run the Notebook

Add a Databricks Notebook activity pointing at a notebook that receives `folder_path/file_name` via base parameters, validates the contract (Chapter 11), merges into silver (Listing 12.1), and exits non-zero on hard failure. The notebook must be idempotent: merge on `order_id`, never blind append.

12.5.3 Close the Failure Loop

Create the Logic App: HTTP trigger → parse ADF error payload → post Teams adaptive card. Add it as the pipeline’s on-failure webhook activity. Test by uploading a contract-violating file: expect a quarantine record (soft failure, no page) and then a corrupted file that fails the notebook outright: expect the Teams card with a working link to the failed run.

12.6 Friday Use Case: Self-Healing Orchestration for Banking Reconciliation

A bank reconciles fifteen systems nightly: core banking, cards, payments, ledgers, and eleven feeds. Files arrive at different times, occasionally late, occasionally malformed. Today’s cron-driven chain fails opaquely and pages humans for transient faults.

12.6.1 The Self-Healing Design

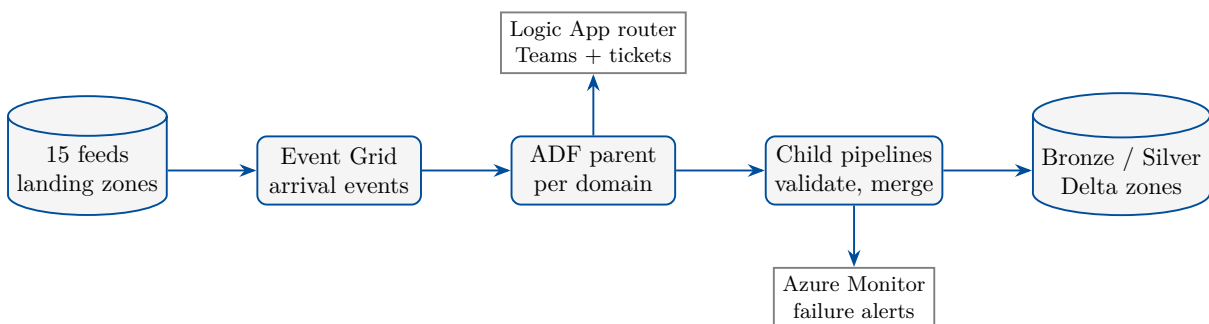


Figure 12.1: Self-healing reconciliation: event-driven fan-out, isolated child pipelines, quarantine routing, and a single notification router.

Self-healing here means five concrete behaviors. (1) *Arrival-driven*: Event Grid triggers replace the schedule; a late feed starts when it arrives, and an Until activity with a deadline covers genuinely missing feeds. (2) *Isolated*: each feed runs in its own child pipeline, so feed 7’s failure never blocks feeds 1–6. (3) *Retry-aware*: transient activities retry with backoff inside Until loops; only exhausted retries escalate. (4) *Data-aware*: contract failures quarantine without paging; structural failures page through the Logic App router. (5) *Auditable*: every child writes the audit table, and the morning reconciliation report is generated from it—the report *is* the health check.

12.6.2 What Self-Healing Is Not

It is not retrying everything forever, and it is not suppressing pages. The design rule: automate the response to *known* failure modes, escalate *novel* ones immediately, and review the quarantine and retry logs weekly so recurring novel failures graduate into known ones.

12.7 Architectural Decision Record Template

Per orchestration domain record: trigger type and arrival SLA, parent/child decomposition, retry policy per activity class, quarantine path and reprocessing owner, notification routing, and the audit schema. For Airflow adoption, record the coexistence boundary: which orchestrator owns which tables.

12.8 Operational Deep Dive: Monitoring the Orchestrator

Send factory diagnostics (PipelineRuns, TriggerRuns, ActivityRuns) to Log Analytics and build three alert classes: **failure alerts** (any pipeline failure, severity by domain criticality), **silence alerts** (expected run missing by deadline—the failure that produces no error), and **drift alerts** (duration or row counts outside the trailing band) [21]. Silence alerts are the ones teams omit and the ones that catch the worst incidents: the trigger that stopped firing alerts on nothing at all.

12.9 Troubleshooting Patterns

Pipeline fires twice per file → two triggers or two subscriptions on the same topic. Trigger never fires → not published, or `blobPathBeginsWith` mismatch (paths are case-sensitive). ForEach slower than expected → sequential default; check the batch count setting. Child pipeline parameter empty → trigger parameter mapping lost in a refactor; the artifact JSON is the source of truth, not the canvas.

12.10 Week 12 Lab Rubric

Criterion	Meets expectation	Needs improvement
Triggering	Event-driven with correct parameter plumbing; partial-upload safe	Polling schedule; partial-file race
Failure design	Quarantine for data faults, page for structural faults, retries bounded	Everything pages, or nothing does
Idempotency	Re-run of the same file changes nothing	Duplicate rows on replay
Auditability	Audit table answers “what ran, what moved, what failed”	Portal screenshots as the only evidence

Table 12.1: Week 12 rubric for the orchestration deliverables.

12.11 Interview Q&A

12.11.1 Concepts and Trade-offs

- Q: When would you use an Until activity instead of a ForEach?**
 A: Until waits on a predicate with a timeout (poll for a file or task state); ForEach iterates a known collection. Waiting is Until’s job.
- Q: What problem does Managed Airflow in ADF solve?**
 A: It runs existing Airflow DAG estates on managed infrastructure with Azure identity integration—no Airflow operations burden, no rewrite.
- Q: Why route failure notifications through a Logic App instead of plain email?**
 A: Structured payloads become rich Teams cards with run links and runbook buttons, and one router workflow centralizes the notification logic.
- Q: What makes an orchestration layer self-healing?**
 A: Arrival-driven triggers, isolated units of work, bounded retries for known faults, quarantine for data faults, and audit-driven reconciliation—with novel failures still paging

humans.

12.12 Further Reading

The ADF triggers and execution documentation [60], the Airflow project [94], and the Logic Apps overview [82] are the primary sources. The Delta Lake paper [1] underpins the medallion zones, and the Data Engineer Handbook [3] collects reconciliation-pattern case studies.

Key Takeaways

- Bronze keeps fidelity, silver keeps validity, gold keeps business meaning; reprocessing flows backward only.
- MERGE by natural key makes silver loads idempotent; blind appends make replays destructive.
- ForEach fans out, Until waits, If routes, Execute Pipeline decomposes.
- Event Grid triggers replace polling; producers must rename-commit so triggers see only complete files.
- Managed Airflow serves DAG estates; Logic Apps serve the human edge of failures.
- Self-healing automates known failure modes and escalates novel ones; the audit table is the health check.

12.13 Exercises

1. **(Conceptual)** Why is bronze append-only, and what reprocessing scenarios depend on it?
2. **(Conceptual)** Explain the double-fire and partial-upload hazards of blob-event triggers and their fixes.
3. **(Conceptual)** Why should one Logic App router own all pipeline notifications?
4. **(Hands-on)** Complete the Thursday project; capture the Teams card and the quarantine record from the two failure tests.
5. **(Hands-on)** Add a ForEach fan-out over five entity folders with parallelism 5; measure wall-clock versus sequential.
6. **(Hands-on)** Write the silence alert (expected run missing by 07:00) as a Log Analytics query and test it.
7. **(Design)** Decompose the 15-feed banking estate into parents and children; define the audit schema.
8. **(Design)** Write the ADR for Managed Airflow versus ADF pipelines for a team migrating 200 DAGs.
9. **(Design)** Design the retry policy matrix: which activity classes retry, how many times, with what backoff.
10. **(Interview)** Prepare a two-minute answer to “a feed was late and nobody noticed until the board meeting—design that away.”

12.14 Milestone 3 Capstone: Automated Batch ELT Pipeline

Capstone Project

Mission: Assemble Part III: an Event Grid-triggered ADF pipeline fires on file upload, runs a Databricks notebook that validates contracts, runs data-quality checks, and engineers ML features, quarantines failures, and bulk-loads the output into a Synapse dedicated pool with COPY—all observable, all idempotent.

Estimated time: 8–10 hours.

What you will practice:

- Event-driven triggering with rename-commit producers.
- Contract validation, quality checks, and quarantine in one notebook.
- COPY with managed identity into a hash-distributed table.
- Alerting, audit, and cost estimation across ADF, Databricks, and Synapse.

Scenario

The retailer from Part II wants near-instant ingestion: when a store’s nightly file lands, quality checks and feature engineering run immediately and the warehouse reflects the data within minutes. Bad files must be quarantined with reasons; the whole flow must survive a rerun without duplicating a single row.

Requirements

- **Trigger:** blob-event trigger on `/raw/incoming/` with folder/file parameters; producers rename-commit.
- **Processing:** Databricks notebook with contract gate, at least three data-quality checks with a demonstrated failing case, and feature engineering into a curated Delta table.
- **Load:** ADF Copy activity with managed identity into a hash-dedicated Synapse table; merge-on-file-id idempotency.
- **Operations:** failure notification via Logic App; ADF diagnostic logs in Log Analytics; one alert rule with evidence.
- **Cost:** monthly estimate across ADF activities, Databricks DBUs, and Synapse DWU-hours with two optimizations.

Reference Architecture

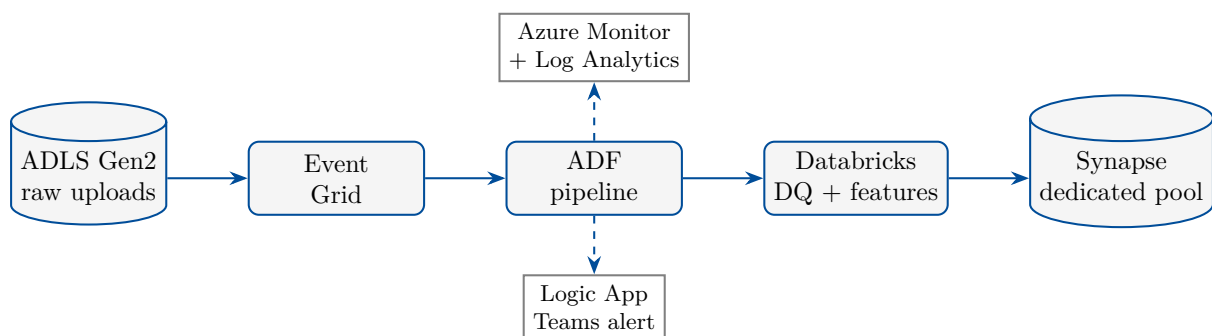


Figure 12.2: Milestone 3 architecture: event-driven ELT from lake arrival to warehouse load with quality gates and notifications.

Implementation Steps

1. Create the Event Grid system topic and the ADF blob-event trigger.
2. Write the Databricks notebook: schema check → quarantine-or-transform → quality checks → curated Delta.
3. Add the COPY load into the hash-distributed target; make the notebook merge idempotent on file id.
4. Wire the Logic App failure path and the Log Analytics alert.
5. Test the happy path, a poison file, and a deliberate rerun; produce the cost estimate.

Deliverables

- Repo with artifacts, notebook, and a README that runs end-to-end from a clean environment.
- Architecture diagram and the audit/alert evidence pack.
- Cost estimate with two documented optimization decisions.

Acceptance Criteria

- Uploading a file starts the pipeline with no manual step; a bad file lands in quarantine with a reason.
- Re-running the same file leaves warehouse counts unchanged.
- All connections use managed identities; no secrets in any artifact.

Stretch Goals

- Add the silence alert for a feed that never arrives.
- Replace the COPY step with a serverless CETAS materialization and compare cost at your volume.
- Rebuild the orchestration as a Managed Airflow DAG and document the trade-offs you experienced.

Part IV

Streaming and Real-Time Analytics

Event Ingestion at Scale: Event Hubs and IoT Hub

Learning Objectives

- Distinguish message queues from event streams, and choose between Azure Service Bus and Event Hubs for a given workload.
- Design an Event Hubs partition strategy, size throughput units, and explain why partition count caps consumer parallelism.
- Implement producer-side idempotency with unique event identifiers and consumer-side checkpointing with at-least-once semantics.
- Configure consumer groups so independent readers share one stream without interfering.
- Enable Event Hubs Capture for automatic batched delivery to ADLS Gen2 and consume Event Hubs through its Kafka-compatible endpoint.
- Position IoT Hub for device management scenarios versus Event Hubs for pure telemetry firehoses.
- Architect a global telemetry ingestion point for five million mobile game clients partitioned by user ID.

Cross-Cloud Equivalence

The partitioned, replayable event log is a cross-cloud primitive. Event Hubs' model—partitions, consumer groups, offsets, retention—is deliberately Kafka-shaped, and every cloud offers an equivalent.

Capability	AWS	Azure	GCP
Event streaming	Kinesis Data Streams / MSK	Event Hubs (+ Kafka endpoint)	Pub/Sub / Managed Kafka
Message queue	SQS / Amazon MQ	Service Bus queues and topics	Pub/Sub (pull)
Device ingestion	IoT Core	IoT Hub	(partner; IoT Core retired)
Stream-to-lake export	Kinesis Firehose	Event Hubs Capture	Pub/Sub to GCS subscriptions
Partition key	Partition key per record	Partition key per event	Ordering keys
Consumer isolation	Enhanced fan-out	Consumer groups	Subscriptions

Fabric: eventstreams into OneLake; **Snowflake:** Snowpipe Streaming; **MongoDB:** change streams as a source.

Because Event Hubs speaks the Kafka protocol, producer and consumer skills transfer to Amazon MSK, Confluent, and self-managed Kafka clusters with only endpoint and credential changes [95].

Key Terms

Partition: an ordered, append-only log shard; the unit of parallelism and ordering in Event Hubs. **Throughput unit (TU):** the capacity slice of a standard Event Hubs namespace (1 MB/s or 1,000 events/s ingress, 2 MB/s egress per TU). **Consumer group:** a named, independent view over a hub's partitions with its own offsets. **Checkpoint:** a consumer's persisted position, enabling resume after restart. **Capture:** automatic batched write of hub traffic to ADLS Gen2 or Blob Storage in Avro format.

13.1 Week 13 Overview: Unbounded Data Arrives

Part IV changes the fundamental assumption: data no longer arrives in files on a schedule—it arrives continuously, unordered, duplicated, and sometimes very late. The ingestion layer's job is to absorb whatever producers throw at it, durably, and let many independent consumers read at their own pace. Azure's answer splits by intent: **Service Bus** for commanded messages (each message consumed once, by one consumer), **Event Hubs** for observed events (each event retained and replayable by many consumers) [18].

Definition

An **event stream** is an append-only, time-ordered, replayable log of immutable facts. A **message queue** is a work-distribution mechanism where each message is claimed and removed by one consumer. Streams remember; queues forget.

13.2 Monday: Decoupling with Service Bus

13.2.1 Queues and Topics

A Service Bus **queue** delivers each message to exactly one competing consumer—work distribution with built-in load leveling, dead-lettering, and scheduled delivery. A **topic** adds subscriptions: each subscription receives its own copy, filtered by rules—publish/subscribe with per-subscriber curation. Choose Service Bus when the message is a *command* or *task*: process this payment, send this email, provision this account. Sessions add ordered processing per key; transactions add atomic multi-entity operations; duplicate detection suppresses producer retries.

13.2.2 Service Bus vs. Event Hubs

The comparison is semantic, not about scale. Service Bus optimizes for *per-message guarantees*: completion, abandonment, dead-letter, TTL. Event Hubs optimizes for *stream throughput and replay*: partitions, offsets, retention windows, multiple consumer groups. A payment instruction belongs in Service Bus; a clickstream belongs in Event Hubs; a telemetry flood from five million devices belongs in Event Hubs; an order workflow belongs in Service Bus.

Dimension	Service Bus	Event Hubs
Semantics	Message consumed once by one consumer	Event retained; replayable by many readers
Ordering	Sessions per key	Per partition
Throughput target	Thousands of messages/s	Millions of events/s
Delivery model	Pull with settlement (complete/abandon)	Pull by offset; consumer checkpoints
Typical payload	Commands, business transactions	Telemetry, logs, clickstream

Table 13.1: Queues distribute work; streams record history.

13.3 Tuesday: Event Hubs—Partitions, Consumer Groups, Throughput

13.3.1 Partitions: Parallelism and Ordering

Every event lands in exactly one partition, chosen by hash of the partition key (or round-robin when no key is set). Within a partition, order is guaranteed; across partitions, it is not. The design consequence: *choose the partition key to be the entity whose ordering you need*—`user_id`, `card_id`, `device_id`—and choose the partition count for peak consumer parallelism, because a partition can be read by only one consumer instance per consumer group at a time. Partition count is set at creation and effectively immutable, so size for the future peak, not today's.

13.3.2 Throughput Units and Consumer Groups

Standard namespaces sell capacity as throughput units; Premium and Dedicated tiers add reserved capacity, longer retention, and stronger isolation. **Consumer groups** let independent applications—a real-time scorer, a data-lake archiver, a fraud rules engine—each read the full stream at their own offset pace. Create one consumer group per application, and never share one between production and experiments.

13.3.3 At-Least-Once and Idempotency

Event Hubs guarantees at-least-once delivery: retries and replays mean the same event can arrive twice. Producers should attach a unique `eventId`; consumers must be idempotent—deduplicate on sink upserts, never assume uniqueness. Consumers persist **checkpoints** (offsets) to Blob Storage so restarts resume from the last committed position rather than in-memory state.

Common Pitfall

Partitioning by a low-cardinality key. Partitioning game telemetry by platform (iOS/Android) creates two hot partitions and thirty idle ones. Partition by `user_id`: high cardinality, even spread, per-user ordering preserved.

13.4 Wednesday: Capture and the Kafka Endpoint

13.4.1 Capture: The Stream-to-Lake Bridge

Event Hubs Capture writes stream traffic to ADLS Gen2 automatically, batched by time or size window, in Avro format, into a folder pattern you define (`namespace/hub/year/month/day/hour/minute`).

It converts the streaming estate into a batch-ingestible bronze zone with zero consumer code—the stream equivalent of the Chapter 12 landing zone. Downstream, the Chapter 9 Databricks jobs or Chapter 7 serverless queries read the captured files exactly like any lake data.

13.4.2 Kafka Compatibility

Event Hubs exposes an Apache Kafka protocol endpoint: existing Kafka producers and consumers—including Spark Structured Streaming’s Kafka source in Chapter 15—connect by changing bootstrap server and credentials, not code [95]. The practical notes: use the `$ConnectionString` username pattern with SASL over SSL, keep secrets in Key Vault, and remember that Kafka consumer groups map to Event Hubs consumer groups.

Tip

Enable Capture from day one even if nothing consumes the files yet. Storage is cheap; the day you need to replay yesterday’s stream into a new model, Capture is the only way that history exists.

13.5 Thursday Hands-on Project: IoT Telemetry Producer with Capture

Create an Event Hub, stream mock temperature telemetry from a Python asyncio producer, and verify captured batches appearing in ADLS Gen2.

13.5.1 Create the Namespace, Hub, and Capture

```

1 $rg = "rg-de-azure-book-week13"
2 az group create --name $rg --location eastus
3 $ns = "ehn-week13-001"
4
5 az eventhubs namespace create --name $ns --resource-group $rg '
6   --location eastus --sku Standard --capacity 2 --enable-auto-inflate '
7   --maximum-throughput-units 10
8
9 az eventhubs eventhub create --name iot-telemetry --namespace-name $ns '
10  --resource-group $rg --partition-count 8 --message-retention-in-days 3
11
12 az eventhubs eventhub update --name iot-telemetry --namespace-name $ns '
13  --resource-group $rg --enable-capture true '
14  --capture-interval 300 --capture-size-limit 314572800 '
15  --destination-name EventHubArchive.AzureBlockBlob '
16  --storage-account "/subscriptions/<sub>/resourceGroups/$rg/providers/
17    Microsoft.Storage/storageAccounts/<adls>" '
18  --blob-container telemetry-capture '
19  --archive-name-format "{Namespace}/{EventHub}/{Year}_{Month}_{Day}/{Hour}_{
20    Minute}"

```

Listing 13.1: Namespace, hub with 8 partitions, and Capture to the lake.

13.5.2 The Producer

```

1 import asyncio, json, random, time, os
2 from azure.eventhub.aio import EventHubProducerClient
3 from azure.eventhub import EventData
4
5 CONN_STR = os.environ["EVENTHUB_CONN_STR"] # fetched from Key Vault
6 HUB_NAME = "iot-telemetry"

```

```

7
8 async def run():
9     producer = EventHubProducerClient.from_connection_string(
10         conn_str=CONN_STR, eventhub_name=HUB_NAME)
11     async with producer:
12         while True:
13             batch = await producer.create_batch()
14             for sensor_id in range(1, 6):
15                 event = {
16                     "deviceId": f"sensor-{sensor_id}",
17                     "eventId": f"{sensor_id}-{int(time.time())}", # dedupe key
18                     "tempC": round(random.uniform(-20, 45), 2),
19                     "ts": time.time(),
20                 }
21                 batch.add(EventData(json.dumps(event)))
22             await producer.send_batch(batch)
23             await asyncio.sleep(1)
24
25 asyncio.run(run())

```

Listing 13.2: Async producer with per-event dedupe keys; connection string from Key Vault, never hardcoded.

13.5.3 Verify

Watch the namespace metrics (incoming messages, throttled requests) in the portal; after the capture interval, confirm Avro files under `telemetry-capture/` in the lake; read one with `fastavro` or the Chapter 7 serverless pool and confirm the event bodies match what the producer sent.

13.6 Friday Use Case: Telemetry for Five Million Mobile Games

A games publisher needs one global ingestion endpoint for five million concurrent mobile clients emitting session, purchase, and error events, feeding both a real-time engagement scorer and the analytics lake.

13.6.1 Design

A Premium Event Hubs namespace in the primary region, geo-paired for disaster recovery; partition count sized at 64 so the future consumer fleet (target: 32 parallel scorer instances with headroom) is never partition-capped. Partition key `user_id`: per-player ordering for session reconstruction, high cardinality for even spread. Two consumer groups—`rt-scoring` for the Chapter 15 Structured Streaming job, `lake-archive` for Capture-based archival—plus a third, `fraud-lab`, created later without touching either. Auto-inflate absorbs launch-day spikes; the Client-side SDK batches events and applies backoff on throttling.

13.6.2 The Numbers That Matter

Estimate peak events per second (concurrent clients $\times$ events per client-minute), divide by TU capacity to size the namespace, then sanity-check egress: two consumer groups double egress bytes. The design review should show these calculations—ingestion architectures are sized on arithmetic, not adjectives.

Common Pitfall

Replay storms. A consumer group that checkpoints rarely and crashes often re-reads hours of events, multiplying egress and lag simultaneously. Checkpoint every N seconds or M events, whichever is smaller for the lag SLO.

13.7 Architectural Decision Record Template

Per stream record: partition key and count with the parallelism and ordering justification, TU/Premium sizing arithmetic, consumer groups and their owners, retention window, Capture configuration, and the idempotency contract (eventId semantics) shared by producers and consumers.

13.8 Operational Deep Dive: IoT Hub and Device Identity

When the producers are devices rather than applications, **IoT Hub** adds what Event Hubs lacks: per-device identity and revocation, device twins for configuration state, cloud-to-device messaging, and protocol adaptation (MQTT, AMQP, HTTPS) [19]. IoT Hub's built-in Event Hubs-compatible endpoint feeds the same downstream consumers. The rule: telemetry from *applications* goes to Event Hubs; telemetry from *devices you must identify, configure, and command* goes through IoT Hub.

13.9 Troubleshooting Patterns

Consumers see duplicates → expected at-least-once behavior; check sink deduplication. One consumer instance idle → more instances than partitions. Throttled requests → TU ceiling; enable auto-inflate or repartition traffic. Capture files missing → storage RBAC for the namespace's managed identity, or capture interval/size windows not yet met. Offset errors after a consumer rewrite → the checkpoint store outlived the code; version checkpoint containers per consumer deployment.

13.10 Week 13 Lab Rubric

Criterion	Meets expectation	Needs improvement
Partitioning	Key and count justified with ordering/parallelism arithmetic	Defaults accepted without reasoning
Idempotency	Unique eventIds; consumer dedupe demonstrated	Uniqueness assumed
Capture	Files verified in the lake and read back	Enabled but never validated
Security	Connection string from Key Vault; per-app consumer groups	Secrets in code; shared consumer group

Table 13.2: Week 13 rubric for the ingestion deliverables.

13.11 Interview Q&A

13.11.1 Concepts and Trade-offs

1. **Q: When do you choose Service Bus over Event Hubs?**

A: When messages are commands needing per-message settlement—complete, abandon, dead-letter—rather than a replayable event log.

2. **Q: Why does partition count cap consumer parallelism?**

A: One partition is read by at most one instance per consumer group; more instances than partitions leaves instances idle.

3. **Q: What does Event Hubs Capture automate?**

A: Batched delivery of stream traffic to ADLS Gen2 as Avro, on time/size windows, with no consumer code.

4. **Q: Why attach a unique eventId to every message?**

A: At-least-once delivery means duplicates; the eventId is the dedupe key that makes sink upserts idempotent.

13.12 Further Reading

The Event Hubs documentation [18] and the Kafka project [95] cover the stream model and protocol compatibility; IoT Hub’s documentation [19] covers device identity. Chapter 14 (Stream Analytics) and Chapter 15 (Structured Streaming) consume what this chapter ingests.

Key Takeaways

- Queues distribute commands; streams record replayable facts. Choose Service Bus or Event Hubs by semantics, not by scale.
- Partition keys carry ordering; partition counts carry parallelism—and cannot practically be changed later.
- One consumer group per application; never share between production and experiments.
- Delivery is at-least-once: unique eventIds, idempotent sinks, durable checkpoints.
- Capture turns the stream into a lake bronze zone for free; enable it on day one.
- The Kafka endpoint makes Event Hubs a drop-in for Kafka tooling, including Chapter 15’s Spark jobs.

13.13 Exercises

1. **(Conceptual)** Explain why a payment instruction belongs in Service Bus and a click-stream in Event Hubs.
2. **(Conceptual)** What happens when a consumer group has more instances than partitions? More partitions than instances?
3. **(Conceptual)** Why is Event Hubs delivery at-least-once, and what does that obligate consumers to do?
4. **(Hands-on)** Complete the Thursday lab; verify captured Avro matches produced events.

5. **(Hands-on)** Add a second consumer group and demonstrate independent replay from an earlier offset.
6. **(Hands-on)** Induce throttling by capping TUs, observe producer backoff, and document the auto-inflate response.
7. **(Design)** Size the namespace for the games publisher: show events/s, TU, and egress arithmetic.
8. **(Design)** Write the ADR for 64 partitions at creation, including the cost of over-provisioning.
9. **(Design)** Design the checkpoint cadence for a consumer with a two-minute lag SLO.
10. **(Interview)** Prepare a two-minute answer to “why did our real-time dashboard show duplicate purchases?”

13.14 Capstone Project: Partitioned Telemetry Ingestion with Capture

Capstone Project

Mission: Deliver a production-shaped ingestion point: an Event Hub partitioned and sized with shown arithmetic, an idempotent Python producer, Capture verified end-to-end into ADLS Gen2, two isolated consumer groups, and a lag/throughput monitoring story.

Estimated time: 3–4 hours.

What you will practice:

- Partition-key and partition-count design with justification.
- Producer batching, dedupe keys, and secret hygiene.
- Capture verification and lake replay.
- Consumer-group isolation and checkpoint discipline.

Scenario

The games publisher’s pilot title launches next quarter: 50,000 expected concurrent clients, five events per client-minute, a real-time scorer and a lake archive as first consumers, and a data-science replay requirement after any model change.

Requirements

- **Stream:** hub with documented partition key/count and TU sizing arithmetic; auto-inflate enabled.
- **Producer:** batched asyncio producer with unique eventIds and Key Vault-sourced credentials.
- **Archive:** Capture into the bronze zone with a date-partitioned folder format; read-back proof.
- **Consumers:** two consumer groups with independent checkpoints; a demonstrated replay from history.

Reference Architecture

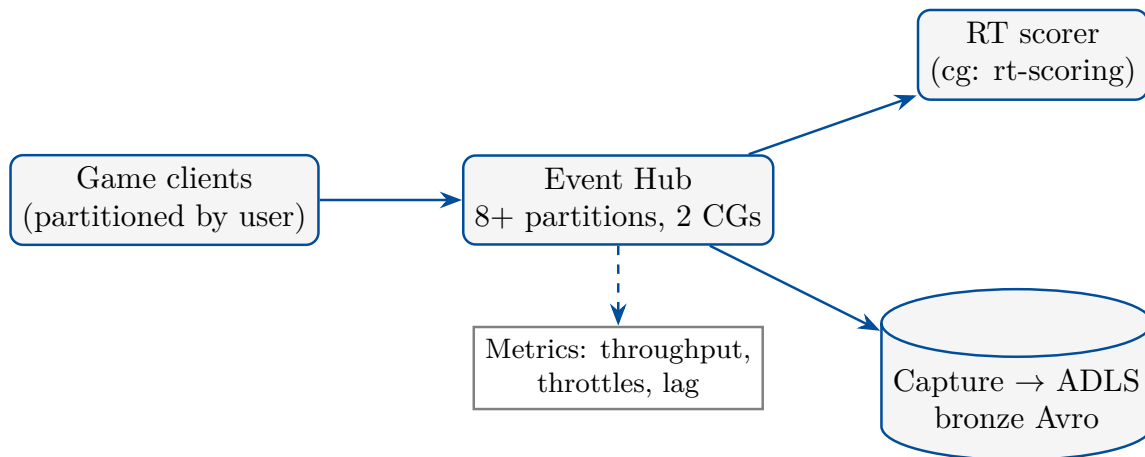


Figure 13.1: Capstone architecture: partitioned hub with isolated consumer groups, real-time scoring, and lake capture.

Implementation Steps

1. Create the namespace, hub, and Capture (Listing 13.1).
2. Run the producer (Listing 13.2) at pilot volume; record metrics.
3. Verify Capture read-back and the date-partitioned layout.
4. Add the second consumer group; demonstrate replay without affecting the first.
5. Write the sizing ADR with the arithmetic.

Deliverables

- CLI scripts, producer code, and the sizing ADR in the repo.
- Metrics screenshots: throughput, throttles, and capture files in the lake.
- Replay demonstration notes for the data-science scenario.

Acceptance Criteria

- No event processed twice by a consumer despite forced retries.
- A consumer restart resumes within the lag SLO from its checkpoint.
- Captured files reconcile with producer counts for a five-minute window.

Stretch Goals

- Connect a Kafka-protocol consumer to the hub and document the configuration deltas.
- Chaos test: kill a consumer instance mid-partition and measure recovery.
- Compare Capture cost against a self-written archiver consumer at your event volume.

Real-Time Processing with Azure Stream Analytics

Learning Objectives

- Explain the Azure Stream Analytics (ASA) architecture: inputs, the SQL-like query, streaming units, and outputs.
- Write ASA queries using tumbling, hopping, sliding, and session windows, and choose the right window for a business question.
- Distinguish event time from arrival time and control it with `TIMESTAMP BY`.
- Configure late-arrival and out-of-order tolerance windows with explicit adjust-versus-drop policies.
- Sink windowed aggregates to Azure SQL Database and Power BI with correct delivery semantics.
- Monitor watermark delay and input-event metrics so silent input stalls are distinguishable from zero traffic.
- Diagnose a dashboard whose daily aggregates are skewed by offline mobile devices uploading late.

Cross-Cloud Equivalence

Managed SQL-over-streams exists on every cloud, and the windowing vocabulary—tumbling, hopping, sliding, session—is identical everywhere; only the engine and billing unit differ.

Capability	AWS	Azure	GCP
Managed SQL	Managed Flink (Kinesis Data Analytics)	Azure Stream Analytics	Dataflow (Beam SQL)
Billing unit	Kinesis Processing Units	Streaming Units (SUs)	Dataflow worker resources
Event source	Kinesis / MSK	Event Hubs / IoT Hub	Pub/Sub
Late-data control	Watermarks in Flink	Late/out-of-order policies	Watermarks + allowed lateness
Stream to warehouse	Kinesis Firehose → Redshift	ASA output → Synapse/SQL	Dataflow → BigQuery

Fabric: eventstreams + Activator; **Snowflake:** Snowpipe Streaming + dynamic tables; **MongoDB:** Atlas Stream Processing.

When a use case outgrows ASA—complex state, stream-stream joins at scale, custom

code—Chapter 15’s Spark Structured Streaming is the escalation path, with the same windowing concepts at a lower level of abstraction.

Key Terms

Streaming Unit (SU): ASA’s blended CPU/memory capacity slice; jobs are sized in SUs and monitored by SU utilization. **Event time:** when the event happened (producer timestamp); **arrival time:** when it reached the hub. **Tumbling window:** fixed-size, non-overlapping, back-to-back time buckets. **Watermark:** the engine’s estimate of event-time progress; the mechanism that decides when a window can close. **Late arrival:** an event whose event time is older than the configured tolerance when it arrives.

14.1 Week 14 Overview: SQL Against the Firehose

Chapter 13 landed unbounded data in Event Hubs. This chapter processes it in motion. Azure Stream Analytics is the serverless option: you write a SQL-like query over streaming inputs, ASA runs it continuously on managed capacity, and results flow to outputs—SQL Database, Synapse, ADLS, Cosmos DB, Power BI—with at-least-once delivery [70]. No cluster, no code deployment, no state management on your side. The price of that simplicity is the boundary of the model: when the logic needs custom state, ML libraries, or multi-stream joins with complex timing, you graduate to Chapter 15.

The conceptual core of the week is time. Streaming correctness is the management of the gap between *when things happened* and *when they were observed*—windows, watermarks, and tolerance policies are the tools.

14.2 Monday: ASA Architecture and Query Language

14.2.1 Inputs, Query, Outputs

An ASA job is three declarations: **inputs** (streaming—Event Hubs, IoT Hub—or *reference data*, slowly changing blobs/tables joined to enrich the stream), the **query** (a SELECT statement executed continuously), and **outputs** (sinks with their own delivery semantics). The query language looks like T-SQL with streaming extensions: windowing functions, **TIMESTAMP BY**, and temporal joins. Because the engine is shared and serverless, capacity is expressed in **Streaming Units**; the operational skill is reading SU utilization—sustained values above 80% mean add SUs or reduce per-event work.

14.2.2 A First Query

```

1 SELECT deviceId,
2     System.Timestamp() AS window_end,
3     AVG(tempC) AS avg_temp,
4     COUNT(*) AS readings
5 INTO sql_telemetry_agg
6 FROM iot_hub_input TIMESTAMP BY ts
7 GROUP BY deviceId, TumblingWindow(minute, 5);

```

Listing 14.1: Continuous aggregation from Event Hubs to Azure SQL, in event time.

TIMESTAMP BY ts promotes the producer’s timestamp to the event-time axis; without it, the engine uses arrival time, and every late or replayed event lands in the wrong window.

Common Pitfall

Forgetting `TIMESTAMP BY`. Arrival-time windows make dashboards that look correct in testing (where data arrives instantly) and silently wrong in production (where it does not). Declare event time explicitly, always.

14.3 Tuesday: The Four Windows

14.3.1 Tumbling and Hopping

Tumbling windows partition time into equal non-overlapping buckets: *count clicks per 5 minutes*. Every event belongs to exactly one window. **Hopping** windows overlap: *a 10-minute window computed every 2 minutes*—a moving average that updates smoothly. Same size, smaller hop, more overlapping windows per event.

14.3.2 Sliding and Session

Sliding windows emit on *content change*, not on a schedule: `SlidingWindow(minute, 10)` produces output whenever the set of events in the trailing 10 minutes changes—the right tool for detectors (“any card used in two cities within 10 minutes,” Chapter 16’s capstone). **Session** windows group events separated by less than a timeout gap—user sessions from clickstreams—with a maximum duration cap.

Window	Emits	Canonical question
Tumbling	On a fixed cadence, disjoint buckets	“KPIs per 5 minutes”
Hopping	Overlapping buckets on a cadence	“Moving 10-minute average every 2 minutes”
Sliding	When window content changes	“Alert when X happens within Y minutes”
Session	When activity gaps exceed a timeout	“Group this user’s activity into sessions”

Table 14.1: Window selection is a statement about the business question, not the data.

Tip

If the requirement says “every N minutes,” reach for tumbling or hopping. If it says “whenever the condition becomes true,” reach for sliding. If it says “a burst of user activity,” reach for session.

14.4 Wednesday: Late and Out-of-Order Events

14.4.1 The Two Disorder Knobs

ASA exposes two policies [93]:

- **Out-of-order tolerance:** how far apart event times may be within the arriving stream. Events outside it are *adjusted* (clamped to the tolerance edge) or *dropped*, per policy.
- **Late-arrival tolerance:** how much older than the current watermark an event may be when it arrives. Same adjust-or-drop choice.

Adjusted events are re-timestamped and counted in a tolerable window; dropped events vanish—counted in metrics, invisible in results. Defaults (seconds) suit connected services; mobile telemetry, where devices buffer offline for hours, needs tolerances measured in the business’s reporting cadence and a deliberate decision about what “the daily total” means for data that arrives tomorrow.

14.4.2 The Silent-Stall Problem

A stalled input—no events arriving—advances no watermark, closes no windows, and emits no rows. On the dashboard, a dead producer is indistinguishable from zero traffic. The fix is operational: alert on **Watermark Delay** and **Input Events** metrics so that “no rows” and “no input” are different alerts with different runbooks.

Common Pitfall

Default tolerances on mobile data. The default late-arrival window silently drops offline-device uploads. Daily aggregates then undercount precisely the least-connected users—and the bias is invisible until someone reconciles against the source system.

14.5 Thursday Hands-on Project: Five-Minute Click Aggregates to Azure SQL

Build an ASA job that reads click events from the Chapter 13 hub, counts clicks in five-minute tumbling windows, and writes aggregates to Azure SQL Database.

14.5.1 Create the Job

```
1 $rg = "rg-de-azure-book-week13"
2 az stream-analytics job create --name asa-clicks --resource-group $rg '
3   --location eastus --streaming-units 3 '
4   --transformation name=clicks query="@transformation.sql"
```

Listing 14.2: Streaming job shell; input/output/query wiring via CLI JSON or the portal.

14.5.2 The Query

```
1 SELECT page,
2       System.Timestamp() AS window_end,
3       COUNT(*) AS clicks,
4       COUNT(DISTINCT userId) AS unique_users
5 INTO sql_click_agg
6 FROM eh_clicks TIMESTAMP BY event_ts
7 GROUP BY page, TumblingWindow(minute, 5);
```

Listing 14.3: Tumbling-window click counts, event time from the producer payload.

14.5.3 Test the Time Policies

Send three event classes with the Chapter 13 producer pattern: in-order events (expect clean windows), an event 30 minutes late with a 5-minute tolerance (expect a drop or adjust per your configured policy), and an out-of-order burst. Capture the job’s **LateInputEvents** and **OutOfOrderEvents** metrics as evidence. Then verify the Azure SQL table receives one row per page per window with matching totals.

14.6 Friday Use Case: The Offline-Mobile Dashboard

A media app’s daily-engagement dashboard is wrong by 8–12% every morning and “heals” by afternoon. Root cause: mobile clients buffer events while offline and upload on reconnect; the ASA job’s default 5-second late-arrival tolerance drops them; the overnight aggregate undercounts; late events then trickle into *later* windows, inflating them.

14.6.1 Diagnosis

The reconciliation pattern is the tell: source-system counts exceed dashboard counts at 08:00 and converge by 14:00. Job metrics confirm **LateInputEvents** spiking in the morning reconnect wave. The dashboard is not broken; the *time policy* encodes an assumption (always-connected clients) the product violates.

14.6.2 The Fix

1. **Policy:** raise late-arrival tolerance to the reporting contract—e.g., 24 hours for a daily report—with the adjust policy, and document what “daily” now means (a window that stays open a day).
2. **Two-speed reporting:** keep the real-time dashboard on short tolerances for operational freshness, and compute the *official* daily number from the lake (Capture files, Chapter 13) where late events are simply reprocessed.
3. **Observability:** alert on late-input-event rate so a policy regression is caught in hours, not at month-end.

Note

The general principle outlives ASA: *fast* answers and *correct* answers to the same question are two pipelines with different time semantics. The stream gives you now-approximately; the lake gives you eventually-exactly. Mature platforms publish both, labeled.

14.7 Architectural Decision Record Template

Per ASA job record: event-time field and its producer-side reliability, both tolerance windows with the business reporting contract they implement, adjust-versus-drop choices, SU sizing with utilization evidence, output delivery semantics, and the watermark-delay alert threshold.

14.8 Operational Deep Dive: Sizing and SU Utilization

SU consumption is driven by per-event work: joins against reference data, `COUNT(DISTINCT)`, UDFs, and high-cardinality `GROUP BY`s are the expensive shapes. Scale by adding SUs (6 is the practical per-job minimum for partitioned sources so partitions spread), parallelize by writing the query to be partition-embarrassing (`PARTITION BY PartitionId`), and re-test with realistic volume—SU utilization at 1% of production load tells you nothing.

14.9 Troubleshooting Patterns

No output, no errors → input stall; check **Input Events** and the watermark delay metric before the query. Output drifting from source → late/out-of-order drops; check those metrics against your tolerance design. SU utilization pinned at 100% → query shape (distinct counts, reference

joins) or undersizing. Duplicate rows in SQL output → at-least-once delivery into a table without a natural unique key; add the window end plus dimensions as the key and use merge semantics.

14.10 Week 14 Lab Rubric

Criterion	Meets expectation	Needs improvement
Time semantics	TIMESTAMP BY set; tolerances configured and tested	Arrival time by accident; default tolerances
Window choice	Window type justified against the business question	Window picked by familiarity
Outputs	Sink key design prevents duplicates	At-least-once duplicates visible in SQL
Observability	Watermark and late-event metrics captured as evidence	No metrics; stalls indistinguishable from zero

Table 14.2: Week 14 rubric for the Stream Analytics deliverables.

14.11 Interview Q&A

14.11.1 Concepts and Trade-offs

1. **Q: Contrast tumbling and hopping windows with an example.**
A: Both are fixed-size; tumbling buckets are disjoint (5-minute counts), hopping overlap (10-minute window every 2 minutes for a moving average).
2. **Q: What does the late-arrival tolerance control?**
A: How much older than the watermark an event may be; beyond it the event is adjusted to the tolerance edge or dropped, per policy.
3. **Q: Why does a silent input stall look like zero traffic?**
A: No events advance no watermark and emit no rows—identical to a genuinely empty period; only input/watermark metrics distinguish them.
4. **Q: How do you fix daily aggregates skewed by late mobile uploads?**
A: Raise and document the late-arrival tolerance for the reporting contract, and compute official dailies from the lake where late data reprocesses cleanly.

14.12 Further Reading

The Stream Analytics overview [70] and the windowing reference [93] are the primary sources. Compare against Spark Structured Streaming’s watermark model [97] ahead of Chapter 15, where the same concepts reappear with explicit state management.

Key Takeaways

- ASA is serverless SQL-over-streams: inputs, one continuous query, outputs; capacity in Streaming Units.
- Event time (TIMESTAMP BY) is the only honest time axis; arrival time lies under real network conditions.

- Tumbling for periodic KPIs, hopping for moving aggregates, sliding for detectors, session for activity bursts.
- Late/out-of-order tolerances encode a business reporting contract; defaults silently drop offline data.
- Watermark-delay and input-event alerts separate “no input” from “zero traffic.”
- At-least-once outputs need idempotent sink design: natural keys and merges, not blind inserts.

14.13 Exercises

1. **(Conceptual)** Explain why arrival-time windows looked correct in your test harness.
2. **(Conceptual)** Which window type answers “did any device exceed 80°C for more than 3 minutes,” and why?
3. **(Conceptual)** What does the adjust policy do to an event that is 10 minutes late under a 5-minute tolerance?
4. **(Hands-on)** Complete the Thursday project with the three disorder tests and metric evidence.
5. **(Hands-on)** Rewrite the click query as a hopping window and describe how the dashboard changes.
6. **(Hands-on)** Add a session-window query that reconstructs user sessions with a 20-minute gap timeout.
7. **(Design)** Specify the two-speed reporting design for the media app: tolerances, refresh cadences, and labels.
8. **(Design)** Write the ADR for ASA versus Structured Streaming for a rule-expressible-in-SQL fraud detector.
9. **(Design)** Design the SQL sink schema and merge logic that make ASA’s at-least-once output idempotent.
10. **(Interview)** Prepare a two-minute answer to “why do morning dashboards always undercount?”

14.14 Capstone Project: Windowed Engagement Aggregates with Explicit Time Policies

Capstone Project

Mission: Ship a production-shaped ASA job: event-time windowed aggregates from Event Hubs to Azure SQL, explicit late/out-of-order policies tested with induced disorder, idempotent sink design, and watermark-delay alerting that distinguishes stalls from silence.

Estimated time: 3–4 hours.

What you will practice:

- Window selection against a stated business question.
- Tolerance configuration as an implemented reporting contract.
- At-least-once-safe sink design.
- Streaming observability: watermark, late events, SU utilization.

Scenario

The media company from the Friday use case wants the fixed version: a real-time engagement dashboard plus a documented policy for how late mobile data is handled, with evidence that the policy works.

Requirements

- **Query:** five-minute tumbling aggregates with event time from the payload.
- **Policy:** late-arrival and out-of-order tolerances set deliberately, with adjust/drop documented and metric-tested.
- **Sink:** Azure SQL table with a key design that tolerates duplicate delivery.
- **Observability:** watermark-delay alert configured and fired in a stall test.

Reference Architecture

Event Hubs → ASA job (tumbling query, explicit policies) → Azure SQL → Power BI, with Azure Monitor watching watermark delay and late-event rates—the Chapter 16 capstone extends this into fraud detection.

Implementation Steps

1. Create the job, input, output, and query (Listings 14.2–14.3).
2. Configure tolerances; run the three disorder tests; capture metrics.
3. Implement the idempotent sink merge; prove duplicate delivery changes nothing.
4. Wire and fire the watermark-delay alert.

Deliverables

- Job definition JSON, query, sink DDL, and merge procedure.
- Metric evidence for the disorder tests and the stall alert.
- A one-page time-policy document written for the dashboard’s consumers.

Acceptance Criteria

- Induced late events follow the documented policy with metric proof.
- A forced output retry produces no duplicate rows in the sink.
- The stall test pages the alert; zero traffic does not.

Stretch Goals

- Add a reference-data join (device registry) and measure its SU cost.
- Parallelize the query with `PARTITION BY PartitionId` and compare utilization.
- Build the lake-side “official daily” job and reconcile it against the stream dashboard for one week.

Advanced Streaming with Spark Structured Streaming on Databricks

Learning Objectives

- Consume Event Hubs from Spark Structured Streaming through the Kafka-compatible endpoint with secrets from Key Vault.
- Explain micro-batch versus continuous processing and their latency/throughput trade-offs.
- Manage streaming state: watermarks, the RocksDB state-store provider, and why unbounded state kills jobs.
- Deliver end-to-end exactly-once semantics via checkpointing plus an idempotent Delta merge sink.
- Write stream-stream joins with watermark constraints that bound state retention on both sides.
- Apply checkpoint-location discipline across deployments so changed queries never replay or corrupt state.
- Design a real-time HL7 anonymization pipeline from Event Hubs through Databricks into Synapse.

Cross-Cloud Equivalence

Structured Streaming’s model—a stream as an unbounded table, micro-batches, watermarks, checkpoints—maps directly onto the streaming engines of the other platforms.

Capability	AWS	Azure	GCP
Spark streaming host	EMR / Glue streaming	Azure Databricks	Dataproc / Serverless Spark
Stream source	Kinesis / MSK connector	Event Hubs (Kafka endpoint)	Pub/Sub / Kafka connector
State store	RocksDB on EMR	Default or RocksDB provider	RocksDB on Dataproc
Exactly-once sink	Delta/Iceberg merge	Delta Lake merge	BigQuery Storage Write API
Checkpoint storage	S3	ADLS Gen2	GCS

Fabric: Spark structured streaming on OneLake; **Snowflake:** Snowpipe Streaming + tasks; **MongoDB:** Atlas Stream Processing

The Structured Streaming programming guide [97] is platform-neutral reading; everything in this chapter about watermarks and state applies identically on EMR and Dataproc.

Key Terms

Micro-batch: the default execution mode—the stream processed as a sequence of small batch jobs. **Watermark:** a threshold on event time beyond which state for old windows/keys is evicted. **State store:** the per-partition store holding aggregation and join state between micro-batches. **Checkpoint:** the durable directory binding offsets, state, and query plan for exactly-once recovery. **Stream-stream join:** a join of two unbounded streams, kept finite by watermark and time-range constraints.

15.1 Week 15 Overview: When SQL-Over-Streams Is Not Enough

Chapter 14’s Stream Analytics covers rules expressible in SQL over bounded windows. This chapter covers everything else: sessionization with custom logic, ML model scoring in the stream, joins between two live streams, and stateful computations ASA cannot express. Spark Structured Streaming treats a stream as an unbounded table and the query as an incremental computation over it [97]—the same DataFrame API as Chapter 9, extended with triggers, watermarks, and checkpoints.

The price of this power is *state ownership*. ASA managed state invisibly; Structured Streaming makes you declare how long state lives (watermarks), where it lives (state-store provider), and how it survives restarts (checkpoints). This chapter is about that ownership.

15.2 Monday: State Management—Watermarks, Stores, and Checkpoints

15.2.1 Why Unbounded State Kills Jobs

A stateful query—a `groupBy` aggregation, a deduplication, a join—keeps state per key. Without eviction, state for every key ever seen accumulates until executors exhaust memory and the job dies, usually weeks after deployment when the key-space has grown. The **watermark** is the eviction rule: `withWatermark("event_time", "10 minutes")` tells the engine it may discard state for windows whose event time is more than ten minutes behind the current maximum. Watermarks are the boundary between a streaming job and a memory leak.

15.2.2 State-Store Providers

The default in-memory state store keeps state in executor JVM memory with a write-ahead log in the checkpoint; it suits moderate state. The **RocksDB provider** spills state to an embedded key-value store on local disk, scaling to hundreds of millions of keys per executor at the cost of serialization overhead. Choose RocksDB when state size is the scaling constraint; measure the per-batch commit latency either way.

15.2.3 Checkpoints Bind Everything

The checkpoint directory holds source offsets, state-store snapshots, and the query plan. This is what makes recovery exact—and what makes the directory sacred. A changed query started from an old checkpoint can replay data, corrupt state, or refuse to start. The discipline: **version checkpoint paths per deployment** (`.../ad-join/v3`), start materially changed queries from fresh locations, and backfill from the source when history matters.

Common Pitfall

Reusing a checkpoint directory after changing the query. Offsets, state schema, and plan are bound together in the checkpoint. New logic plus old state is undefined behavior; version the path and reprocess deliberately.

15.3 Tuesday: Micro-Batch, Continuous, and Exactly-Once

15.3.1 Execution Modes

Micro-batch (the default) processes the stream in sub-second-to-minutes batches: high throughput, hundred-millisecond-to-second latencies, and the mature path for stateful operations. **Continuous processing** runs long-lived operators for single-digit-millisecond latency, but with a restricted operator set and at-least-once semantics—appropriate for stateless, ultra-low-latency forwarding, rare in practice. The realistic choice for nearly all production streaming is micro-batch with a tuned trigger interval.

15.3.2 End-to-End Exactly-Once

Exactly-once is a property of the *pipeline*, not the engine. The recipe: (1) replayable source (Event Hubs offsets survive), (2) checkpointed query (offsets and state recover together), (3) **idempotent sink**—a Delta merge keyed on the event’s unique id, or a sink that tracks which batch wrote which rows. The weaker alternative—at-least-once delivery plus downstream deduplication—moves the dedup cost to every consumer forever; prefer paying it once, in the sink.

Definition

End-to-end exactly-once means each input record affects the final output exactly once, across failures. It requires replayable sources, deterministic processing with checkpointed state, and sinks that are idempotent or transactional with the checkpoint.

15.4 Wednesday: Event Hubs via Kafka and Stream-Stream Joins

15.4.1 Consuming the Hub

Event Hubs’ Kafka endpoint plugs straight into Spark’s Kafka source; the connection string comes from Key Vault via Databricks secrets:

```

1 from pyspark.sql import functions as F
2
3 EH_CONN = dbutils.secrets.get("kv-scope", "eventhubs-conn-str")
4 kafka_options = {
5     "kafka.bootstrap.servers": "<ns>.servicebus.windows.net:9093",
6     "subscribe": "iot-telemetry",
7     "kafka.security.protocol": "SASL_SSL",
8     "kafka.sasl.mechanism": "PLAIN",
9     "kafka.sasl.jaas.config": (
10         "kafkashaded.org.apache.kafka.common.security.plain."
11         'PlainLoginModule required username="{ConnectionString}" '
12         f'password="{EH_CONN}";'
13     ),
14 }
15

```

```

16 raw = (spark.readStream.format("kafka").options(**kafka_options).load()
17         .select(F.from_json(F.col("value").cast("string"),
18                             "deviceId STRING, tempC DOUBLE, ts DOUBLE").alias("e"))
19         .select("e.*")
20         .withColumn("event_time", F.from_unixtime("ts").cast("timestamp")))
21
22 agg = (raw.withWatermark("event_time", "10 minutes")
23         .groupBy(F.window("event_time", "5 minutes"), "deviceId")
24         .agg(F.avg("tempC").alias("avg_temp")))
25
26 (agg.writeStream.format("delta")
27     .outputMode("append")
28     .option("checkpointLocation",
29            "abfss://checkpoints@<adls>.dfs.core.windows.net/iot/v1")
30     .toTable("iot_agg"))

```

Listing 15.1: Structured Streaming from the Event Hubs Kafka endpoint.

15.4.2 Stream-Stream Joins with Bounded State

Joining two live streams—ad impressions to ad clicks—requires both sides to declare watermarks *and* a time-range join condition, so the engine knows when join state for a key can be evicted:

```

1 impressions = impressions.withWatermark("impression_ts", "10 minutes")
2 clicks = clicks.withWatermark("click_ts", "20 minutes")
3
4 joined = impressions.join(
5     clicks,
6     F.expr("""click_ad_id = impression_ad_id
7             AND click_ts BETWEEN impression_ts
8             AND impression_ts + INTERVAL 30 MINUTES"""),
9     "leftOuter",
10 )
11
12 (joined.writeStream.format("delta")
13     .outputMode("append")
14     .option("checkpointLocation",
15            "abfss://checkpoints@<adls>.dfs.core.windows.net/ad-join/v1")
16     .toTable("ad_attribution"))

```

Listing 15.2: Watermarks on both sides bound how long join state is retained.

Without the watermark-plus-interval pair, the join retains state forever—the unbounded-state failure wearing a join’s clothes.

Tip

Set the two watermarks asymmetrically on purpose: the side expected to arrive later (here, clicks after impressions) gets the longer watermark. The interval condition then expresses the business window—“a click counts if within 30 minutes of the impression.”

15.5 Thursday Hands-on Project: Event Hubs to Delta with Windowed Aggregation

Implement Listing 15.1 end-to-end against the Chapter 13 hub: consume telemetry through the Kafka endpoint, aggregate into five-minute windows with a ten-minute watermark, and append results into a Delta table.

15.5.1 Setup

Create the Databricks secret scope backed by Key Vault; store the Event Hubs connection string; create the checkpoints container in the lake. Run the Chapter 13 producer so the stream has live data.

15.5.2 Run and Observe

Start the stream; in the Spark UI watch `inputRate` versus `processingRate`—when processing lags input, micro-batches grow. Stop the query, restart it, and confirm it resumes from the checkpoint with no reprocessing beyond the watermark. Query `iot_agg` from the Chapter 7 serverless pool to close the lakehouse loop.

15.5.3 Break It Deliberately

(1) Send events with `ts` twenty minutes old: watch them evicted by the watermark and count them in the stream metrics. (2) Change the aggregation and restart from the same checkpoint: observe the failure or corruption risk, then restart from a versioned path (`/iot/v2`) and document the difference.

15.6 Friday Use Case: Real-Time HL7 Anonymization

A hospital network must stream HL7 admission/discharge/transfer messages into analytics, but PHI may never persist unmasked. Design: producers publish to Event Hubs partitioned by facility; a Structured Streaming job parses HL7 segments, tokenizes patient identifiers via a lookup against a Key Vault-backed token service (batch the lookups per micro-batch to bound latency), writes the anonymized stream to the curated Delta zone, and lets Synapse serverless views serve analysts—the Chapter 23 compliance chapter owns the full erasure story this design must support.

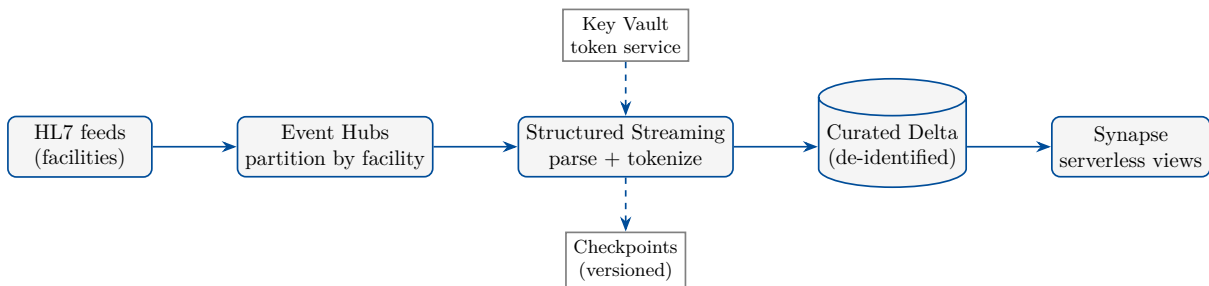


Figure 15.1: Real-time HL7 anonymization: tokenize in motion, persist only de-identified data, serve through governed views.

Common Pitfall

PHI in checkpoints and logs. The state store and the driver’s logs are persistence too. If PHI enters the stream, it enters checkpoints; design the tokenization to happen before any stateful operator, and scope checkpoint containers with the same ACL rigor as curated data.

15.7 Architectural Decision Record Template

Per streaming job record: trigger interval with the latency SLO, watermark with the late-data contract, state-store provider with state-size evidence, checkpoint path versioning policy, sink idempotency mechanism, and the backfill procedure for changed logic.

15.8 Operational Deep Dive: Lag, Restarts, and Backfills

Operate a stream on three signals: **input vs. processing rate** (growing gap → scale or optimize), **batch duration** (approaching the trigger interval → you are at capacity), and **state size** (growing without bound → watermark too generous or keys unbounded). For deploys, the runbook is: stop, bump the checkpoint path version, deploy, start, and—if the change altered semantics—backfill the affected window from the lake’s captured bronze data.

15.9 Troubleshooting Patterns

Job OOMs after weeks of health → state growth; audit watermarks and key cardinality. Restart reprocesses hours of data → checkpoint lost or path reused after a query change. Kafka auth failures → the `$ConnectionString` JAAS literal misquoted or the secret rotated without a job restart. Delta sink duplicates → sink is append-only without idempotency keys; exactly-once requires the merge.

15.10 Week 15 Lab Rubric

Criterion	Meets expectation	Needs improvement
State discipline	Watermarks on every stateful op; state size monitored	Unbounded state discovered post-mortem
Checkpointing	Versioned paths; restart resumes exactly; changed query starts fresh	Checkpoint reused across logic changes
Delivery	Idempotent Delta sink demonstrated under forced retry	Append-only sink with visible duplicates
Security	Secrets via Key Vault scope; checkpoint ACLs considered	Connection strings in notebooks

Table 15.1: Week 15 rubric for the streaming deliverables.

15.11 Interview Q&A

15.11.1 Concepts and Trade-offs

- Q: Micro-batch vs. continuous processing—what is the trade-off?**
A: Micro-batch offers full operators and exactly-once at sub-second latency; continuous offers millisecond latency with a restricted operator set and at-least-once semantics.
- Q: Why does unbounded state kill a streaming job, and how do watermarks evict it?**
A: State per key accumulates until memory is exhausted; the watermark tells the engine it may evict state older than the tolerance behind the event-time frontier.

3. **Q: What must both sides of a stream-stream join declare?**

A: A watermark each, plus a time-range join condition, so join state retention is bounded on both sides.

4. **Q: Contrast checkpointing plus an idempotent Delta sink with at-least-once plus downstream dedup.**

A: The former pays dedup cost once in the sink and gives consumers clean data; the latter pushes dedup to every consumer forever.

15.12 Further Reading

The Structured Streaming programming guide [97] is the canonical reference. The Spark project [96] and Delta Lake paper [1] cover the engine and the sink; the Event Hubs documentation [18] covers the Kafka endpoint configuration used here.

Key Takeaways

- Structured Streaming treats streams as unbounded tables; state, not syntax, is the new responsibility.
- Every stateful operator needs a watermark; unbounded state is a memory leak with a time delay.
- RocksDB scales the state store; measure commit latency when you switch.
- Exactly-once is replayable source + checkpointed state + idempotent sink; anything less is at-least-once with marketing.
- Stream-stream joins need watermarks on both sides and an interval condition.
- Checkpoint paths are versioned deployment artifacts; changed logic starts fresh and back-fills deliberately.

15.13 Exercises

1. **(Conceptual)** Explain what the checkpoint directory contains and why each part matters at recovery.
2. **(Conceptual)** Why does a deduplication query need a watermark just as much as a windowed aggregation?
3. **(Conceptual)** What changes when you switch the state store to RocksDB, and what would you measure before and after?
4. **(Hands-on)** Complete the Thursday project including both deliberate-breaking experiments; record the outcomes.
5. **(Hands-on)** Add a deduplication on `eventId` with `dropDuplicates` and a watermark; show the state-size difference versus no watermark.
6. **(Hands-on)** Implement the ad-attribution join from Listing 15.2 with simulated late clicks and verify the 30-minute attribution window.
7. **(Design)** Write the ADR for the HL7 pipeline: tokenization placement, checkpoint ACLs, and the erase-support story.

8. **(Design)** Design the backfill procedure for a changed aggregation: which window, from which bronze data, into which table version.
9. **(Design)** Specify SLOs (lag, batch duration, state size) with alert thresholds for the `iot_agg` job.
10. **(Interview)** Prepare a two-minute answer to “the stream was fine for a month and then OOMed overnight—why?”

15.14 Capstone Project: Exactly-Once Windowed Aggregation from Event Hubs

Capstone Project

Mission: Deliver an exactly-once streaming aggregation: Kafka-endpoint consumption, watermark-bounded state, versioned checkpoints, an idempotent Delta sink, and operational evidence—rates, state size, and a clean restart.

Estimated time: 4–5 hours.

What you will practice:

- Key Vault-scoped Kafka configuration against Event Hubs.
- Watermark and state-store selection with measurements.
- Checkpoint versioning and restart behavior.
- Idempotent sinks proven under forced failure.

Scenario

The games publisher from Chapter 13 wants five-minute engagement aggregates in a Delta table for live dashboards, with a guarantee that a consumer crash or deploy never double-counts a session.

Requirements

- **Source:** Kafka endpoint with Key Vault-scoped credentials; partition-aligned consumption.
- **State:** watermark matching the documented late-data contract; state size graphed over a run.
- **Delivery:** Delta sink with merge-based idempotency; forced-crash test with zero duplicates.
- **Deployment:** versioned checkpoint paths; a documented query-change procedure.

Reference Architecture

Figure 15.1’s backbone applies with the games schema: hub → Structured Streaming (windowed aggregation) → curated Delta → serverless views, checkpoints versioned per deployment.

Implementation Steps

1. Configure the secret scope and Kafka options (Listing 15.1).
2. Run the windowed aggregation; record rate and state-size metrics.
3. Crash the consumer mid-batch; restart; prove no duplicates via the sink’s merge key.

4. Change the aggregation; deploy to a new checkpoint version; backfill the affected window.

Deliverables

- Notebook, job definition, and checkpoint-versioning runbook.
- Metrics evidence: input/processing rates, batch durations, state size.
- The duplicate-proof test write-up.

Acceptance Criteria

- Forced mid-batch failure followed by restart produces exactly-once output.
- Late events beyond the watermark are evicted and counted, not silently absorbed.
- A changed query never starts from a stale checkpoint.

Stretch Goals

- Switch to the RocksDB provider and benchmark commit latency at 10× key cardinality.
- Add a stream-stream join to the pipeline with asymmetric watermarks.
- Implement Trigger.AvailableNow for a cost-optimized hourly micro-batch variant and compare lag.

Business Intelligence, Power BI, and Reverse ETL

Learning Objectives

- Explain the Power BI architecture—Desktop, Service, semantic models, and dataflows—and where the data engineer’s responsibility ends and the analyst’s begins.
- Choose among Import, DirectQuery, and Dual storage modes based on freshness, latency, and source-load requirements.
- Design a star-schema semantic model with role-playing dimensions, correct relationships, and measures instead of calculated columns.
- Write basic DAX measures using CALCULATE, filter context, and time-intelligence functions.
- Implement row-level security with static and dynamic roles and validate the result with “View as role.”
- Connect Power BI to a Synapse dedicated pool with DirectQuery and tune the model so queries stay fast at the source.
- Design the BI access layer for 500 analysts where regional managers see only their own regions.

Cross-Cloud Equivalence

The BI layer is the least portable part of the stack—each cloud has a native analytics front end—but the underlying concepts (extracted versus live models, row-level security, measure semantics) transfer directly.

Capability	AWS	Azure	GCP
BI service	Amazon QuickSight	Power BI	Looker / Looker Studio
Live query mode	Direct query to Redshift/Athena	DirectQuery	Live connection to BigQuery
Extracted cache	SPICE	Import mode (in-memory)	Extracted data / BI Engine
Row-level security	QuickSight RLS	Power BI RLS roles	Looker access filters
Metrics layer	QuickSight datasets	Semantic model + DAX	LookML

Fabric: Power BI with DirectLake over OneLake; **Snowflake:** Snowsight + partner BI; **MongoDB:** Atlas Charts + BI Connector

Power BI’s **DirectLake** mode in Microsoft Fabric is the emerging fourth storage mode: the engine reads Delta tables in OneLake directly, combining Import-mode speed with Di-

rectQuery freshness. When evaluating Azure BI architecture after this chapter, DirectLake is the first alternative to benchmark.

Key Terms

Semantic model: the governed, in-memory or live-connected data model behind Power BI reports—tables, relationships, measures, and security. **Import mode:** data is extracted and cached in the VertiPaq engine; refreshes are scheduled. **DirectQuery:** visuals generate SQL against the source at query time; nothing is cached. **RLS:** row-level security, filtering rows per user through DAX role rules. **DAX:** Data Analysis Expressions, the measure language of semantic models.

16.1 Week 16 Overview: The Last Mile of the Data Platform

Every pipeline in this book terminates, eventually, in front of a human. Power BI is where Azure data platforms most often meet their users, and the meeting point is a **semantic model**: a curated, governed, business-named layer between the warehouse and the report [61]. Data engineers who dismiss BI as “the analysts’ job” discover that storage-mode decisions, model shapes, and RLS design are data-engineering decisions—they determine warehouse load, query latency, and whether a regional manager can see another region’s revenue.

Week 16 closes Part IV with the three decisions that dominate a semantic model’s behavior: where the data lives at query time (Import, DirectQuery, Dual), how the model is shaped (star schema, measures, relationships), and who can see which rows (RLS). The Thursday project wires a live DirectQuery dashboard against the Chapter 5 dedicated pool; the Friday use case scales the design to 500 analysts.

Note

Terminology check: Microsoft renamed Power BI “datasets” to *semantic models*. Older documentation, job descriptions, and the DP-203 exam pool may use either word; they are the same object.

16.2 Monday: Power BI Architecture

16.2.1 Desktop, Service, Semantic Models, Dataflows

Power BI Desktop is the authoring tool: connect, shape with Power Query, model, and build report pages. Publishing pushes two artifacts to the **Power BI Service**: the report (visuals) and the semantic model (tables, relationships, measures, security). The service handles refresh scheduling, sharing, app workspaces, and governance. **Dataflows** are reusable, cloud-executed Power Query transformations—the right home for shaping logic that more than one model needs, so that twenty models do not re-implement the same customer cleanup.

The architectural principle that matters to data engineers: *many reports should hang off few certified semantic models*. A certified model owned by the data platform team, marked as the endorsed source for a domain, prevents the spreadsheet-ification of the warehouse—fifty near-identical revenue numbers, none reconciled.

16.2.2 The Serving Contract

A semantic model is a contract between the data platform and its consumers. Tables and measures get business names, not `stg_ord_ln_v3`; relationships are documented; RLS is tested

like code. When the warehouse schema changes, the model absorbs the change—this is why Chapter 24’s expand–contract migrations keep a view shim between the physical tables and the model.

16.3 Tuesday: Import, DirectQuery, and Dual

16.3.1 Import Mode

Import extracts data into the VertiPaq columnar engine inside the service. Queries hit memory, not the source: sub-second visuals, full DAX, no load on the warehouse. The costs are staleness (data is as fresh as the last scheduled refresh—eight per day on Pro, forty-eight on Premium) and capacity memory. Import is the default and the right choice for most reporting.

16.3.2 DirectQuery Mode

DirectQuery stores only metadata; every visual interaction generates SQL against the source [10]. Freshness is real-time and there is no extract to govern, but every filter click is a warehouse query: latency depends on the source, concurrency multiplies load, and some DAX patterns are not translatable. DirectQuery is the right choice when data must be current to the minute, when the dataset is too large to cache, or—as in this week’s project—when a governed warehouse is the single source of truth and duplicating it adds risk, not value.

16.3.3 Dual Mode and Aggregation Tables

Dual mode marks a table as both: serve from cache when possible, fall back to DirectQuery. Combined with **aggregation tables**—small imported pre-aggregates over a large DirectQuery fact—Dual gives dashboard-speed summaries with drill-through to live detail. This is the pattern behind the “real-time dashboard over a large fact” requirement that naive DirectQuery models fail.

Mode	Strengths	Costs	Choose when
Import	Fastest visuals; full DAX; no source load	Staleness; capacity memory	Default; refresh cadence acceptable
DirectQuery	Real-time; no extract; single source of truth	Source latency and load; some DAX limits	Freshness SLAs; huge datasets; governed warehouse
Dual + aggregations	Cached summaries, live detail	Model complexity	Large DirectQuery facts behind dashboards

Table 16.1: Storage-mode selection for semantic models.

Common Pitfall

DirectQuery over an unoptimized source. A DirectQuery report is a load generator pointed at your warehouse. Every slicer change is a SQL query per visual per user. Benchmark the source under realistic visual concurrency before publishing.

16.4 Wednesday: RLS, DAX, and Reverse ETL

16.4.1 Row-Level Security

RLS restricts rows per user through roles containing DAX filter expressions [64]. A **static** role hard-codes the filter: `[Region] = "West"`. A **dynamic** role derives it from the caller: `[ManagerEmail] = USERPRINCIPALNAME()` against a mapping table. Dynamic RLS is the only design that scales to 500 users—one role, one mapping table, no role-per-region sprawl.

Rules that keep RLS honest: filters should flow from dimension to fact through single-direction relationships; bidirectional relationships for RLS propagation are a last resort because they also change measure semantics; and every role must be tested with *View as role* plus a test account that belongs to it.

16.4.2 DAX Fundamentals

DAX measures are evaluated in a **filter context**: the set of filters applied by the visual, slicers, and report filters at evaluation time. The engine function is `CALCULATE`, which modifies filter context. Time-intelligence functions (`SAMEPERIODLASTYEAR`, `DATESYTD`) require a marked date table.

```

1 Revenue = SUM ( FactSales[NetAmount] )
2
3 Revenue LY =
4 CALCULATE ( [Revenue], SAMEPERIODLASTYEAR ( DimDate[Date] ) )
5
6 Revenue YTD =
7 CALCULATE ( [Revenue], DATESYTD ( DimDate[Date] ) )
8
9 Revenue YoY % =
10 DIVIDE ( [Revenue] - [Revenue LY], [Revenue LY] )

```

Listing 16.1: Core DAX measures for a sales model.

Tip

Measures over calculated columns, always, for anything aggregated. A calculated column is computed at refresh and stored per row; a measure is computed at query time in filter context. “Total revenue” as a column cannot respond to a region slicer; as a measure, it does so for free.

16.4.3 Reverse ETL: Warehouse Truth Back to Operations

16.4.4 The Pattern

Reverse ETL queries the curated warehouse—say, `gold.churn_risk_customers`—and upserts the segment into the system of action: a Dynamics 365 marketing list, a Salesforce campaign, a support-tool priority flag. ADF or Synapse pipelines execute the sync; the CRM API is the sink. The pattern completes the loop: models score in the warehouse, humans act in the operational tool, and the action’s outcome lands back in the lake as new training data.

16.4.5 Identity Resolution Before Sync

The warehouse’s `customer_key` means nothing to the CRM. Identity resolution—matching warehouse entities to CRM contacts by email, phone, or a persisted crosswalk table—must run before the sync, with explicit handling for unmatched records (create, skip, or route to an

exceptions queue). Syncing unresolved identities creates duplicate CRM contacts at machine speed—the fastest way to make the sales team distrust the data platform.

16.4.6 Sync Frequency as a Trade-Off

Freshness costs API calls. Nightly syncs are cheap and coarse; hourly syncs multiply API consumption and risk rate limits; event-driven syncs (a Function fired when the gold table changes) are freshest and most complex. Choose by the action's urgency: a churn-risk flag that drives a weekly email cadence needs nightly, not real-time; a fraud hold needs minutes.

Sync cadence	Mechanism	Right for
Nightly batch	ADF pipeline, scheduled	Marketing segments, weekly cadences
Hourly micro-batch	Tumbling-window trigger	Priority flags, daily workflows
Event-driven	Function on gold-table change	Fraud holds, entitlement changes

Table 16.2: Reverse-ETL cadence trades freshness against API cost and rate limits.

16.5 Thursday Hands-on Project: DirectQuery Dashboard with RLS

This project connects Power BI Desktop to the Chapter 5 dedicated pool in DirectQuery mode, builds a sales monitoring dashboard, and enforces a regional RLS role.

16.5.1 Connect and Model

1. Resume the Chapter 5 pool (`az synapse sql pool resume`).
2. In Power BI Desktop: Get Data → Azure Synapse Analytics SQL → the pool endpoint → **DirectQuery**.
3. Import the star: `fact_sales`, `dim_customer`, `dim_date`, `dim_store`. Verify relationships (single direction, dims to fact), mark `dim_date` as a date table.
4. Author the measures from Listing 16.1.

16.5.2 Build and Secure the Dashboard

Create a revenue-by-region bar chart, a revenue-vs-LY line, and a date slicer. Then define the RLS role:

```

1 -- Role: RegionalManager, applied to DimStore
2 [Region] IN
3     SELECT COLUMNS (
4         FILTER ( ManagerRegions,
5                 ManagerRegions[ManagerEmail] = USERPRINCIPALNAME() ),
6         "Region", ManagerRegions[Region]
7     )

```

Listing 16.2: Dynamic RLS role filtering stores through the manager mapping table.

Publish to the service, open *Semantic model* → *Security*, assign a test account to **RegionalManager**, and validate with *View as role*: the West manager sees only West revenue, and the totals reconcile against the unfiltered model.

16.5.3 Performance Check

Open Performance Analyzer, interact with the dashboard, and record the DirectQuery durations. If any visual exceeds three seconds, the fix lives in the warehouse (distribution, column-store, resource class)—Chapter 6’s toolkit—not in the report.

16.6 Friday Use Case: 500 Analysts, One Governed Model

An enterprise centralizes sales data in Synapse. Five hundred analysts query it; regional sales managers must see only their own regions; executives see everything; finance sees all regions but not customer identities.

16.6.1 Target Design

One certified semantic model in a dedicated workspace, DirectQuery against the gold star schema. Dynamic RLS with a **ManagerRegions** mapping table maintained by the sales-ops team through a small app—not by editing Power BI roles. A second role, **FinanceRestricted**, adds a DAX filter on customer-level columns via object-level security for the PII columns. Deployment pipelines move the model dev → test → prod, with RLS membership wired per environment.

16.6.2 Operating Model

- **Certification:** the model is endorsed; analyst-built models over exports are unsupported.
- **Access:** role membership through Entra ID security groups, never individual emails.
- **Audit:** RLS membership changes and model deployments logged; quarterly access reviews tie back to the mapping table’s owner.
- **Load:** warehouse concurrency sized for dashboard peaks (Chapter 5); aggregations in Dual mode absorb the top dashboards.

Common Pitfall

Role sprawl. Fifty static roles—one per region—guarantee that a reorg breaks security silently. One dynamic role plus a governed mapping table is the design that survives organizational change.

16.7 Architectural Decision Record Template

For each semantic model record: storage mode per table with the freshness SLA that justifies it; the RLS design (dynamic rule, mapping-table owner, test accounts); the measure inventory owned by the platform team; and the certification status with the refresh or source-latency budget.

16.8 Operational Deep Dive: Refresh, Endorsement, and Tenant Settings

Import models fail operationally, not visually: a failed 06:00 refresh means yesterday’s data presented as today’s. Alert on refresh failures, monitor duration trends, and stage large models behind incremental refresh. At the tenant level, disable publish-to-web, restrict export of

underlying data where policy requires, and use endorsement labels (promoted, certified) as the discoverability mechanism that keeps analysts on governed models.

16.9 Troubleshooting Patterns

Blank visuals for one user mean RLS filters them to nothing—test with their principal name. Slow DirectQuery visuals mean the source plan is bad; capture the generated SQL and run it in the pool. Numbers that change when a slicer is added usually mean a missing relationship or a measure ignoring filter context incorrectly. Refresh failures after a schema change mean the model’s source query referenced a renamed column—the Chapter 24 view shim exists to prevent exactly this.

16.10 Week 16 Lab Rubric

Criterion	Meets expectation	Needs improvement
Storage mode	DirectQuery justified with a freshness requirement; Performance Analyzer evidence	Mode chosen by default; no latency measurement
Modeling	Star schema, marked date table, measures not columns	Snowflake drift, calculated-column aggregates
Security	Dynamic RLS validated with View-as-role and a test account	Static role, untested, or membership by individual email
Governance	Model published to a governed workspace with endorsement	Report left in a personal workspace

Table 16.3: Week 16 rubric for the Power BI deliverables.

16.11 Interview Q&A

16.11.1 Concepts and Trade-offs

1. **Q: When does DirectQuery beat Import mode?**
A: When data must be current to the minute, when the dataset exceeds cache capacity, or when the warehouse is the governed source of truth and duplication adds risk.
2. **Q: How does RLS restrict what a regional sales manager sees?**
A: A role applies a DAX filter—ideally dynamic via `USERPRINCIPALNAME()` against a mapping table—that the engine applies to every query the user runs.
3. **Q: What is the difference between a measure and a calculated column?**
A: A column is computed at refresh and stored per row; a measure is computed at query time in the current filter context and responds to slicers.
4. **Q: Why mark a table as a date table?**
A: Time-intelligence functions like `SAMEPERIODLASTYEAR` require a contiguous, marked date table to compute correctly.

16.12 Further Reading

Start with the Power BI documentation hub [61], the DirectQuery guide [10], and the RLS reference [64]. The star-schema discipline behind every good model is Chapter 5’s Kimball

material [8]. When the model outgrows DirectQuery latency, Chapter 6’s optimization toolkit and Fabric’s DirectLake mode are the next reading.

Key Takeaways

- The semantic model is a governed contract: business names, documented relationships, tested security—many reports, few certified models.
- Import caches for speed; DirectQuery queries the source for freshness; Dual with aggregation tables combines both for large facts.
- A DirectQuery report is a load generator against the warehouse; benchmark before publishing.
- Dynamic RLS with a governed mapping table is the only design that scales; validate every role with View as role.
- Measures, not calculated columns, for anything aggregated; CALCULATE and filter context are the heart of DAX.
- Refresh failures are operational incidents—alert on them like pipeline failures.

16.13 Exercises

1. **(Conceptual)** Explain the filter context that evaluates [Revenue] when a user selects one region and one quarter.
2. **(Conceptual)** Why does bidirectional cross-filtering for RLS propagation risk incorrect measure results?
3. **(Conceptual)** A 2 TB fact table backs a dashboard viewed at 09:00 daily. Compare Import, DirectQuery, and Dual-with-aggregations designs.
4. **(Hands-on)** Complete the Thursday project and capture Performance Analyzer timings for three visuals.
5. **(Hands-on)** Add an `Orders` count measure and a YoY% measure; break then fix them by removing the date-table marking.
6. **(Hands-on)** Extend the RLS model with a second mapping dimension (product line) and verify the two roles compose.
7. **(Design)** Design the workspace, endorsement, and deployment-pipeline layout for the 500-analyst enterprise.
8. **(Design)** Write the ADR choosing DirectQuery over Import for a fraud-monitoring dashboard with a 60-second freshness SLA.
9. **(Design)** Specify the audit evidence you would keep to prove RLS correctness during a compliance review.
10. **(Interview)** Prepare a two-minute answer to “why are there three different revenue numbers in this company?”

16.14 Milestone 4 Capstone: Real-Time Fraud Detection

Capstone Project

Mission: Assemble Part IV: simulated credit-card swipes stream into Event Hubs partitioned by card; an ASA sliding-window query detects cards used in two cities within ten minutes; alerts land in Azure SQL and on a live Power BI dashboard—with tested late-data policies and an honest cost estimate.

Estimated time: 8–10 hours.

What you will practice:

- Partitioned event ingestion with idempotent producers (Chapter 13).
- Sliding-window detection queries with explicit time policies (Chapter 14).
- Streaming observability: watermark delay, late events, input stalls.
- Streaming-to-dashboard delivery without overwhelming Power BI push limits.

Scenario

A card issuer needs same-minute detection of the impossible-travel pattern: one physical card used in two distant cities within ten minutes. Detection must tolerate devices uploading late, must never double-alert on producer retries, and must show alerts live to the fraud operations team.

Requirements

- **Ingestion:** hub partitioned by `card_id`; producer attaches unique `eventId` per swipe.
- **Detection:** ASA sliding-window query with explicit late-arrival and out-of-order policies.
- **Serving:** alerts to Azure SQL (idempotent key design) and a Power BI streaming dashboard fed by *aggregates*, not raw swipes.
- **Operations:** watermark-delay alert; at least three automated fraud-rule tests (duplicate swipe, borderline window, out-of-order event).
- **Cost:** Event Hubs TUs, ASA SUs, and Power BI capacity estimated with two optimizations.

Reference Architecture

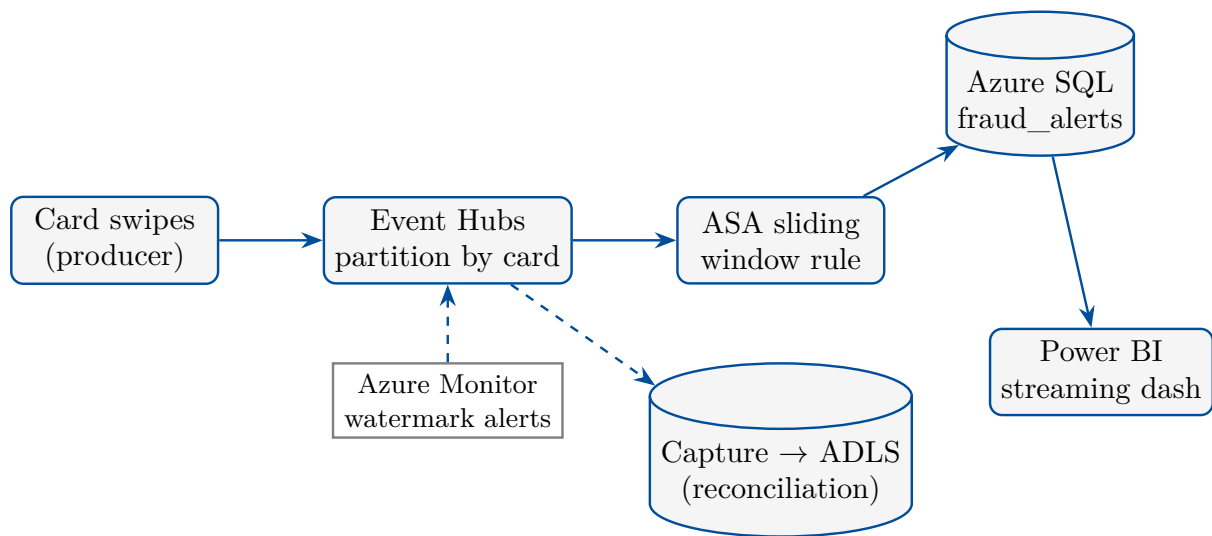


Figure 16.1: Milestone 4 architecture: partitioned ingestion, sliding-window detection, idempotent alert sink, live dashboard, lake reconciliation.

Implementation Steps

1. Create the hub (card-partitioned) and the swipe producer with dedupe keys.
2. Author the ASA job: `GROUP BY card_id, SlidingWindow(minute, 10) HAVING COUNT(DISTINCT city) > 1, event time via TIMESTAMP BY.`
3. Sink alerts to Azure SQL with a key design that tolerates redelivery; build the streaming dashboard.
4. Run the three rule tests; wire and fire the watermark-delay alert.
5. Reconcile a day of stream alerts against Capture files; produce the cost estimate.

Deliverables

- Producer, job definition, sink DDL, and tests in the repo with a README that runs from clean.
- Evidence pack: rule-test results, watermark alert firing, stream/lake reconciliation.
- Cost estimate with two documented optimizations.

Acceptance Criteria

- A two-city swipe pattern raises exactly one alert despite forced producer retries.
- Late and out-of-order events follow the documented policy with metric proof.
- The dashboard shows alerts within the freshness SLO without raw-swipe volume.

Stretch Goals

- Reimplement the rule in Structured Streaming (Chapter 15) and compare operational burden.

- Add a reverse-ETL hop: confirmed fraud flags pushed back to the card system of record.
- Add a historical-features join (card's 30-day profile via reference data) to reduce false positives.

Part V

Machine Learning and MLOps

Feature Engineering and In-Database Machine Learning

Learning Objectives

- Explain where feature engineering belongs in a data platform and why feature definitions must be shared between training and scoring.
- Train a classification model with Synapse Spark (or SynapseML) on warehouse data and export it in ONNX format.
- Score live rows natively in a dedicated SQL pool with the T-SQL PREDICT function.
- Eliminate data movement from the scoring path and reason about when in-database scoring beats a real-time endpoint.
- Keep training and scoring feature schemas aligned and detect skew before it degrades a model silently.
- Build the Thursday churn model end-to-end: Spark training, ONNX export, SQL scoring.
- Architect a SQL-only daily inventory forecasting system for 1,000 retail stores.

Cross-Cloud Equivalence

In-database scoring—pushing the model to the data rather than shipping data to a model server—exists in every analytical engine, typically via ONNX or a native PREDICT-style function.

Capability	AWS	Azure	GCP
In-database scoring	Redshift ML (SageMaker)	Synapse T-SQL PREDICT (ONNX)	BigQuery ML
Spark training	EMR + SageMaker	Synapse Spark / SynapseML	Dataproc + Vertex AI
Feature store	SageMaker Feature Store	AML managed feature store	Vertex AI Feature Store
Model format for SQL	(Auto via SageMaker)	ONNX	Native BQML models
Batch scoring	Redshift batch transform	PREDICT over pool tables	ML.PREDICT

Fabric: SynapseML + MLflow in Fabric notebooks; **Snowflake:** Snowpark ML + SQL inference; **MongoDB:** Atlas Vector Search

ONNX is the portable artifact: train in Spark (or scikit-learn, PyTorch, TensorFlow), export once, score in SQL, in an AML endpoint (Chapter 18), or at the edge—one model, many runtimes.

Key Terms

Feature: a measurable model input derived from raw data. **ONNX:** Open Neural Network Exchange, the portable model format PREDICT requires. **PREDICT:** the T-SQL function that scores rows against a stored ONNX model inside the database engine. **Training-serving skew:** divergence between feature computation at training time and at scoring time. **SynapseML:** Microsoft’s distributed ML library for Spark on Synapse and Databricks.

17.1 Week 17 Overview: Models Where the Data Lives

Part V embeds machine learning into the data platform rather than bolting it on. The shortest path from warehouse data to a scored row does not leave the warehouse: train where compute is rich (Spark), export the model as a portable artifact (ONNX), and score where the data sits (T-SQL PREDICT in the dedicated pool) [51]. Every hop you remove—export jobs, feature APIs, network round-trips—is a failure mode deleted and a latency budget returned.

The week’s conceptual weight sits in **feature engineering**: the features are the model’s interface to reality, and their definitions must be identical at training and scoring time. In-database scoring makes this discipline vivid: the scoring features come from SQL, so the training features had better be derivable by the same SQL.

17.2 Monday: Feature Engineering as Platform Discipline

17.2.1 Features Are Contracts

A feature—`orders_last_30d`, `avg_basket_90d`, `days_since_last_login`—is a versioned, testable data transformation with an owner and a consumer (the model). Treating features as ad-hoc SQL inside training notebooks guarantees the central ML production failure: the model trains on one definition and scores on another. The platform answer is to compute features once, store them in governed feature tables (the gold zone’s feature schema, or Chapter 18’s managed feature store), and have both training and scoring read from that single definition.

17.2.2 Point-in-Time Correctness

The subtlest feature bug is **leakage**: features computed with information that would not have existed at prediction time. A churn feature `total_refunds` computed over all history includes refunds that happened *after* the prediction date. The defense is point-in-time joins: every feature row carries an `as_of` timestamp, and training sets are assembled with temporal discipline—the same event-time thinking as Chapter 14, applied to ML.

Common Pitfall

Notebook-local features. When the training notebook computes features inline and the scoring path recomputes them in SQL, the two will drift—not if, but when. Persist feature tables; train and score from the same columns.

17.3 Tuesday: T-SQL PREDICT—Scoring Without a Server

17.3.1 The Mechanism

PREDICT scores rows inside the dedicated SQL pool [51]: the ONNX model is stored in a table (as a hex-encoded varbinary), and the function joins it against feature columns:

```

1  -- model bytes land here once, from the Spark export step
2  CREATE TABLE ml_models (
3      model_id INT IDENTITY PRIMARY KEY,
4      model_name VARCHAR(100),
5      model_bytes VARBINARY(MAX),
6      created_at DATETIME2 DEFAULT GETDATE()
7  );
8
9  SELECT c.customer_id,
10         p.score AS churn_probability
11  FROM dbo.customer_features c
12  CROSS APPLY (
13      SELECT score FROM PREDICT(
14          MODEL = (SELECT TOP 1 model_bytes FROM ml_models
15                  WHERE model_name = 'churn_v3' ORDER BY model_id DESC),
16          DATA = c AS d,
17          RUNTIME = ONNX
18      ) WITH (score REAL)
19  ) p;

```

Listing 17.1: Storing an ONNX model and scoring live rows with PREDICT.

Scoring happens where the data already lives: no export, no endpoint, no network. For batch scoring over millions of warehouse rows—nightly churn scores, monthly risk ratings—this is structurally the cheapest and most operationally boring design, which is a compliment.

17.3.2 When In-Database Scoring Wins and Loses

Wins: batch or interactive scoring over warehouse-resident features; SQL-native teams; governance-sensitive data that must not leave the database perimeter. Loses: sub-100-millisecond scoring of single records from applications (that is Chapter 19’s managed online endpoint), features that need live computation (the warehouse only knows what ETL told it), and models too large or exotic for the ONNX runtime inside SQL.

17.4 Wednesday: Training with Synapse Spark and Exporting ONNX

17.4.1 The Training Path

Synapse Spark pools (or Databricks, Chapter 9) read warehouse or lake features, train with Spark MLlib or SynapseML [67], evaluate against a holdout, and export ONNX:

```

1  from pyspark.ml import Pipeline
2  from pyspark.ml.feature import VectorAssembler
3  from pyspark.ml.classification import LogisticRegression
4  import onnxmltools
5
6  features = ["orders_30d", "avg_basket_90d", "days_since_login",
7             "support_tickets_30d"]
8  df = spark.table("gold.churn_training")  # point-in-time-correct features
9
10 train, test = df.randomSplit([0.8, 0.2], seed=42)

```

```

11 pipe = Pipeline(stages=[
12     VectorAssembler(inputCols=features, outputCol="features"),
13     LogisticRegression(labelCol="churned", featuresCol="features"),
14 ])
15 model = pipe.fit(train)
16
17 auc = BinaryClassificationEvaluator(labelCol="churned") \
18     .evaluate(model.transform(test))
19 print(f"holdout AUC = {auc:.3f}")
20
21 onnx_model = onnxmltools.convert_sparkml(
22     model, "churn_v3",
23     [(f, FloatTensorType([1])) for f in features]) # illustrative signature
24 with open("churn_v3.onnx", "wb") as f:
25     f.write(onnx_model.SerializeToString())

```

Listing 17.2: Train logistic regression churn model in Synapse Spark and export ONNX.

17.4.2 The Schema Alignment Contract

PREDICT binds model inputs to table columns *by name and type*. The training feature list and the scoring view's columns must match exactly—order, names, types. Encode the contract as the `dbo.customer_features` view used by both paths, version it with the model (`churn_v3` expects view version 3), and test the pairing in CI: a model registered against the wrong feature view is a silent wrong-answer generator.

Tip

Store the model's expected feature list and types as metadata beside the model bytes, and write a validation query that diffs it against `sys.columns` of the scoring view. Run it in the deployment pipeline before promoting a model.

17.5 Thursday Hands-on Project: Churn Model from Spark to SQL

Train a logistic-regression churn model on customer features in Synapse Spark, export ONNX, load it into the dedicated pool, and score live customers with PREDICT.

17.5.1 Steps

1. Build `gold.churn_training` (point-in-time features + label) and `dbo.customer_features` (the scoring view)—same column definitions, enforced.
2. Train and export with Listing 17.2; record holdout AUC.
3. Load the model bytes into `ml_models` (OPENROWSET BULK from the lake).
4. Score with Listing 17.1; verify score distributions between Spark-side evaluation and SQL-side scoring match on the same rows.
5. Induce a schema mismatch (rename one feature in the view) and capture the failure—this is the regression test for the alignment contract.

17.6 Friday Use Case: SQL-Only Inventory Forecasting for 1,000 Stores

A retailer needs daily per-store demand forecasts. Constraint from the platform team: the entire pipeline must be SQL-operable—no model servers, no Python on the scoring path—because the operations team that owns it is a SQL team.

17.6.1 Architecture

Nightly: Synapse Spark trains a global forecasting model (store, product features plus calendar features) and exports ONNX weekly. The pool stores the model; a scheduled stored procedure computes tomorrow’s features from the silver zone and scores all store-product pairs with PREDICT into `gold.inventory_forecast`. Chapter 7’s serverless views expose the forecast to Power BI; Chapter 16’s reverse ETL pushes reorder alerts to the operations tool. Every component is already in this book; the design work is the schema contract and the retraining cadence.

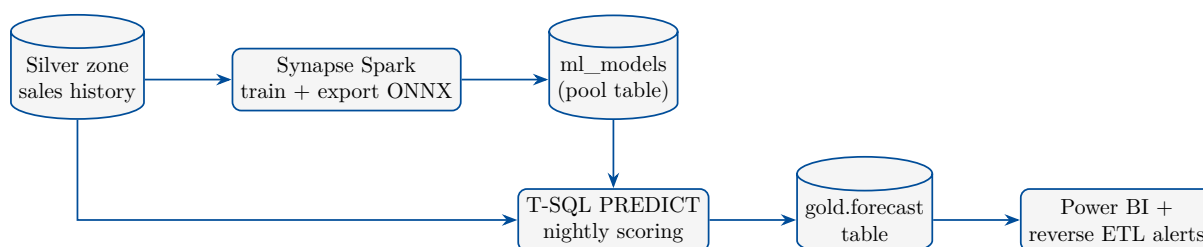


Figure 17.1: SQL-only forecasting: Spark trains weekly, PREDICT scores nightly, and the whole serving path is operable by a SQL team.

Common Pitfall

Letting “SQL-only” degrade into “never retrained.” The operability constraint covers the *scoring* path, not the *learning* path. Schedule retraining, gate promotion on a metric (Chapter 19), and monitor drift—an unmaintained model is a slow-motion incident.

17.7 Architectural Decision Record Template

Per model record: feature table of record with its point-in-time discipline, the training environment and cadence, the ONNX export verification step, the scoring path (in-database versus endpoint) with the latency and data-residency justification, and the schema-alignment test that pairs model version to feature view version.

17.8 Operational Deep Dive: Scoring SLAs and Model Inventory

Treat scored tables like any data product: a freshness SLA (scores land by 06:30), a volume check (row count matches the eligible population), and a distribution check (today’s score histogram versus the trailing band—a shift with no upstream change is a model or feature problem). Keep a model inventory—model, version, feature view version, trained-on window, holdout metric, promoted-by—queryable from the `ml_models` table itself.

17.9 Troubleshooting Patterns

PREDICT binding errors → feature name/type mismatch between the view and the ONNX signature; run the alignment validator. Scores shifted after a pool maintenance window → the model table was repopulated from the wrong artifact; the model inventory query settles it. Spark and SQL disagree on the same rows → preprocessing (scaling, encoding) done in training but omitted in SQL features—preprocessing must live upstream of both paths, in the feature view.

17.10 Week 17 Lab Rubric

Criterion	Meets expectation	Needs improvement
Feature discipline	One feature definition feeds training and scoring; point-in-time correct	Inline notebook features; leakage unexamined
Model lifecycle	ONNX export verified; model + view versioned together	Model file passed around without metadata
Scoring	PREDICT scores reconcile with Spark-side evaluation	Scores accepted unverified
Operability	SQL-operable scoring path with freshness and distribution checks	Manual notebook scoring

Table 17.1: Week 17 rubric for the feature-engineering deliverables.

17.11 Interview Q&A

17.11.1 Concepts and Trade-offs

1. **Q: What does T-SQL PREDICT eliminate from the scoring path?**
A: Data movement—no export jobs, no endpoints, no network round-trips; rows are scored where they already live.
2. **Q: When is in-database scoring preferable to a real-time endpoint?**
A: For batch/interactive scoring over warehouse-resident features, SQL-native operations teams, and data that must not leave the database perimeter.
3. **Q: In what format must a model be stored for Synapse PREDICT?**
A: ONNX, stored as varbinary in a table and referenced by the PREDICT function at scoring time.
4. **Q: What must be true of training and scoring feature schemas?**
A: They must match by name, type, and semantics—enforced by one shared feature view versioned with the model.

17.12 Further Reading

The PREDICT tutorial [51] and the SynapseML documentation [67] are the primary sources. The ONNX ecosystem and the Azure Machine Learning documentation [83] connect this chapter to Chapters 18–19, where the same models get endpoints, pipelines, and monitoring.

Key Takeaways

- Features are versioned contracts; one feature table feeds both training and scoring.
- Point-in-time correctness prevents leakage—the silent way models lie.
- PREDICT plus ONNX scores rows in-place; batch scoring never needs a server.
- In-database scoring wins on operability and data gravity; endpoints win on latency and live features.
- Model version and feature-view version are one deployment unit; test the pairing.
- SQL-only scoring is legitimate architecture when retraining and monitoring are still engineered in.

17.13 Exercises

1. **(Conceptual)** Explain training-serving skew with a concrete feature example and its silent failure mode.
2. **(Conceptual)** Why does leakage survive model validation, and how do point-in-time joins catch it?
3. **(Conceptual)** When would you keep preprocessing out of the ONNX graph, and where must it then live?
4. **(Hands-on)** Complete the Thursday project, including the induced schema-mismatch test.
5. **(Hands-on)** Add a calibration step to the training pipeline and compare SQL-side score distributions before/after.
6. **(Hands-on)** Write the model/feature alignment validator query and wire it into a deployment script.
7. **(Design)** Design the `ml_models` inventory schema and the freshness/distribution checks for scored tables.
8. **(Design)** Write the ADR choosing PREDICT over an AML endpoint for the inventory forecaster.
9. **(Design)** Extend the Friday design with a drift trigger that queues retraining (preview Chapter 19).
10. **(Interview)** Prepare a two-minute answer to “the model was 91% accurate in training and useless in production—what happened?”

17.14 Capstone Project: In-Database Churn Scoring with a Schema Contract

Capstone Project

Mission: Ship a governed in-database scoring loop: point-in-time feature tables, Spark training with ONNX export, PREDICT scoring in the dedicated pool, a versioned model/feature contract with an automated alignment test, and scoring SLAs an operations team can run.

Estimated time: 4–6 hours.

What you will practice:

- Point-in-time feature assembly and leakage discipline.
- ONNX export, registration, and in-pool scoring.
- Model/feature version pairing with automated validation.
- Score freshness, volume, and distribution monitoring.

Scenario

A subscription business with a churn-risk reverse-ETL loop (Chapter 16) wants nightly churn scores for every active customer, consumed by the retention team’s SQL tooling. The platform team owns training; the SQL team owns scoring; the contract between them must be machine-checked, not meeting-checked.

Requirements

- **Features:** one governed feature view, point-in-time correct, used by both training and scoring.
- **Model:** logistic regression or better, holdout AUC recorded, ONNX artifact registered with metadata.
- **Contract:** automated alignment validation gating promotion; induced-mismatch regression test.
- **Operations:** scoring completes by 06:30 with freshness, volume, and distribution checks on the output.

Reference Architecture

Figure 17.1 with the subscription schema: silver interactions → Spark training → `ml_models` → PREDICT → `gold.churn_scores` → reverse ETL to Dynamics 365.

Implementation Steps

1. Assemble the point-in-time training set and the scoring view from one definition.
2. Train, evaluate, and export ONNX (Listing 17.2).
3. Register the model with metadata; score with PREDICT (Listing 17.1).
4. Build the alignment validator; pass it, then prove it catches a mismatch.
5. Add the three output checks; document the operations runbook.

Deliverables

- Training notebook, ONNX artifact, registration script, and scoring procedure.
- The alignment validator and its regression test.
- The scoring SLA runbook with the three checks.

Acceptance Criteria

- SQL-side scores reconcile with Spark-side evaluation on identical rows.

- The alignment test blocks a mismatched model/view pairing in the deployment path.
- The scoring table meets its freshness and volume checks for three consecutive nights.

Stretch Goals

- Add a challenger model and an automated champion/challenger comparison on holdout AUC.
- Compute a simple drift metric on one feature and chart it for two weeks.
- Port the scoring to a serverless SQL pool over Delta features and compare operability.

Machine Learning Pipelines with Azure ML and Databricks

Learning Objectives

- Navigate the Azure Machine Learning workspace: studio, compute instances, compute clusters, datastores, and environments.
- Choose among AutoML, the Designer, and custom notebooks for a given team and problem.
- Register ADLS Gen2 data via datastores secured with the workspace managed identity.
- Build an AML pipeline from versioned components—preprocess, train, evaluate, register—and execute it from the Python SDK.
- Register features in the managed feature store so training and serving share one definition.
- Integrate Databricks for heavy feature computation feeding AML training.
- Design a secure workflow for data scientists training PyTorch models on GPU clusters over governed lake data.

Cross-Cloud Equivalence

Managed ML platforms share a universal anatomy: a workspace, versioned data references, reusable environments, pipeline components, and a model registry.

Capability	AWS	Azure	GCP
ML platform	SageMaker	Azure Machine Learning	Vertex AI
AutoML	SageMaker Autopilot	Azure AutoML	Vertex AutoML
Pipelines	SageMaker Pipelines	AML pipelines (components)	Vertex Pipelines (Kubeflow)
Feature store	SageMaker Feature Store	AML managed feature store	Vertex AI Feature Store
Model registry	SageMaker Model Registry	AML Model Registry	Vertex Model Registry
Notebook compute	Studio notebooks	Compute instances	Workbench instances

Fabric: built-in ML with MLflow on lakehouses; **Snowflake:** Snowpark ML; **MongoDB:** partner MLOps over Atlas.

MLflow is the lingua franca underneath: AML, Databricks, and Fabric all track experiments with it, so experiment-tracking skills transfer even where the platform does not.

Key Terms

Workspace: the AML management boundary—compute, data, experiments, models, endpoints. **Compute instance:** a managed single-node development VM; **compute cluster:** autoscaling multi-node training compute. **Environment:** a versioned, reproducible software context (Docker image plus conda/pip spec). **Component:** a reusable, versioned pipeline step with declared inputs, outputs, and environment. **Datastore:** a registered, credential-managed reference to storage such as ADLS Gen2.

18.1 Week 18 Overview: The ML Platform Layer

Chapter 17 embedded a trained model into SQL. Chapters 18–19 build the platform that produces, ships, and watches such models continuously. Azure Machine Learning is that platform: a workspace unifying compute, data access, experiment tracking, pipelines, a model registry, and endpoints [83]. Databricks remains in the picture where it belongs—heavy feature computation over the lake—with AML owning training orchestration, registration, and serving.

The discipline of the week is *reproducibility through versioning*: versioned data assets, versioned environments, versioned components, versioned models. A pipeline built from versioned pieces can answer the only question that matters when a model misbehaves in production: *exactly what produced this artifact?*

18.2 Monday: Workspace, Compute, and Data Access

18.2.1 Compute Instances vs. Compute Clusters

The **compute instance** is a per-developer managed VM for notebooks—always-on, single-node, your desk in the cloud. The **compute cluster** is the training fleet: autoscaling, multi-node, GPU-capable, and—critically—*able to scale to zero*. The cost rule mirrors Chapter 9’s job clusters: development on instances, training on clusters, and a zero minimum node count so idle time bills nothing.

18.2.2 Datastores and Managed Identity

A datastore registers ADLS Gen2 (or other storage) in the workspace. Register it with the workspace’s **managed identity** and grant that identity **Storage Blob Data Reader** on the lake: training jobs then read governed data with no keys, no service-principal secrets, and an auditable identity [86]. Data assets—named, versioned references over datastore paths—give pipelines stable inputs (`azureml:features:3`) even as the underlying folders change.

Common Pitfall

Notebook-mounted storage with personal credentials. A datastore with a personal account key in a notebook bypasses every governance control the lake has and dies when the person changes roles. Managed-identity datastores are the only production pattern.

18.3 Tuesday: AutoML, Designer, or Custom Code

18.3.1 The Three Authoring Modes

AutoML sweeps algorithms and hyperparameters against a dataset and metric, producing a ranked leaderboard with explainability—the right tool for baselines and for teams establishing

“how hard is this problem, really.” The **Designer** composes drag-and-drop pipelines from modules—accessible, but its artifacts are the least code-reviewable of the three. **Custom notebooks and components** give full control (PyTorch, custom losses, exotic features) at full responsibility. The mature pattern uses AutoML to set the baseline bar and custom components to beat it; a custom model that cannot beat AutoML’s leaderboard entry has not earned its complexity.

18.3.2 The Thursday Preview: AutoML to Endpoint in One Afternoon

The fastest path through the platform: point AutoML at a registered data asset, let it sweep, deploy the best run to a **managed online endpoint**, and test with a REST payload. This is Week 18’s original project, and it remains the fastest way to establish the end-to-end skeleton (data asset → trained model → registered model → endpoint) before replacing the middle with your own pipeline.

18.4 Wednesday: AML Pipelines—Components and Environments

18.4.1 Pipelines from Versioned Parts

An AML pipeline is a DAG of **components** [71]. Each component declares inputs, outputs, a command, and an **environment**; each is versioned independently and reused across pipelines. Environments pin the software world (base image plus dependencies) so a pipeline that ran in March reproduces in October. The canonical pipeline shape is Preprocess → Train → Evaluate → (conditionally) Register.

```

1 from azure.ai.ml import dsl, Input
2 from azure.ai.ml.entities import Environment
3
4 env = Environment(
5     name="churn-env", version=3,
6     image="mcr.microsoft.com/azureml/openmpi4.1.0-ubuntu22.04",
7     conda_file="conda.yml",
8 )
9
10 @dsl.pipeline()
11 def churn_pipeline(raw: Input):
12     prep = preprocess_component(input_data=raw)
13     train = train_component(train_data=prep.outputs.clean)
14     eval_ = evaluate_component(model=train.outputs.model,
15                               test_data=prep.outputs.test)
16     return {"model": train.outputs.model}
17
18 ml_client.jobs.create_or_update(
19     churn_pipeline(raw=Input(path="azureml:features:1"))

```

Listing 18.1: An AML pipeline assembled from registered components via the v2 SDK.

18.4.2 The Managed Feature Store

AML’s managed feature store operationalizes Chapter 17’s contract: feature sets are versioned, materialized to offline (lake) and online (cache) stores, and carry point-in-time-correct retrieval for training-set assembly. Features stop being notebook-local code and become cataloged, monitored assets with owners—the ML counterpart of the medallion gold zone.

18.4.3 Databricks in the Picture

Where feature computation is heavy—sessionization over billions of events, graph features—Databricks does the work on the lake and writes the feature tables; AML pipelines then consume them as data assets. The boundary rule: Databricks owns *feature materialization*, AML owns *training orchestration and the registry*; the feature table in the governed lake is the contract between them.

18.5 Thursday Hands-on Project: Preprocess, Train, Evaluate, Register

Build the canonical AML pipeline from the Python SDK and execute it from a notebook: preprocess the churn features, train, evaluate, and register the model only if it clears a metric gate.

18.5.1 Provision the Workspace

```

1 $rg = "rg-de-azure-book-week18"
2 az group create --name $rg --location eastus
3 az extension add --name ml
4 az ml workspace create --name aml-week18 --resource-group $rg --location eastus
5
6 az ml compute create --name cpu-cluster --resource-group $rg '
7   --workspace-name aml-week18 --type AmlCompute '
8   --size Standard_DS3_v2 --min-instances 0 --max-instances 4 '
9   --idle-time-before-scale-down 300

```

Listing 18.2: Workspace with a managed identity and a scale-to-zero training cluster.

Register the ADLS datastore with the workspace managed identity and grant it read access to the feature container.

18.5.2 Author and Run the Pipeline

Define `preprocess`, `train`, and `evaluate` components (each a small script plus environment), assemble them as in Listing 18.1, and submit. In the studio, inspect the pipeline graph, per-step logs, and the tracked metrics.

18.5.3 Gate Registration on a Metric

Extend the evaluate component to emit `auc`, and add a condition: register only when $AUC \geq 0.80$. Run once with a deliberately degraded feature set (drop the strongest feature) and confirm the gate blocks registration—the failing case is the deliverable that proves the gate exists.

18.6 Friday Use Case: GPU PyTorch over a Governed Lake

A media company’s data scientists must train custom PyTorch recommendation models on GPU clusters against customer interaction data in the governed ADLS lake—without copying data to unmanaged storage and without any secrets in notebooks.

18.6.1 Design

Workspace managed identity holds **Storage Blob Data Reader** on the lake; a data asset versions each training snapshot. GPU compute clusters (NC-family) scale from zero per job.

Environments pin CUDA/PyTorch versions; MLflow tracking captures every run; the registry holds promoted models with lineage back to the exact data asset, environment, and code version. Purview (Chapter 22) sees the lake side; AML’s lineage covers the model side. The security review can trace every model to the data and identity that produced it—which is the actual requirement behind “secure access to governed data.”

Common Pitfall

Export-first workflows. “Export the lake to CSV for training” breaks lineage, residency, and freshness in one step. The datastore pattern exists precisely so governed data is read *in place* by an auditable identity.

18.7 Architectural Decision Record Template

Per pipeline record: component inventory with owners, environment pinning policy, data asset versions consumed, the metric gate and its threshold rationale, compute sizing (SKU, min/max nodes, idle scale-down), and the Databricks/AML responsibility boundary.

18.8 Operational Deep Dive: Pipeline Schedules and Reuse

AML pipelines run on schedules or via Event Grid triggers (data-arrival-driven training). Because components are versioned and cached, unchanged steps reuse prior outputs—re-running a pipeline after a code change to `train` does not re-run `preprocess`. Monitor pipeline duration per step; the step that doubled is usually the one whose data doubled, which is a feature-table question, not an ML question.

18.9 Troubleshooting Patterns

Job fails at environment build → dependency conflict in the conda spec; pin versions. Job fails on data access → the workspace identity lacks the lake RBAC role or the datastore fell back to key auth. Gate never passes after a data change → inspect the feature table before questioning the threshold. Cluster never scales → max instances or regional quota; check both before resizing the job.

18.10 Week 18 Lab Rubric

Criterion	Meets expectation	Needs improvement
Reproducibility	Versioned data, environment, components; pipeline re-runs identically	Local notebooks and unpinned dependencies
Security	Managed-identity datastore; no secrets in notebooks	Personal keys or mounted storage
Gating	Metric gate blocks a demonstrated failing model	Every run registers a model
Cost	Cluster scales to zero; idle time bounded	Always-on GPU cluster

Table 18.1: Week 18 rubric for the AML pipeline deliverables.

18.11 Interview Q&A

18.11.1 Concepts and Trade-offs

1. **Q: When would you choose AutoML over a custom training notebook?**

A: For baselines and well-structured tabular problems—AutoML sets the bar a custom model must beat to justify its complexity.

2. **Q: Why use AML compute clusters instead of compute instances for training?**

A: Clusters autoscale across nodes for training and scale to zero when idle; instances are single-node development desks.

3. **Q: How does AML securely access governed ADLS training data?**

A: Through a datastore bound to the workspace managed identity with lake RBAC—no keys, full audit trail.

4. **Q: What does model registration give you beyond saving a file?**

A: Versioned identity with lineage to data, environment, code, and metrics—the answer to “what exactly produced this artifact.”

18.12 Further Reading

The Azure ML overview [83], the pipelines concept guide [71], and the examples repository [20] are the primary sources. Chapter 19 continues with endpoints, drift, and the retraining loop; Chapter 17 supplies the feature contract this pipeline consumes.

Key Takeaways

- Instances for development, clusters for training, minimum nodes zero: ML compute economics mirror Spark’s.
- Managed-identity datastores are the only governed path from the lake to training.
- AutoML sets the baseline; custom components must beat it to justify their complexity.
- Versioned components plus pinned environments make pipelines reproducible and cache-efficient.
- The managed feature store turns features into cataloged, point-in-time-correct assets.
- Registration is lineage: every production model answers “what data, code, and environment made me.”

18.13 Exercises

1. **(Conceptual)** Explain why a zero-minimum-node cluster changes team behavior, not just cost.
2. **(Conceptual)** What breaks when two pipelines share a component version and one team edits it?
3. **(Conceptual)** Why does the Designer produce less reviewable artifacts than components-as-code?
4. **(Hands-on)** Complete the Thursday project, including the gated-registration failing case.

5. **(Hands-on)** Run an AutoML sweep on the same data asset and compare its best AUC against your pipeline’s model.
6. **(Hands-on)** Change only the `train` component and observe step-level caching on re-run; record which steps executed.
7. **(Design)** Design the Databricks/AML boundary for the media company: which features are materialized where, and why.
8. **(Design)** Write the ADR for the metric gate: threshold, metric choice, and the override process.
9. **(Design)** Specify an Event Grid trigger that retrains when the feature table’s drift metric crosses a threshold (preview Chapter 19).
10. **(Interview)** Prepare a two-minute answer to “why did this model change behavior after a re-run with no code changes?”

18.14 Capstone Project: Gated Training Pipeline over Governed Features

Capstone Project

Mission: Deliver a reproducible training pipeline: versioned data assets read via managed identity, pinned environments, componentized preprocess/train/evaluate steps, a metric-gated registration, and AutoML as the recorded baseline.

Estimated time: 5–6 hours.

What you will practice:

- Workspace, compute, and datastore provisioning with identity-based access.
- Component authoring and pipeline assembly with the v2 SDK.
- Metric gates with a proven failing case.
- Baseline discipline: AutoML versus custom, on record.

Scenario

The subscription business graduates from Chapter 17’s hand-run training to a pipeline the platform team can schedule. Data science owns components; the platform team owns the gate; compliance requires full lineage from model to training data.

Requirements

- **Access:** all data read through the managed-identity datastore; zero secrets in code.
- **Reproducibility:** pinned environment; versioned components and data assets; re-run produces identical metrics.
- **Gate:** registration only at $AUC \geq 0.80$, with the failing case demonstrated.
- **Baseline:** an AutoML run on the same asset, with the comparison recorded in the model description.

Reference Architecture

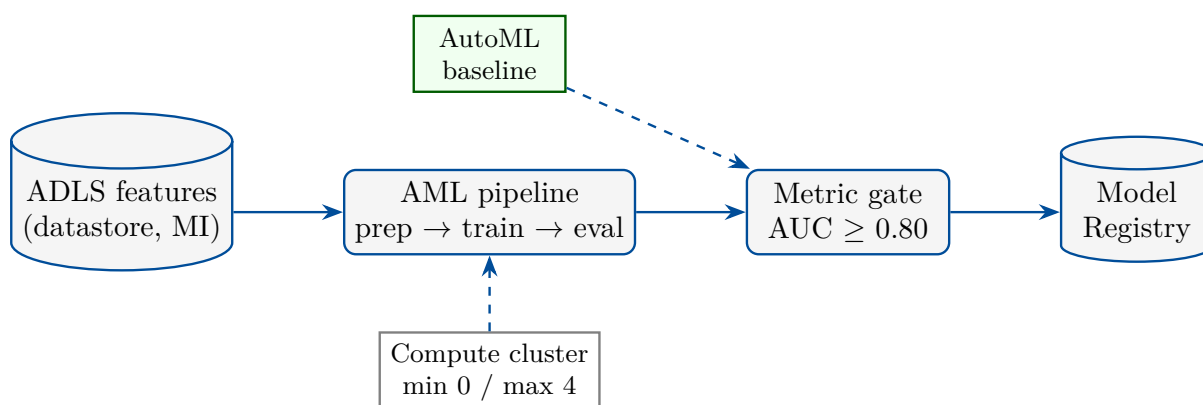


Figure 18.1: Capstone architecture: governed features feed a gated pipeline; AutoML provides the recorded baseline.

Implementation Steps

1. Provision workspace and compute (Listing 18.2); register the datastore and data asset.
2. Author the three components and assemble the pipeline (Listing 18.1).
3. Add the metric gate; demonstrate pass and fail runs.
4. Run the AutoML baseline; record the comparison in the registry description.
5. Verify lineage: registry entry resolves to data asset, environment, code, and metrics.

Deliverables

- Component and pipeline code, environment specs, and provisioning scripts.
- Gate pass/fail evidence and the AutoML comparison record.
- A README that reproduces the pipeline from a clean workspace.

Acceptance Criteria

- A re-run with unchanged inputs reuses cached steps and reproduces metrics.
- A degraded feature set is blocked from registration by the gate.
- Every registered model traces to its data asset version and environment.

Stretch Goals

- Add the managed feature store and swap the pipeline input to a versioned feature set.
- Schedule the pipeline and add step-duration regression alerting.
- Move feature materialization to a Databricks job and consume its output as the data asset.

Model Serving, Monitoring, and the Retraining Loop

Learning Objectives

- Choose between managed online endpoints (real-time) and batch endpoints (offline scoring) from the workload’s latency and volume profile.
- Deploy a registered model with blue-green traffic shifting and safe rollback.
- Implement production data collection and distinguish input drift, prediction drift, and concept drift.
- Wire a drift alert through Azure Monitor and Event Grid to an automated retraining pipeline.
- Gate model promotion on metric thresholds plus human approval—never on recency.
- Architect the full MLOps lifecycle for a production recommendation engine.
- Keep training and serving features aligned through the managed feature store at serving time.

Cross-Cloud Equivalence

Model serving splits everywhere into real-time endpoints and batch scoring, and monitoring splits into infrastructure health, data drift, and model quality. The vocabulary below is the translation table.

Capability	AWS		Azure		GCP
Real-time serving	SageMaker	end-points	AML managed online endpoints		Vertex AI endpoints
Batch scoring	SageMaker	batch transform	AML batch endpoints		Vertex batch prediction
Model monitoring	SageMaker	Model Monitor	AML model monitoring / drift		Vertex Model Monitoring
Registry and stages	SageMaker	registry + approval	AML registry + tags/stages		Vertex registry + aliases
Retrain trigger	EventBridge	→ pipeline	Event Grid → AML pipeline		Eventarc → pipeline

Fabric: MLflow models + SynapseML serving; **Snowflake:** Snowpark ML registry + SQL inference; **MongoDB:** in-app scoring

The MLOps loop in this chapter—monitor, detect drift, retrain, gate, promote—is platform-neutral. Memorize the loop and the vendor product names become interchangeable details.

Key Terms

Managed online endpoint: AML’s hosted real-time scoring service with auth, scaling, and blue-green deployments. **Batch endpoint:** AML’s offline scoring over large inputs on a schedule. **Data drift:** statistical change in production input distributions versus the training baseline. **Blue-green deployment:** two deployments behind one endpoint, with traffic shifted gradually and rollback by shifting back. **Promotion gate:** the criteria (metric threshold, tests, approval) a model must pass before serving production traffic.

19.1 Week 19 Overview: The Model Is Not Done When It Trains

Chapter 18 produced a registered model. This chapter operates it. The production reality of ML is that models degrade: the world changes, input distributions shift, and yesterday’s champion quietly becomes wrong. MLOps is the discipline that notices: serve with safe deployment mechanics, collect what the model sees, measure drift, retrain on evidence, and promote through gates. The data engineer’s role is central—drift is a data problem, collection is a pipeline problem, and retraining is an orchestration problem.

19.2 Monday: Managed Endpoints—Real-Time and Batch

19.2.1 Online Endpoints

A managed online endpoint hosts one or more *deployments*—a model, an environment, a scoring script, and compute—behind a REST URI with key or Microsoft Entra token authentication. AML manages provisioning, scaling, and TLS; you manage the model and the scoring contract. This is the serving path for per-request latency budgets: fraud checks at swipe time, recommendations at page load.

19.2.2 Batch Endpoints

Batch endpoints score large inputs asynchronously: point the job at a data asset, AML fans the scoring across compute, and results land in storage. Nightly churn scoring for two million customers—Chapter 17’s workload, at platform scale—belongs here. The cost profile matches the duty cycle: compute exists only while scoring.

Dimension	Online endpoint	Batch endpoint
Latency budget	Milliseconds per request	Minutes per job
Input	Single/small payloads	Data assets, millions of rows
Cost shape	Always-on minimum instances	Per-job compute, scales to zero
Failure mode	Latency/error rate SLIs	Job completion and output freshness
Canonical use	Fraud at swipe, recommendations	Nightly churn, risk, forecasts

Table 19.1: Choose the endpoint family from the workload’s latency and volume, not from fashion.

19.3 Tuesday: Safe Deployment—Blue-Green and Gated Promotion

19.3.1 Traffic Shifting

Endpoints host multiple deployments with percentage-based traffic splitting. The safe promotion pattern: deploy the challenger at 5% traffic, compare its latency, error rate, and score distribution against the champion, shift to 100% over hours, and keep the champion warm until the challenger has survived a full business cycle. Rollback is a traffic shift, not a redeploy—seconds, not an incident.

19.3.2 The Promotion Gate

What earns 100% of traffic is a policy decision made explicit: a metric threshold on a holdout (Chapter 18’s gate), passing shadow-traffic comparison, a human approval recorded in the registry, and a rollback plan. “Newest model wins” is not a policy; it is how a bad retrain reaches production at 03:00 on a schedule.

Common Pitfall

Promoting on training metrics alone. A model that aces last month’s holdout can fail on this month’s traffic. Shadow and canary evaluation on *live* inputs is the only promotion evidence that includes the present.

19.4 Wednesday: Drift Detection and the Retraining Loop

19.4.1 Three Kinds of Drift

- **Input (data) drift:** production feature distributions diverge from the training baseline—a new channel, a pricing change, a broken upstream feed. Detected by comparing feature histograms/statistics over time.
- **Prediction drift:** the output distribution shifts—the model now scores 40% of customers high-risk instead of 8%. Often the first visible symptom.
- **Concept drift:** the relationship between features and outcomes changes—the definition of “a good customer” moved. Detectable only with delayed ground truth, which is why scoring pipelines must persist inputs *and* eventual outcomes for later join-back.

19.4.2 The Automated Loop

AML model monitoring compares production data (collected from endpoint logging) against the training baseline and emits drift signals. The production wiring: monitor → Azure Monitor alert → Event Grid event → Logic App or direct trigger → the Chapter 18 retraining pipeline → metric gate → human approval → blue-green promotion [83]. Automation handles detection and retraining; humans handle promotion, because the decision to change what customers experience is a business decision with an audit trail.

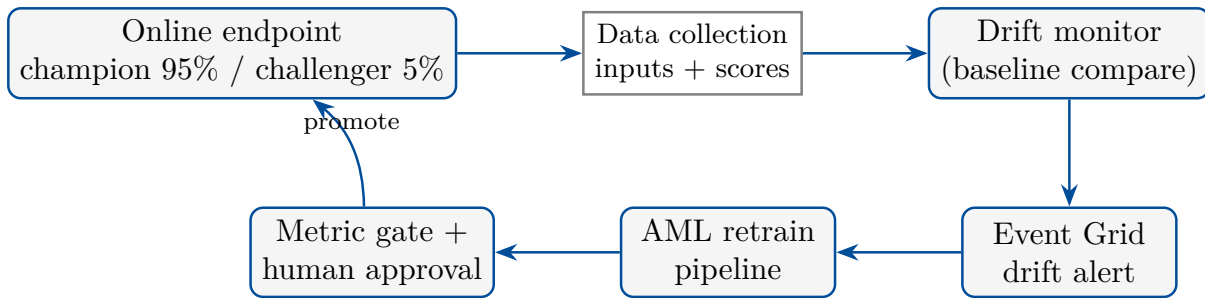


Figure 19.1: The MLOps loop: serve, collect, monitor, alert, retrain, gate, promote. Humans own the promotion edge.

Tip

Persist every scoring request’s features and the eventual outcome with a join key. Without that table, concept drift is invisible and retraining has no fresh labels; with it, every MLOps question becomes a SQL query.

19.5 Thursday Hands-on Project: Deploy, Watch, Drift, Retrain

Take Chapter 18’s registered churn model to production: deploy to a managed online endpoint, test with a REST payload, configure monitoring with the training data as baseline, inject drift, and watch the loop fire.

19.5.1 Deploy and Test

```

1 az ml online-endpoint create --name churn-ep '
2   --resource-group $rg --workspace-name aml-week18 --auth-mode key
3
4 az ml online-deployment create --name blue --endpoint-name churn-ep '
5   --resource-group $rg --workspace-name aml-week18 '
6   --model azureml:churn-model:3 --instance-type Standard_DS3_v2 '
7   --instance-count 1
8
9 az ml online-endpoint update --name churn-ep '
10  --resource-group $rg --workspace-name aml-week18 --traffic blue=100
11
12 az ml online-endpoint invoke --name churn-ep '
13  --resource-group $rg --workspace-name aml-week18 '
14  --request-file sample-payload.json

```

Listing 19.1: Endpoint, deployment, and a scored test payload.

19.5.2 Monitor and Inject Drift

Enable data collection on the deployment and configure a model monitor with the training data asset as baseline. Then point a synthetic traffic generator at the endpoint and shift one feature’s distribution (for example, double `support_tickets_30d`). The monitor’s drift metric should cross the alert threshold within its evaluation cadence; the Event Grid wiring should enqueue a retraining run.

19.5.3 Close the Loop Manually

Let the retraining pipeline run, watch the gate evaluate, approve the promotion, shift traffic blue → green, and record the whole cycle in the model’s registry description. A loop demonstrated once by hand is a loop you can automate with confidence.

19.6 Friday Use Case: MLOps for a Production Recommendation Engine

A retailer’s recommendation engine serves 30 M requests/day. Symptoms of decay arrive quarterly: click-through sags, and nobody can say when it started. The MLOps design:

- **Serving:** online endpoint with blue-green; P99 latency and error-rate SLIs with alerts.
- **Collection:** every request’s features, served recommendations, and the click outcome joined within 24 hours into `gold.reco_outcomes`.
- **Monitoring:** weekly input-drift comparison against the training baseline; daily prediction-distribution check; monthly concept-drift evaluation on joined outcomes.
- **Retraining:** drift alert → Event Grid → retraining pipeline over the managed feature store (training and serving features from one definition—Chapter 17’s contract, now load-bearing).
- **Promotion:** metric gate on a fresh holdout plus a one-day shadow comparison, then human approval, then staged traffic shift.

Common Pitfall

Drift alerts without a retraining runbook. An alert that pages someone who then improvises is not a loop. The alert’s runbook names the pipeline, the gate, the approver, and the rollback—before the first alert fires.

19.7 Architectural Decision Record Template

Per served model record: endpoint family and SKU with the latency/volume justification, traffic-shifting policy, collection schema (features, scores, outcomes, join keys), drift metrics and thresholds with the baseline definition, the retraining trigger path, and the promotion gate with its approvers.

19.8 Operational Deep Dive: Endpoint SLIs and Cost

Track four SLIs per online endpoint: request latency (P50/P99), error rate, saturation (CPU/GPU utilization), and traffic. Cost follows instance-hours; autoscaling rules should scale on utilization *and* request rate, and deployments with near-zero traffic should be scaled to the minimum or consolidated onto shared inference pools. Batch endpoints need job-duration and output-freshness SLIs instead—the batch analogue of Chapter 14’s silence alert: a scoring job that did not run produces no error, only absence.

19.9 Troubleshooting Patterns

Endpoint 500s after deploy → scoring script/environment mismatch with the registered model’s expectations; the alignment validator from Chapter 17 catches this pre-deploy. Latency doubled

after promotion → the challenger’s compute SKU differs or the model is materially larger. Drift alerts daily → threshold set tighter than natural weekday/weekend seasonality; compare against seasonal baselines. Retraining loop fires but gate always fails → the gate’s holdout is stale; refresh evaluation data with the drifted distribution.

19.10 Week 19 Lab Rubric

Criterion	Meets expectation	Needs improvement
Serving	Endpoint family justified; blue-green with measured comparison	Single deployment, big-bang promotion
Monitoring	Input + prediction drift with baseline; collection schema complete	No collection; drift discovered by users
Loop	Drift alert triggers retraining; gate and approval evidenced	Alert exists; nothing downstream
Cost	Autoscaling or scale-to-zero configured with rationale	Fixed large instances around the clock

Table 19.2: Week 19 rubric for the serving and monitoring deliverables.

19.11 Interview Q&A

19.11.1 Concepts and Trade-offs

1. **Q: Real-time vs. batch managed endpoints—which fits nightly scoring?**
A: Batch endpoints: millions of rows, minutes of latency tolerance, compute that scales to zero between runs.
2. **Q: What is data drift, and how does AML detect it?**
A: Statistical divergence of production inputs from the training baseline; model monitoring compares collected production data against the baseline on a cadence.
3. **Q: What should trigger a retraining pipeline in a mature setup?**
A: Evidence—drift metrics crossing thresholds or quality decay on joined outcomes—not a calendar alone.
4. **Q: What should gate model promotion to production?**
A: A metric threshold on fresh data, shadow/canary comparison on live traffic, human approval, and a rollback plan.

19.12 Further Reading

The AML documentation [83] covers endpoints and monitoring; the examples repository [20] includes deployment and monitoring samples. Chapter 18’s pipeline [71] is the retraining half of this chapter’s loop, and Chapter 20 extends the platform to unstructured data and generative workloads.

Key Takeaways

- Match the endpoint family to the workload: online for per-request latency, batch for volume scoring.

- Blue-green traffic shifting makes rollback a configuration change, not an incident.
- Collect features, scores, and outcomes with join keys—the table every MLOps question depends on.
- Input, prediction, and concept drift are different failures with different detectors.
- Automation retrains; humans promote. The gate is metric plus evidence plus approval.
- A drift alert without a runbook is noise; wire the loop before you need it.

19.13 Exercises

1. **(Conceptual)** Why can concept drift not be detected from inputs alone?
2. **(Conceptual)** Explain why “newest model wins” fails as a promotion policy.
3. **(Conceptual)** What is the batch-endpoint analogue of an online endpoint’s error-rate SLI?
4. **(Hands-on)** Complete the Thursday project: deploy, drift injection, alert, retrain, gate, promote.
5. **(Hands-on)** Add a 5% challenger deployment and compare its score distribution against the champion’s on live traffic.
6. **(Hands-on)** Build the outcomes join-back table and compute a one-week concept-drift report.
7. **(Design)** Write the drift-alert runbook for the recommendation engine: pipeline, gate, approver, rollback.
8. **(Design)** Design seasonal-aware drift thresholds for a business with strong weekly cycles.
9. **(Design)** Write the ADR choosing batch endpoints plus PREDICT (Chapter 17) versus online endpoints for the churn use case.
10. **(Interview)** Prepare a two-minute answer to “the model’s accuracy looks fine but the business metric is falling—what do you check?”

19.14 Capstone Project: The Full MLOps Loop for Churn

Capstone Project

Mission: Close the loop on Chapters 17–18: serve the churn model behind a managed online endpoint with blue-green deployments, collect production traffic, detect drift against the training baseline, and demonstrate the alert → retrain → gate → promote cycle end-to-end.

Estimated time: 5–7 hours.

What you will practice:

- Endpoint deployment, traffic shifting, and invocation testing.
- Production data collection with outcome join-back.
- Drift monitoring with a deliberate-drift experiment.
- Event-driven retraining with human-gated promotion.

Scenario

The subscription business’s churn model is live and—leadership suspects—aging. You are to prove the platform can *notice* decay and respond, with evidence at each edge of the loop.

Requirements

- **Serving:** online endpoint with two deployments and a documented traffic-shift policy.
- **Collection:** features, scores, and outcomes persisted with join keys.
- **Detection:** drift monitor against the training baseline; a synthetic drift experiment that provably fires.
- **Response:** Event Grid → retraining pipeline → metric gate → recorded human approval → blue-green promotion.

Reference Architecture

Figure 19.1 is the reference; the collection store lands in the gold zone so the Chapter 22 governance layer catalogs production ML data alongside everything else.

Implementation Steps

1. Deploy the endpoint and blue deployment (Listing 19.1); invoke with a test payload.
2. Enable collection; verify the feature/score/outcome table populates.
3. Configure the monitor; run the synthetic drift experiment; capture the alert.
4. Execute the retraining pipeline from the alert path; gate, approve, and shift traffic.
5. Write the loop’s evidence pack into the registry description.

Deliverables

- Endpoint/deployment definitions, monitoring configuration, and Event Grid wiring.
- The drift-experiment evidence: metrics, alert, retrain run, gate decision.
- A one-page operations runbook for the loop.

Acceptance Criteria

- Injected drift produces an alert and a retraining run without manual detection.
- Promotion requires the gate and a recorded approval; rollback is a traffic shift.
- The outcomes table supports a concept-drift query over any trailing window.

Stretch Goals

- Add seasonal-aware thresholds and show the false-alert reduction.
- Automate challenger shadow evaluation with a comparison report attached to the approval request.
- Extend monitoring to the Chapter 16 reverse-ETL consumer: does retention action volume track score distribution?

Unstructured Data and Generative AI Data Products

Learning Objectives

- Extract key-value pairs and tables from scanned documents with Azure AI Document Intelligence.
- Apply Azure AI Language (NER, key phrases, PII detection) and Translator inside data pipelines.
- Trigger document processing with an Azure Function on ADLS Gen2 upload and persist structured JSON to the lake.
- Index extracted metadata in Azure AI Search so unstructured content becomes queryable.
- Design retrieval-augmented generation (RAG) data products over enterprise content with Azure OpenAI.
- Govern PII exposed by OCR and embeddings, and evaluate hallucination risk in generated outputs.
- Complete the Part V milestone: an end-to-end MLOps pipeline from feature extraction to nightly batch scoring.

Cross-Cloud Equivalence			
Document AI, search, and managed foundation models exist on every cloud; the pipeline pattern—ingest, extract, index, retrieve, generate—is identical across vendors.			
Capability	AWS	Azure	GCP
Document extraction	Textract	AI Document Intelligence	Document AI
NLP / PII detection	Comprehend	AI Language	Cloud Natural Language
Translation	Amazon Translate	Azure Translator	Cloud Translation
Search / vectors	OpenSearch / Kendra	AI Search (vector + hybrid)	Vertex AI Search
Foundation models	Bedrock	Azure OpenAI Service	Vertex AI (Gemini)
Event-driven processing	Lambda	Azure Functions	Cloud Functions
Fabric: AI skills over OneLake; Snowflake: Cortex (LLM functions in SQL); MongoDB: Atlas Vector Search.			
The RAG pattern taught here—chunk, embed, index, retrieve, ground the model—is the			

same architecture whether the vector store is AI Search, OpenSearch, pgvector, or Atlas Vector Search.

Key Terms

Document Intelligence: Azure’s document extraction service—layout, tables, key-value pairs, and prebuilt invoice/receipt/ID models. **AI Search:** Azure’s search service with keyword, vector, and hybrid (semantic) retrieval over indexed content. **RAG:** retrieval-augmented generation—grounding a language model’s answers in retrieved enterprise documents. **Embedding:** a vector representation of text enabling semantic similarity search. **Skillset:** AI Search’s enrichment pipeline applied at indexing time (OCR, entity extraction, translation).

20.1 Week 20 Overview: The Other 80% of Enterprise Data

Everything so far assumed data arrives as rows. Enterprise reality is messier: invoices are scanned PDFs, contracts are Word documents, support tickets are prose, product catalogs are images. Week 20 brings unstructured content into the platform with Azure’s AI services [72], then goes one step further: once content is extracted, embedded, and indexed, it can *ground generative models*—the retrieval-augmented generation pattern that turned search indexes into the data layer of the generative AI wave [84]. The data engineer’s job is unchanged in kind: move content reliably, extract faithfully, govern what is sensitive, and make it findable.

20.2 Monday: Document Intelligence and Vision

20.2.1 Layout, Prebuilt, and Custom Models

Document Intelligence offers three depths. **Read/Layout** extracts text, tables, and structure from any document. **Prebuilt models**—invoices, receipts, IDs, business cards—return typed fields (`VendorName`, `InvoiceTotal`, line items) with confidence scores, no training required. **Custom models** learn your forms from a handful of labeled samples. Choose the shallowest model that meets the field-level accuracy bar; prebuilt invoice extraction beats a custom model you have to maintain, until your invoices’ layouts defeat it.

20.2.2 Confidence Is Part of the Schema

Extraction is probabilistic. Every extracted field carries a confidence; the pipeline design must decide what happens below the threshold—route to human review, flag in the output schema, or quarantine like a contract violation (Chapter 11). Treating OCR output as deterministic text is the unstructured-data version of trusting external input.

Common Pitfall

OCR-exposed PII. Scanning user-uploaded documents surfaces names, addresses, account numbers, and health data that never appeared in your structured schemas. Classify extracted output with the same Purview rules (Chapter 22) as any other zone, and run AI Language’s PII detection over extracted text before it reaches searchable indexes.

20.3 Tuesday: Language, Translator, and Enrichment

20.3.1 Language Understanding as a Transform

AI Language provides named-entity recognition, key-phrase extraction, sentiment, summarization, and PII detection as callable transforms. In a data pipeline they are just another hop: support tickets land in bronze, a Function or Databricks job calls the Language endpoint, and the silver table gains `entities`, `key_phrases`, `sentiment`, and `pii_redacted_text` columns. Translator plays the same role for multilingual estates—translate at ingestion so downstream search and analytics operate on one language, preserving originals in bronze.

20.3.2 Batching and Cost

Cognitive services bill per text record or page; the cost lever is batching (submit documents in batches, cache by content hash so reprocessed files do not rebill) and selectivity (extract only the fields the data product needs). A pipeline that re-OCRs a 50 PB corpus on every retry has a retry-shaped cost bug.

20.4 Wednesday: Event-Driven Document Processing and Search

20.4.1 The Function-on-Upload Pattern

The canonical unstructured pipeline: upload to ADLS Gen2 → Event Grid BlobCreated → Azure Function → Document Intelligence → structured JSON to the silver zone. This is Chapter 12’s event-driven orchestration applied to documents; the Function is the right-sized compute because extraction latency is dominated by the service call, not by local work.

```

1 import azure.functions as func
2 from azure.ai.documentintelligence import DocumentIntelligenceClient
3 from azure.identity import DefaultAzureCredential
4 import json, os
5
6 client = DocumentIntelligenceClient(
7     endpoint=os.environ["DOCINTEL_ENDPOINT"],
8     credential=DefaultAzureCredential())
9
10 def main(event: func.EventGridEvent, outputBlob: func.Out[str]):
11     blob_url = event.get_json()["url"]
12     poller = client.begin_analyze_document_from_url(
13         "prebuilt-invoice", blob_url)
14     result = poller.result()
15     fields = {k: {"value": v.content, "confidence": v.confidence}
16               for doc in result.documents
17               for k, v in doc.fields.items()}
18     outputBlob.set(json.dumps({
19         "source": blob_url,
20         "model": "prebuilt-invoice",
21         "fields": fields}))

```

Listing 20.1: Event Grid-triggered Function extracting an invoice and landing JSON in the lake.

20.4.2 Making It Searchable

Azure AI Search indexes the extracted JSON—full-text over content, vector indexes over embeddings, hybrid queries combining both, with semantic ranking on top [16]. Indexers can crawl the lake directly with a skillset that applies OCR and entity extraction *at index time*, which

suits content that needs search but not a curated silver schema. The design decision: extract-then-index when the structured fields are themselves a data product (downstream SQL, BI); index-time enrichment when search *is* the product.

20.5 Thursday Hands-on Project: Invoice Extraction Function

Build the Function from Listing 20.1: a scanned-PDF invoice uploaded to the lake is extracted to key-value JSON within seconds, with low-confidence fields flagged.

20.5.1 Provision

```

1 $rg = "rg-de-azure-book-week20"
2 az group create --name $rg --location eastus
3
4 az cognitiveservices account create --name docintel-wk20 '
5   --resource-group $rg --kind FormRecognizer --sku S0 '
6   --location eastus --custom-domain docintel-wk20
7
8 az functionapp create --name func-wk20-doc --resource-group $rg '
9   --consumption-plan-location eastus --runtime python --runtime-version 3.11 '
10  --functions-version 4 --storage-account <stfunc> --os-type Linux '
11  --assign-identity "[system]"

```

Listing 20.2: Document Intelligence and a Function App with a managed identity.

Grant the Function’s managed identity **Cognitive Services User** on the Document Intelligence account and blob access to the lake; wire the Event Grid subscription on **BlobCreated** for the **incoming/invoices/** prefix.

20.5.2 Test the Confidence Path

Upload a clean invoice (fields extracted, confidences high, JSON lands in silver) and a deliberately degraded scan—crumpled, low DPI. Verify low-confidence fields carry their scores and route to the review queue; verify the Function is idempotent against duplicate Event Grid deliveries by keying the output blob name on the source blob’s content hash.

20.6 Friday Use Case: Terabytes of User Images, Searchable

A marketplace ingests terabytes of seller-uploaded images. Requirements: OCR any text in images, tag objects and scenes, detect policy-violating content, and make all of it searchable by support and trust-and-safety teams.

20.6.1 Architecture

Upload → Event Grid → Function fan-out → Vision (captioning, tags, OCR) and Content Moderator → extracted metadata JSON to silver → AI Search index (keyword over tags/text, vector over image-caption embeddings) → an internal search app. PII detection runs over OCR text before indexing; moderation verdicts gate public visibility. The Chapter 13 principles apply at the ingestion edge; the Chapter 22 governance applies to what OCR reveals.

20.6.2 From Search to RAG

The same index grounds a generative assistant: a trust-and-safety analyst asks “show me listings that look like counterfeit handbags from new sellers,” the query embeds and retrieves matching content from AI Search, and Azure OpenAI synthesizes an answer with citations to the retrieved

listings [84]. The data product discipline is what makes the AI safe: grounded answers, citations, access control enforced at retrieval (security filters per user), and evaluation of groundedness before the assistant ships.

Common Pitfall

Ungrounded generation over enterprise data. A language model answering from parametric memory will invent plausible policy numbers and listing IDs. RAG exists to make answers checkable: if the assistant cannot cite the retrieved document, it does not answer. Enforce citation requirements in the system prompt and in evaluation.

20.7 Architectural Decision Record Template

Per unstructured pipeline record: extraction model choice (read/prebuilt/custom) with the accuracy evidence, confidence thresholds and the human-review route, PII handling of extracted text, index schema (keyword/vector/hybrid) with the query patterns it serves, and—for generative surfaces—the grounding, citation, and evaluation policy.

20.8 Operational Deep Dive: Evaluation as Pipeline Testing

Generative data products need evaluation pipelines the way ETL needs data-quality tests. Maintain a golden set of analyst questions with known-correct answers; run the assistant against it on every index or prompt change; measure groundedness and citation accuracy, not vibes. A model or prompt change that regresses the golden set blocks deployment—the Chapter 19 promotion gate, applied to prompts and indexes.

20.9 Troubleshooting Patterns

Extraction quality collapsed after a vendor redesign → prebuilt model drift; evaluate a custom model on the new layout. Function timeouts on large PDFs → extraction is slow for hundred-page scans; switch to asynchronous pattern with a completion queue. Search results stale → indexer cadence or change detection policy on the lake datasource. Assistant answers ungrounded → retrieval returning irrelevant chunks; fix chunking and top-*k* before touching the prompt.

20.10 Week 20 Lab Rubric

Criterion	Meets expectation	Needs improvement
Extraction	Confidence-aware output with a review route for low scores	OCR text trusted blindly
Eventing	Idempotent Function keyed on content hash	Duplicate processing per Event Grid retry
PII	Extracted text classified/redacted before indexing	PII searchable in the index
Generative	Grounded answers with citations and golden-set evaluation	Ungrounded answers, untested prompts

Table 20.1: Week 20 rubric for the unstructured-data deliverables.

20.11 Interview Q&A

20.11.1 Concepts and Trade-offs

1. **Q: Which service extracts key-value pairs from scanned invoices?**

A: Azure AI Document Intelligence—the prebuilt invoice model returns typed fields with confidence scores, no training needed.

2. **Q: Why trigger document processing with a Function on blob upload?**

A: Event-driven processing removes polling; the Function is right-sized because extraction latency lives in the service call, not local compute.

3. **Q: How do you make extracted document metadata searchable?**

A: Land structured JSON in the lake and index it in AI Search—keyword, vector, or hybrid depending on query patterns.

4. **Q: What PII risks arise when OCR-ing user-uploaded documents?**

A: OCR surfaces PII that structured schemas never declared; extracted text must be classified and redacted before it reaches searchable indexes.

20.12 Further Reading

The Document Intelligence [72], AI Search [16], and Azure OpenAI [84] documentations are the primary sources. The ADF documentation [76] covers calling cognitive services from pipelines, and Chapter 22's Purview material governs what extraction exposes.

Key Takeaways

- Choose the shallowest extraction model that meets the accuracy bar: read, then prebuilt, then custom.
- Confidence scores are part of the output schema; low-confidence fields need a human-review route.
- Event Grid plus Functions makes document processing event-driven; content-hash keys make it idempotent.
- Extract-then-index when fields are a data product; index-time enrichment when search is the product.
- OCR and embeddings expose PII—classify and redact before indexing.
- RAG makes generative answers checkable; golden-set evaluation gates every prompt or index change.

20.13 Exercises

1. **(Conceptual)** When does a custom Document Intelligence model beat the prebuilt invoice model?
2. **(Conceptual)** Explain why extraction output must be treated as probabilistic, and what that implies for schema design.
3. **(Conceptual)** Compare keyword, vector, and hybrid retrieval for the trust-and-safety search app.

4. **(Hands-on)** Complete the Thursday project with both the clean and degraded invoices.
5. **(Hands-on)** Add AI Language PII detection as a second hop and verify redaction before indexing.
6. **(Hands-on)** Build the golden-set evaluation harness and demonstrate a blocked regression.
7. **(Design)** Design the security-trimmed RAG index: per-user document visibility enforced at retrieval.
8. **(Design)** Write the ADR for extract-then-index versus index-time enrichment for the marketplace.
9. **(Design)** Specify the moderation gate: which verdicts block visibility, and how do appeals reprocess?
10. **(Interview)** Prepare a two-minute answer to “why did the assistant quote a policy that does not exist?”

20.14 Milestone 5 Capstone: End-to-End MLOps Pipeline

Capstone Project

Mission: Assemble Part V: ADF orchestrates Synapse feature extraction → Azure ML pipeline (preprocess, train, evaluate) → metric-gated registration → a batch endpoint scoring new data nightly, with drift-triggered retraining and full lineage.

Estimated time: 8–10 hours.

What you will practice:

- Cross-service orchestration (ADF driving Synapse and AML).
- Versioned pipeline components with pinned environments (Chapter 18).
- Metric-gated registration and batch serving (Chapters 18–19).
- Drift detection wired to automated retraining (Chapter 19).

Scenario

The retailer’s data science team ships a churn model that scores the full customer base nightly. Requirements: no manual steps from feature refresh to scored output, a metric gate before any registration, and a drift alert that retrains without human detection—while every model remains traceable to its training data.

Requirements

- **Orchestration:** ADF pipeline runs Synapse feature extraction, then invokes the AML pipeline; failure paths notify.
- **Training:** AML components with pinned environments; datastore access via managed identity only.
- **Gating:** registration requires $AUC \geq 0.80$ plus recorded approval; batch endpoint created from the registered model.
- **Monitoring:** drift monitor on production inputs; endpoint/job metrics in Azure Monitor; one tested alert.

- **Tests:** at least three automated checks—metric threshold, feature-schema validation, batch-output sanity—with a failing case.

Reference Architecture

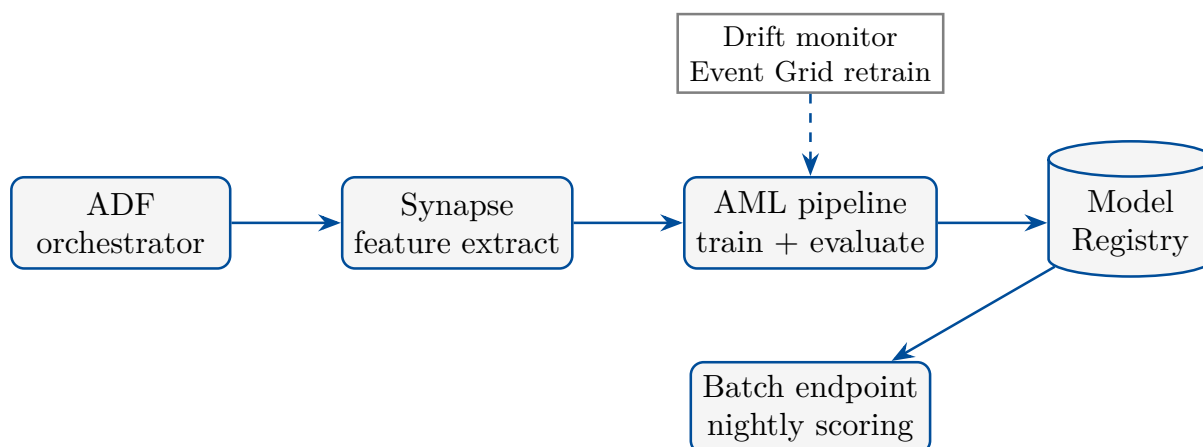


Figure 20.1: Milestone 5 architecture: ADF-orchestrated feature extraction, gated AML training, nightly batch scoring, drift-triggered retraining.

Implementation Steps

1. Register the ADLS datastore with the workspace managed identity; version the feature asset.
2. Build the AML pipeline with the metric gate (Chapter 18).
3. Create the batch endpoint from a registered model; schedule nightly scoring from ADF.
4. Configure the drift monitor and the Event Grid retraining path; fire it with the synthetic drift experiment.
5. Produce the cost estimate (AML compute, storage, ADF) with two optimizations.

Deliverables

- All pipeline definitions, component code, and orchestration JSON in the repo.
- Evidence pack: gate pass/fail, drift alert, batch output sanity checks, lineage query results.
- README that runs the system end-to-end from a clean workspace.

Acceptance Criteria

- A clean-environment run executes train → register → nightly batch scoring with no manual steps.
- A degraded training run is blocked by the gate; a drift event triggers retraining automatically.
- Every scored batch traces to a model version, a data asset version, and a pipeline run.

Stretch Goals

- Add this chapter's document pipeline as a feature source (extracted invoice fields as churn features).
- Replace nightly batch with an online endpoint behind blue-green for a real-time consumer.
- Add a champion/challenger evaluation on joined outcomes before promotion.

Part VI

Security, Governance, and Exam Prep

Zero-Trust Data Platforms: Identity, Networking, and Secrets

Learning Objectives

- Apply zero-trust principles—verify explicitly, least privilege, assume breach—to data platform architecture.
- Distinguish Microsoft Entra ID identities: users, service principals, and system- versus user-assigned managed identities.
- Deploy Private Endpoints and disable public network access so data services exist only inside your VNet.
- Contrast Private Endpoints with service endpoints and explain the DNS requirements that make private access actually work.
- Centralize secrets in Azure Key Vault with rotation policies and Event Grid-driven expiry notifications.
- Evaluate customer-managed keys versus Microsoft-managed keys for regulatory and key-custody requirements.
- Design a no-exfiltration data perimeter for a healthcare provider using Synapse data exfiltration protection.

Cross-Cloud Equivalence			
Zero-trust networking and identity-first access are cloud-universal; the product names differ but the architecture—private connectivity, no long-lived secrets, centralized keys—is identical.			
Capability	AWS	Azure	GCP
Identity plane	IAM	Microsoft Entra ID + Azure RBAC	Cloud IAM
Workload identity	IAM roles for EC2/EKS	Managed identities	Service account impersonation
Private service access	PrivateLink / VPC endpoints	Private Endpoints	Private Service Connect
Secrets manager	Secrets Manager	Azure Key Vault	Secret Manager
Key custody (CMK)	KMS	Key Vault / Managed HSM	Cloud KMS / HSM
Bastion access	Systems Manager Session Manager	Azure Bastion	IAP TCP forwarding
Fabric: Entra ID + OneLake security + private links; Snowflake: private connectivity (PrivateLink) + Tri-Secret; MongoDB: private endpoints			

The exam and interview favorite—“why is a managed identity better than a service principal secret?”—has the same answer on every cloud: no secret material exists to leak, rotate, or commit to git.

Key Terms

Zero trust: a security model that verifies every request explicitly, grants least privilege, and assumes the network is hostile. **Managed identity:** an Entra ID identity whose lifecycle Azure manages—no credentials in code or configuration. **Private Endpoint:** a network interface that binds a PaaS resource to a private IP inside your VNet. **Service endpoint:** a VNet-optimized route to a PaaS service that still targets the public IP. **CMK:** customer-managed keys—encryption keys you own in Key Vault, with your rotation and revocation policy.

21.1 Week 21 Overview: Part VI Opens with the Perimeter

Month 6 turns the platform from functional to defensible. The security model of the previous twenty weeks—managed identities on linked services, RBAC on storage, Key Vault for secrets—now becomes the explicit subject. Zero trust is the organizing principle: *no implicit trust from network location*; every request authenticated, authorized, and encrypted; the blast radius of any single compromise minimized by least privilege [92, 86].

For data platforms, zero trust resolves to four concrete engineering programs: identity (everything authenticates as a managed identity), networking (nothing is publicly reachable), secrets (nothing long-lived exists outside Key Vault), and custody (keys you can rotate and revoke). This chapter builds all four; Chapter 23 adds the data-privacy layer on top.

21.2 Monday: Entra ID, Service Principals, and Managed Identities

21.2.1 The Identity Spectrum

A **service principal** is an application identity in Entra ID, authenticated by a client secret or certificate that *you* create, store, rotate, and eventually leak into a log file. A **managed identity** is a service principal whose credential lifecycle Azure owns: the resource (a VM, a Function, a Data Factory) simply *has* an identity, requests tokens from a local endpoint, and never touches secret material. **System-assigned** identities live and die with one resource; **user-assigned** identities are standalone objects shared across resources (useful when many Functions need one identity, or when RBAC must exist before the resource does).

21.2.2 The Access Pattern

The zero-trust access pattern for any service-to-service call: caller has a managed identity → identity holds an RBAC role on the target at the tightest scope → caller requests a token → target validates. No connection strings, no keys, no rotation calendar. Chapters 5, 10, 11, and 18 already used this pattern for COPY loads, linked services, and AML datastores; this section names the principle they were applying.

Common Pitfall

Broad roles to silence 403s. Granting **Owner** or account-wide **Contributor** because a pipeline “couldn’t read storage” converts a scoped authorization question into a standing breach. The correct response to a 403 is to identify the missing data-plane action—often **Storage Blob Data Reader** on one container—not to widen the role.

21.3 Tuesday: Private Networking—Endpoints, DNS, and Bastion

21.3.1 Private Endpoint vs. Service Endpoint

A **service endpoint** extends your VNet’s identity to a PaaS service over the Azure backbone—the service still has its public IP; traffic is merely routed optimally, and the service firewall must allow your subnet. A **Private Endpoint** puts a private IP *inside* your VNet that maps to one specific resource; disable public network access on that resource and it ceases to exist on the internet [85]. Zero trust wants the latter.

21.3.2 DNS: The Step Everyone Forgets

A private endpoint without DNS is a trap: clients resolve `stadls.dfs.core.windows.net` to the *public* IP, traffic fails or leaks, and the misconfiguration is invisible in the portal. The complete pattern is private endpoint + **private DNS zone** (`privatelink.dfs.core.windows.net`) + VNet link + the zone group registration on the endpoint. Chapter 24’s Terraform listing encodes all four; hand-built setups routinely omit the third or fourth.

21.3.3 Administration Without Exposure

With data services private, administrator access needs its own path: **Azure Bastion** provides browser-based RDP/SSH to VMs without public IPs on the VMs or NSG openings. The admin plane becomes: Bastion → jump VM → private endpoints. Nothing in the estate answers a public SYN packet.

Tip

Prove the perimeter from *outside*: after disabling public access, attempt the data-plane call from your workstation and confirm it fails, then from the jump VM and confirm it succeeds. A private endpoint nobody tested is a hope, not a control.

21.4 Wednesday: Key Vault, Rotation, and CMK

21.4.1 Secrets, Keys, Certificates

Azure Key Vault is the custody layer for three object classes: **secrets** (connection strings, API keys), **keys** (RSA/EC keys for encryption, including CMKs), and **certificates** (with managed issuance and renewal) [9]. The zero-trust rule: applications reference secrets *by URI*; the vault enforces access via RBAC; managed identities do the authenticating. A rotated secret must require no code change—consumers read the current version at startup or per use.

21.4.2 Automated Rotation

Key Vault rotation policies plus Event Grid’s **SecretNearExpiry** event close the loop: when a secret nears expiry, an event fires; a Function or Logic App regenerates the credential at the

source (for example, a new storage key), writes the new value as a new secret version, and consumers pick it up on next read. The syllabus of Chapter 12's Logic Apps and Chapter 11's Event Grid converge here: rotation is just another event-driven pipeline, one whose payload happens to be a credential.

21.4.3 Customer-Managed Keys

At-rest encryption is on by default with Microsoft-managed keys. **CMK** moves key custody to your Key Vault: you control rotation, can revoke access by disabling the key (an instant cryptographic kill switch), and can produce the custody evidence regulated industries require. The operational price is real: key availability becomes your availability dependency, and a deleted key without purge protection is a data-loss event. CMK is a compliance answer, not a default posture.

Common Pitfall

Key Vault references without purge protection. Disabling purge protection saves nothing and turns an accidental key deletion into permanent ciphertext. Enable soft-delete *and* purge protection on any vault holding CMKs.

21.5 Thursday Hands-on Project: A Private-Only Storage Account

Create an ADLS Gen2 account, bind it to a private endpoint, disable public access, and prove the access pattern from a VM inside the VNet.

21.5.1 Build the Perimeter

```

1 $rg = "rg-de-azure-book-week21"
2 az group create --name $rg --location eastus
3 az network vnet create --name vnet-data --resource-group $rg '
4   --address-prefix 10.20.0.0/16 --subnet-name snet-data --subnet-prefix
   10.20.1.0/24
5
6 $adlsId = az storage account show --name <adls> --resource-group $rg --query id
   -o tsv
7
8 az network private-endpoint create --name pep-adls-dfs --resource-group $rg '
9   --vnet-name vnet-data --subnet snet-data '
10  --private-connection-resource-id $adlsId '
11  --group-id dfs --connection-name adls-dfs-conn
12
13 az network private-dns zone create --resource-group $rg '
14   --name "privatelink.dfs.core.windows.net"
15 az network private-dns link vnet create --resource-group $rg '
16   --zone-name "privatelink.dfs.core.windows.net" --name link-data '
17   --virtual-network vnet-data --registration-enabled false
18
19 az network private-endpoint dns-zone-group create --resource-group $rg '
20   --endpoint-name pep-adls-dfs --name dfszone '
21   --private-dns-zone "privatelink.dfs.core.windows.net" --zone-name dfs
22
23 az storage account update --name <adls> --resource-group $rg '
24   --public-network-access Disabled --default-action Deny

```

Listing 21.1: Private endpoint with DNS zone group, then public access disabled.

21.5.2 Prove It

Deploy a small VM into `snet-data` with Azure Bastion for access. From your workstation: `az storage fs list` against the account fails (no public path). From the VM via Bastion: the same call succeeds over the private IP, and `nslookup <adls>.dfs.core.windows.net` returns a 10.20.x.x address—the DNS proof that makes the perimeter real.

21.6 Friday Use Case: A No-Exfiltration Perimeter for Healthcare

A healthcare provider’s compliance requirement: PHI processed in Synapse must not be copyable to any storage outside the governed perimeter, even by a credentialed insider.

21.6.1 The Design

Every data service (ADLS, Synapse, Key Vault, SQL) sits behind private endpoints with public access disabled. The Synapse workspace enables **data exfiltration protection**: the workspace’s managed VNet blocks outbound connections to arbitrary endpoints, so a notebook cannot `pip install` from a personal repository or POST PHI to an external URL. Egress that *is* required flows through an explicit firewall allowlist, logged. CMK with your rotation evidence satisfies key-custody requirements; Bastion mediates administration; Defender for Cloud (Chapter 23) watches the control plane. The insider threat is bounded: a credentialed user can read what their role allows, but the data has no network path out.

Common Pitfall

Perimeter without egress control. Private endpoints stop inbound exposure; they say nothing about a notebook exfiltrating to attacker-controlled storage. Exfiltration protection and egress allowlisting are the outbound half of the same control—deploy both or the perimeter is a one-way valve.

21.7 Architectural Decision Record Template

Per platform record: identity model per service (which managed identities, which RBAC roles, at what scopes), the private endpoint inventory with DNS evidence, public-access posture per resource, Key Vault topology (one vault per environment, RBAC mode), CMK scope and rotation ownership, and the egress policy with its allowlist owner.

21.8 Operational Deep Dive: Auditing the Perimeter

The perimeter drifts unless audited: a monthly automated check that lists every data resource with `publicNetworkAccess` not Disabled, every storage account with shared-key authorization enabled, every Key Vault without purge protection, and every role assignment broader than the approved set. Azure Policy can enforce most of these at deploy time (Chapter 23); the audit catches what predates the policy. Feed both into the same report so posture is one page, not two systems.

21.9 Troubleshooting Patterns

Service unreachable after private endpoint creation → DNS zone/link/group incomplete; check resolution first. Intermittent private access → two DNS paths disagreeing (custom DNS for-

warders); trace from the failing client. Pipeline fails after public access disabled → the Azure IR is outside the perimeter; use a SHIR inside the VNet (Chapter 10) or a managed VNet IR. Key Vault reference failing after rotation → consumer pinned a secret *version* instead of the rolling URI.

21.10 Week 21 Lab Rubric

Criterion	Meets expectation	Needs improvement
Identity	Managed identities everywhere; scoped RBAC	Service-principal secrets; broad roles
Networking	Private endpoints + DNS proven from inside and outside	Endpoint without DNS; public access left enabled
Secrets	Key Vault references; rotation via events; purge protection	Secrets in variables; no rotation story
Verification	Outside-fails/inside-succeeds evidence captured	Controls asserted but never tested

Table 21.1: Week 21 rubric for the zero-trust deliverables.

21.11 Interview Q&A

21.11.1 Concepts and Trade-offs

1. **Q: Why prefer managed identities over service principal client secrets?**
A: Azure owns the credential lifecycle—no secret to store, rotate, or leak; tokens are issued to the resource on demand.
2. **Q: Private Endpoint vs. Service Endpoint—what is the key difference?**
A: A private endpoint gives the resource a private IP in your VNet and enables disabling public access; a service endpoint only optimizes routing to the public IP.
3. **Q: What does the Key Vault SecretNearExpiry event enable?**
A: Automated rotation: an event-driven pipeline regenerates the credential at the source and publishes a new secret version with no code change.
4. **Q: What does VNet data-exfiltration protection in Synapse guard against?**
A: Outbound paths—notebooks or jobs copying data to endpoints outside the governed perimeter, even by credentialed users.

21.12 Further Reading

The Entra ID [92], Azure RBAC [86], Private Link [85], and Key Vault [9] documentations are the primary sources. The Well-Architected security pillar [52] frames the zero-trust review; Chapter 23 adds policy enforcement and the privacy layer.

Key Takeaways

- Zero trust for data platforms: managed identities, private-only networking, vault-custodied secrets, owned keys.
- A 403 is a scoping question; broad roles are a standing breach, not a fix.

- A private endpoint without its private DNS zone is a trap—test name resolution, not just connectivity.
- Disable public access *and* control egress; perimeters are two-way.
- Rotation must be event-driven and code-invisible; `SecretNearExpiry` is the trigger.
- CMK buys custody and revocation at the price of availability responsibility; protect the vault with purge protection.

21.13 Exercises

1. **(Conceptual)** Explain the token flow when an ADF managed identity reads ADLS Gen2.
2. **(Conceptual)** Why does disabling public network access require DNS changes, not just firewall changes?
3. **(Conceptual)** When is a user-assigned managed identity preferable to system-assigned?
4. **(Hands-on)** Complete the Thursday project, including the outside-fails/inside-succeeds proof.
5. **(Hands-on)** Implement a rotation pipeline for a storage key using `SecretNearExpiry` and a Function.
6. **(Hands-on)** Write the monthly posture-audit script and run it against your lab estate.
7. **(Design)** Design the egress allowlist for the healthcare Synapse workspace: what legitimately needs outbound?
8. **(Design)** Write the ADR for CMK versus Microsoft-managed keys for the PHI estate.
9. **(Design)** Plan the migration of a service-principal-based estate to managed identities with zero downtime.
10. **(Interview)** Prepare a two-minute answer to “how would you exfiltrate data from your own platform—and what stops you?”

21.14 Capstone Project: Private-Only Lakehouse Perimeter

Capstone Project

Mission: Convert the lab lakehouse into a zero-trust perimeter: private endpoints and DNS for every data service, public access disabled, managed identities only, Key Vault custody with rotation, Bastion-mediated administration, and an audited posture report.

Estimated time: 4–6 hours.

What you will practice:

- Private endpoint + DNS deployment done completely and verifiably.
- Identity-first access with scoped RBAC.
- Rotation automation and CMK custody.
- Posture auditing as a repeatable artifact.

Scenario

The healthcare provider’s security review blocks go-live until the analytics estate is provably unreachable from the internet and provably incapable of unsanctioned egress. Your Part II lakehouse is the conversion target.

Requirements

- **Network:** private endpoints + DNS zones for ADLS, Synapse, Key Vault; public access disabled; outside-fails/inside-succeeds proof.
- **Identity:** every service-to-service call on managed identities; no shared-key or SAS paths enabled.
- **Secrets:** all secrets in Key Vault by URI reference; one rotation demonstrated via events.
- **Egress:** Synapse exfiltration protection enabled; the allowlist documented.
- **Audit:** the posture script output committed as evidence.

Reference Architecture

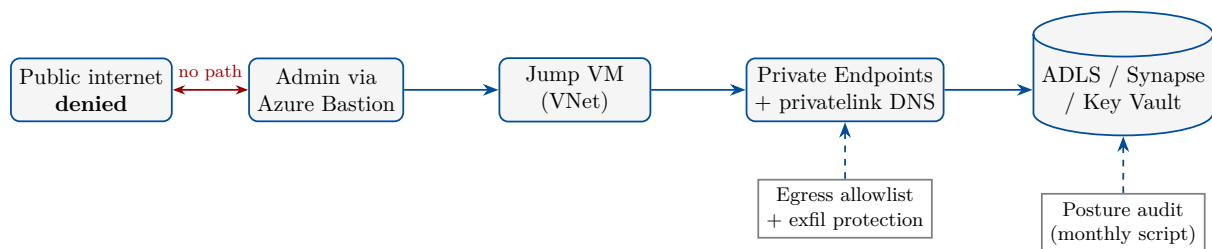


Figure 21.1: Capstone perimeter: no public path in, allowlisted egress out, every hop authenticated by identity.

Implementation Steps

1. Deploy endpoints and DNS for each service (Listing 21.1).
2. Disable public access and shared-key authorization; retest all pipelines over managed identities.
3. Configure the vault, rotation policy, and one Event Grid rotation demo.
4. Enable exfiltration protection; document the allowlist.
5. Run the posture audit; fix what it finds; commit the clean report.

Deliverables

- CLI/Terraform artifacts for the perimeter and the DNS evidence.
- The access proofs and the rotation demo recording.
- The posture report and the ADR set for identity, network, secrets, and custody.

Acceptance Criteria

- No data service resolves publicly; every in-VNet access path works.
- A rotated secret propagates to consumers with zero code or config change.
- The posture audit reports zero public-access, shared-key, or purge-protection violations.

Stretch Goals

- Convert the entire perimeter to Terraform (preview of Chapter 24) and destroy/rebuild it from code.
- Add Defender for Cloud recommendations to the posture report.
- Simulate an insider exfiltration attempt from a notebook and document which control stops it.

Observability, FinOps, and Governance at Scale

Learning Objectives

- Operate Microsoft Purview beyond the basics: custom classification rules, sensitivity labels, data policies, and access provisioning.
- Trace lineage from lake to report across Synapse, Databricks, ADF, and Power BI for impact analysis.
- Gate pipeline loads with automated data-quality tests (dbt tests or Great Expectations checkpoints) and route failures to quarantine.
- Build the observability layer: Azure Monitor metrics, Log Analytics diagnostics, and alerts for pipelines, streams, and warehouses.
- Apply FinOps discipline: cost allocation by tags, anomaly alerts, and per-product unit economics.
- Define reliability SLOs for data products—freshness, completeness, correctness—with error budgets.
- Architect centralized governance over a decentralized data mesh for a five-domain conglomerate.

Cross-Cloud Equivalence			
Catalogs, lineage, quality gates, and cost observability are the cross-cloud operating layer; each cloud pairs a native catalog with the same open quality tooling.			
Capability	AWS	Azure	GCP
Catalog and governance	Glue Data Catalog / Lake Formation	Microsoft Purview	Dataplex
Lineage	Glue / SageMaker lineage	Purview lineage	Dataplex lineage
Data quality tests	dbt / Great Expectations / Deequ	dbt / Great Expectations in ADF or Databricks	dbt / Great Expectations / Dataplex rules
Monitoring	CloudWatch	Azure Monitor + Log Analytics	Cloud Operations
Cost management	Cost Explorer + budgets	Cost Management + budgets	Cloud Billing + budgets
Sensitivity labels	Macie findings	Purview + MIP labels to Power BI	DLP inspection
Fabric: Purview hub + OneLake data catalog; Snowflake: Horizon Catalog + object tagging; MongoDB: Atlas data federation			

dbt and Great Expectations are deliberately platform-neutral: the same `dbt test` or GE checkpoint runs against Synapse, Snowflake, BigQuery, or Redshift—quality tooling is the one layer you can learn once [4, 6].

Key Terms

Purview data map: the automatically scanned, searchable graph of data assets across the estate. **Lineage:** the recorded flow of data from source through transformations to reports. **Sensitivity label:** a classification (e.g., Confidential) that travels with data into Synapse and Power BI. **Data quality gate:** automated tests that must pass before a load completes. **FinOps:** the operating practice of making cloud cost an engineering metric with owners.

22.1 Week 22 Overview: The Operating Layer

By this point the platform ingests, stores, transforms, streams, trains, and serves. What remains is *running it*: knowing what data exists and who touched it (governance), knowing when it breaks before users do (observability), knowing what it costs and why (FinOps), and knowing how reliable it promises to be (SLOs). These are not four disciplines but one: the operating layer of a data platform, and the difference between a platform that scales to fifty teams and one that collapses into tickets.

22.2 Monday: Purview Deep Dive—Classifications, Labels, Policies

22.2.1 Custom Classification Rules

Built-in classifiers recognize common PII (emails, credit cards, national IDs). Proprietary identifiers—your employee ID format, your claim numbers—need **custom classification rules**: regex or dictionary-based rules with a match-confidence threshold, applied during scans [55]. A rule that matches `EMP-\d{6}` across the whole lake turns “where does employee data live?” from an audit-week archaeology project into a catalog query.

22.2.2 Sensitivity Labels That Travel

Purview sensitivity labels (via Microsoft Information Protection) attach to columns and assets and *propagate*: a column labeled Confidential-PII in the lake carries the label into Synapse views and Power BI semantic models, where label-aware policies (export restrictions, watermarking) apply. The label is the governance mechanism that survives the journey from data to dashboard—without it, governance ends at the lake boundary.

22.2.3 Data Policies and Access Provisioning

Purview data policies let governance teams grant read access to registered sources through self-service access requests with approval workflows—access becomes a catalog operation with an audit trail, rather than a ticket that ends with an engineer hand-editing RBAC. Keep the platform team’s role as policy *approver*, not permission *typist*.

Common Pitfall

Scanning after serving. Running the first Purview scan after dashboards ship means PII was discoverable only after it was already queryable by analysts. Scan and classify *before* any serving view is published—Chapter 8’s capstone enshrined this order.

22.3 Tuesday: Lineage and Quality Gates

22.3.1 Lineage for Impact Analysis

Purview ingests lineage from ADF pipelines, Databricks jobs, Synapse, and Power BI [33]. The payoff is impact analysis in both directions: *upstream*—this report is wrong; which dataset fed it?—and *downstream*—this schema change is proposed; which reports break? Chapter 24’s expand–contract migrations depend on exactly this downstream query to know when the contract step is safe.

22.3.2 Quality Gates in the Pipeline

Lineage tells you what happened; quality gates decide what is *allowed* to happen. Run `dbt test` or Great Expectations checkpoints as a pipeline step *before* the load: uniqueness and not-null tests on keys, accepted-value tests on enums, referential tests on foreign keys, freshness tests on source arrival. A failed gate halts the load, routes the batch to quarantine, emits a metric, and pages on-call—silently loading bad data is worse than a delayed pipeline, because bad data compounds through every downstream product.

```

1 import great_expectations as gx
2
3 context = gx.get_context()
4 batch = context.sources.spark_default.query_data_asset(
5     "orders_silver", query="SELECT * FROM silver.orders WHERE _batch_id = '{bid
6     }'"
7 )
8 results = context.run_checkpoint("orders_silver_ckpt") # keys unique, amounts
9             >= 0
10 if not results.success:
11     quarantine(batch_id="{bid}", results=results)
12     raise SystemExit(f"quality gate failed for batch '{{bid}}'")

```

Listing 22.1: A Great Expectations checkpoint gate inside a Databricks job.

Tip

Start with five tests: primary key unique and not null, one referential check, one accepted-values check, one freshness check. A gate with five running tests beats a designed gate with fifty unwritten ones.

22.4 Wednesday: Observability and FinOps

22.4.1 The Observability Baseline

Azure Monitor plus Log Analytics is the estate-wide telemetry plane [21]. The data-platform baseline, assembled from earlier chapters: ADF pipeline/trigger/activity diagnostics, Databricks job duration and cluster metrics, Synapse utilization and queue depth, Event Hubs throughput and lag, ASA watermark delay, endpoint SLIs, and—the one that catches what errors miss—

silence alerts for expected runs that never happened. One dashboard, one alert taxonomy (sev by domain criticality), one runbook index.

22.4.2 FinOps as Engineering

Cost Management with tags gives allocation: every resource tagged **product**, **environment**, **domain** so spend decomposes by owner [68]. Budgets with action-group alerts make overruns events, not invoice surprises. The practices that move the number are the ones this book repeated: pause idle dedicated pools (Chapter 5), job clusters that terminate (Chapter 9), partition-pruned serverless queries (Chapter 7), scale-to-zero ML compute (Chapter 18). Review the top ten cost movers monthly with the same rigor as the top ten slow queries—they are often the same list.

22.4.3 Data Product SLOs

Define per product: **freshness** (data newer than X by time Y), **completeness** (row counts within band), **correctness** (quality tests green). Publish them, measure them, and let error budgets arbitrate: a product inside budget ships features; a product burning budget ships fixes. This is the SRE loop applied to data, and it converts “the pipeline feels flaky” into an engineering decision with a threshold.

22.5 Thursday Hands-on Project: Custom Classification and a Quality Gate

Define a custom Purview classification for a proprietary employee ID format, scan the lake so tagged columns appear automatically, then add a dbt-style quality gate to a pipeline and prove it blocks a bad batch.

22.5.1 Custom Classification

In the Purview portal: create classification **ContosoEmployeeId** with regex rule **EMP-\d{6}** and a 85% confidence threshold; create a scan rule set including it; scan the lab storage account. Verify that a column of synthetic employee IDs is discovered, classified, and searchable—and that the classification shows up in lineage and search results.

22.5.2 The Quality Gate

Add three dbt tests (or the GE checkpoint of Listing 22.1) to the Chapter 10 pipeline’s sink: **customer_id** unique and not null, **status** in accepted values. Run the happy path (load succeeds), then a poisoned batch (duplicate keys): the gate must fail, the batch must land in quarantine, the alert must fire, and the audit row must record the failure reason. All four outcomes are the deliverable.

22.6 Friday Use Case: Governed Data Mesh for a Five-Domain Conglomerate

A conglomerate’s five business units (retail, logistics, finance, insurance, media) each own their data engineering. Central IT is a bottleneck; a free-for-all is a compliance incident. The data mesh answer: **decentralized ownership, centralized governance**.

22.6.1 What Is Centralized, What Is Not

Centralized (platform team)	Decentralized (domain teams)
Purview catalog, classification rules, label taxonomy	Data product design, schemas, pipelines
Self-service infrastructure (Transform modules, CI templates)	Product-level SLOs within platform minimums
Quality-gate framework and quarantine patterns	Test definitions per product
Cross-domain access policies and audit	Domain-local access approval
Cost guardrails (tags, budgets, alerts)	Cost within the domain's budget

Table 22.1: The mesh boundary: the platform owns the rails; domains own the trains.

Every domain publishes data products—versioned, SLO-bearing, cataloged assets—into the shared estate; every product passes the same gates and carries the same labels; access flows through Purview policies with domain-owner approval. The platform team's leverage is that compliance is the *path of least resistance*: the self-service templates are already governed, so doing it right is easier than doing it around.

Common Pitfall

Mesh as rebranded anarchy. If domains can publish uncataloged, unlabeled, untested products, you have five data swamps with a governance slide deck. The mesh works only when the central mechanisms—catalog, gates, labels, budgets—are enforced by the delivery path, not by policy documents.

22.7 Architectural Decision Record Template

Record platform-wide: the classification taxonomy with custom rules, the label propagation policy, the quality-gate minimum test set, the alert taxonomy and runbook index, the tagging standard with budget thresholds, and the SLO template every product adopts.

22.8 Operational Deep Dive: The Monthly Platform Review

One meeting, four pages: reliability (SLO attainment per product, top incidents), quality (gate failure rate, quarantine backlog age), governance (unclassified assets, unscanned sources, stale access reviews), and cost (top movers, budget variance, optimization actions). The review's power is cadence: the same four pages monthly make drift—in reliability, in cost, in classification coverage—visible while it is still cheap to fix.

22.9 Troubleshooting Patterns

Assets missing from the catalog → the source was registered but the scan rule set excludes the file type or the managed identity lacks read. Lineage gaps → a transform ran outside ADF/Databricks (an ad-hoc notebook); lineage is only as complete as the execution paths you standardize. Gate always green → tests query a stale snapshot or the wrong environment; test the gate with a poisoned batch regularly, like a fire drill. Cost anomaly unexplained → untagged resources; make the tag policy deny untagged deployments.

22.10 Week 22 Lab Rubric

Criterion	Meets expectation	Needs improvement
Classification	Custom rule discovered proprietary IDs across the lake	Only built-in classifiers used
Quality gates	Poisoned batch blocked, quarantined, alerted, audited	Tests exist but never wired into the pipeline
Lineage	Impact query answered from lake to report	Lineage partial, unexamined
Cost	Tags, budget, and one optimization evidenced	Cost discovered at invoice time

Table 22.2: Week 22 rubric for the operating-layer deliverables.

22.11 Interview Q&A

22.11.1 Concepts and Trade-offs

- Q: How do sensitivity labels flow from Purview to Synapse and Power BI?**
 A: Labels attach to classified columns and propagate with the data, so label-aware policies apply at the semantic-model and report layer.
- Q: Why run dbt tests or GE checkpoints before the load step?**
 A: Bad data compounds through every downstream product; a gate before load quarantines failures while a gate after load only documents them.
- Q: What does Purview lineage show when a report suddenly breaks?**
 A: The upstream chain—which dataset, pipeline, and job fed the report—turning impact analysis from archaeology into a query.
- Q: In a data mesh, what stays centralized?**
 A: The rails: catalog, classification, label taxonomy, delivery templates, gate framework, access policy, and cost guardrails. Domains own products.

22.12 Further Reading

The Purview documentation [55] and lineage guide [33] cover governance; dbt Core [4] and Great Expectations [6] cover quality gates; Azure Monitor [21] and cost budgets [68] cover observability and FinOps. The Data Engineer Handbook [3] collects mesh case studies.

Key Takeaways

- Custom classification rules turn “where does our proprietary ID live” into a catalog query.
- Labels that propagate govern the dashboard; labels that stop at the lake govern nothing users touch.
- Quality gates belong before the load; quarantine with reasons, alert, and audit every failure.
- Observability’s highest-value alert is the silence alert: the run that never happened.
- FinOps is tags, budgets, and monthly review of the top movers—which are usually also the slowest queries.

- A data mesh centralizes the rails and decentralizes the trains; enforced delivery paths, not policy PDFs, keep it honest.

22.13 Exercises

1. **(Conceptual)** Why must governance mechanisms be embedded in the delivery path in a mesh?
2. **(Conceptual)** Compare a quality gate before versus after the load in terms of blast radius.
3. **(Conceptual)** What does an error budget change about how teams respond to a flaky pipeline?
4. **(Hands-on)** Complete the Thursday project; capture classification results and the four gate outcomes.
5. **(Hands-on)** Build the silence alert for a daily pipeline and demonstrate it by disabling the trigger.
6. **(Hands-on)** Tag the lab estate, create a budget with an alert, and produce the cost-by-domain report.
7. **(Design)** Write the SLO definitions for three data products from earlier chapters, with measurement queries.
8. **(Design)** Design the conglomerate’s self-service template: what is prefilled so compliance is the default?
9. **(Design)** Write the ADR for dbt tests versus Great Expectations for your platform.
10. **(Interview)** Prepare a two-minute answer to “how do you know your data platform is healthy right now?”

22.14 Capstone Project: The Governed, Observable, Costed Platform

Capstone Project

Mission: Retrofit the operating layer onto the Part II–III platform: Purview classification including one custom rule, lineage from lake to report, a quality gate with a proven poisoned-batch path, the observability baseline with a silence alert, and a tagged cost model with a budget.

Estimated time: 5–7 hours.

What you will practice:

- Custom classification and label propagation.
- Quality gates wired into a real pipeline.
- Alert taxonomy including the silence case.
- Cost allocation, budgets, and a documented optimization.

Scenario

The retailer’s platform works but is unopinionated: nothing is classified, quality is hoped for, failures are discovered by users, and the invoice is a surprise. You are the platform’s first operations-focused engineer.

Requirements

- **Governance:** full scan; built-in PII plus one custom rule; labels visible in a Power BI model.
- **Quality:** at least three gating tests with quarantine, alert, and audit on the poisoned path.
- **Observability:** diagnostic settings to Log Analytics; failure, drift, and silence alerts; one dashboard.
- **FinOps:** tags on every resource; a budget with action group; one optimization with quantified impact.

Reference Architecture

The platform map of Parts II–III with the operating layer drawn as a horizontal band: Purview over the lake and warehouse; quality gates inside pipelines; Monitor/Log Analytics under everything; Cost Management beside everything.

Implementation Steps

1. Scan and classify; create the custom rule; verify label propagation.
2. Wire the quality gate into one pipeline; run happy and poisoned batches.
3. Enable diagnostics; create the three alert classes; fire each once.
4. Tag the estate; create the budget; produce the cost report and the optimization.
5. Assemble the four-page monthly review from real outputs.

Deliverables

- Classification and label evidence; gate code and poisoned-batch evidence pack.
- Alert definitions, the dashboard, and the Log Analytics queries.
- The cost report, the budget export, and the first monthly review.

Acceptance Criteria

- A poisoned batch is blocked, quarantined, alerted, and audited—demonstrated.
- A disabled trigger pages the silence alert within its evaluation window.
- Every resource carries the tagging standard; the budget alert is armed.

Stretch Goals

- Enforce tags with Azure Policy so untagged deployments are denied.
- Add SLO attainment tracking for two products with error-budget burn rates.
- Simulate a mesh handoff: a second “domain” onboards using only your templates.

Compliance, Data Privacy, and Certification Exam Preparation

Learning Objectives

- Apply Dynamic Data Masking in Azure SQL and Synapse and verify masked views with a read-only user.
- Distinguish Always Encrypted from Transparent Data Encryption and identify which threats each addresses.
- Use Azure Policy and Microsoft Defender for Cloud for continuous auditing and threat detection.
- Execute a GDPR right-to-erasure via crypto-shredding across lake, warehouse, streaming, and cache copies.
- Handle legal-hold exceptions during erasure without breaking the audit trail.
- Design a pipeline that tokenizes third-party PII with Key Vault before loading Synapse.
- Plan a certification preparation program for DP-203/DP-700 using domain weighting and mock-exam feedback.

Cross-Cloud Equivalence

Data-privacy primitives—masking, client-side encryption, erasure, and policy audit—exist on every platform, and certification exams test them as a family.

Capability	AWS	Azure	GCP
Dynamic masking	Redshift dynamic masking	SQL/Synapse Dynamic Data Masking	BigQuery column-level + masking
Client-side encryption	client-side encryption SDKs	Always Encrypted	client-side encryption / CSEK
At-rest encryption	KMS-backed TDE equivalents	TDE (+ CMK via Key Vault)	Google-managed / CMEK
Policy enforcement	AWS Config + SCPs	Azure Policy	Organization Policy
Threat detection	GuardDuty / Security Hub	Microsoft Defender for Cloud	Security Command Center
PII discovery	Macie	Purview classifications	Sensitive Data Protection

Fabric: OneLake security + MIP labels; **Snowflake:** masking policies + tag-based access; **MongoDB:** client-side field-level encryption

Crypto-shredding—encrypt per data subject, erase by destroying the subject’s key—is the cross-platform answer to “delete everywhere, including backups,” because backups become

unreadable ciphertext when the key is gone.

Key Terms

Dynamic Data Masking (DDM): presentation-layer masking for non-privileged users; the underlying data is unchanged. **Always Encrypted**: client-side column encryption where the database never sees plaintext or keys. **TDE**: transparent data encryption of the database files at rest. **Crypto-shredding**: rendering a subject's data unreadable everywhere by destroying their encryption key. **Legal hold**: a compliance directive suspending deletion for specified data subjects.

23.1 Week 23 Overview: Privacy Engineering and the Exam

The final technical week covers the compliance layer—masking, encryption, policy audit, and the erasure machinery privacy law actually requires—and this chapter doubles as your certification preparation. The pairing is deliberate: DP-203 (Data Engineering on Microsoft Azure, retired in 2025) and its successor DP-700 (Fabric Data Engineer Associate) test exactly the design judgment this book has practiced, and the privacy topics are the most scenario-heavy part of both exams [39, 53]. Learn the material as engineering; prepare for the exam as a separate, mechanical discipline.

23.2 Monday: Dynamic Data Masking

23.2.1 Presentation-Layer Protection

DDM defines masking rules on columns—`default()` (xxxx), `email()` (aXXX@XX.com), `partial()`—and applies them at query time for users without the UNMASK permission [38]. The crucial property: *data at rest and for privileged users is unmasked*. DDM is a convenience control for read-only consumers—support tools, analysts who need shapes but not values—not a security boundary. A user who can run `SELECT ... WHERE email LIKE 'a%'` repeatedly can infer masked values; treat DDM as defense-in-depth, never as the PII control.

```

1 ALTER TABLE dbo.Customers
2   ALTER COLUMN Email ADD MASKED WITH (FUNCTION = 'email()');
3 ALTER TABLE dbo.Customers
4   ALTER COLUMN Phone ADD MASKED WITH (FUNCTION = 'partial(0,"XXX-",4)');
5
6 CREATE USER ro_user WITHOUT LOGIN;
7 GRANT SELECT ON dbo.Customers TO ro_user;
8 -- EXECUTE AS USER = 'ro_user'; SELECT Email, Phone FROM dbo.Customers;
9 -- yields aXXX@XX.com, XXX-1234 ; privileged users see plaintext

```

Listing 23.1: Masking email and phone; verifying as a read-only user.

23.3 Tuesday: Always Encrypted vs. TDE

23.3.1 Two Encryptions, Two Threat Models

TDE encrypts database files at rest: the threat is stolen media or backup files; the database engine decrypts transparently, so anyone with query access sees plaintext. **Always Encrypted** encrypts sensitive columns *client-side*: keys live in Key Vault, decryption happens in the client driver, and the database—including its administrators—only ever holds ciphertext [13]. The exam-favorite distinction: Always Encrypted protects data *from the DBA*; TDE does not. The

engineering price: encrypted columns support limited query shapes (equality, and with secure enclaves, more), so encrypt the columns whose sensitivity justifies the restriction—national IDs, not every string.

Property	TDE	Always Encrypted
Encrypts	Whole database files at rest	Selected columns, end-to-end
Plaintext visible to	Anyone with query access	Only clients holding the key
Protects from DBA	No	Yes
Query restrictions	None	Equality-only (more with enclaves)
Key custody	Service or CMK	Always client-side (Key Vault)

Table 23.1: TDE and Always Encrypted answer different threat models; use both when both threats are real.

23.4 Wednesday: Policy, Defender, and the Legal-Hold Interaction

23.4.1 Continuous Audit

Azure Policy enforces configuration at deploy time and audits continuously: deny public network access, require tags, require CMK for in-scope resources. Microsoft Defender for Cloud adds the threat plane: vulnerability assessment, anomalous-activity alerts (unusual data access patterns, suspicious extraction volume), and a secure score that doubles as the platform’s security backlog [91]. Together they convert Chapter 21’s posture script from a point-in-time check into standing controls.

23.4.2 Legal Hold Meets Erasure

Privacy operations collide: a GDPR erasure request arrives for a subject under litigation hold. The compliant behavior is *deferral with evidence*: the subject is flagged, erasure propagation is suspended for their records, and the audit trail records who requested, who deferred, under which hold, and when it resumed. Azure Policy effects, resource locks, and Purview retention labels can all technically block deletion—the design work is making that block *intentional, recorded, and reversible*, with reprocessing queued for when the hold lifts.

Common Pitfall

Erasing under legal hold. Deleting a held subject’s data is spoliation—often a worse legal outcome than a late erasure. The erasure pipeline’s first step is always the hold check, and its output is always an auditable record, including when the answer is “not now.”

23.5 Thursday Hands-on Project: Masking and a Crypto-Shredding Erasure Drill

Two drills in one day: implement DDM with a verified read-only view, then execute a full right-to-erasure for one synthetic data subject across every copy of their data.

23.5.1 Drill 1: DDM

Apply Listing 23.1 to a PII table; query as a privileged user and as `ro_user`; capture both result sets as the evidence that the mask works and that its boundary is the UNMASK permission.

23.5.2 Drill 2: Crypto-Shredding Erasure

The drill assumes the privacy architecture: each subject’s PII fields are encrypted with a per-subject key in Key Vault (envelope encryption—the data key encrypts the payload, the Key Vault key wraps the data key). To erase subject *S*:

1. **Hold check:** confirm *S* is not under legal hold; record the check.
2. **Destroy the key:** delete *S*’s key from Key Vault. Every copy—raw and curated ADLS zones, Synapse dedicated and serverless tables, events inside the Event Hubs retention window, derived tables, Redis caches, *backups and Delta time-travel history*—is now permanently unreadable ciphertext.
3. **Prove it:** attempt decryption paths; document that none succeed.
4. **Handle the non-encryptable:** identifiers used as join keys (which must be pseudonymous tokens, never raw PII) are purged per-store via the documented runbook; soft-delete and backup windows are inventoried so the erasure certificate can state when the last copy expires.
5. **Certify:** write the erasure record—who, when, what copies, which windows, which hold check—to the immutable audit log.

Note

Naive DELETE is not erasure: soft-delete windows, blob versioning, backups, Delta time travel, and stream retention all keep copies after the delete commits. Crypto-shredding is the only design where “every copy” includes the ones you cannot reach.

23.6 Friday Use Case: Tokenized Third-Party PII Ingestion

A retailer ingests customer files from a third-party data broker. Design: files land in an isolated raw zone; a Function tokenizes PII fields on arrival—replacing emails and names with keyed hashes, with the tokenization keys in Key Vault and the mapping table in an Always Encrypted column—and only the tokenized payload proceeds to silver and Synapse. Purview classifies both zones; the raw zone carries the tightest ACLs, the shortest lifecycle, and CMK; erasure is crypto-shredding the subject’s wrapping key. The analytics estate *never holds raw PII*, which is the strongest possible answer to most privacy review questions.

Common Pitfall

Tokens that are re-identifiable. An unsalted hash of an email is a rainbow table away from plaintext. Tokenization keys must be keyed (HMAC) with vault-custodied secrets, rotated, and never co-located with the data they protect.

23.7 Certification Preparation: DP-203 Legacy and DP-700 Reality

DP-203 was retired in 2025; **DP-700** (Implementing Data Engineering Solutions Using Microsoft Fabric) is its successor for the Azure data-engineering credential [53]. The preparation method is identical either way, and this book’s twenty-four weeks map onto both exams’ domains.

23.7.1 Domain-Weighted Study Plan

Pull the current skills-measured document for your target exam and score yourself per domain: ingestion and transformation (this book’s Parts III–IV), storage and serving (Parts I–II), monitoring and optimization (Chapters 8, 22), security and governance (Chapters 21–23), and for DP-700 specifically, Fabric’s lakehouse, warehouse, eventstream, and DirectLake surfaces. Weight revision time by *domain weight* $\times$ *your weakness*, not by comfort—the failure mode is re-reading what you already know.

23.7.2 Mock-Exam Loop

Take a full-length mock under time pressure; classify every miss as *knowledge gap* (study the topic), *misread* (slow down on negatives like “all EXCEPT”), or *judgment gap* (the “best” answer among working ones—usually the one with least operational toil and least privilege). Re-mock the weak domains after one week; schedule the real exam when two consecutive full mocks clear 80%. The week following this program’s Week 24 is the right window: material is fresh, and the capstone work is your scenario-practice.

Exam skill area	Book chapters	Evidence you know it
Design and implement storage	1–4, 6	You chose tiers, redundancy, formats on evidence
Ingest and transform	5, 9–12	You built event-driven, idempotent pipelines
Streaming	13–16	You reason about watermarks and exactly-once
Serving, BI, ML	5–8, 17–20	You shipped models and semantic models
Security and governance	21–23	You built the perimeter and the erasure drill
Monitoring, optimization, cost	8, 22	You run the monthly review
Fabric specifics (DP-700)	6, 7 (DirectLake), cross-cloud boxes	You can map every Azure service to its Fabric surface

Table 23.2: Mapping this book to the certification skill areas.

23.8 Architectural Decision Record Template

Per privacy design record: which columns are masked, encrypted client-side, or tokenized with the threat model each addresses; key custody and rotation ownership; the erasure architecture (crypto-shredding scope, per-store purge runbook, retention-window inventory); the legal-hold procedure; and the audit-log schema for privacy operations.

23.9 Operational Deep Dive: The Privacy Operations Calendar

Privacy controls decay without a calendar: quarterly—erasure drill on a synthetic subject (like Thursday’s), DDM permission review, tokenization-key rotation; monthly—Defender recommendation triage, policy-compliance report, hold-list reconciliation with legal; continuously—anomalous-access alerts, classification coverage as new sources onboard. The erasure SLA (typically 30 days) is measured from request, so the drill’s timing evidence is the SLA’s foundation.

23.10 Troubleshooting Patterns

Masked values visible to a report user → the user was granted UNMASK by a broad role assignment. Always Encrypted insert failures → the client driver lacks key access or the column encryption key rotated without re-encryption. Erasure drill finds plaintext in a forgotten copy → the data inventory was incomplete; fix the inventory, then re-drill. Defender alerts ignored → alert fatigue; tune to actionable severities or the one real alert drowns.

23.11 Week 23 Lab Rubric

Criterion	Meets expectation	Needs improvement
Masking	Masked view verified as read-only user; UNMASK scope reviewed	Masking asserted, never verified
Encryption	DDM vs. TDE vs. Always Encrypted chosen per threat model	One control applied to every threat
Erasure	Full crypto-shredding drill with certificate and hold check	DELETE-only erasure; copies unexamined
Audit	Immutable record of every privacy operation	Operations performed, evidence absent

Table 23.3: Week 23 rubric for the compliance deliverables.

23.12 Interview Q&A

23.12.1 Concepts and Trade-offs

1. **Q: Who still sees unmasked values when Dynamic Data Masking is enabled?**
A: Users with the UNMASK permission—DDM is presentation-layer; the data and privileged access are unchanged.
2. **Q: Always Encrypted vs. TDE—which protects data from database admins?**
A: Always Encrypted: keys are client-side in Key Vault and the engine holds only ciphertext; TDE decrypts transparently for anyone with query access.
3. **Q: Why tokenize PII before loading it into Synapse?**
A: So the analytics estate never holds raw PII—analysts work on tokens, and the mapping stays in a tightly controlled, separately encrypted store.
4. **Q: Why is naive DELETE not GDPR erasure?**
A: Soft-delete windows, versioning, backups, Delta time travel, and stream retention all keep copies; crypto-shredding renders them unreadable by destroying the key.

23.13 Further Reading

Dynamic Data Masking [38], Always Encrypted [13], and Defender for Cloud [91] cover the privacy layer. For certification, study the current skills outline for DP-700 [53] (and the DP-203 record for legacy context [39]), then return to each cited chapter for hands-on depth.

Key Takeaways

- DDM is presentation-layer defense-in-depth; Always Encrypted is the control that hides data from administrators; TDE protects files at rest.
- Azure Policy prevents misconfiguration at deploy time; Defender detects threats at run time; you need both.
- Erasure means every copy, including backups and time travel—crypto-shredding is the scalable answer.
- Legal holds suspend erasure by design, with an auditable deferral and a reprocessing queue.
- Tokenize third-party PII at the boundary so analytics never holds the raw values.
- Certification prep is domain weighting plus mock-exam feedback loops; the engineering is already done—practice the exam format mechanically.

23.14 Exercises

1. **(Conceptual)** Construct the inference attack that defeats DDM, and name the control that actually prevents it.
2. **(Conceptual)** Why does Always Encrypted restrict query shapes, and which column types deserve that cost?
3. **(Conceptual)** Explain why the hold check precedes key destruction in the erasure pipeline.
4. **(Hands-on)** Complete both Thursday drills; produce the masking evidence and the erasure certificate.
5. **(Hands-on)** Enable Defender for Cloud on the lab subscription and triage the top five recommendations.
6. **(Hands-on)** Write the Azure Policy that denies public network access on storage accounts and test a violating deployment.
7. **(Design)** Design the tokenization service for the broker pipeline: key custody, rotation, and the mapping store.
8. **(Design)** Write the ADR for crypto-shredding versus per-store purge for a platform with 90-day backup retention.
9. **(Design)** Build your personal eight-week DP-700 study plan from the skills outline with domain weightings.
10. **(Interview)** Prepare a two-minute answer to “a customer demands deletion, but backups keep their data for 90 days—what do you tell legal?”

23.15 Capstone Project: The Erasure-Capable Privacy Architecture

Capstone Project

Mission: Build the privacy architecture end-to-end at lab scale: tokenized ingestion, DDM and Always Encrypted where each fits, per-subject encryption keys, a full erasure drill with certificate, a legal-hold deferral demonstration, and the policy/Defender audit layer.

Estimated time: 6–8 hours.

What you will practice:

- Layered privacy controls chosen per threat model.
- Crypto-shredding across every storage copy, proven.
- Legal-hold interaction with a preserved audit trail.
- Continuous compliance via Azure Policy and Defender.

Scenario

The broker-ingestion retailer must pass a privacy audit: demonstrate that any subject can be erased within 30 days across every copy, that PII is never queryable in raw form by analysts, and that holds suspend erasure with evidence.

Requirements

- **Ingestion:** tokenization at the boundary; raw zone isolated, CMK-encrypted, short-lived.
- **Controls:** DDM for read-only SQL users; Always Encrypted for the token-mapping table; TDE throughout.
- **Erasure:** per-subject keys; drill proves unreadability in lake, warehouse, stream retention, and caches; certificate issued.
- **Hold:** one subject under hold is deferred with the full audit record and a reprocessing entry.
- **Audit:** the deny-public-access policy and Defender triage report committed as evidence.

Reference Architecture

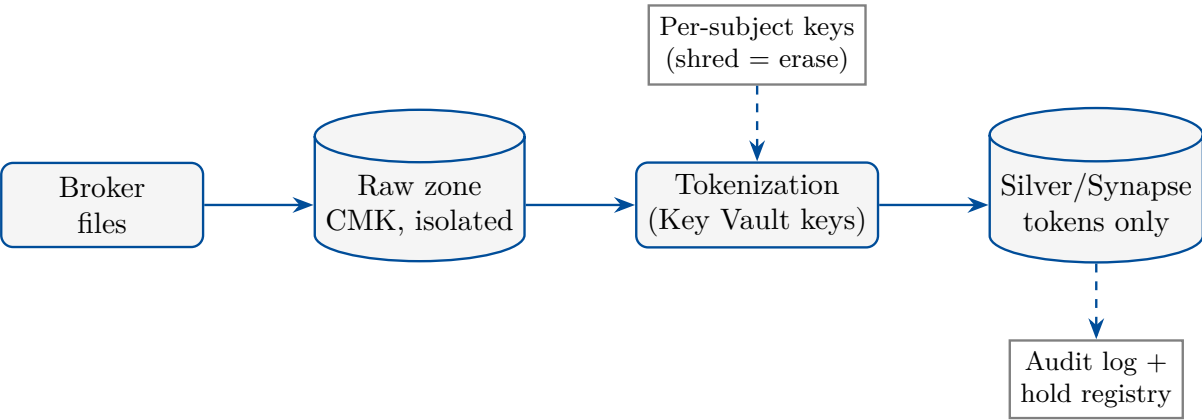


Figure 23.1: Capstone privacy architecture: tokenize at the boundary, hold only tokens downstream, erase by destroying keys.

Implementation Steps

1. Build the tokenization boundary and the isolated raw zone.
2. Apply DDM, Always Encrypted, and TDE per the threat-model table.
3. Run the erasure drill for one synthetic subject; produce the certificate.
4. Place a second subject under hold; demonstrate deferral and resume.
5. Deploy the policy, triage Defender, and assemble the audit pack.

Deliverables

- Tokenization code, key-custody configuration, and the zone ACL evidence.
- The erasure drill record, certificate, and hold-deferral record.
- The policy definition, compliance report, and Defender triage.

Acceptance Criteria

- Post-erasure, no path—including time travel and backups—yields the subject's plaintext.
- The held subject's records survive with an auditable deferral and are erased on resume.
- Analysts can query the full analytics estate without ever seeing raw PII.

Stretch Goals

- Automate the erasure SLA clock: request timestamp to certificate, with alerting at day 25.
- Add Purview retention labels to the raw zone and reconcile them with the erasure runbook.
- Run the quarterly privacy calendar once end-to-end and time every drill.

Production DataOps and the Enterprise Capstone

Learning Objectives

- Provision ADLS Gen2, Synapse, and Data Factory with Terraform’s `azurerm` provider, and read Bicep as the Azure-native alternative.
- Manage Terraform state in an Azure Storage backend with locking, and keep state files out of version control.
- Structure a dbt project—models, tests, documentation—against Synapse or Fabric.
- Build a GitHub Actions pipeline that validates Terraform and runs dbt tests on every pull request, with `apply` gated on review.
- Deploy zero-downtime schema changes with the expand–contract pattern coordinated through dbt and lineage.
- Design environment promotion (dev → test → prod) for data platform artifacts.
- Complete the program’s final capstone: a secure, governed, multi-subscription data mesh deployed entirely from code.

Cross-Cloud Equivalence

Infrastructure as code, SQL-based transformation, and CI/CD are the most portable skills in this book—the same Terraform workflow and the same dbt project structure run against every cloud.

Capability	AWS	Azure	GCP
Multi-cloud IaC	Terraform provider	Terraform provider	Terraform provider
Native IaC	CloudFormation / CDK	Bicep / ARM templates	Deployment Manager (legacy) / Config Controller
Transform-as-code	dbt on Redshift/Athena	dbt on Synapse/Fabric	dbt on BigQuery
CI/CD	CodePipeline / GitHub Actions	GitHub Actions / Azure DevOps	Cloud Build / GitHub Actions
State backend	S3 + DynamoDB lock	Azure Storage + lease lock	GCS backend

Fabric: Git integration + deployment pipelines; **Snowflake:** dbt + Terraform provider; **MongoDB:** Atlas Terraform provider

Everything in this chapter is vendor-neutral discipline: declarative infrastructure, reviewed plans, tested transformations, and schema changes that never break a consumer mid-flight.

Key Terms

IaC: infrastructure as code—declarative, versioned, reviewable environment definitions. **Terraform state:** the recorded mapping of configuration to real resources; shared state needs a remote backend with locking. **dbt:** SQL transformation framework with models, tests, and documentation as code. **Expand–contract:** zero-downtime schema evolution: add new shape, migrate, switch reads, then remove the old. **Environment promotion:** the same artifacts progressing dev → test → prod through gates.

24.1 Week 24 Overview: The Program Becomes a Platform

The final week is about *repeatability*. Every resource built by hand in Chapters 1–23 becomes code; every deployment becomes a reviewed, tested pipeline run; every schema change becomes a migration that never breaks a consumer. This is DataOps: the application of software-engineering delivery discipline to data platforms. The chapter ends with the program’s final capstone—the multi-subscription, zero-trust, governed data mesh that assembles everything.

24.2 Monday: Terraform and Bicep for Data Platforms

24.2.1 Terraform with the azurerm Provider

Terraform declares desired state; the `azurerm` provider maps it to Azure Resource Manager [7]. The data-platform skeleton:

```

1 terraform {
2   required_providers {
3     azurerm = { source = "hashicorp/azurerm", version = "~> 4.0" }
4   }
5   backend "azurerm" {
6     resource_group_name = "rg-tfstate"
7     storage_account_name = "sttfstate001"
8     container_name      = "tfstate"
9     key                 = "platform.tfstate"
10  }
11 }
12 provider "azurerm" { features {} }
13
14 resource "azurerm_storage_account" "adls" {
15   name                        = "stadlsdemo001"
16   resource_group_name        = azurerm_resource_group.rg.name
17   location                   = azurerm_resource_group.rg.location
18   account_tier               = "Standard"
19   account_replication_type   = "LRS"
20   is_hns_enabled             = true
21   public_network_access_enabled = false
22 }
23
24 resource "azurerm_data_factory" "adf" {
25   name                        = "adf-demo-001"
26   resource_group_name        = azurerm_resource_group.rg.name
27   location                   = azurerm_resource_group.rg.location
28   identity { type = "SystemAssigned" }
29 }

```

Listing 24.1: Terraform for the storage and orchestration foundation.

Two details carry the production weight. First, the **remote state backend** on Azure Storage with lease-based locking: state is the source of truth about what exists, so it must be shared,

locked against concurrent applies, and never committed to git. Second, explicit security defaults: `public_network_access_enabled = false` written *in code*, because several resource types default to public.

24.2.2 Bicep: The Azure-Native Alternative

Bicep transpiles to ARM templates and offers first-day support for every Azure resource property, tight portal integration, and no state file (ARM tracks deployments) [90]. Choose Bicep for Azure-only estates with ARM-native teams; choose Terraform for multi-cloud posture, the provider ecosystem (Databricks, dbt Cloud, Snowflake), and the plan/apply workflow maturity. Many organizations standardize on Terraform for data platforms and Bicep for foundational landing zones.

Common Pitfall

terraform apply without review. A destructive apply on a storage account is a data-loss event. The workflow is: `plan` on every pull request, review the plan like code, `apply` only from the protected main branch after merge—and state locking makes two applies at once impossible.

24.3 Tuesday: dbt—Transformations as Reviewed Code

24.3.1 Models, Tests, Documentation

A dbt project is a directory of SQL **models** (SELECT statements materialized as views/tables/incremental tables), **tests** (generic—unique, not-null, accepted-values, relationships—plus custom SQL tests), and generated **documentation** with lineage [4]. Against Synapse or Fabric, dbt becomes the transformation layer the warehouse was missing: versioned SQL, reviewed in PRs, tested in CI, documented automatically. The `ref()` graph is a dependency DAG you did not have to draw—and, as Wednesday shows, the safety net for schema changes.

24.3.2 Incremental Models

Full refreshes do not scale; dbt **incremental** models append or merge new rows using a watermark column. The pattern pairs with Chapter 12’s idempotency: incremental models with unique keys merge rather than duplicate, and `-full-refresh` remains the recovery path when logic changes invalidate history.

24.4 Wednesday: CI/CD and Zero-Downtime Schema Evolution

24.4.1 The Pipeline

The data platform’s CI/CD workflow on GitHub Actions: on pull request—`terraform fmt/validate/plan`, `dbt build` against a CI schema, linting; on merge to main—`terraform apply`, ADF publish (the npm ADFUtilities pattern), `dbt deploy` to the next environment. Environment promotion is the same artifact set advancing through dev → test → prod with gates between: plan review, test evidence, and for prod, an approval.

```
1 name: pr-validate
2 on: [pull_request]
3 jobs:
4   terraform:
5     runs-on: ubuntu-latest
```

```

6   steps:
7     - uses: actions/checkout@v4
8     - uses: hashicorp/setup-terraform@v3
9     - run: terraform -chdir=infra init -backend=false
10    - run: terraform -chdir=infra validate
11    - run: terraform -chdir=infra plan -input=false
12  dbt:
13    runs-on: ubuntu-latest
14    steps:
15      - uses: actions/checkout@v4
16      - run: pip install dbt-synapse
17      - run: dbt build --target ci           # models + tests against CI schema

```

Listing 24.2: Pull-request validation workflow (excerpt).

24.4.2 Expand–Contract Migrations

A breaking schema change—renaming `customer_email`—ships as four ordered releases: (1) **expand**: add the new nullable column alongside the old; (2) **migrate**: dual-write new writes and backfill history (dbt incremental or ADF copy); (3) **switch reads**: a backward-compatible view shim so consumers keep one stable interface; (4) **contract**: drop the old column only after Purview lineage and the dbt `ref()` graph prove no consumer reads it. Every intermediate state is valid; no consumer breaks between releases.

Common Pitfall

Contracting first. The failure story is canonical: a release dropped `customer_email` while the dbt model and the Power BI dataset still referenced it; dashboards failed at 09:00 and the fix needed a hot rollback and a full dbt re-run. Never drop until the dependency check—lineage plus `ref()` graph—returns zero readers, and ship the view shim in the same release as the column change.

24.5 Thursday Hands-on Project: Terraform + dbt + GitHub Actions

Create a GitHub repository containing Terraform that provisions an ADLS Gen2 account and an ADF instance, a dbt project with three models and five tests, and a workflow that validates Terraform and runs dbt tests on every pull request.

24.5.1 Build the Repository

1. `infra/`: the Terraform from Listing 24.1—resource group, HNS-enabled ADLS account with public access disabled, ADF with a system-assigned identity, and the remote state backend.
2. `transform/`: a dbt project with staging, intermediate, and mart models over the lab orders data; five tests: unique and not-null on the order key, accepted values on status, a relationship test to customers, a freshness test on source arrival.
3. `.github/workflows/`: Listing 24.2, extended with the dbt job against a CI schema.

24.5.2 Prove the Gates

Open three pull requests: a valid change (both jobs green, plan output posted), a Terraform change that would enable public access (plan review catches it—this is why humans read plans),

and a dbt change violating a test (dbt job red, merge blocked by branch protection). The red PRs are the deliverables: gates that have never fired are gates you have not verified.

24.6 Friday: Program Review and Certification Scheduling

Close the program deliberately. Re-run Chapter 23’s exam-mapping table against yourself; the capstones you shipped are the scenario practice the exam assumes. Identify the two weakest domains by mock-exam evidence, schedule both full-length mocks for the coming weekend and the next, and book the exam for when two consecutive mocks clear 80%. Then do the more durable thing: keep the capstone repositories green. A platform you can still deploy from code in three months is worth more to your next employer than the badge.

24.7 Milestone 6 Capstone: The Secure, Governed, Multi-Subscription Data Mesh

Capstone Project

Mission: Deploy the program’s final architecture entirely from code: three subscriptions—ingestion with PII tokenization, analytics with Synapse and ADLS, governance with Purview—peered VNet, private endpoints everywhere, zero public access, dbt-gated loads, and a GitHub Actions pipeline that plans, tests, and applies under review.

Estimated time: 12–16 hours.

What you will practice:

- Terraform modules per subscription with a locked remote state backend.
- Zero-trust networking: private endpoints, DNS, exfiltration protection (Chapter 21).
- Tokenized PII ingestion with event-driven processing (Chapters 13, 20, 23).
- Governance-as-default: Purview, dbt gates, budgets (Chapter 22).

Scenario

The conglomerate from Chapter 22 sanctions the mesh pilot: Subscription A ingests streaming PII and tokenizes it with Functions and Key Vault; Subscription B hosts the Synapse and ADLS analytics estate; Subscription C runs Purview governance. All traffic is private; all deployment is code; all loads are quality-gated. You are the platform team.

Requirements

- **IaC:** one Terraform module per subscription; remote state with locking; nothing deployed by hand; `plan` on PRs, `apply` post-review.
- **Network:** VNet peering, private DNS zones, private endpoints on every data service; `public_network_access_enabled = false` in code, verified from outside.
- **Ingestion:** Event Hubs → tokenization Function (Key Vault keys) → analytics estate; PHI never persists unmasked.
- **Quality:** dbt project with at least three models and five tests gating the tokenized load; poisoned-batch demonstration.
- **Governance:** sources registered and scanned in Purview; classifications evidenced before serving; cost budget per subscription with alerts.

- **Review:** the Chapter 22 four-page monthly review produced from this estate’s real outputs.

Reference Architecture

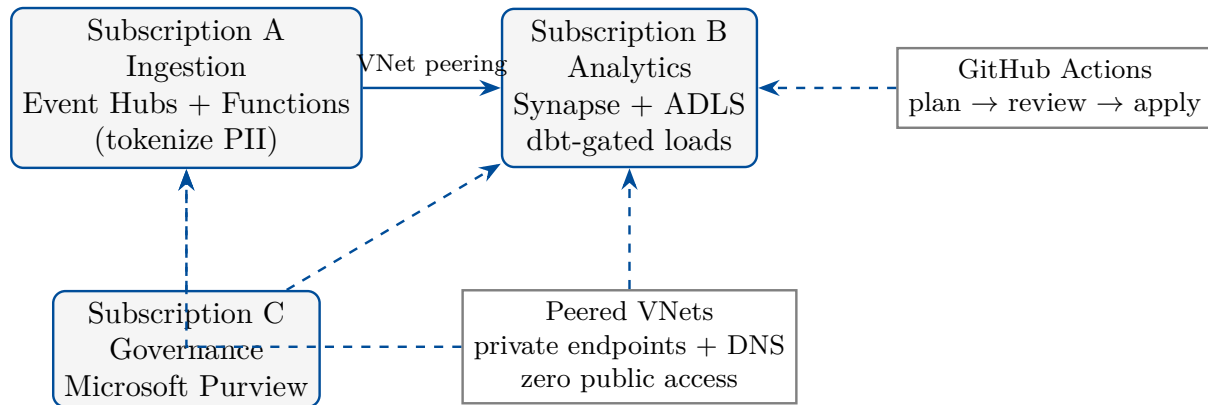


Figure 24.1: Final capstone: a three-subscription, zero-trust, code-deployed data mesh with governance as the default path.

Implementation Steps

1. Write the three Terraform modules; configure the locked state backend; deploy Subscription C (governance) first.
2. Peer the VNets; create private DNS zones; deploy endpoints with DNS zone groups—the Listing 21.1 pattern as code.
3. Deploy the tokenization Function with Key Vault references; wire Event Hubs ingestion.
4. Deploy Synapse with public access disabled; register and scan sources in Purview.
5. Add the dbt project and the GitHub Actions gates; run the happy path and the poisoned batch.
6. Prove zero public reachability from outside; produce the cost budgets and the monthly review.

Deliverables

- The repository: Terraform modules, dbt project, workflows, and a README that deploys everything from clean.
- Evidence: plan/apply history, dbt test results (pass and blocked), the outside-fails network proof, Purview classifications.
- The cost report across three subscriptions and the four-page monthly review.

Acceptance Criteria

- `terraform destroy` followed by `apply` rebuilds the entire estate with no manual step.
- No resource is publicly reachable; all cross-subscription traffic flows over private endpoints.
- A poisoned tokenized batch is blocked by the dbt gate, quarantined, alerted, and audited.

- A destructive plan (e.g., deleting a storage account) is visible in PR review and blocked by process.

Stretch Goals

- Add a fourth domain subscription using only your modules and templates—the mesh handoff test.
- Implement one expand–contract migration on a served table with the lineage-gated contract step.
- Add Defender for Cloud and the Chapter 23 privacy calendar to the governance subscription.

24.8 Key Takeaways for the Whole Program

- Storage decisions are architecture decisions; identity, networking, and lifecycle are designed, not defaulted.
- Serve logically when you can, provision when you must, and always measure bytes, seconds, and dollars.
- Pipelines are software: parameterized, tested, idempotent, and wired to fail loudly.
- Streams are state: watermarks, checkpoints, and exactly-once are engineering, not configuration.
- Models are data products: versioned features, gated promotion, and monitored drift.
- Governance, privacy, and cost are the operating layer that makes everything else survivable at scale.
- If it is not in code with a reviewed plan and a tested pipeline, it does not exist—that is the DataOps bar this book set out to teach.

24.9 Exercises

1. **(Conceptual)** Why does Terraform state need locking, and what failure does the lease prevent?
2. **(Conceptual)** Explain each expand–contract step and what breaks if step 4 precedes step 3.
3. **(Conceptual)** Why run `terraform plan` on PRs but `apply` only after merge and review?
4. **(Hands-on)** Complete the Thursday project with all three gate-proving pull requests.
5. **(Hands-on)** Convert one hand-built lab environment to Terraform and verify zero drift on plan.
6. **(Hands-on)** Add a dbt exposure documenting a Power BI dependency, then use lineage plus `ref()` to approve a contract step.
7. **(Design)** Design the environment-promotion gates for the capstone estate: what evidence advances `dev` → `test` → `prod`.
8. **(Design)** Write the ADR for Terraform versus Bicep for this estate.
9. **(Design)** Build your final two-week certification plan from two mock-exam result sets.

10. **(Interview)** Prepare a two-minute answer to “walk me through how a column rename reaches production in your platform.”

Azure CLI Reference for Data Engineers

This appendix is a working reference for the Azure CLI commands used throughout the book, organized by service. All examples assume Windows PowerShell with backtick line continuation, a signed-in session (`az login`), and a subscription selected with `az account set`. Replace placeholder names before running.

Note

Commands are shown in their simplest useful form. Production deployments belong in Terraform or Bicep (Chapter 24); the CLI is for learning, inspection, and one-off operations. Every command shown mutating a resource should be run first in a sandbox subscription.

A.1 Account, Identity, and Access

Command	Purpose
<code>az login</code>	Interactive sign-in
<code>az account set -subscription <id></code>	Select the working subscription
<code>az group create -n <rg> -l eastus</code>	Create a resource group
<code>az ad signed-in-user show</code>	Show the current identity
<code>az role assignment create -assignee <id> \</code> <code>-role "Storage Blob Data Reader"</code> <code>-scope <resource-id></code>	Grant an RBAC role at a scope
<code>az role assignment list -scope <resource-id></code>	Audit role assignments

Prefer object IDs with `-assignee-object-id` and `-assignee-principal-type ServicePrincipal` for managed identities; it avoids Graph lookup ambiguity.

A.2 Storage and ADLS Gen2

```

1 az storage account create --name <adls> --resource-group $rg '
2   --location eastus --sku Standard_GZRS --kind StorageV2 --hns true '
3   --min-tls-version TLS1_2 --allow-blob-public-access false
4
5 az storage fs create --name raw --account-name <adls> --auth-mode login
6 az storage fs directory create --name bronze -f raw '
7   --account-name <adls> --auth-mode login
8 az storage fs file upload -s ./orders.csv '

```

```

9  -p bronze/year=2026/month=08/orders.csv -f raw '
10  --account-name <adls> --auth-mode login
11 az storage fs file list -f raw --path bronze '
12  --account-name <adls> --auth-mode login --recursive
13
14 az storage account management-policy create '
15  --account-name <adls> --resource-group $rg --policy @lifecycle.json
16
17 az storage blob service-properties update --account-name <adls> '
18  --enable-delete-retention true --delete-retention-days 14 '
19  --enable-versioning true
20
21 az storage account update --name <adls> --resource-group $rg '
22  --public-network-access Disabled --default-action Deny

```

Listing A.1: Core ADLS Gen2 lifecycle commands.

Tip

Use `--auth-mode login` so data-plane operations execute as your Entra identity under RBAC instead of the account key. Scripts that pass account keys are credential-handling liabilities.

A.3 Synapse Analytics

Command	Purpose
<code>az synapse workspace create ...</code>	Workspace with storage and SQL admin
<code>az synapse sql pool create</code> <code>-performance-level DW100c</code>	Provision a dedicated pool
<code>az synapse sql pool pause / resume</code>	Stop/start compute billing
<code>az synapse sql pool list -workspace-name</code> <code><ws></code>	Inventory pools
<code>az synapse workspace firewall-rule create</code>	Allow a client IP range
<code>az synapse spark pool create</code>	Provision a Spark pool

The pause/resume pair is the cost-control command set from Chapter 5; automate it rather than relying on memory.

A.4 Data Factory, Databricks, and Event Hubs

```

1 az datafactory create --name <adf> --resource-group $rg --location eastus
2 az datafactory show --name <adf> --resource-group $rg '
3  --query identity.principalId
4
5 az databricks workspace create --name <dbx> --resource-group $rg '
6  --location eastus --sku premium
7
8 az eventhubs namespace create --name <ns> --resource-group $rg '
9  --sku Standard --capacity 2 --enable-auto-inflate '
10  --maximum-throughput-units 10
11 az eventhubs eventhub create --name <hub> --namespace-name <ns> '
12  --resource-group $rg --partition-count 8 --message-retention-in-days 3
13
14 az stream-analytics job create --name <job> --resource-group $rg '
15  --location eastus --streaming-units 3

```

Listing A.2: Orchestration and streaming estate commands.

A.5 Networking, Key Vault, and Security

```

1 az network vnet create --name vnet-data --resource-group $rg '
2   --address-prefix 10.20.0.0/16 --subnet-name snet-data '
3   --subnet-prefix 10.20.1.0/24
4
5 az network private-endpoint create --name pep-adls-dfs '
6   --resource-group $rg --vnet-name vnet-data --subnet snet-data '
7   --private-connection-resource-id <adls-id> '
8   --group-id dfs --connection-name adls-dfs-conn
9
10 az network private-dns zone create --resource-group $rg '
11   --name "privatelink.dfs.core.windows.net"
12
13 az keyvault create --name <kv> --resource-group $rg --location eastus '
14   --enable-rbac-authorization true --enable-purge-protection true
15 az keyvault secret set --vault-name <kv> --name my-secret --value <value>
16 az keyvault secret show --vault-name <kv> --name my-secret --query value -o tsv

```

Listing A.3: Private networking and secrets.

A.6 Monitoring and Cost

Command	Purpose
<code>az monitor diagnostic-settings create</code>	Route resource logs to Log Analytics
<code>az monitor metrics alert create</code>	Threshold alert on a metric
<code>az monitor log-analytics query</code>	Run a KQL query from the shell
<code>az consumption budget create</code>	Budget with alert thresholds
<code>az resource list -tag product=orders</code>	Inventory by cost-allocation tag

A.7 Universal Flags and Query Patterns

- `-query "<jmespath>" -o tsv` extracts fields for scripting; `-o table` for humans; `-o json` for full fidelity.
- `-what-if` is available on deployments (not direct resource commands); use Bicep/ARM deployment what-if for previewing changes.
- `az find "<fuzzy description>"` searches the command surface when you remember the intent but not the verb.
- `az <command> -help` is consistently excellent; consult it before searching the web.

A.8 Cross-References

The chapters that use each command family: storage (Chapters 1, 5), Synapse (Chapters 5–8), ADF and Databricks (Chapters 9–12), Event Hubs and ASA (Chapters 13–16), ML workspaces (Chapters 18–19), and networking and Key Vault (Chapters 21–23). For infrastructure as code, see Chapter 24 and [7, 90].

Appendix B

Spark SQL and Delta Lake Quick Reference

This appendix consolidates the Spark, SQL, and Delta Lake syntax used across the book into one lookup. Examples run on Azure Databricks or Synapse Spark unless noted; T-SQL fragments are Synapse dedicated or serverless pool syntax.

B.1 Reading and Writing

```
1 LAKE = "abfss://curated@<adls>.dfs.core.windows.net"
2
3 df = spark.read.parquet(f"{LAKE}/orders/")
4 df_json = (spark.read.schema("order_id STRING, amount DOUBLE")
5           .json(f"{LAKE}/raw/orders/"))
6 df_csv = (spark.read.option("header", True)
7           .option("inferSchema", False)          # declare schemas; inference costs
8           .a pass
9           .csv(f"{LAKE}/raw/orders/"))
10 (df.write.format("delta").mode("overwrite")
11   .option("overwriteSchema", "true")
12   .partitionBy("order_year", "order_month")
13   .save(f"{LAKE}/orders_v2/"))
14
15 spark.sql(f"CREATE TABLE IF NOT EXISTS orders_v2 "
16           f"USING DELTA LOCATION '{LAKE}/orders_v2/'")
```

Listing B.1: The lake read/write patterns used throughout Parts II–V.

B.2 DataFrame Operations and the Shuffle Boundary

Operation	Type	Notes
<code>select</code> , <code>filter</code> , <code>withColumn</code>	Narrow	No shuffle; stays in partition
<code>groupBy().agg()</code>	Wide	Shuffle by group key
<code>join()</code>	Wide	Shuffle unless broadcast or co-located
<code>broadcast(dim)</code>	Hint	Avoids the shuffle for small dims
<code>distinct()</code> , <code>dropDuplicates()</code>	Wide	State/shuffle on the full key
<code>repartition(n)</code> / <code>coalesce(n)</code>	Wide	Reshape partitions before writes
<code>cache()</code>	Eager marker	Reuse across multiple actions only

Key configuration: `spark.sql.shuffle.partitions` (default 200—tune to data volume), `spark.sql.adaptive` (skew and coalesce handling), and Photon on Databricks for vectorized scans (Chapter 11).

B.3 Delta Lake Operations

```

1 MERGE INTO silver.orders AS tgt
2 USING staging.batch AS src ON tgt.order_id = src.order_id
3 WHEN MATCHED AND src.updated_at > tgt.updated_at THEN UPDATE SET *
4 WHEN NOT MATCHED THEN INSERT *;
5
6 SELECT * FROM silver.orders VERSION AS OF 12;
7 SELECT * FROM silver.orders TIMESTAMP AS OF '2026-08-01T00:00:00';
8
9 DESCRIBE HISTORY silver.orders;
10 DESCRIBE DETAIL silver.orders;    -- file counts, sizes, partitioning
11
12 OPTIMIZE silver.orders ZORDER BY (customer_id);
13 VACUUM silver.orders RETAIN 168 HOURS;    -- 7 days; check time-travel consumers
    first

```

Listing B.2: Delta maintenance and time travel (Spark SQL).

Common Pitfall

VACUUM under the retention horizon of a time-travel consumer. Files removed by VACUUM break `VERSION AS OF` queries older than the retention. Coordinate the horizon with audit and reprocessing requirements (Chapters 6, 23).

B.4 Structured Streaming

```

1 stream = (spark.readStream.format("kafka").options(**kafka_options).load()
2           .select(F.from_json(F.col("value").cast("string"), schema).alias("e"))
3           .select("e.*")
4           .withColumn("event_time", F.to_timestamp("ts")))
5
6 agg = (stream.withWatermark("event_time", "10 minutes")
7         .groupBy(F.window("event_time", "5 minutes"), "deviceId")
8         .agg(F.avg("tempC").alias("avg_temp")))
9
10 (agg.writeStream.format("delta").outputMode("append")
11     .option("checkpointLocation", "abfss://checkpoints@<adls>/iot/v1")
12     .trigger(processingTime="1 minute")    # or availableNow=True for batch-
    style

```

```
13 .toTable("iot_agg"))
```

Listing B.3: The Chapter 15 streaming skeleton, condensed.

Remember: version checkpoint paths per deployment; never restart changed logic from a stale checkpoint.

B.5 Synapse T-SQL Essentials

```
1 -- bulk load (dedicated pool)
2 COPY INTO stg_sales
3 FROM 'https://<adls>.dfs.core.windows.net/raw/sales/*.parquet'
4 WITH (FILE_TYPE='PARQUET', CREDENTIAL=(IDENTITY='Managed Identity'));
5
6 -- distributed table creation
7 CREATE TABLE fact_sales
8 WITH (DISTRIBUTION = HASH(customer_id), CLUSTERED COLUMNSTORE INDEX)
9 AS SELECT * FROM stg_sales;
10
11 -- serverless: query the lake in place
12 SELECT * FROM OPENROWSET(
13     BULK 'https://<adls>.dfs.core.windows.net/gold/fact_sales/',
14     FORMAT = 'DELTA') AS f
15 WHERE f.filepath(1) = '2026';
16
17 -- serverless: materialize back to the lake
18 CREATE EXTERNAL TABLE curated.orders_clean
19 WITH (LOCATION='orders_clean/', DATA_SOURCE=ds_curated,
20     FILE_FORMAT=parquet_ff)
21 AS SELECT ... ;
22
23 -- in-database scoring (dedicated pool)
24 SELECT * FROM PREDICT(MODEL = @model, DATA = dbo.features AS d,
25     RUNTIME = ONNX) WITH (score REAL);
```

Listing B.4: Dedicated and serverless pool statements used in Chapters 5–8.

B.6 Useful DMVs

DMV	Answers
sys.dm_pdw_exec_requests	What ran, how long, which label
sys.dm_pdw_request_steps	Where data moved (shuffle steps)
sys.pdw_nodes_column_store_row_groups	Rowgroup health per table
sys.dm_pdw_wlm_waiting_requests	Concurrency queue depth
sys.workload_management_workload_classification	Classifier routing rules

B.7 Cross-References

Chapters 5–7 (warehouse and lakehouse SQL), 9 and 12 (Spark and Delta), 15 (streaming), and 17 (PREDICT). Primary documentation: Spark [96], Structured Streaming [97], Delta Lake [1, 5], and Synapse best practices [26, 66].

Governance and Production-Readiness Checklists

This appendix collects the book’s operating discipline into printable checklists. Use them as go-live gates for new data products, as audit preparation, and as the skeleton of your platform’s monthly review (Chapter 22). Each item cites the chapter that teaches it.

Note

A checklist is a minimum bar, not a ceiling. When a product handles regulated data, extend the privacy section with your compliance team’s requirements before first load, not after first incident.

C.1 New Data Product Go-Live

Storage and Layout

- ☐ Storage account, container, and zone follow the naming convention (Ch. 1).
- ☐ Redundancy (LRS/ZRS/GZRS) chosen against an RPO/RTO statement (Ch. 1).
- ☐ Lifecycle policy defined per prefix with tested prefixes (Ch. 1).
- ☐ Formats are columnar (Delta/Parquet) in curated zones; file sizes managed with OPTIMIZE/VACUUM (Ch. 7, 9).
- ☐ Grain declared for every fact table; distribution keys chosen with skew evidence (Ch. 5, 6).

Pipelines and Quality

- ☐ Pipeline is parameterized for dev/test/prod from one artifact set (Ch. 10).
- ☐ Sinks are idempotent; a demonstrated rerun changes nothing (Ch. 10, 12).
- ☐ At least three data-quality tests gate the load; poisoned-batch path demonstrated (Ch. 22).
- ☐ Failure path notifies a human; audit table records every run (Ch. 10, 12).
- ☐ Contract version recorded per source; quarantine and reprocessing runbook exists (Ch. 11).

Security and Privacy

- ☐ All service-to-service access uses managed identities; zero secrets in code (Ch. 21).
- ☐ Private endpoints plus private DNS; public access disabled; outside-fails proof captured (Ch. 21).
- ☐ PII classified in Purview *before* any serving view exposes it (Ch. 8, 22).
- ☐ Masking/encryption/tokenization chosen per threat model (Ch. 23).
- ☐ Erasure path designed (crypto-shredding or per-store purge) with hold interaction (Ch. 23).

Observability and Cost

- ☐ Diagnostic settings route to Log Analytics; failure, drift, and silence alerts armed (Ch. 22).
- ☐ Freshness/completeness/correctness SLOs published with measurement queries (Ch. 22).
- ☐ Resources tagged (`product`, `environment`, `domain`); budget alert set (Ch. 22).
- ☐ Monthly cost estimate documented with at least two optimization decisions (all capstones).
- ☐ Idle-cost discipline verified: pools paused, clusters terminate, compute scales to zero (Ch. 5, 9, 18).

C.2 Schema Change (Expand–Contract) Gate

- ☐ Expand: new column/shape added alongside the old; nothing removed (Ch. 24).
- ☐ Migrate: dual-write active; backfill completed and reconciled (Ch. 24).
- ☐ Switch: view shim presents one stable interface to consumers (Ch. 24).
- ☐ Contract: Purview lineage plus dbt `ref()` graph show zero readers of the old shape (Ch. 22, 24).
- ☐ Each step shipped as a separate reviewed release; rollback path stated (Ch. 24).

C.3 Streaming Job Production Gate

- ☐ Partition key and count justified with ordering/parallelism arithmetic (Ch. 13).
- ☐ Producers attach unique event IDs; consumers are idempotent and checkpointed (Ch. 13).
- ☐ Event time declared (`TIMESTAMP BY` / watermarks); late/out-of-order policy documented (Ch. 14, 15).
- ☐ State bounded by watermarks; state-store provider chosen with size evidence (Ch. 15).
- ☐ Checkpoint paths versioned per deployment; change procedure documented (Ch. 15).
- ☐ Watermark-delay and input-stall alerts distinguish silence from zero traffic (Ch. 14).
- ☐ Capture or replay path exists for reconciliation and backfill (Ch. 13, 14).

C.4 ML Model Promotion Gate

- ☐ Model registered with lineage: data asset, environment, code, metrics (Ch. 18).

- ☐ Feature schema alignment validated; training and serving share one definition (Ch. 17).
- ☐ Metric gate passed on fresh holdout; AutoML baseline comparison recorded (Ch. 18).
- ☐ Shadow/canary evaluation on live traffic completed (Ch. 19).
- ☐ Human approval recorded; rollback is a traffic shift, not a redeploy (Ch. 19).
- ☐ Drift monitor armed with the retraining runbook attached (Ch. 19).

C.5 Monthly Platform Review Agenda

1. **Reliability:** SLO attainment per product; top three incidents with root causes (Ch. 22).
2. **Quality:** gate failure rate; quarantine backlog age; tests added this month (Ch. 22).
3. **Governance:** unclassified assets; unscanned sources; stale access reviews (Ch. 8, 22).
4. **Cost:** top ten movers; budget variance; optimizations executed and planned (Ch. 22).
5. **Security:** posture report—public access, shared keys, purge protection, role breadth (Ch. 21).
6. **Privacy:** erasure SLA performance; hold register reconciled with legal (Ch. 23).

C.6 Quarterly Drills

- ☐ Erasure drill on a synthetic subject with certificate (Ch. 23).
- ☐ Restore drill: recover one deleted table and one deleted container (Ch. 1).
- ☐ Region-failure tabletop for RA-GRS/GZRS estates with the runbook (Ch. 1).
- ☐ Chaos drill: kill a streaming consumer mid-batch; measure recovery against the SLO (Ch. 15).
- ☐ Gate fire drill: poisoned batch through every gated pipeline (Ch. 22).
- ☐ Rebuild drill: `terraform destroy`/`apply` one non-production estate (Ch. 24).

Cross-Cloud Service Equivalence

This appendix is a living reference that maps conceptually equivalent services and ideas across the seven platforms studied in this book series: Amazon Web Services (AWS), Microsoft Azure, Microsoft Fabric, Google Cloud Platform (GCP), MongoDB, Snowflake, and Databricks. The names differ, but the underlying data-engineering concepts—durable object storage, tiered life-cycle economics, immutability, managed relational engines, elastic compute, governed lakehouse tables, and disciplined data organization—recur everywhere.

Note

This appendix grows with each chapter. Every chapter contributes the equivalence table introduced in its cross-cloud callout, so by the end of the book you hold a single consolidated Rosetta Stone for moving fluently between clouds. Databricks appears throughout as the lakehouse-platform column: it runs on AWS, Azure, and GCP alike, so its entries describe the platform layer rather than the underlying cloud. Where a clean one-to-one mapping does not exist, the prose notes the closest analog and the caveat.

D.1 Free Tiers and Environment Setup

Chapter 0’s account-and-tooling setup has a counterpart on every platform, and the guardrail pattern—separate the admin identity, set a budget on day one, name and tag everything for deletion—is identical everywhere.

Setup step	AWS	Azure	GCP
Free offer	Free Tier (12 mo + always free)	Free account (\$200/30 days + always free)	Free trial (\$300/90 days + Always Free)
Identity model	IAM / IAM Identity Center	Microsoft Entra ID + RBAC	Cloud IAM
CLI	AWS CLI v2	Azure CLI (az login)	gcloud CLI (gcloud init)
Browser shell	AWS CloudShell	Azure Cloud Shell	Cloud Shell
Cost guardrail	AWS Budgets + alerts	Cost Management budgets	Budgets & alerts

Microsoft Fabric starts from a Microsoft 365/Power BI work account with a trial capacity; **Snowflake** offers a 30-day/\$400 trial guarded by Resource Monitors; **MongoDB Atlas** has a forever-free M0 cluster behind database users and an IP allowlist. **Databricks** offers a time-limited free trial on each cloud (plus the serverless free tier and the Community Edition for learning); the first-day disciplines are the same—create a workspace in its own resource group, attach

cluster policies before users arrive, and route billing through tags or `system.billing.usage` from the start.

D.2 Object and Data-Lake Storage

The foundational layer covered in Chapter 1. Every platform offers durable, HTTP-addressable object storage with tiered pricing, lifecycle transitions, versioning, and write-once-read-many (WORM) controls.

Capability	AWS	Azure	GCP
Object storage	Amazon S3	ADLS Gen2 / Blob	Cloud Storage
Hot tier	S3 Standard	Hot	Standard
Infrequent access	S3 Standard-IA	Cool	Nearline
Archive (instant)	Glacier Instant Re-trieval	Cold	Coldline
Deep archive	Glacier Deep Archive	Archive	Archive
Lifecycle rules	Lifecycle policies	Lifecycle management	Object Lifecycle Mgmt
Versioning	S3 Versioning	Blob versioning	Object Versioning
Immutability (WORM)	Object Lock	Immutable blob storage	Bucket Lock
Cross-region copy	CRR / SRR	Object replication	Dual/multi-region + Transfer
Encryption at rest	SSE-S3 / SSE-KMS	SSE + CMK (Key Vault)	Google-managed / CMEK

The three remaining platforms approach storage differently. **Microsoft Fabric** exposes a single logical lake, *OneLake*, and uses *shortcuts* to virtualize S3, ADLS Gen2, and Google Cloud Storage in place without copying. **Snowflake** abstracts storage entirely: table data lives in cloud object storage as immutable, columnar *micro-partitions* that the service manages on your behalf. **MongoDB** persists documents through the *WiredTiger* storage engine, balancing an internal cache against the operating-system page cache rather than exposing storage tiers. **Databricks** does not own a storage tier at all: it reads and writes the customer’s S3, ADLS Gen2, or GCS buckets through Unity Catalog *external locations* and *volumes*, with Delta Lake supplying the transactional table layer on top—proof that the lake and the compute plane are separable purchases.

D.3 Managed Relational Databases

The focus of Chapter 2 for the three hyperscalers. Each provides a fully managed relational engine, a cloud-native high-performance variant, high availability, read replicas, and a migration service.

Capability	AWS	Azure	GCP
Managed relational	Amazon RDS	Azure SQL Database	Cloud SQL
Cloud-native engine	Amazon Aurora	SQL Hyperscale	AlloyDB
Managed PostgreSQL	RDS / Aurora PostgreSQL	Azure DB for PostgreSQL	Cloud SQL / AlloyDB
High availability	Multi-AZ	Zone-redundant HA	Regional HA
Read scaling	Read Replicas	Read replicas / geo-replicas	Read Replicas
Global / geo	Aurora Database Global	Active geo-replication	Cross-region replicas
Serverless compute	Aurora Serverless v2	Azure SQL Serverless	AlloyDB (elastic)
Migration service	DMS + SCT	Azure Database Migration Svc	Database Migration Svc

Snowflake, **Fabric**, and **MongoDB** are not managed relational engines, but they solve overlapping problems: Snowflake and Fabric provide SQL analytics over warehouse/lakehouse storage, while MongoDB provides a distributed document store whose schema design (Chapter 2) replaces rigid relational normalization with intentional, access-driven modeling. **Databricks** is likewise not an OLTP engine; its *Lakehouse Federation* feature queries external relational systems (SQL Server, PostgreSQL, MySQL) in place so operational data can be joined or ingested without a separate ETL hop.

D.4 Elastic Compute and Analytical Warehouses

Snowflake’s Chapter 2 topic—separating compute from storage and scaling it elastically—has a counterpart in every analytics platform.

Concept	Snowflake	AWS	Azure / GCP
Compute unit	Virtual Warehouse	Redshift RA3 / Serverless	Synapse pool / Big-Query slots
Elastic concurrency	Multi-cluster warehouse	Concurrency scaling	Autoscale / on-demand
Pause when idle	Auto-suspend / resume	Pause / resume	Auto-pause / serverless
Billing unit	Credits	RPU / node-hours	DWU / slot-hours
Result reuse	Result cache	Result cache	Result / BI cache

Microsoft Fabric meters all workloads against a shared pool of Capacity Units (CU) on an F-SKU, an approach closer to a single elastic budget than to per-engine warehouses. **MongoDB Atlas** scales through cluster tiers with optional auto-scaling of compute and storage. **Databricks** bills Databricks Units (DBU) on top of VM cost: all-purpose clusters for interactive work, job clusters that terminate on completion for pipelines, and serverless SQL warehouses that scale to zero—the same pause-when-idle discipline with more explicit knobs.

D.5 Data Organization and the Medallion Pattern

Fabric’s Chapter 2 topic—the Bronze/Silver/Gold medallion—is a portable design pattern rather than a product. Every platform implements the same progressive-refinement idea with its own primitives.

Layer	Fabric	Data lake (AWS/Azure/GCP)	Snowflake
Bronze (raw)	Bronze Lakehouse	raw/ (S3/ADLS/GCS) prefix	RAW schema / stage
Silver (validated)	Silver Lakehouse	validated/ prefix	STAGING schema
Gold (curated)	Gold Lakehouse	curated/ prefix	ANALYTICS schema
Storage format	Delta-Parquet	Parquet / Delta / Iceberg	Micro-partitions

MongoDB realizes the same separation with distinct collections or databases (for example `raw_events`, `validated`, `curated`) governed by schema-design patterns rather than a lake layout. **Databricks** is where the medallion vocabulary was popularized: `bronze/silver/gold` schemas of Delta tables under a Unity Catalog catalog, with expectations and quarantine as the quality gates between layers and Unity Catalog grants as the access boundary per layer.

D.6 Document Data Modeling

MongoDB’s Chapter 2 schema-design patterns generalize to every document and wide-column store.

Concept	MongoDB	Amazon namoDB	Dy-	Azure Cosmos DB
Primary access unit	Document / collection	Item / table		Item / container
Partitioning	Shard key	Partition key		Partition key
One-table modeling	Embedding + patterns	Single-table design		Embedding + partitioning
Time-series grouping	Bucket Pattern	Composite sort keys		Bucketing by key
Optional attributes	Attribute Pattern	Sparse attributes		Sparse properties

Google Cloud’s closest analogs are **Firestore** and **Bigtable**. In **Snowflake** and **Fabric**, semi-structured documents are stored in **VARIANT** (Snowflake) or JSON columns and queried with path expressions rather than modeled as collections. **Databricks** follows the same lakehouse pattern: JSON payloads land as strings or **VARIANT** columns in Delta tables and are shredded with `from_json` and schema hints during the bronze-to-silver pass.

D.7 Globally Distributed and NoSQL Databases

Chapter 3 branches by platform into high-scale NoSQL (AWS DynamoDB, Azure Cosmos DB), globally consistent NewSQL (GCP Cloud Spanner), and advanced document modeling (MongoDB). They all answer the same question: how to scale a database horizontally across partitions and regions.

Concept	DynamoDB	Cosmos DB	Cloud Spanner	MongoDB
Model	Key-value / doc	Multi-model	Distributed SQL	Document
Partitioning	Partition key	Partition key	Split by key range	Shard key
Throughput unit	RCU / WCU	Request Units (RU)	Nodes / processing units	Cluster tier
Secondary indexes	GSI / LSI	Automatic indexing	Secondary indexes	Indexes
Change stream	DynamoDB Streams	Change Feed	Change streams	Change streams
Expiry	TTL	TTL	(manual)	TTL index
Multi-region	Global Tables	Multi-region writes	Multi-region (synchronous)	Zones / global clusters
Consistency	Eventual / strong	5 tunable levels	External (True-Time)	Tunable read/write concern

Snowflake and **Fabric** are analytical platforms rather than OLTP key-value stores; they ingest from these operational databases and store semi-structured payloads in **VARIANT**/JSON columns. Google’s dedicated NoSQL analogs are **Firestore** (document) and **Bigtable** (wide-column). **Databricks** deliberately stays out of operational NoSQL: it consumes change feeds from Cosmos DB, DynamoDB, and MongoDB into Delta tables, leaving point reads and sub-millisecond serving to the operational stores.

D.8 Managed Apache Spark

Microsoft Fabric’s Chapter 3 topic—managed Spark pools and notebooks—has a first-class equivalent on every cloud.

Capability	Fabric	AWS	Azure / GCP
Managed Spark	Starter / Custom pools	Glue / EMR / EMR Serverless	Synapse Spark / Databricks; Dataproc
Notebook UX	Fabric Notebooks	Glue Studio / EMR Studio	Synapse / Databricks; Vertex AI
Serverless option	Autoscaling pools	Glue / EMR Serverless	Synapse serverless; Dataproc Serverless
Lake format	Delta on OneLake	Delta / Iceberg / Hudi	Delta; Delta / Iceberg

Snowflake offers **Snowpark** for DataFrame-style processing, and **MongoDB** integrates through the Spark Connector. **Databricks** is the reference implementation this book uses in Chapter 9: the Databricks Runtime with the vectorized Photon engine, a built-in cluster manager (no YARN to run), autoscaling clusters plus serverless compute, and Delta as the default table format. The Spark API, Catalyst optimizer, and shuffle mechanics are identical on EMR, Dataproc, Fabric, and Databricks—only the operational wrapper and the billing unit change.

D.9 Query Acceleration and Materialized Views

Snowflake’s Chapter 3 acceleration tools—materialized views, the Search Optimization Service, and transient/temporary tables—map to comparable features in every warehouse.

Feature	Snowflake	AWS Redshift	Azure Synapse / GCP BigQuery
Materialized views	Materialized Views	Materialized views	Materialized views
Point-lookup accel.	Search Optimization Svc	Sort keys / zone maps	(indexing) / search indexes
Result reuse	Result cache	Result cache	Result-set cache / BI Engine
Ephemeral tables	Transient / temporary	Temp tables	Temp tables

Fabric accelerates reads through Direct Lake and its Warehouse engine, while **MongoDB** relies on secondary and compound indexes plus the in-memory working set. **Databricks SQL** pairs materialized views and streaming tables with a result cache on serverless warehouses, and adds file-level acceleration—OPTIMIZE, Z-order and liquid clustering, bloom filters, and deletion vectors—that the next sections cover in detail.

D.10 In-Memory Caching and Hybrid Ingestion

The AWS and Azure Chapter 4 topic—low-latency caching in front of a durable store, fed by hybrid (edge-to-cloud) ingestion—has a managed Redis-compatible equivalent on every cloud.

Capability	AWS	Azure	GCP / others
Managed cache	ElastiCache (Redis/Memcached)	Cache for Redis	Memorystore (Redis/Memcached)
Real-time API layer	AppSync / API Gateway	API Management / SignalR	API Gateway / Apigee
Hybrid ingestion	DataSync / Snow family / IoT Core	Data Box / IoT Hub / Arc	Transfer Appliance / IoT Core
Streaming intake	Kinesis / MSK	Event Hubs	Pub/Sub

Snowflake and **Fabric** rely on result and Direct Lake caches rather than a standalone Redis tier, while **MongoDB** serves hot data from its in-memory working set and Atlas can pair with an external cache. **Databricks** caches at a different layer: the disk cache (formerly Delta cache) keeps hot Parquet blocks on cluster-local SSDs, and predictive I/O accelerates repeated scans without any application-managed cache keys.

D.11 Wide-Column and Key-Value NoSQL

GCP’s Chapter 4 topic—Cloud Bigtable—is a wide-column, high-throughput, low-latency NoSQL store. Its peers span the managed and open-source ecosystems.

Characteristic	GCP Bigtable	AWS	Azure / open source
Data model	Wide-column	DynamoDB (key-value/doc)	Cosmos DB (multi-model)
HBase-compatible	HBase API	Keyspaces (Cassandra)	Managed Instance for Cassandra
Scaling unit	Nodes / clusters	On-demand / provisioned	RU/s (Cosmos)
Best fit	Time-series, IoT, ad-tech	Serverless key-value	Global multi-model

Snowflake and **Fabric** address these workloads analytically rather than as operational key-value stores; **MongoDB** covers the document end of the same NoSQL spectrum. **Databricks** likewise serves these workloads analytically: Bigtable and Cassandra estates are typically migrated by bulk export to Delta tables, not by replacing the serving path.

D.12 Data Protection: Cloning, Time Travel, and Migration

Snowflake’s Chapter 4 topic—zero-copy cloning, Time Travel, and Fail-safe—plus MongoDB’s relational-to-document migration map to data-protection and migration tooling across the clouds.

Capability	Snowflake	AWS / Azure	GCP / Fabric
Instant clone	Zero-copy clone	Aurora clone / SQL DB copy	BigQuery table clone; OneLake shortcut
Point-in-time recovery	Time Travel	PITR (RDS/SQL DB backups)	PITR; Delta time travel
Retention safety net	Fail-safe (7 days)	Automated backups	Backups; Delta VACUUM window
Schema migration	—	DMS / SCT	Database Migration Service

MongoDB migrations use **Relational Migrator** plus the embed-vs-reference patterns from Chapter 4, and **Delta Lake** exposes historical versions through **VERSION AS OF** time-travel queries. **Databricks** operationalizes the same protection set on Delta: **RESTORE** for point-in-time recovery, **VACUUM** as the retention safety-valve cutoff, and *shallow clones* as the zero-copy clone analog for dev/test copies of production tables.

D.13 MPP Data Warehouses

Chapter 5’s dedicated SQL pool is one of the most portable ideas in analytics engineering: a control node coordinates compute nodes over physically distributed tables. The same mental model applies to Redshift’s ra3 clusters and BigQuery’s slot-based execution.

Capability	AWS	Azure	GCP
MPP warehouse	Amazon Redshift	Synapse dedicated SQL pool	BigQuery (slot-based)
Sizing unit	ra3 nodes / RPUs	cDWU (pauseable)	Slots / autoscaling editions
Distribution	DISTKEY / DISTSTYLE	Hash / round-robin / replicated	Automatic (no user control)
Bulk load from lake	COPY from S3	COPY INTO from ADLS Gen2	LOAD DATA from Cloud Storage
Serverless SQL over lake	Athena / Redshift Spectrum	Synapse serverless SQL pool	BigQuery external tables

Fabric: T-SQL MPP warehouse on OneLake; **Snowflake:** virtual warehouses; **MongoDB:** Atlas SQL (not MPP); **Databricks:** SQL warehouses (serverless / pro / classic) with the Photon engine over Delta tables.

The Kimball discipline—grain declarations, conformed dimensions, slowly changing dimensions—is engine-neutral. A star schema designed well in Chapter 5 ports to Redshift, BigQuery, Snowflake, a Fabric warehouse, or a Databricks SQL warehouse with only dialect changes.

D.14 Warehouse Physical Design and Workload Isolation

Chapter 6’s optimization levers—columnar storage, distribution, workload groups, precomputed results—exist on every warehouse; only the knobs differ. Synapse exposes physical design explicitly, while BigQuery and Snowflake automate more of it: the trade-off is control versus toil.

Capability	AWS (Redshift)	Azure (Synapse)	GCP (BigQuery)
Columnar storage	Columnar by default	Clustered column-store index	Columnar by default
Physical layout	Sort keys + dist keys	Distribution + partitioning	Clustering + partitioning
Workload isolation	WLM queues	Workload groups + classifiers	Slot reservations
Precomputed results	Materialized views	Materialized views + result-set cache	Materialized views + BI Engine
Query cache	Result cache	Result-set cache	Cached query results

Fabric: warehouse with automatic tuning; **Snowflake:** micro-partition pruning + resource monitors; **MongoDB:** indexes + covered queries; **Databricks:** OPTIMIZE, Z-order, liquid clustering, predictive optimization.

Databricks’ *liquid clustering* deserves special attention: like Iceberg’s sort-order evolution and BigQuery’s clustering, it makes physical layout rewritable without rewriting the table, which removes the classic “wrong distribution key” lock-in that Chapter 6 warns about.

D.15 Serverless SQL over the Lake

Chapter 7’s pattern—query the lake in place, pay per byte scanned, materialize back with CETAS—exists on every major cloud, and the billing model makes format and partitioning discipline a cross-cloud skill rather than a Synapse trick.

Capability	AWS	Azure	GCP
Serverless lake SQL	Amazon Athena	Synapse serverless SQL pool	BigQuery external tables
Query-in-place API	S3 Select / Athena	OPENROWSET	External data sources
Materialize to lake	CTAS into S3	CETAS	Export / CREATE TABLE AS
Open table format	Delta / Iceberg / Hudi on S3	Delta Lake on ADLS Gen2	BigLake tables / Iceberg
Billing lever	Per TB scanned	Per TB scanned	Per TB scanned (on-demand)

Fabric: SQL analytics endpoint over OneLake; **Snowflake:** external tables / Iceberg;

MongoDB: Atlas Data Federation over S3; **Databricks:** serverless SQL warehouses over UC Delta tables.

The lakehouse thesis—warehouse-class reliability on lake storage—originated with Delta Lake [1, 2] and is now the default architecture vocabulary across all seven platforms in this series.

D.16 Data Catalogs and Governance Planes

Chapter 8’s governance plane—a metastore-backed catalog, automated classification, lineage, and in-place sharing—has a native equivalent on every cloud. Purview’s distinction is scope: it catalogs Azure, AWS, GCP, and on-premises sources in one data map.

Capability	AWS	Azure	GCP
Catalog / data map	Glue Data Catalog + Lake Formation	Microsoft Purview	Dataplex
PII discovery	Macie	Purview classifications	Sensitive Data Protection
Lineage	Glue / CloudTrail analysis	Purview lineage	Dataplex lineage
Cross-tenant sharing	AWS Data Exchange / S3 sharing	Azure Data Share	Analytics Hub / BigQuery sharing
Clean room	AWS Clean Rooms	Data Share + confidential compute	BigQuery data clean rooms

Fabric: Purview hub + OneLake catalog; **Snowflake:** Horizon Catalog + listings;

MongoDB: Atlas auditing; **Databricks:** Unity Catalog + Delta Sharing (open protocol).

Unity Catalog’s distinctive move is one governance model—grants, row filters, column masks, lineage, audit—spanning tables, files, notebooks, models, and functions; the hyperscalers split the same job across a catalog plus a policy engine. Delta Sharing’s distinction is the open protocol: recipients consume shares with pandas, Spark, or Power BI without any Databricks account.

D.17 Managed Visual ETL and Orchestration

Chapter 10’s four-part ADF object model—connection (linked service), data pointer (dataset), activity graph (pipeline), schedule (trigger)—has a direct analog in every managed ETL service.

Capability	AWS	Azure	GCP
Managed ETL service	AWS Glue	Azure Data Factory	Cloud Data Fusion
Visual transform engine	Glue Studio / DataBrew	Mapping Data Flows	Data Fusion wrangler
Orchestration	Step Functions / MWAA	ADF pipelines and triggers	Cloud Composer (Airflow)
Serverless transform compute	Glue jobs (Spark)	Data Flows on managed Spark	Dataflow (Beam)
Connection abstraction	Glue connections	Linked services	Data Fusion connections

Fabric: pipelines + dataflows gen2; **Snowflake:** tasks/streams + partner tools;
MongoDB: Atlas Triggers; **Databricks:** Lakeflow Declarative Pipelines + Workflows.

Mapping Data Flows compile to Spark under the hood, so the execution concepts from Chapter 9 (partitions, shuffles) apply underneath the visual canvas—the abstraction leaks upward when performance matters. Databricks’ declarative pipelines share the same philosophy: define datasets and quality expectations, let the platform handle execution, retries, and incremental refresh.

D.18 Change Data Capture, Migration, and Hybrid Connectivity

Chapter 11’s integration problems—reaching private networks and streaming database changes—are universal. Every cloud offers an agent/relay for private connectivity and a replication service for change data capture.

Capability	AWS	Azure	GCP
Private data movement agent	Glue connectors / DataSync agent	Self-Hosted Integration Runtime	VPN/Interconnect + remote agent
Database migration	AWS DMS (+SCT)	Azure DMS (+DMA)	Database Migration Service
CDC stream	DMS ongoing replication	Azure SQL CDC / DMS CDC	Datastream
Event fabric	EventBridge	Event Grid	Eventarc
Schema enforcement	Glue Schema Registry	Contract checks in ADF/Databricks	Dataflow / Dataplex validation

Fabric: on-premises data gateway; **Snowflake:** Snowpipe + connectors;
MongoDB: change streams (CDC-native); **Databricks:** Lakeflow Connect, Auto Loader, AUTO CDC.

Data contracts—schema, versioning, quarantine—are platform-neutral governance. Databricks encodes them directly in the pipeline: Auto Loader’s schema inference with hints and evolution, the `_rescued_data` column for malformed rows, and DLT expectations that warn, drop, or fail on contract violations.

D.19 Medallion Orchestration and Scheduling

Chapter 12’s medallion pipelines need a scheduler with dependency graphs and failure semantics on every platform. Bronze/silver/gold is the same idea whether the lake is S3, ADLS, or GCS.

Capability	AWS	Azure	GCP
Medallion zones	S3 raw/curated + Glue	ADLS Gen2 + Delta Lake	GCS + BigLake
Managed Airflow	MWAA	Managed Airflow in ADF	Cloud Composer
Control-flow orchestration	Step Functions	ADF pipeline activities	Workflows
Event trigger	EventBridge → Lambda/Glue	Event Grid → ADF trigger	Eventarc → Workflows
Notification work-flows	SNS / Step Functions	Logic Apps	Cloud Functions + Workflows

Fabric: OneLake medallion + pipelines; **Snowflake:** tasks/streams DAGs;
MongoDB: Atlas Triggers; **Databricks:** Workflows with repair-run partial recovery + job clusters.

Airflow’s DAG model—tasks, dependencies, retries, backfills—is itself the cross-cloud skill: MWAA, Cloud Composer, and Managed Airflow in ADF all execute the same Python DAG files [94], and Databricks Workflows integrates with external orchestrators through the Jobs API when an estate standardizes on Airflow.

D.20 Event Streaming Backbones

Chapter 13’s partitioned, replayable event log is a cross-cloud primitive. Event Hubs’ model—partitions, consumer groups, offsets, retention—is deliberately Kafka-shaped, and every cloud offers an equivalent.

Capability	AWS	Azure	GCP
Event streaming	Kinesis Data Streams / MSK	Event Hubs (+ Kafka endpoint)	Pub/Sub / Managed Kafka
Message queue	SQS / Amazon MQ	Service Bus queues and topics	Pub/Sub (pull)
Device ingestion	IoT Core	IoT Hub	(partner; IoT Core retired)
Stream-to-lake export	Kinesis Firehose	Event Hubs Capture	Pub/Sub to GCS subscriptions
Consumer isolation	Enhanced fan-out	Consumer groups	Subscriptions

Fabric: eventstreams into OneLake; **Snowflake:** Snowpipe Streaming;
MongoDB: change streams as a source; **Databricks:** `kafka` format source against the Event Hubs Kafka endpoint.

Because Event Hubs speaks the Kafka protocol, producer and consumer skills transfer to Amazon MSK, Confluent, and self-managed clusters with only endpoint and credential changes [95]—which is exactly how Chapter 15’s Databricks consumer attaches.

D.21 Managed SQL over Streams

Chapter 14’s managed SQL-over-streams exists on every cloud, and the windowing vocabulary—tumbling, hopping, sliding, session—is identical everywhere; only the engine and billing unit differ.

Capability		AWS	Azure	GCP
Managed SQL	stream	Managed Flink (Kinesis Data Analytics)	Azure Stream Analytics	Dataflow (Beam SQL)
Billing unit		Kinesis Processing Units	Streaming Units (SUs)	Dataflow worker resources
Event source		Kinesis / MSK	Event Hubs / IoT Hub	Pub/Sub
Late-data control		Watermarks in Flink	Late/out-of-order policies	Watermarks + allowed lateness
Stream to warehouse		Firehose → Redshift	ASA output → Synapse/SQL	Dataflow → BigQuery

Fabric: eventstreams + Activator; **Snowflake:** Snowpipe Streaming + dynamic tables;
MongoDB: Atlas Stream Processing; **Databricks:** Structured Streaming / DLT streaming tables.

When a use case outgrows ASA—complex state, stream-stream joins at scale, custom code—Spark Structured Streaming (Chapter 15) is the escalation path, with the same windowing concepts at a lower level of abstraction. Databricks exposes the full state-design surface: `withWatermark`, `dropDuplicates`, and `flatMapGroupsWithState` with RocksDB-backed state.

D.22 Spark Structured Streaming

Chapter 15’s model—a stream as an unbounded table, micro-batches, watermarks, checkpoints—maps directly onto the streaming engines of the other platforms.

Capability	AWS	Azure	GCP
Spark streaming host	EMR / Glue streaming	Azure Databricks	Dataproc / Serverless Spark
Stream source	Kinesis / MSK connector	Event Hubs (Kafka endpoint)	Pub/Sub / Kafka connector
State store	RocksDB on EMR	Default or RocksDB provider	RocksDB on Dataproc
Exactly-once sink	Delta/Iceberg merge	Delta Lake merge	BigQuery Storage Write API
Checkpoint storage	S3	ADLS Gen2	GCS

Fabric: structured streaming on OneLake; **Snowflake:** Snowpipe Streaming + tasks;
MongoDB: Atlas Stream Processing; **Databricks:** native—the reference implementation used in this chapter.

The Structured Streaming programming guide [97] is platform-neutral reading; everything Chapter 15 teaches about watermarks and state applies identically on EMR and Dataproc. The batch-unification trigger (`availableNow`) is the same idea as Dataflow’s unified batch/stream thesis and Flink’s batch mode.

D.23 Business Intelligence and Semantic Models

Chapter 16’s BI layer is the least portable part of the stack—each cloud has a native analytics front end—but the underlying concepts (extracted versus live models, row-level security, measure semantics) transfer directly.

Capability	AWS	Azure	GCP
BI service	Amazon QuickSight	Power BI	Looker / Looker Studio
Live query mode	Direct query to Redshift/Athena	DirectQuery	Live connection to BigQuery
Extracted cache	SPICE	Import mode (in-memory)	Extracted data / BI Engine
Row-level security	QuickSight RLS	Power BI RLS roles	Looker access filters
Metrics layer	QuickSight datasets	Semantic model + DAX	LookML

Fabric: Power BI with DirectLake over OneLake; **Snowflake:** Snowsight + partner BI;

MongoDB: Atlas Charts + BI Connector; **Databricks:** AI/BI dashboards + Genie natural-language spaces.

Power BI’s **DirectLake** mode in Microsoft Fabric is the emerging fourth storage mode: the engine reads Delta tables in OneLake directly, combining Import-mode speed with DirectQuery freshness. Databricks’ AI/BI makes the mirror-image bet: dashboards read governed Delta tables in place and inherit Unity Catalog lineage, so a tile traces back to the pipeline that produced it.

D.24 Feature Engineering and In-Database Machine Learning

Chapter 17’s in-database scoring—pushing the model to the data rather than shipping data to a model server—exists in every analytical engine, typically via ONNX or a native PREDICT-style function.

Capability	AWS	Azure	GCP
In-database scoring	Redshift ML (SageMaker)	Synapse T-SQL PREDICT (ONNX)	BigQuery ML
Spark training	EMR + SageMaker	Synapse Spark / SynapseML	Dataproc + Vertex AI
Feature store	SageMaker Feature Store	AML managed feature store	Vertex AI Feature Store
Model format for SQL	(auto via SageMaker)	ONNX	Native BQML models
Batch scoring	Redshift batch transform	PREDICT over pool tables	ML.PREDICT

Fabric: SynapseML + MLflow in notebooks; **Snowflake:** Snowpark ML + SQL inference;

MongoDB: Atlas Vector Search scoring; **Databricks:** Feature Engineering in UC + Mosaic AI serving.

ONNX is the portable artifact: train in Spark, export once, score in SQL, in an AML endpoint (Chapter 19), or at the edge. Databricks’ feature store makes the anti-skew contract explicit: one governed Delta definition feeds both training sets (via point-in-time joins) and online serving lookups.

D.25 Managed Machine Learning Platforms

Chapter 18’s managed ML platforms share a universal anatomy: a workspace, versioned data references, reusable environments, pipeline components, and a model registry.

Capability	AWS	Azure	GCP
ML platform	SageMaker	Azure Machine Learning	Vertex AI
AutoML	SageMaker Autopilot	Azure AutoML	Vertex AutoML
Pipelines	SageMaker Pipelines	AML pipelines (components)	Vertex Pipelines (Kubeflow)
Feature store	SageMaker Feature Store	AML managed feature store	Vertex AI Feature Store
Model registry	SageMaker Model Registry	AML Model Registry	Vertex Model Registry

Fabric: built-in ML with MLflow on lakehouses; **Snowflake:** Snowpark ML;
MongoDB: partner MLOps over Atlas; **Databricks:** managed MLflow + UC model registry.

MLflow is the lingua franca underneath: AML, Databricks, and Fabric all track experiments with it, so experiment-tracking skills transfer even where the platform does not. Databricks adds alias-based lifecycle (champion/challenger) and registry lineage from training data to deployed model inside Unity Catalog.

D.26 Model Serving, Monitoring, and Retraining

Chapter 19’s serving splits everywhere into real-time endpoints and batch scoring, and monitoring splits into infrastructure health, data drift, and model quality.

Capability	AWS	Azure	GCP
Real-time serving	SageMaker endpoints	AML managed online endpoints	Vertex AI endpoints
Batch scoring	SageMaker batch transform	AML batch endpoints	Vertex batch prediction
Model monitoring	SageMaker Model Monitor	AML model monitoring / drift	Vertex Model Monitoring
Registry stages	Registry + approval states	Registry tags/stages	Registry + aliases
Retrain trigger	EventBridge → pipeline	Event Grid → AML pipeline	Eventarc → pipeline

Fabric: MLflow models + SynapseML serving; **Snowflake:** Snowpark ML registry;
MongoDB: in-app scoring over Atlas; **Databricks:** Model Serving + inference tables + Lakehouse Monitoring.

The MLOps loop—monitor, detect drift, retrain, gate, promote—is platform-neutral. The Databricks distinction: inference logs are Delta tables in your own catalog, so monitoring is a SQL/BI problem over data you already govern, not a separate observability silo.

D.27 Unstructured Data and Generative AI

Chapter 20’s document AI, search, and managed foundation models exist on every cloud; the pipeline pattern—ingest, extract, index, retrieve, generate—is identical across vendors.

Capability	AWS	Azure	GCP
Document extraction	Textract	AI Document Intelligence	Document AI
NLP / PII detection	Comprehend	AI Language	Cloud Natural Language
Search / vectors	OpenSearch / Kendra	AI Search (vector + hybrid)	Vertex AI Search
Foundation models	Bedrock	Azure OpenAI Service	Vertex AI (Gemini)
Event-driven processing	Lambda	Azure Functions	Cloud Functions

Fabric: AI skills over OneLake; **Snowflake:** Cortex (LLM functions in SQL);
MongoDB: Atlas Vector Search; **Databricks:** Foundation Model APIs + Mosaic AI Vector Search.

The RAG pattern taught in Chapter 20—chunk, embed, index, retrieve, ground the model—is the same architecture whether the vector store is AI Search, OpenSearch, pgvector, Atlas Vector Search, or Mosaic AI Vector Search; evaluation with LLM judges (MLflow evaluate, Bedrock evaluations, Azure AI eval) is likewise a shared discipline.

D.28 Zero-Trust Networking, Identity, and Secrets

Chapter 21’s architecture—private connectivity, no long-lived secrets, centralized keys—is cloud-universal; the product names differ, the design does not.

Capability	AWS	Azure	GCP
Identity plane	IAM	Microsoft Entra ID + Azure RBAC	Cloud IAM
Workload identity	IAM roles for EC2/EKS	Managed identities	Service account impersonation
Private service access	PrivateLink / VPC endpoints	Private Endpoints	Private Service Connect
Secrets manager	Secrets Manager	Azure Key Vault	Secret Manager
Key custody (CMK)	KMS	Key Vault / Managed HSM	Cloud KMS / HSM

Fabric: Entra ID + OneLake security; **Snowflake:** PrivateLink + Tri-Secret;
MongoDB: Atlas private endpoints + CSFLE; **Databricks:** secret scopes, OAuth M2M, private connectivity.

The exam and interview favorite—“why is a managed identity better than a service principal secret?”—has the same answer on every cloud: no secret material exists to leak, rotate, or commit to git. Databricks’ PAT tokens are the cautionary contrast: they are long-lived secrets, which is why cluster policies and OAuth machine-to-machine service principals are the production pattern.

D.29 Observability, FinOps, and Quality Gates

Chapter 22’s operating layer—catalogs, lineage, quality gates, and cost observability—pairs a native catalog with the same open quality tooling on every cloud.

Capability	AWS	Azure	GCP
Catalog and governance	Glue Data Catalog / Lake Formation	Microsoft Purview	Dataplex
Data quality tests	dbt / Great Expectations / Deequ	dbt / Great Expectations in ADF or Databricks	dbt / Great Expectations / Dataplex rules
Monitoring	CloudWatch	Azure Monitor + Log Analytics	Cloud Operations
Cost management	Cost Explorer + budgets	Cost Management + budgets	Cloud Billing + budgets
Sensitivity labels	Macie findings	Purview + MIP labels to Power BI	DLP inspection

Fabric: Purview hub + Capacity Metrics; **Snowflake:** Horizon + resource monitors;

MongoDB: Atlas auditing; **Databricks:** `system.billing.usage` + `system.access.audit` + Lakehouse Monitoring.

dbt and Great Expectations are deliberately platform-neutral: the same `dbt test` or GE checkpoint runs against Synapse, Snowflake, BigQuery, Redshift, or Databricks SQL—quality tooling is the one layer you can learn once [4, 6].

D.30 Data Privacy and Compliance Primitives

Chapter 23’s privacy primitives—masking, client-side encryption, erasure, and policy audit—exist on every platform, and certification exams test them as a family.

Capability	AWS	Azure	GCP
Dynamic masking	Redshift dynamic masking	SQL/Synapse Dynamic Data Masking	BigQuery column-level + masking
Client-side encryption	Client-side encryption SDKs	Always Encrypted	Client-side encryption / CSEK
At-rest encryption	KMS-backed TDE equivalents	TDE (+ CMK via Key Vault)	Google-managed / CMEK
Policy enforcement	AWS Config + SCPs	Azure Policy	Organization Policy
Threat detection	GuardDuty / Security Hub	Microsoft Defender for Cloud	Security Command Center
PII discovery	Macie	Purview classifications	Sensitive Data Protection

Fabric: OneLake security + MIP labels; **Snowflake:** masking policies + tags;

MongoDB: client-side field-level encryption; **Databricks:** UC row filters + column masks.

Crypto-shredding—encrypt per data subject, erase by destroying the subject’s key—is the cross-platform answer to “delete everywhere, including backups,” because backups become unreadable ciphertext when the key is gone.

D.31 DataOps: Infrastructure as Code and CI/CD

Chapter 24’s disciplines—declarative infrastructure, SQL-based transformation, reviewed pipelines—are the most portable skills in this book. The same Terraform workflow and the same dbt project structure run against every cloud.

Capability	AWS		Azure		GCP	
Multi-cloud IaC	Terraform provider	<code>aws</code>	Terraform provider	<code>azurearm</code>	Terraform provider	<code>google</code>
Native IaC	CloudFormation	/	Bicep / ARM templates	tem-	Deployment Manager (legacy)	
Transform-as-code	dbt on Redshift/Athena		dbt on Synapse/Fabric	on	dbt on BigQuery	
CI/CD	CodePipeline / GitHub Actions	/	GitHub Actions / Azure DevOps	/	Cloud Build / GitHub Actions	
State backend	S3 + DynamoDB lock		Azure Storage + lease lock		GCS backend	

Fabric: Git integration + deployment pipelines; **Snowflake:** dbt + Terraform provider;

MongoDB: Atlas Terraform provider; **Databricks:** Asset Bundles (`databricks.yml`) + Terraform provider.

The lesson that transfers everywhere: *the merge is the deploy, and the pipeline is the only writer to production*. Databricks Asset Bundles add a validate-before-deploy step (`databricks bundle validate`) that mirrors `terraform plan` and Bicep what-if—the same reviewed-plan discipline in native tooling.

Tip

Use this appendix as a study aid for the book's lockstep, cross-cloud approach: when you learn a concept on one platform, immediately locate its equivalent here and predict how the other six would express the same idea. That habit turns seven separate vocabularies into one mental model.

Appendix E

Environment Setup Reference

This appendix consolidates the environment requirements scattered through the book into one reference you can revisit before any chapter. Chapter 0 walks through each step with narrative and screenshots-in-words; this appendix is the checklist and command summary. Everything assumes Windows with PowerShell, a sandbox subscription, and the book’s naming and tagging conventions.

Note

Install once, verify always. Run `az account show -output table` at the start of every session and before any destructive command. Most lab accidents come from the wrong subscription, not from the wrong command.

E.1 Azure Subscription and Microsoft Entra ID

The labs assume the management hierarchy from Chapter 0: one Microsoft Entra tenant, one subscription used only for this book, and one resource group per chapter named `rg-de-azbook-chNN-dev` so cleanup is a single `az group delete`.

- **Free account:** \$200 credit for 30 days, selected services free for 12 months, and always-free quotas [32]. The credit is not a safety net—a forgotten dedicated pool or premium cluster can exhaust it in days.
- **Identity separation:** never run daily labs as Global Administrator. Use a normal user account with **Contributor** on the lab resource groups, MFA enabled, and a separate break-glass admin account [92, 86].
- **Data-plane roles:** several labs need data-plane RBAC in addition to Contributor, notably **Storage Blob Data Contributor** on ADLS Gen2 accounts and Cosmos DB data-plane roles for keyless access. Owner on the resource is never a substitute.
- **Automation identity:** create resource-group-scoped service principals only when a lab requires non-interactive runs (Chapters 18, 24); store the secret in Key Vault, never in source control.

E.2 Azure CLI

The primary command-line tool for every chapter [46].

```
1 winget install --exact --id Microsoft.AzureCLI
2 az version
```

```

3 az login
4 az account list --output table
5 az account set --subscription "00000000-0000-0000-0000-000000000000"
6 az account show --output table

```

Listing E.1: Installing and verifying Azure CLI on Windows.

Useful extensions used by later chapters install on demand with `az extension add -name <name>` (for example `databricks`, `datafactory`, `kusto`). Azure Cloud Shell is the no-install fallback for locked-down machines [58], but copy commands and outputs back into your lab notes—its file system and network path are not your machine’s.

E.3 Azure PowerShell

The Az module family is the first-class alternative when an organization standardizes on PowerShell automation [45].

```

1 Install-Module -Name Az -Scope CurrentUser -Repository PSGallery -Force
2 Connect-AzAccount
3 Set-AzContext -Subscription "00000000-0000-0000-0000-000000000000"
4 Get-AzContext | Select-Object Name, Subscription, Tenant

```

Listing E.2: Installing the Az module and selecting a context.

Common Pitfall

Split contexts. `az account set` configures Azure CLI; `Set-AzContext` configures Azure PowerShell. They are independent and can silently point at different subscriptions in the same terminal. Check both before destructive operations.

E.4 Python SDKs and Local Tooling

Chapters with Python code assume Python 3.11+ with a per-book virtual environment. The recurring packages are:

Package	Used for
<code>azure-identity</code>	DefaultAzureCredential sign-in for all SDK clients
<code>azure-storage-file-datalake</code>	ADLS Gen2 file-system operations (Ch. 1, 7)
<code>azure-cosmos</code>	Cosmos DB NoSQL reads/writes (Ch. 3)
<code>azure-eventhub</code>	Event Hubs producers and consumers (Ch. 13)
<code>azure-ai-ml</code>	Azure ML workspaces, pipelines, endpoints (Ch. 17–19)
<code>redis</code>	Cache-aside client for Azure Cache for Redis (Ch. 4)
<code>pyodbc</code> / <code>sqlalchemy</code>	Azure SQL and Synapse connectivity (Ch. 2, 5)
<code>faker</code> , <code>pyarrow</code>	Synthetic data generation and Parquet writing (Ch. 5–6)
<code>databricks-cli</code>	/ Workspace automation (Ch. 9, 12, 15, 24)
<code>databricks-sdk</code>	
<code>dbt-core</code>	+ Transformations as code (Ch. 22, 24)
<code>dbt-fabric</code> / <code>dbt-databricks</code>	

Keep credentials out of code: `DefaultAzureCredential` picks up the `az login` session locally and the managed identity in Azure, so the same code runs in both places.

E.5 Bicep and Terraform

Chapter 24 introduces infrastructure as code; install both tools early so later chapters can show declarative alternatives to portal clicks.

```

1 az bicep install && az bicep version
2 winget install --exact --id Hashicorp.Terraform
3 terraform version

```

Listing E.3: Installing Bicep and Terraform on Windows.

Bicep is Azure-native and needs no state file; Terraform is the cross-cloud constant and stores state in an Azure Storage account backend with lease locking [90, 7]. Chapter 24’s lab wires both into GitHub Actions with OIDC workload identity—no stored secrets.

E.6 Cost Guardrails

Guardrails are part of the environment, not an afterthought. Before Chapter 1, create a monthly subscription budget with alerts at 25%, 50%, 75%, and 90% of a small amount [68], tag every resource group with `project`, `environment`, `chapter`, and `owner`, and rehearse deletion once:

```

1 az consumption budget create --budget-name budget-de-azbook-monthly '
2   --amount 25 --time-grain Monthly --category Cost '
3   --time-period start-date=2026-08-01 end-date=2027-08-01
4
5 az group delete --name rg-de-azbook-chNN-dev --yes --no-wait

```

Listing E.4: The budget-and-cleanup habit every chapter assumes.

Tip

End every lab session with two measurements: the resource list in the chapter’s resource group (`az resource list -resource-group <rg> -output table`) and the current forecast in Cost Management. Pause or delete anything that bills while idle—dedicated pools, Databricks clusters, VMs—before closing the terminal.

E.7 Per-Chapter Tool Requirements

The table below lists what each chapter expects to be installed or provisioned *before* Monday. “CLI” means Azure CLI; “Python” means the virtual environment above.

Ch.	Primary services	Tooling prerequisites
0	Entra ID, Cost Management	CLI, Azure PowerShell, budget, MFA
1	ADLS Gen2	CLI; Python: <code>azure-storage-file-datalake</code> ; AzCopy optional
2	Azure SQL, PostgreSQL Flexible Server	CLI; Python: <code>pyodbc</code> ; <code>sqlcmd</code> optional
3	Cosmos DB for NoSQL	CLI; Python: <code>azure-cosmos</code>
4	Azure Cache for Redis, API Management	CLI; Python: <code>redis</code> , <code>pyodbc</code>
5	Synapse dedicated SQL pool	CLI (<code>az synapse</code>); Python: <code>faker</code> , <code>pyarrow</code>
6	Synapse dedicated SQL pool (existing)	Chapter 5 pool; T-SQL client
7	Synapse serverless SQL pool	Chapter 5 workspace; ADLS data from Ch. 1
8	Microsoft Purview, Data Share	Purview account; scan credentials; Ch. 1 lake
9	Azure Databricks	CLI <code>databricks</code> extension; workspace; ABFS access
10	Azure Data Factory	CLI <code>datafactory</code> extension; Ch. 1 lake; Ch. 5 pool
11	Azure DMS, Self-Hosted IR, Event Grid	Source Azure SQL DB; DMS; contract JSON schemas
12	Databricks, ADF, Logic Apps, Event Grid	Ch. 9 workspace; Teams webhook for notifications
13	Event Hubs, IoT Hub	Python: <code>azure-eventhub</code> (asyncio); Ch. 1 lake for Capture
14	Azure Stream Analytics	Ch. 13 hub; target Azure SQL DB
15	Databricks Structured Streaming	Ch. 9 workspace; Ch. 13 hub Kafka endpoint
16	Power BI, Synapse	Power BI Desktop; Ch. 5 pool; gateway if on-prem
17	Synapse Spark, AML, ONNX	<code>synapseml</code> , <code>onnxmltools</code> ; Ch. 5 pool
18	Azure ML, Databricks	Python: <code>azure-ai-ml</code> ; AML workspace + compute cluster
19	AML managed endpoints, monitoring	Ch. 18 registered model; Application Insights
20	AI Document Intelligence, AI Search, Functions	Azure Functions Core Tools; Python: <code>azure-ai-*</code>
21	Private Endpoints, Key Vault, Bastion	VNet + VM for tests; Key Vault; Event Grid
22	Purview, Azure Monitor, dbt	dbt installed; Log Analytics workspace
23	DDM, Always Encrypted, Defender, Policy	Ch. 2/5 databases; Key Vault for CMK
24	Terraform, Bicep, dbt, GitHub Actions	Terraform + Bicep; GitHub repo; OIDC app registration

Chapters are cumulative by design: the Chapter 5 warehouse, Chapter 9 Databricks workspace, and Chapter 13 event hub are reused downstream, so tearing them down early forces rebuilds later. Pause rather than delete shared infrastructure until the part of the book that uses it is done.

Appendix F

Lab Solution Sketches

This appendix gives a compact solution sketch for every chapter's Thursday hands-on project (Chapters 0–24): the approach, the key commands or code shape, and the failure modes that actually occur. These are reference solutions, not the only solutions—any design that meets the chapter's rubric and acceptance criteria is valid.

Note

Attempt each lab before reading its sketch. The learning value is in the debugging; the sketch exists to unblock you after a genuine attempt and to model what “done” looks like. All sketches assume the Chapter 0 environment: signed-in Azure CLI, the book's naming conventions, and a sandbox subscription with a budget.

F.1 Part I: Core Data Infrastructure and Storage

Chapter 0: Verify the Setup

Approach. Chapter 0 has no Thursday project; its deliverable is a verified environment: a free account, MFA on a non-admin daily identity, Azure CLI and the Az module authenticated to the correct subscription, a monthly budget with alerts, and a tagged resource group created, listed, and deleted from PowerShell.

Key commands. `az login; az account set -subscription <id>; az group create -n rg-de-azbook-ch00-dev -l eastus -tags ...; az consumption budget create ...; az group delete -n <rg> -yes -no-wait.`

Common failure modes. Signing in with the wrong tenant (personal vs. work account with the same email); CLI and Az module pointing at different subscriptions; forgetting that the budget alerts rather than blocks; deleting the wrong resource group because naming was ad hoc.

Chapter 1: Deploy ADLS Gen2 with Lifecycle Management

Approach. Create a StorageV2 account with `-hns true`, create the `raw/validated/curated` file systems, enable soft delete and versioning, then apply a lifecycle management policy that archives blobs prefixed for cold data after 30 days. Upload sample files with `az storage fs file upload -auth-mode login` and verify POSIX-style ACLs on a directory.

Common failure modes. Forgetting `-hns true` (cannot be enabled later without recreating); `AuthorizationPermissionMismatch` because the caller has Contributor but not Storage Blob

Data Contributor; a lifecycle policy that never fires because the `prefixMatch` does not match the uploaded paths; globally-duplicate account names.

Chapter 2: Serverless Azure SQL with Geo-Replica and Failover

Approach. Deploy a logical server and a Serverless database, open a firewall rule for the client IP only, create a secondary server in a paired region, and add an active geo-replica. Drive a small Python workload (inserts and point reads) against the primary, fail over manually, and confirm the workload reconnects through the secondary.

Common failure modes. Firewall rule missing so `pyodbc` times out; geo-replica created with a different admin login so post-failover authentication fails; expecting the failover to be free of application change—without a failover group, connection strings must point at the new server; auto-pause making the first query after idle look like an outage.

Chapter 3: Forum Container with Python Writes and Multi-Region Writes

Approach. Create a Cosmos DB for NoSQL account (Session consistency), one database, and one container partitioned by `/postId` holding posts, comments, and reactions as discriminated item types. Write items with the Python SDK using `DefaultAzureCredential` or a key from Key Vault, then add a second write region and observe conflict behavior with last-writer-wins.

Common failure modes. Choosing a low-cardinality partition key and throttling at 429s despite spare account RU/s; cross-partition queries where a point read was intended; forgetting that multi-region writes make conflicts possible—no conflict logic means silent LWW; committing the account key to source control.

Chapter 4: Redis Cache-Aside for Azure SQL Query Results

Approach. Deploy Azure Cache for Redis (Basic/Standard is enough), then write a Python read path: check Redis for a key derived from the query, on miss read Azure SQL, serialize the result with a TTL, and return it. Measure p95 latency for cache hit versus miss, then delete a key and watch the origin query reappear.

Common failure modes. Caching without a TTL so stale results survive writes; using the access keys in code instead of Key Vault or managed identity; non-SSL port 6379 blocked by policy—use 6380; mistaking a cache stampede after flush for a database outage.

F.2 Part II: Data Warehousing and Analytics

Chapter 5: Ten Million Rows into a Star Schema

Approach. Provision a Synapse workspace plus a DW100c dedicated pool, grant the workspace managed identity `Storage Blob Data Contributor` on the lake, generate 10M synthetic order rows as partitioned Parquet with `faker/pyarrow`, COPY into a round-robin staging table, CTAS into a hash-distributed clustered-columnstore fact with replicated dimensions, then run the SCD2 MERGE and verify a two-version customer. Pause the pool.

Common failure modes. COPY authorization errors—managed identity missing the data-plane role; `MAXERRORS` rejects clustering on one folder (schema drift in one file); SCD2 MERGE producing duplicate current rows because the close-and-insert ran twice; leaving the pool running and discovering it on the bill.

Chapter 6: Benchmarking Distribution Types

Approach. Create three copies of a 50M-row orders table—hash on the join key, hash on an unrelated key, and round-robin—then run the same join-heavy query against each and record elapsed time and `sys.dm_pdw_request_steps` data-movement operations. The correct hash key wins because joins execute locally without shuffle moves.

Common failure modes. Benchmarking once and trusting it—run warm and cold; comparing tables with different rowgroup quality (small loads leave open rowgroups); drawing conclusions at DW100c that invert at DW1000c; forgetting the result-set cache and measuring a cached answer.

Chapter 7: CSV In, Parquet Out with CETAS

Approach. Against the Chapter 5 workspace’s serverless pool: define an external data source and file format, profile the raw CSVs with `OPENROWSET`, write typed, cleaned rows back to the lake as partitioned Parquet with CETAS, then build external tables and views over the result as the logical-warehouse serving layer. Compare bytes scanned for CSV versus Parquet.

Common failure modes. Schema inference misreading types—declare columns explicitly; unfiltered scans burning the per-TB budget (use `filepath()` partition filters); CETAS failing because the target folder already exists; managed identity lacking lake write access.

Chapter 8: Scan, Classify, Trace (Purview)

Approach. Register the lab ADLS Gen2 account and Synapse workspace in Purview, grant the Purview managed identity read access, run a scan, and confirm built-in classifiers tag `PII_Email`-style columns. Trigger an ADF copy and trace lineage from lake file through pipeline to dataset. Finish by creating a Data Share snapshot for a synthetic partner.

Common failure modes. Scan finds nothing because the managed identity lacks storage data-plane read; classifications missed because the data does not resemble the built-in patterns (custom classifiers are Chapter 22); lineage gaps when assets were registered after the pipeline ran; expecting scans to be instant on first registration.

F.3 Part III: Batch Processing and ETL Orchestration

Chapter 9: Distributed Aggregation to Delta (Databricks)

Approach. Deploy a Databricks workspace, create an all-purpose cluster (small, autoterminate at 20 minutes), read the governed lake over ABFS using the cluster’s credential passthrough or a service principal secret from a secret scope, run a grouped aggregation in PySpark, and write the result as a Delta table. Repeat as a job on a job cluster and compare cost and startup behavior.

Common failure modes. Writing to DBFS root instead of the governed ADLS account; `AnalysisException` on path because the storage credential is wrong, not the path; all-purpose cluster left running—autotermination unset; confusion between table created in the workspace metastore versus data files actually in the lake.

Chapter 10: Join, Clean, and Load with Mapping Data Flows

Approach. In ADF, create linked services for the lake and the Synapse pool, datasets for orders and customers, then a Mapping Data Flow: join on `customer_id`, filter nulls, normalize

timestamps with derived columns, and sink into the pool. Orchestrate with a pipeline that runs the data flow activity with an `onFailure` path to a notification.

Common failure modes. Debug session never started so the data preview is empty; join skew making the flow crawl—broadcast the small side; no `onFailure` path so the pipeline fails silently overnight; staging settings pointing at storage the IR cannot reach.

Chapter 11: DMS CDC with a Contract Gate

Approach. Enable CDC on a source Azure SQL table, configure an Azure DMS task streaming changes to ADLS Gen2 as JSON/Parquet deltas, then land a validation step (Databricks notebook or ADF data flow) that checks each batch against a versioned JSON contract and routes violations to a `quarantine/` prefix with the reason attached.

Common failure modes. CDC not enabled at the database level so the task captures nothing; clock skew between source and landing making ordering look wrong; contract checks treating optional-field additions as failures (they are backward-compatible); quarantine with no alerting, so violations accumulate invisibly.

Chapter 12: Event-Driven Databricks Orchestration with Failure Notification

Approach. Wire the canonical pipeline: an Event Grid blob-created event on the raw zone fires an ADF storage-event trigger; the pipeline runs a Databricks notebook activity with the file path as a parameter; an `onFailure` path calls a Logic App that posts a rich card with the run link to Teams. Test success, a data failure (bad schema), and a platform failure (stopped cluster).

Common failure modes. Trigger filtering too broad (every blob in the account fires it); Event Grid subscription created but the ADF trigger unpublished; notebook activity failing on authentication because the linked service uses an expiring PAT instead of a managed identity; Logic App card with no run URL, forcing portal archaeology during an incident.

F.4 Part IV: Streaming and Real-Time Analytics

Chapter 13: IoT Telemetry Producer with Capture

Approach. Create an Event Hubs namespace and hub (4 partitions), enable Capture to ADLS Gen2 on a 5-minute/300 MB window, then run a Python asyncio producer sending temperature events with a unique `eventId`, partition key, and `event_time`. Verify captured Avro files appear in the lake and that consumer group offsets advance.

Common failure modes. SAS policy with Send-only rights used by a consumer; batch send failing on the 1 MB message-size cap; Capture writing nothing because the window had not elapsed or the identity lacked lake write; more consumer instances than partitions, leaving readers idle.

Chapter 14: Five-Minute Click Aggregates to Azure SQL

Approach. Build an ASA job: input from the Chapter 13 hub, query counting clicks per page in five-minute tumbling windows with a documented late-arrival policy, output to an Azure SQL table with a unique key on (window end, page) so restarts upsert idempotently. Start the job, produce events, and chart the aggregates.

Common failure modes. Interpreting zero output rows as zero traffic when the input has stalled—watch watermark and input metrics; output primary-key violations because the sink is

not idempotent; SU autoscale lagging a burst so late data piles up; timestamp by arrival time instead of the event's own `event_time`.

Chapter 15: Event Hubs to Delta with Windowed Aggregation

Approach. In Databricks, read the hub through its Kafka endpoint (`org.apache.kafka.common.security.pl` with the connection string), parse JSON with an explicit schema, aggregate with `window(col("event_time"), "5 minutes")` plus `withWatermark("event_time", "10 minutes")`, and write to a Delta table with a checkpoint location in the lake. Kill and restart the stream to prove exactly-once replay.

Common failure modes. Missing or shared checkpoint directory corrupting state across query versions; watermark omitted so state grows until the executor dies; reading with the Event Hubs SDK connector config but Kafka-protocol options (or vice versa); micro-batch latency blamed on the engine when the trigger interval was the default.

Chapter 16: DirectQuery Dashboard with RLS

Approach. Connect Power BI Desktop to the Chapter 5 pool in DirectQuery mode, build a sales monitoring page (KPI cards, trend, by-region matrix) with measures rather than calculated columns, mark the date table, then define a dynamic RLS role filtering a region mapping table by `USERPRINCIPALNAME()`. Publish and validate with “View as role” and a test account.

Common failure modes. Import mode chosen by habit, breaking the freshness requirement; calculated columns where measures were needed (static at refresh, ignore slicers); time-intelligence functions returning blanks because the date table is not marked; RLS tested only with the author's admin account, which bypasses nothing in Desktop but everything in service if workspace roles are wrong.

F.5 Part V: Machine Learning and MLOps

Chapter 17: Churn Model from Spark to SQL

Approach. In Synapse Spark, assemble churn features, train logistic regression, export the model to ONNX with `onnxmltools`, load it as `varbinary` into a models table in the dedicated pool, and score customers with T-SQL `PREDICT`. Verify scores match the Spark-side predictions on a sample.

Common failure modes. Feature schema drift between training and `PREDICT` (name, type, order must match exactly); unsupported operator in the ONNX conversion—stick to the documented model types; model stored as hex string instead of binary; scoring wrong rows because the feature view joined on an unversioned snapshot.

Chapter 18: Preprocess, Train, Evaluate, Register (AML Pipeline)

Approach. From the Python SDK, define an AML pipeline with preprocess, train, and evaluate components on a compute cluster; wire a condition so registration happens only when the metric gate (for example $AUC \geq 0.80$) passes; submit from a notebook and inspect the run in AML studio. Data access goes through a datastore bound to the workspace managed identity.

Common failure modes. Training on a compute instance that cannot autoscale or scale to zero; datastore using account keys instead of identity, defeating the audit trail; environment image rebuilt on every run because dependencies were not pinned; metric gate comparing against a hardcoded value with no recorded rationale.

Chapter 19: Deploy, Watch, Drift, Retrain

Approach. Deploy the Chapter 18 registered model to an AML managed online endpoint, score with a REST payload, enable data collection, and configure model monitoring with the training data as baseline. Inject synthetic drift (shift a feature distribution), watch the monitor flag it, and trigger the retraining pipeline from the alert.

Common failure modes. Endpoint deployed with no data collection, so drift is undetectable; baseline set from stale training data; retraining triggered on a calendar rather than evidence; promotion without a rollback plan—always keep the previous version deployable.

Chapter 20: Invoice Extraction Function

Approach. Create an event-driven Azure Function triggered by blob uploads to an `invoices/` prefix; the Function calls AI Document Intelligence’s prebuilt invoice model, writes typed key-value JSON to the lake, flags fields below a confidence threshold for review, and indexes the results in AI Search for the retrieval demo.

Common failure modes. Polling instead of event trigger, adding minutes of latency; treating OCR confidence as uniform—low-confidence fields need a human review path; indexing raw OCR text containing PII without classification and redaction; Function scaling bill surprising you because the trigger filter matched every blob in the account.

F.6 Part VI: Security, Governance, and Exam Prep

Chapter 21: A Private-Only Storage Account

Approach. Deploy an ADLS Gen2 account with `-allow-blob-public-access false` and public network access disabled, attach a private endpoint in a lab VNet with private DNS zone integration, and prove the access pattern: from a VM inside the VNet, `az storage fs file list` succeeds; from your laptop, it fails. Access uses managed identity or RBAC, never keys.

Common failure modes. Private endpoint created but DNS resolves to the public IP (missing `privatelink.dfs.core.windows.net` zone link); disabling public access before the private path works, locking everyone out; service endpoints mistaken for private endpoints—the public IP is still reachable; Key Vault firewall forgetting the Function/VM identity.

Chapter 22: Custom Classification and a Quality Gate

Approach. Extend Chapter 8’s Purview estate with a custom classification rule for a proprietary employee-ID format (regex plus column-name hints), rescan, and confirm automatic tagging. Then add a dbt (or Great Expectations) quality gate before the load step of a pipeline and prove a bad batch is blocked and quarantined while a good batch passes.

Common failure modes. Custom rule too broad, tagging everything (test against known negatives); quality gate placed after the load, documenting failure instead of preventing it; gate failures with no notification path; scan schedule never re-run so new columns stay unclassified.

Chapter 23: Masking and a Crypto-Shredding Erasure Drill

Approach. Drill 1: apply Dynamic Data Masking to email and phone columns, create a read-only test user, and verify masked output; then grant UNMASK to a privileged role and verify the difference. Drill 2: pick one synthetic data subject, enumerate every copy of their data (tables, lake files, Delta history, backups), and execute erasure by destroying the subject-specific encryption key—crypto-shredding—then prove the copies are unreadable.

Common failure modes. Believing DDM protects data from admins (it is presentation-layer only); DELETE declared as erasure while soft delete, versioning, backups, and Delta time travel retain copies; per-subject keys not actually separated, so one key destroys many subjects; no audit evidence of the erasure for the regulator.

Chapter 24: Terraform + dbt + GitHub Actions

Approach. One repository: Terraform (azurerm provider, storage-account backend with lease locking) provisioning an ADLS Gen2 account and an ADF instance; a dbt project with three models (staging, intermediate, mart) and five tests; a GitHub Actions workflow using OIDC workload identity that runs `terraform fmt -check`, `terraform validate`, `terraform plan` on pull requests and `dbt build` against a dev target.

Common failure modes. Storing a service principal secret in repository secrets instead of OIDC federation; state backend without locking so two runs corrupt state; pipeline deploying to prod from any branch—protect the environment with reviewers; dbt tests running against the wrong target because profiles were parameterized per environment incorrectly.

Tip

When a lab fails, debug in this order: identity (am I in the right subscription and tenant?), authorization (management-plane role *and* data-plane role?), network (firewall, private endpoint, DNS), then the service itself. That sequence resolves the large majority of lab failures in this book without touching the code.

Certification Exam Mapping: DP-203 and DP-700

This appendix maps the Microsoft data-engineering certification objectives onto the chapters of this book, then turns the map into a study strategy and a self-assessment checklist. Chapter 23 explains the context: DP-203 (Data Engineering on Microsoft Azure) was retired in 2025 and **DP-700** (Implementing Data Engineering Solutions Using Microsoft Fabric) is its successor credential [39, 53]. The preparation method is identical for both, and this book’s twenty-four weeks were sequenced to cover both exams’ domains.

Note

Exam skills-measured documents change. Before booking, download the current version for your target exam and reconcile it against the tables below—the mapping is durable, the exact percentages are not.

G.1 How the Exams Differ

DP-203 tested the classic Azure data stack directly: ADLS Gen2, Synapse, Databricks, ADF, Stream Analytics, and the security/monitoring wraparound. DP-700 tests the same engineering competencies—storage design, ingestion, transformation, orchestration, real-time, security, optimization—expressed through Microsoft Fabric surfaces: lakehouses, warehouses, eventstreams, pipelines, notebooks, and Power BI with DirectLake. The cross-cloud boxes throughout this book (and the consolidated tables in the Cross-Cloud Service Equivalence appendix) are the bridge: each maps an Azure service to its Fabric counterpart, which is precisely the translation DP-700 demands.

G.2 DP-203 Domain Mapping

The historical DP-203 domains and their book coverage:

DP-203 domain	Weight	Book chapters
Design and implement data storage	≈40–45%	Ch. 1 (ADLS Gen2, tiers, lifecycle), Ch. 2 (Azure SQL/PostgreSQL), Ch. 3 (Cosmos DB), Ch. 5–7 (Synapse dedicated and serverless, lakehouse), Ch. 12 (Delta Lake medallion)
Design and develop data processing	≈25–30%	Ch. 9 (Databricks/Spark), Ch. 10 (ADF, Mapping Data Flows), Ch. 11 (CDC, DMS), Ch. 13–15 (Event Hubs, Stream Analytics, Structured Streaming)
Design and implement data security	≈10–15%	Ch. 21 (zero trust, private endpoints, Key Vault), Ch. 23 (masking, Always Encrypted, Defender, Policy), Ch. 8 (Purview)
Monitor and optimize data storage and processing	≈10–15%	Ch. 6 (distribution, workload management), Ch. 8 (governance telemetry), Ch. 22 (observability, FinOps)

If you maintain legacy DP-203-era knowledge for a team or an interview loop, the chapters above remain the correct study path; only the credential is retired, not the platform.

G.3 DP-700 Domain Mapping

DP-700 organizes the same competencies around the Fabric platform:

DP-700 domain	Book chapters	Fabric surface to learn
Implement and manage an analytics solution	Ch. 0–1, 21–24	Workspace and capacity management, OneLake security, Git integration, deployment pipelines
Ingest and transform data	Ch. 9–15	Pipelines and dataflows gen2, notebooks and Spark jobs, eventstreams, shortcuts, medallion on OneLake
Monitor and optimize an analytics solution	Ch. 5–8, 16, 22	Capacity Metrics app, warehouse and lakehouse optimization, semantic model and DirectLake performance

Fabric-specific surfaces that this Azure book teaches conceptually but that you should rehearse hands-on in a Fabric trial capacity: OneLake shortcuts (the cross-cloud boxes call them out under AWS/Azure/GCP storage), DirectLake storage mode (Chapter 16’s cross-cloud box), and eventstreams with Activator (Chapter 14’s cross-cloud box).

G.4 Objective-to-Chapter Quick Reference

Use this table in the final two weeks: find the objective, rebuild the lab from memory, and check the sketch in the Lab Solution Sketches appendix only after your attempt.

Representative objective	Chapters	Evidence you know it
Design data partitioning, tiers, redundancy, and lifecycle	1, 12	You chose tiers and formats on evidence, not habit
Design and implement relational and NoSQL stores	2, 3	You can defend partition keys and consistency levels
Design star schemas, distributions, and SCD patterns	5, 6	You benchmarked distribution choices with numbers
Implement serverless lake querying and CETAS	7	You cut bytes scanned with Parquet and pruning
Build batch pipelines and Mapping Data Flows	9, 10	You built idempotent, parameterized pipelines
Implement CDC and hybrid data movement	11	You gated a CDC stream with a schema contract
Implement streaming ingestion and processing	13–15	You reason about watermarks and exactly-once
Serve data with BI, RLS, and semantic models	16	You built a DirectQuery dashboard with dynamic RLS
Operationalize ML: features, pipelines, serving, drift	17–20	You shipped and monitored a model end-to-end
Design security: identities, private networking, secrets	21, 23	You built the private-only perimeter and the erasure drill
Implement governance: catalog, lineage, quality gates	8, 22	You blocked a bad batch with a gate, not a memo
Automate with IaC, dbt, and CI/CD	24	The pipeline is the only writer to prod

G.5 Study Strategy

Follow the Chapter 23 method:

1. **Weight by weakness, not comfort.** Score yourself per domain; allocate revision time by *domain weight* $\times$ *your weakness*. The failure mode is re-reading what you already know.
2. **Rebuild, do not re-read.** For every weak domain, rebuild the corresponding Thursday lab from memory in a fresh resource group. Fluency under time pressure comes from rebuilding, not highlighting.
3. **Mock under time.** Take a full-length practice exam in one sitting. Classify every miss as a *knowledge gap* (study it), a *misread* (slow down on “all EXCEPT” questions), or a *judgment gap*—the “best” answer among working options is usually the one with the least operational toil and the least privilege.
4. **Re-mock the weak domains after one week.** Schedule the real exam when two consecutive full mocks clear 80% with no domain below 70%. The week after Week 24 is the right window: material is fresh and the capstone is your scenario practice.
5. **Translate to Fabric for DP-700.** For each Azure service in this book, state its Fabric surface from memory using the cross-cloud boxes; that translation skill is the core of the successor exam.

G.6 Self-Assessment Checklist

Check each item only when you can do it unaided in the lab environment, not merely explain it.

Storage and Databases

- ☐ Deploy ADLS Gen2 with HNS, lifecycle policy, soft delete, and immutability (Ch. 1)
- ☐ Choose redundancy (LRS/ZRS/GRS/GZRS) and an access tier from requirements (Ch. 1)
- ☐ Deploy Azure SQL Serverless with a geo-replica and execute failover (Ch. 2)
- ☐ Design a Cosmos DB container: partition key, RU budget, consistency level (Ch. 3)
- ☐ Implement cache-aside with TTL and invalidation reasoning (Ch. 4)

Warehousing and Analytics

- ☐ Declare the grain, model a star schema, implement SCD2 with MERGE (Ch. 5)
- ☐ Benchmark hash/round-robin/replicated distributions and read the DMS plan (Ch. 6)
- ☐ Query the lake with OPENROWSET and materialize with CETAS (Ch. 7)
- ☐ Explain columnstore, workload groups, and result-set caching (Ch. 6)

Ingestion, Transformation, and Orchestration

- ☐ Build a Mapping Data Flow and parameterize it for dev/test/prod (Ch. 10)
- ☐ Configure CDC from a relational source to the lake with a contract gate (Ch. 11)
- ☐ Orchestrate event-driven Databricks notebooks with failure notification (Ch. 12)
- ☐ Explain partitions, shuffles, and Catalyst well enough to debug a slow job (Ch. 9)

Streaming

- ☐ Produce to Event Hubs with partition keys, event IDs, and Capture (Ch. 13)
- ☐ Write ASA tumbling/hopping windows with a documented late-arrival policy (Ch. 14)
- ☐ Implement Structured Streaming with watermarks, checkpoints, and idempotent Delta sinks (Ch. 15)
- ☐ Contrast Service Bus and Event Hubs settlement semantics (Ch. 13)

Serving, ML, and GenAI

- ☐ Build a DirectQuery report with dynamic RLS and a marked date table (Ch. 16)
- ☐ Train in Spark, export ONNX, and score with T-SQL PREDICT (Ch. 17)
- ☐ Build an AML pipeline with a metric gate and model registration (Ch. 18)
- ☐ Deploy, monitor drift, and trigger evidence-based retraining (Ch. 19)
- ☐ Assemble a document-extraction and RAG pipeline with PII controls (Ch. 20)

Security, Governance, and DataOps

- ☐ Build a private-only data service: private endpoint, DNS, no public access (Ch. 21)
- ☐ Explain managed identity versus service principal secrets, with rotation (Ch. 21)
- ☐ Run Purview scans with custom classifications and trace lineage (Ch. 8, 22)

- ☐ Enforce a pre-load quality gate and quarantine path (Ch. 22)
- ☐ Implement masking, Always Encrypted, and a crypto-shredding erasure drill (Ch. 23)
- ☐ Ship Terraform + dbt + CI/CD with OIDC and reviewed plans (Ch. 24)

Tip

The interview question bank appendix doubles as exam preparation: every question marked *intermediate* or *advanced* is phrased the way scenario-based exam items are—a constraint set and a judgment call, not a definition.

Interview Question Bank

This appendix consolidates the Interview Q&A sections from all twenty-five chapters into one bank, reorganized by domain and tagged by difficulty: **[F]** foundational (definitions and core trade-offs), **[I]** intermediate (design judgment with constraints), and **[A]** advanced (failure modes, migration risk, and cost reasoning). Answers are abbreviated to interview length; the chapter cross-reference points to the full treatment.

Tip

Practice out loud with a timer: sixty seconds for foundational, two minutes for intermediate, and a whiteboard sketch for advanced. Interviewers score structure—state the constraint, name the options, choose, and say what would change your mind.

H.1 Storage and Databases (Chapters 1–4)

1. **[F, Ch. 1] Q: Why is ADLS Gen2 preferred for analytics lakes over a flat Blob Storage namespace?**
A: It adds a hierarchical namespace, atomic directory operations, and path-level ACLs while retaining Blob Storage scale and durability.
2. **[F, Ch. 1] Q: What does soft delete protect that versioning does not?**
A: Soft delete protects deleted blobs or containers during a retention window; versioning primarily protects prior object states after overwrite.
3. **[I, Ch. 1] Q: When would you choose ZRS instead of GZRS?**
A: When zone failure is in scope but cross-region replication cost or data-residency constraints make geo-replication unsuitable.
4. **[I, Ch. 1] Q: Why might the Archive tier be wrong for data that is rarely read?**
A: Rare reads may still be urgent. Archive requires rehydration, so RTO and operational approval can make Cool or Cold a better fit.
5. **[F, Ch. 2] Q: When would you choose vCore over DTU for Azure SQL Database?**
A: When you need explicit control over cores, hardware family, storage, reserved capacity, hybrid licensing, or enterprise capacity planning.
6. **[I, Ch. 2] Q: Why is Serverless not always cheaper?**
A: Always-active workloads may pay continuously, auto-pause may never trigger, and cold-start latency can force a minimum compute setting.

7. **[I, Ch. 2] Q: Why use an auto-failover group instead of only active geo-replication?**
A: A failover group provides listener endpoints and group-level failover for related databases, simplifying application connection management.
8. **[I, Ch. 2] Q: What does PostgreSQL Flexible Server HA protect against?**
A: Server or zone failures—not bad application writes; backups and PITR are still required.
9. **[A, Ch. 2] Q: What makes Hyperscale different from Business Critical?**
A: Hyperscale separates compute from distributed page and log storage for very large databases and fast restore; Business Critical focuses on local SSD replicas and low-latency IO.
10. **[A, Ch. 2] Q: What is the largest risk in a zero-downtime SQL Server migration?**
A: Unrehearsed cutover dependencies: compatibility gaps, logins, jobs, connection strings, downstream ETL, and rollback rules can break despite a successful data copy.
11. **[F, Ch. 3] Q: What is a Request Unit?**
A: Cosmos DB's normalized measure of CPU, memory, and IO used by reads, writes, queries, and maintenance operations.
12. **[F, Ch. 3] Q: What makes a good partition key?**
A: High cardinality, even distribution, alignment with common point reads and queries, and support for transaction boundaries within a logical partition.
13. **[I, Ch. 3] Q: Why is Session consistency common for global apps?**
A: It provides read-your-writes within a user session while allowing lower latency and more regional independence than Strong consistency.
14. **[I, Ch. 3] Q: When would you avoid multi-region writes?**
A: When the workload cannot tolerate conflict-resolution complexity, or when single-write failover meets latency and availability goals.
15. **[I, Ch. 3] Q: Why use the Change Feed for data engineering?**
A: It turns operational item changes into scalable downstream projections for functions, streams, lakes, caches, search, and materialized views.
16. **[A, Ch. 3] Q: What is the biggest risk in a high-scale leaderboard?**
A: A hot partition or global aggregation path can throttle the system even when total account RU/s appears sufficient.
17. **[F, Ch. 4] Q: What is the cache-aside pattern?**
A: The application checks the cache first, reads the origin on a miss, stores the result with a TTL, and returns the value.
18. **[F, Ch. 4] Q: Why is Redis not usually the system of record?**
A: It is optimized for in-memory speed; durable databases should own authoritative state unless the Redis tier and design explicitly meet durability requirements.
19. **[I, Ch. 4] Q: When should you prefer Cosmos DB Integrated Cache over Redis?**
A: When repeated Cosmos DB reads can use the dedicated gateway and accept a bounded staleness window without application-managed cache keys.
20. **[I, Ch. 4] Q: What causes a cache stampede?**
A: Many callers miss the same key at once and all query the origin, typically after a popular key expires or the cache is flushed.

21. **[I, Ch. 4] Q: What belongs in API Management for ingestion?**
A: Authentication, authorization, quotas, rate limits, validation, transformations, routing, timeout policy, and diagnostics at the API boundary.
22. **[A, Ch. 4] Q: What metric is more useful than cache hit ratio alone?**
A: Hit ratio combined with p95/p99 latency, origin load, stale-read incidents, errors, evictions, and business correctness metrics.

H.2 Warehousing and Analytics (Chapters 5–7)

1. **[F, Ch. 5] Q: Why must you declare the grain before designing a fact table?**
A: The grain defines what one row means; without it, joins fan out and aggregates double-count silently.
2. **[F, Ch. 5] Q: When is SCD Type 2 required instead of Type 1?**
A: When the business must reconstruct what was known at a past point in time, such as the customer's segment when an order was placed.
3. **[I, Ch. 5] Q: Which fact type models a process with milestones such as order, ship, deliver?**
A: An accumulating snapshot—one row per process instance, updated as each milestone completes.
4. **[I, Ch. 5] Q: When does a serverless SQL pool beat a dedicated pool on cost?**
A: When queries are infrequent or bursty, data is columnar and partition-pruned, and the bytes scanned per month cost less than keeping a provisioned pool running.
5. **[F, Ch. 6] Q: Why is clustered columnstore the default for large fact tables?**
A: Columnar compression cuts IO roughly 10×, and analytics queries touch few columns, so a CCI reads only what is referenced.
6. **[I, Ch. 6] Q: What problem does hash distribution solve for large joins?**
A: It co-locates rows with equal keys on the same distribution, so joins execute locally without Data Movement Service shuffles.
7. **[I, Ch. 6] Q: What do workload classifiers map incoming queries to?**
A: Workload groups—based on login, role, label, or time—so each class of work gets reserved resources, caps, and importance.
8. **[I, Ch. 6] Q: When does a materialized view beat adding an index?**
A: When many queries repeat the same aggregation; a materialized view precomputes it, while an index only speeds row lookup and barely helps scans.
9. **[F, Ch. 7] Q: How is serverless SQL billed, and how do you cut that cost?**
A: Per TB of data processed. Use Parquet/Delta, partition pruning via `filepath()`, explicit schemas, and appropriately sized files.
10. **[F, Ch. 7] Q: What does CETAS do?**
A: `CREATE EXTERNAL TABLE AS SELECT` materializes a serverless query result back to the lake as Parquet or Delta—serverless ETL.
11. **[I, Ch. 7] Q: What is the logical data warehouse pattern?**
A: A serving layer of external tables and views over lake files, so BI queries a warehouse-shaped interface without copying data out of the lake.
12. **[I, Ch. 7] Q: What does Delta Lake add to plain files on ADLS Gen2?**
A: A transaction log that provides ACID commits, time travel, schema enforcement, and

MERGE upserts over object storage.

H.3 Governance and Catalogs (Chapters 8 and 22)

1. **[F, Ch. 8] Q: What does a Purview scan automatically discover?**
A: Assets, schemas, and—via built-in and custom classifiers—sensitive data categories, organized into a searchable data map.
2. **[I, Ch. 8] Q: How does lineage help impact analysis before a schema change?**
A: The downstream query shows which views, models, and reports read a column, so you know exactly what breaks before you change it.
3. **[I, Ch. 8] Q: Why use Azure Data Share instead of copying files to a partner?**
A: Sharing in place keeps data governed: invitations, terms, snapshot schedules, and revocation replace an ungoverned copy in someone else's storage.
4. **[A, Ch. 8] Q: What is a data clean room, and what must it never expose?**
A: An architecture where parties match overlapping data via tokenized keys and exchange only aggregates; it must never expose row-level raw PII across the boundary.
5. **[I, Ch. 22] Q: How do sensitivity labels flow from Purview to Synapse and Power BI?**
A: Labels attach to classified columns and propagate with the data, so label-aware policies apply at the semantic-model and report layer.
6. **[I, Ch. 22] Q: Why run dbt tests or GE checkpoints before the load step?**
A: Bad data compounds through every downstream product; a gate before load quarantines failures while a gate after load only documents them.
7. **[I, Ch. 22] Q: What does Purview lineage show when a report suddenly breaks?**
A: The upstream chain—which dataset, pipeline, and job fed the report—turning impact analysis from archaeology into a query.
8. **[A, Ch. 22] Q: In a data mesh, what stays centralized?**
A: The rails: catalog, classification, label taxonomy, delivery templates, gate framework, access policy, and cost guardrails. Domains own products.

H.4 Batch Processing, ETL, and Orchestration (Chapters 9–12)

1. **[F, Ch. 9] Q: Why use job clusters instead of all-purpose clusters in production?**
A: They terminate on completion and bill 2–3× less per DBU; all-purpose clusters are priced for interactivity and keep billing while idle.
2. **[I, Ch. 9] Q: What does a Spark partition control during a shuffle?**
A: Parallelism and memory per task: each shuffle partition becomes one task; too few causes spill, too many floods the driver.
3. **[I, Ch. 9] Q: How do Delta Live Tables differ from plain Delta tables?**
A: DLT is a declarative pipeline framework—you define tables and expectations; the engine orchestrates computation, retries, and incremental refresh.
4. **[I, Ch. 9] Q: What is DBFS, and why should production data not live there?**
A: The workspace's root storage abstraction; production data belongs in governed ADLS accounts over ABFS, not workspace-scoped storage.

5. **[F, Ch. 10] Q: What is the difference between a linked service and a dataset?**
A: The linked service is the connection and credential to a system; the dataset is a parameterized pointer to specific data within it.
6. **[I, Ch. 10] Q: When do Mapping Data Flows beat hand-written Spark?**
A: For standard join/clean/conform transforms built by SQL-strong analysts; visual authoring and managed Spark beat code when the logic is boring.
7. **[I, Ch. 10] Q: What happens in ADF when no onFailure path is defined?**
A: The activity fails, the pipeline run fails, and nothing downstream runs—no notification unless external monitoring catches it.
8. **[I, Ch. 10] Q: How do you parameterize a pipeline for dev/test/prod promotion?**
A: Parameterize endpoints, paths, and environment names; keep one artifact set and swap parameter files per environment.
9. **[I, Ch. 11] Q: Why would you need a Self-Hosted Integration Runtime?**
A: When sources sit in private networks; the SHIR dials outbound to ADF, so no inbound firewall exposure is needed.
10. **[F, Ch. 11] Q: How does CDC differ from a nightly full extract?**
A: CDC streams deltas from the transaction log—minutes of latency, low source load, deletes representable—versus hours of latency and full table scans.
11. **[I, Ch. 11] Q: Is adding an optional field a breaking contract change?**
A: No—it is backward-compatible. Renaming a field or changing its type breaks existing consumers and requires a major version.
12. **[I, Ch. 11] Q: Who owns a data contract?**
A: The producer owns and versions it; consumers pin to versions, and breaking changes ship with a migration window.
13. **[I, Ch. 12] Q: When would you use an Until activity instead of a ForEach?**
A: Until waits on a predicate with a timeout (poll for a file or task state); ForEach iterates a known collection. Waiting is Until's job.
14. **[I, Ch. 12] Q: What problem does Managed Airflow in ADF solve?**
A: It runs existing Airflow DAG estates on managed infrastructure with Azure identity integration—no Airflow operations burden, no rewrite.
15. **[I, Ch. 12] Q: Why route failure notifications through a Logic App instead of plain email?**
A: Structured payloads become rich Teams cards with run links and runbook buttons, and one router workflow centralizes the notification logic.
16. **[A, Ch. 12] Q: What makes an orchestration layer self-healing?**
A: Arrival-driven triggers, isolated units of work, bounded retries for known faults, quarantine for data faults, and audit-driven reconciliation—with novel failures still paging humans.

H.5 Streaming and Real-Time (Chapters 13–15)

1. **[I, Ch. 13] Q: When do you choose Service Bus over Event Hubs?**
A: When messages are commands needing per-message settlement—complete, abandon, dead-letter—rather than a replayable event log.

2. **[I, Ch. 13] Q: Why does partition count cap consumer parallelism?**
A: One partition is read by at most one instance per consumer group; more instances than partitions leaves instances idle.
3. **[F, Ch. 13] Q: What does Event Hubs Capture automate?**
A: Batched delivery of stream traffic to ADLS Gen2 as Avro, on time/size windows, with no consumer code.
4. **[I, Ch. 13] Q: Why attach a unique eventId to every message?**
A: At-least-once delivery means duplicates; the eventId is the dedupe key that makes sink upserts idempotent.
5. **[F, Ch. 14] Q: Contrast tumbling and hopping windows with an example.**
A: Both are fixed-size; tumbling buckets are disjoint (five-minute counts), hopping overlap (ten-minute window every two minutes for a moving average).
6. **[I, Ch. 14] Q: What does the late-arrival tolerance control?**
A: How much older than the watermark an event may be; beyond it the event is adjusted to the tolerance edge or dropped, per policy.
7. **[A, Ch. 14] Q: Why does a silent input stall look like zero traffic?**
A: No events advance no watermark and emit no rows—identical to a genuinely empty period; only input and watermark metrics distinguish them.
8. **[A, Ch. 14] Q: How do you fix daily aggregates skewed by late mobile uploads?**
A: Raise and document the late-arrival tolerance for the reporting contract, and compute official dailies from the lake where late data reprocesses cleanly.
9. **[I, Ch. 15] Q: Micro-batch versus continuous processing—what is the trade-off?**
A: Micro-batch offers full operators and exactly-once at sub-second latency; continuous offers millisecond latency with a restricted operator set and at-least-once semantics.
10. **[I, Ch. 15] Q: Why does unbounded state kill a streaming job, and how do watermarks evict it?**
A: State per key accumulates until memory is exhausted; the watermark tells the engine it may evict state older than the tolerance behind the event-time frontier.
11. **[A, Ch. 15] Q: What must both sides of a stream-stream join declare?**
A: A watermark each, plus a time-range join condition, so join state retention is bounded on both sides.
12. **[A, Ch. 15] Q: Contrast checkpointing plus an idempotent Delta sink with at-least-once plus downstream dedup.**
A: The former pays the dedup cost once in the sink and gives consumers clean data; the latter pushes dedup to every consumer forever.

H.6 Business Intelligence and Serving (Chapter 16)

1. **[I, Ch. 16] Q: When does DirectQuery beat Import mode?**
A: When data must be current to the minute, when the dataset exceeds cache capacity, or when the warehouse is the governed source of truth and duplication adds risk.
2. **[I, Ch. 16] Q: How does RLS restrict what a regional sales manager sees?**
A: A role applies a DAX filter—ideally dynamic via `USERPRINCIPALNAME()` against a mapping table—that the engine applies to every query the user runs.

3. **[F, Ch. 16] Q: What is the difference between a measure and a calculated column?**

A: A column is computed at refresh and stored per row; a measure is computed at query time in the current filter context and responds to slicers.

4. **[F, Ch. 16] Q: Why mark a table as a date table?**

A: Time-intelligence functions like `SAMEPERIODLASTYEAR` require a contiguous, marked date table to compute correctly.

H.7 Machine Learning and Generative AI (Chapters 17–20)

1. **[F, Ch. 17] Q: What does T-SQL `PREDICT` eliminate from the scoring path?**

A: Data movement—no export jobs, no endpoints, no network round-trips; rows are scored where they already live.

2. **[I, Ch. 17] Q: When is in-database scoring preferable to a real-time endpoint?**

A: For batch or interactive scoring over warehouse-resident features, SQL-native operations teams, and data that must not leave the database perimeter.

3. **[F, Ch. 17] Q: In what format must a model be stored for Synapse `PREDICT`?**

A: ONNX, stored as varbinary in a table and referenced by the `PREDICT` function at scoring time.

4. **[I, Ch. 17] Q: What must be true of training and scoring feature schemas?**

A: They must match by name, type, and semantics—enforced by one shared feature view versioned with the model.

5. **[I, Ch. 18] Q: When would you choose AutoML over a custom training notebook?**

A: For baselines and well-structured tabular problems—AutoML sets the bar a custom model must beat to justify its complexity.

6. **[F, Ch. 18] Q: Why use AML compute clusters instead of compute instances for training?**

A: Clusters autoscale across nodes for training and scale to zero when idle; instances are single-node development desks.

7. **[I, Ch. 18] Q: How does AML securely access governed ADLS training data?**

A: Through a datastore bound to the workspace managed identity with lake RBAC—no keys, full audit trail.

8. **[I, Ch. 18] Q: What does model registration give you beyond saving a file?**

A: Versioned identity with lineage to data, environment, code, and metrics—the answer to “what exactly produced this artifact.”

9. **[F, Ch. 19] Q: Real-time versus batch managed endpoints—which fits nightly scoring?**

A: Batch endpoints: millions of rows, minutes of latency tolerance, and compute that scales to zero between runs.

10. **[I, Ch. 19] Q: What is data drift, and how does AML detect it?**

A: Statistical divergence of production inputs from the training baseline; model monitoring compares collected production data against the baseline on a cadence.

11. **[A, Ch. 19] Q: What should trigger a retraining pipeline in a mature setup?**

A: Evidence—drift metrics crossing thresholds or quality decay on joined outcomes—not

a calendar alone.

12. **[A, Ch. 19] Q: What should gate model promotion to production?**
A: A metric threshold on fresh data, shadow or canary comparison on live traffic, human approval, and a rollback plan.
13. **[F, Ch. 20] Q: Which service extracts key-value pairs from scanned invoices?**
A: Azure AI Document Intelligence—the prebuilt invoice model returns typed fields with confidence scores, no training needed.
14. **[I, Ch. 20] Q: Why trigger document processing with a Function on blob upload?**
A: Event-driven processing removes polling; the Function is right-sized because extraction latency lives in the service call, not local compute.
15. **[I, Ch. 20] Q: How do you make extracted document metadata searchable?**
A: Land structured JSON in the lake and index it in AI Search—keyword, vector, or hybrid depending on query patterns.
16. **[A, Ch. 20] Q: What PII risks arise when OCR-ing user-uploaded documents?**
A: OCR surfaces PII that structured schemas never declared; extracted text must be classified and redacted before it reaches searchable indexes.

H.8 Security, Privacy, and Compliance (Chapters 21 and 23)

1. **[F, Ch. 21] Q: Why prefer managed identities over service principal client secrets?**
A: Azure owns the credential lifecycle—no secret to store, rotate, or leak; tokens are issued to the resource on demand.
2. **[I, Ch. 21] Q: Private Endpoint versus Service Endpoint—what is the key difference?**
A: A private endpoint gives the resource a private IP in your VNet and enables disabling public access; a service endpoint only optimizes routing to the public IP.
3. **[I, Ch. 21] Q: What does the Key Vault SecretNearExpiry event enable?**
A: Automated rotation: an event-driven pipeline regenerates the credential at the source and publishes a new secret version with no code change.
4. **[A, Ch. 21] Q: What does VNet data-exfiltration protection in Synapse guard against?**
A: Outbound paths—notebooks or jobs copying data to endpoints outside the governed perimeter, even by credentialed users.
5. **[F, Ch. 23] Q: Who still sees unmasked values when Dynamic Data Masking is enabled?**
A: Users with the UNMASK permission—DDM is presentation-layer; the data and privileged access are unchanged.
6. **[I, Ch. 23] Q: Always Encrypted versus TDE—which protects data from database admins?**
A: Always Encrypted: keys are client-side in Key Vault and the engine holds only ciphertext; TDE decrypts transparently for anyone with query access.
7. **[I, Ch. 23] Q: Why tokenize PII before loading it into Synapse?**
A: So the analytics estate never holds raw PII—analysts work on tokens, and the mapping

stays in a tightly controlled, separately encrypted store.

8. **[A, Ch. 23] Q: Why is a naive DELETE not GDPR erasure?**

A: Soft-delete windows, versioning, backups, Delta time travel, and stream retention all keep copies; crypto-shredding renders them unreadable by destroying the key.

Note

For system-design rounds, the capstones are the material: Chapter 5 (star schema with SCD2), Chapter 12 (event-driven orchestration), Chapter 15 (exactly-once streaming), Chapter 21 (private-only perimeter), Chapter 23 (erasure-capable architecture), and Chapter 24 (the multi-subscription data mesh) each rehearse an end-to-end architecture you can draw and defend.

Glossary

This glossary grows with each chapter and collects the cross-cloud data-engineering terms introduced so far. Definitions are platform-neutral unless a platform is named explicitly.

Access tier A storage pricing class that trades retrieval cost and latency for lower at-rest price (for example Azure Hot, Cool, Cold, and Archive). See also *storage class*.

AlloyDB Google Cloud’s PostgreSQL-compatible engine with a columnar accelerator for analytical queries, positioned as the cloud-native tier above Cloud SQL.

Apache Spark A distributed data-processing engine; Microsoft Fabric runs it on Starter or Custom pools, with counterparts in AWS Glue/EMR, Azure Synapse/Databricks, and GCP Dataproc.

Aurora Amazon’s cloud-native relational engine with distributed storage, Global Databases, and a serverless (v2) compute option.

Availability SLA The contractual uptime a provider guarantees for a service, distinct from *durability*, which concerns data loss rather than reachability.

Bigtable Google Cloud’s wide-column, high-throughput, low-latency NoSQL store for time-series, IoT, and ad-tech workloads; exposes the HBase API and scales by nodes and clusters.

Bucket (storage) The top-level container for objects in AWS S3 and Google Cloud Storage; Azure calls the equivalent a *container*.

Bucket Pattern A MongoDB schema-design pattern that groups many small time-series readings into a single document to cut document count and metadata overhead.

Capped collection A fixed-size MongoDB collection that overwrites its oldest documents when full, useful for high-volume logs and bounded activity feeds.

Change Feed / Streams An ordered log of item-level changes used for event-driven processing; called Change Feed in Cosmos DB, DynamoDB Streams in DynamoDB, and change streams in MongoDB.

Checkpoint In WiredTiger, a periodic flush of a consistent snapshot of cached data to disk, bounding crash-recovery time together with the journal.

Clustering key A user-defined column set that Snowflake uses to co-locate related rows within micro-partitions, improving partition pruning on selective queries.

CMEK / CMK Customer-managed encryption keys (GCP CMEK; Azure customer-managed keys via Key Vault), giving the customer control over the key that encrypts data at rest.

Cloud Spanner Google Cloud’s horizontally scalable, globally distributed relational database offering external (strong) consistency backed by TrueTime.

- Consistency level** A tunable guarantee about read recency; Azure Cosmos DB offers five (Strong, Bounded Staleness, Session, Consistent Prefix, Eventual).
- Cosmos DB** Azure’s globally distributed multi-model NoSQL database, billed in Request Units (RU) and supporting multiple APIs (NoSQL, MongoDB, Cassandra).
- Cross-Region Replication (CRR)** Asynchronous copying of objects to a bucket in another region for disaster recovery; Azure’s equivalent user-configured feature is *object replication*.
- DynamoDB** Amazon’s managed key-value and document NoSQL database, with capacity measured in read/write capacity units (RCU/WCU) or on-demand, and Global Tables for multi-region.
- Durability** The probability that stored data is not lost, commonly quoted as “eleven nines” (99.999999999%) for cloud object storage.
- ElastiCache** Amazon’s managed in-memory cache (Redis- and Memcached-compatible) placed in front of a durable store to serve hot data at sub-millisecond latency; Azure’s analog is Cache for Redis and GCP’s is Memorystore.
- Embedding vs referencing** The MongoDB modeling choice between nesting related data in one document (embedding) or linking documents by reference (and joining with `lookup`).
- Fail-safe** Snowflake’s non-configurable seven-day recovery window, available only to Snowflake support, that follows the Time Travel retention period as a last-resort safety net.
- Global Secondary Index (GSI)** A DynamoDB index with its own partition/sort key enabling queries on non-primary attributes; a Local Secondary Index (LSI) shares the table’s partition key.
- Hierarchical namespace (HNS)** The Azure Data Lake Storage Gen2 feature that adds true directories and atomic directory operations on top of Blob storage.
- Journal (WAL)** The write-ahead log that records changes before they are checkpointed, enabling crash recovery in MongoDB’s WiredTiger engine.
- JSON Schema validation** MongoDB’s rule mechanism (via `jsonSchema` in `createCollection`) that enforces required fields, data types, and value constraints.
- Interleaved table** A Cloud Spanner layout that physically co-locates child rows with their parent row to make parent-child joins fast.
- Lakehouse** An architecture that stores open table formats (such as Delta) in a data lake while offering warehouse-like SQL and governance; the core storage abstraction in Microsoft Fabric.
- Lifecycle policy** A rule set that automatically transitions or expires objects across storage tiers based on age or other conditions.
- Medallion architecture** A progressive-refinement data layout with Bronze (raw), Silver (validated), and Gold (curated/enriched) layers.
- Materialized view** A precomputed, automatically maintained query result stored for fast reads, trading maintenance/storage cost for lower query latency (Snowflake, Redshift, Synapse, BigQuery).
- Memorystore** Google Cloud’s managed in-memory cache service (Redis and Memcached), the GCP counterpart to AWS ElastiCache and Azure Cache for Redis.

- Micro-partition** Snowflake's internal storage unit: an immutable, compressed, columnar file of roughly 50–500 MB of uncompressed data, annotated with per-column metadata for pruning.
- Multi-AZ** An AWS deployment spanning multiple Availability Zones for high availability; Azure's analog is zone-redundant HA and GCP's is regional HA.
- Multi-cluster warehouse** A Snowflake virtual warehouse that adds and removes compute clusters automatically to absorb concurrent query load.
- Object Lock** The AWS S3 WORM feature (governance mode, compliance mode, and legal hold) that prevents object deletion or overwrite for a retention period.
- OneLake** Microsoft Fabric's single, tenant-wide logical data lake shared by all workspaces.
- Partition pruning** Skipping storage units (micro-partitions, files, or partitions) that cannot match a query's predicates, reducing scanned data.
- Point-in-time recovery (PITR)** Restoring a database or table to any moment within a retention window, using continuous backups or transaction logs; realized as Snowflake Time Travel, RDS/SQL Database PITR, and Delta Lake time travel.
- Read replica** A read-only copy of a database that offloads read traffic and can serve as a failover or geo-local endpoint.
- Relational Migrator** MongoDB's tool for migrating relational schemas and data to document collections, applying embed-versus-reference decisions from schema-design patterns.
- Request Unit (RU)** Azure Cosmos DB's normalized currency for throughput, abstracting the CPU, memory, and IOPS consumed by a database operation.
- Search Optimization Service (SOS)** A Snowflake feature that builds a persistent search access path to accelerate selective point-lookup queries on high-cardinality columns.
- Shortcut (Fabric)** A OneLake reference that virtualizes external data (S3, ADLS Gen2, GCS) in place so it can be queried without copying.
- Storage class** The AWS/GCP term for a pricing tier of object storage (for example S3 Standard, Standard-IA, Glacier; GCP Standard, Nearline, Coldline, Archive).
- SSE-KMS** Server-side encryption in AWS S3 using keys managed in AWS Key Management Service.
- Time Travel** Snowflake's ability to query, clone, or restore data as it existed at an earlier point within a retention period (up to 90 days on higher editions); Delta Lake offers an analogous `VERSION AS OF / TIMESTAMP AS OF` capability.
- Transient table** A Snowflake table that persists across sessions but has no Fail-safe period, reducing storage cost for reproducible staging data; a temporary table lives only for one session.
- TrueTime** Google's globally synchronized clock (with bounded uncertainty) that lets Cloud Spanner assign commit timestamps and provide external consistency.
- TTL (Time to Live)** An expiry mechanism that automatically deletes items or documents after a set time (DynamoDB TTL, Cosmos DB TTL, MongoDB TTL index).
- Versioning** Retaining prior versions of an object or blob so that overwrites and deletes are recoverable.

Virtual warehouse Snowflake's unit of elastic compute, sized XS–6XL and billed in credits, fully decoupled from storage.

Wide-column store A NoSQL data model that stores rows as sparse, dynamically wide sets of column families keyed by a row key (Bigtable, HBase, Cassandra), optimized for high write throughput and range scans.

WiredTiger MongoDB's default storage engine, using a configurable internal cache alongside the operating-system page cache, with journaling and checkpointing.

WORM Write-once-read-many storage that prevents modification or deletion for a retention period; implemented as Object Lock (AWS), immutable blob storage (Azure), or Bucket Lock (GCP).

Z-ordering A Delta Lake optimization that co-locates related information in the same files by multi-dimensional clustering, improving data skipping; applied via `OPTIMIZE ... ZORDER BY` and complemented by `VACUUM` to reclaim obsolete files.

Zero-copy clone Snowflake's instant, storage-free copy of a table, schema, or database that shares underlying micro-partitions until either copy is modified (copy-on-write).

Bibliography

- [1] Michael Armbrust, Tathagata Das, Liwen Sun, Burak Yavuz, Shixiong Zhu, Mukul Murthy, Joseph Torres, Herman van Hovell, Adrian Ionescu, Alicja Luszczak, et al. Delta lake: High-performance acid table storage over cloud object stores. *Proceedings of the VLDB Endowment*, 13(12):3411–3424, 2020.
- [2] Michael Armbrust, Ali Ghodsi, Reynold Xin, and Matei Zaharia. Lakehouse: A new generation of open platforms that unify data warehousing and advanced analytics. *CIDR*, 2021.
- [3] DataExpert.io. Data engineer handbook. <https://github.com/DataExpert-io/data-engineer-handbook>, 2025.
- [4] dbt Labs. dbt core: transformation workflow with sql and software engineering practices. <https://github.com/dbt-labs/dbt-core>, 2025.
- [5] Delta Lake Project. Delta lake: an open-source storage framework. <https://github.com/delta-io/delta>, 2025.
- [6] Great Expectations. Great expectations: data quality validation framework. https://github.com/great-expectations/great_expectations, 2025.
- [7] HashiCorp. Terraform azurearm provider documentation. <https://registry.terraform.io/providers/hashicorp/azurerm/latest/docs>, 2025.
- [8] Ralph Kimball and Margy Ross. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. Wiley, 3 edition, 2013.
- [9] Microsoft. About azure key vault. <https://learn.microsoft.com/azure/key-vault/general/overview>, 2025.
- [10] Microsoft. About using directquery in power bi. <https://learn.microsoft.com/power-bi/connect-data/desktop-directquery-about>, 2025.
- [11] Microsoft. Access tiers for blob data. <https://learn.microsoft.com/azure/storage/blobs/access-tiers-overview>, 2025.
- [12] Microsoft. Active geo-replication for azure sql database. <https://learn.microsoft.com/azure/azure-sql/database/active-geo-replication-overview>, 2025.
- [13] Microsoft. Always encrypted overview. <https://learn.microsoft.com/sql/relational-databases/security/encryption/always-encrypted-database-engine>, 2025.
- [14] Microsoft. Auto-failover groups overview and best practices. <https://learn.microsoft.com/azure/azure-sql/database/auto-failover-group-sql-db>, 2025.

- [15] Microsoft. Azure api management policies. <https://learn.microsoft.com/azure/api-management/api-management-policies>, 2025.
- [16] Microsoft. Azure cognitive search documentation. <https://learn.microsoft.com/azure/search/>, 2025.
- [17] Microsoft. Azure cosmos db for nosql python sdk. <https://learn.microsoft.com/python/api/overview/azure/cosmos-readme>, 2025.
- [18] Microsoft. Azure event hubs: a big data streaming platform and event ingestion service. <https://learn.microsoft.com/azure/event-hubs/event-hubs-about>, 2025.
- [19] Microsoft. Azure iot hub documentation. <https://learn.microsoft.com/azure/iot-hub/>, 2025.
- [20] Microsoft. Azure machine learning examples repository. <https://github.com/Azure/azureml-examples>, 2025.
- [21] Microsoft. Azure monitor overview. <https://learn.microsoft.com/azure/azure-monitor/overview>, 2025.
- [22] Microsoft. Azure sql database purchasing models. <https://learn.microsoft.com/azure/azure-sql/database/purchasing-models>, 2025.
- [23] Microsoft. Azure storage redundancy. <https://learn.microsoft.com/azure/storage/common/storage-redundancy>, 2025.
- [24] Microsoft. Azure synapse analytics workload management. <https://learn.microsoft.com/azure/synapse-analytics/sql-data-warehouse/sql-data-warehouse-workload-management>, 2025.
- [25] Microsoft. Best practices for azure cache for redis. <https://learn.microsoft.com/azure/azure-cache-for-redis/cache-best-practices>, 2025.
- [26] Microsoft. Best practices for dedicated sql pools in azure synapse analytics. <https://learn.microsoft.com/azure/synapse-analytics/sql/best-practices-dedicated-sql-pool>, 2025.
- [27] Microsoft. Cetas with synapse sql. <https://learn.microsoft.com/azure/synapse-analytics/sql/develop-tables-cetas>, 2025.
- [28] Microsoft. Change feed in azure cosmos db. <https://learn.microsoft.com/azure/cosmos-db/change-feed>, 2025.
- [29] Microsoft. Configure multi-region writes in azure cosmos db. <https://learn.microsoft.com/azure/cosmos-db/how-to-multi-master>, 2025.
- [30] Microsoft. Consistency levels in azure cosmos db. <https://learn.microsoft.com/azure/cosmos-db/consistency-levels>, 2025.
- [31] Microsoft. Copy statement in azure synapse analytics. <https://learn.microsoft.com/sql/t-sql/statements/copy-into-transact-sql>, 2025.
- [32] Microsoft. Create your azure free account today. <https://azure.microsoft.com/free/>, 2025.
- [33] Microsoft. Data lineage in microsoft purview. <https://learn.microsoft.com/azure/purview/concept-data-lineage>, 2025.
- [34] Microsoft. Data migration assistant. <https://learn.microsoft.com/sql/dma/dma-overview>, 2025.

- [35] Microsoft. Data transformation expressions in mapping data flow. <https://learn.microsoft.com/azure/data-factory/data-transformation-functions>, 2025.
- [36] Microsoft. Dedicated gateway in azure cosmos db. <https://learn.microsoft.com/azure/cosmos-db/dedicated-gateway>, 2025.
- [37] Microsoft. Define your naming convention. <https://learn.microsoft.com/azure/cloud-adoption-framework/ready/azure-best-practices/resource-naming>, 2025.
- [38] Microsoft. Dynamic data masking in azure sql. <https://learn.microsoft.com/azure/azure-sql/database/dynamic-data-masking-overview>, 2025.
- [39] Microsoft. Exam dp-203: Data engineering on microsoft azure. <https://learn.microsoft.com/credentials/certifications/exams/dp-203/>, 2025.
- [40] Microsoft. Get started with azcopy. <https://learn.microsoft.com/azure/storage/common/storage-use-azcopy-v10>, 2025.
- [41] Microsoft. Global distribution in azure cosmos db. <https://learn.microsoft.com/azure/cosmos-db/distribute-data-globally>, 2025.
- [42] Microsoft. Health insurance portability and accountability act on microsoft azure. <https://learn.microsoft.com/azure/compliance/offerings/offering-hipaa-us>, 2025.
- [43] Microsoft. High availability in azure database for postgresql flexible server. <https://learn.microsoft.com/azure/postgresql/flexible-server/concepts-high-availability>, 2025.
- [44] Microsoft. Hyperscale service tier for azure sql database. <https://learn.microsoft.com/azure/azure-sql/database/service-tier-hyperscale>, 2025.
- [45] Microsoft. Install azure powershell. <https://learn.microsoft.com/powershell/azure/install-azure-powershell>, 2025.
- [46] Microsoft. Install the azure cli on windows. <https://learn.microsoft.com/cli/azure/install-azure-cli-windows>, 2025.
- [47] Microsoft. Integrated cache in azure cosmos db. <https://learn.microsoft.com/azure/cosmos-db/integrated-cache>, 2025.
- [48] Microsoft. Integration runtime in azure data factory. <https://learn.microsoft.com/azure/data-factory/concepts-integration-runtime>, 2025.
- [49] Microsoft. Introduction to azure blob storage. <https://learn.microsoft.com/azure/storage/blobs/storage-blobs-introduction>, 2025.
- [50] Microsoft. Introduction to azure data lake storage gen2. <https://learn.microsoft.com/azure/storage/blobs/data-lake-storage-introduction>, 2025.
- [51] Microsoft. Machine learning model scoring with predict in azure synapse analytics. <https://learn.microsoft.com/azure/synapse-analytics/machine-learning/tutorial-sql-pool-model-scoring-predict>, 2025.
- [52] Microsoft. Microsoft azure well-architected framework. <https://learn.microsoft.com/azure/well-architected/>, 2025.
- [53] Microsoft. Microsoft certified: Fabric data engineer associate. <https://learn.microsoft.com/credentials/certifications/data-engineer/>, 2025.
- [54] Microsoft. Microsoft cloud adoption framework for azure. <https://learn.microsoft.com/azure/cloud-adoption-framework/>, 2025.

- [55] Microsoft. Microsoft purview documentation. <https://learn.microsoft.com/azure/purview/>, 2025.
- [56] Microsoft. Optimize costs by automating azure blob storage access tiers. <https://learn.microsoft.com/azure/storage/blobs/lifecycle-management-overview>, 2025.
- [57] Microsoft. Organize your resources with azure management groups, subscriptions, and resource groups. <https://learn.microsoft.com/azure/cloud-adoption-framework/ready/azure-setup-guide/organize-resources>, 2025.
- [58] Microsoft. Overview of azure cloud shell. <https://learn.microsoft.com/azure/cloud-shell/overview>, 2025.
- [59] Microsoft. Partitioning and horizontal scaling in azure cosmos db. <https://learn.microsoft.com/azure/cosmos-db/partitioning-overview>, 2025.
- [60] Microsoft. Pipeline execution and triggers in azure data factory. <https://learn.microsoft.com/azure/data-factory/concepts-pipeline-execution-triggers>, 2025.
- [61] Microsoft. Power bi documentation. <https://learn.microsoft.com/power-bi/>, 2025.
- [62] Microsoft. Quickstart: Azure cosmos db for nosql. <https://learn.microsoft.com/azure/cosmos-db/nosql/quickstart-portal>, 2025.
- [63] Microsoft. Request units in azure cosmos db. <https://learn.microsoft.com/azure/cosmos-db/request-units>, 2025.
- [64] Microsoft. Row-level security (rls) with power bi. <https://learn.microsoft.com/power-bi/enterprise/service-admin-rls>, 2025.
- [65] Microsoft. Serverless compute tier for azure sql database. <https://learn.microsoft.com/azure/azure-sql/database/serverless-tier-overview>, 2025.
- [66] Microsoft. Serverless sql pool in azure synapse analytics. <https://learn.microsoft.com/azure/synapse-analytics/sql/on-demand-workspace-overview>, 2025.
- [67] Microsoft. Synapseml: a distributed machine learning library. <https://microsoft.github.io/SynapseML/>, 2025.
- [68] Microsoft. Tutorial: Create and manage azure budgets. <https://learn.microsoft.com/azure/cost-management-billing/costs/tutorial-acm-create-budgets>, 2025.
- [69] Microsoft. Welcome to azure cosmos db. <https://learn.microsoft.com/azure/cosmos-db/introduction>, 2025.
- [70] Microsoft. Welcome to azure stream analytics. <https://learn.microsoft.com/azure/stream-analytics/stream-analytics-introduction>, 2025.
- [71] Microsoft. What are azure machine learning pipelines? <https://learn.microsoft.com/azure/machine-learning/concept-ml-pipelines>, 2025.
- [72] Microsoft. What is azure ai document intelligence? <https://learn.microsoft.com/azure/ai-services/document-intelligence/>, 2025.
- [73] Microsoft. What is azure api management? <https://learn.microsoft.com/azure/api-management/api-management-key-concepts>, 2025.
- [74] Microsoft. What is azure cache for redis? <https://learn.microsoft.com/azure/azure-cache-for-redis/cache-overview>, 2025.

- [75] Microsoft. What is azure data box? <https://learn.microsoft.com/azure/databox/data-box-overview>, 2025.
- [76] Microsoft. What is azure data factory? <https://learn.microsoft.com/azure/data-factory/introduction>, 2025.
- [77] Microsoft. What is azure data share? <https://learn.microsoft.com/azure/data-share/overview>, 2025.
- [78] Microsoft. What is azure database for postgresql flexible server? <https://learn.microsoft.com/azure/postgresql/flexible-server/overview>, 2025.
- [79] Microsoft. What is azure database migration service? <https://learn.microsoft.com/azure/dms/dms-overview>, 2025.
- [80] Microsoft. What is azure databricks? <https://learn.microsoft.com/azure/databricks/introduction/>, 2025.
- [81] Microsoft. What is azure event grid? <https://learn.microsoft.com/azure/event-grid/overview>, 2025.
- [82] Microsoft. What is azure logic apps? <https://learn.microsoft.com/azure/logic-apps/logic-apps-overview>, 2025.
- [83] Microsoft. What is azure machine learning? <https://learn.microsoft.com/azure/machine-learning/overview-what-is-azure-machine-learning>, 2025.
- [84] Microsoft. What is azure openai service? <https://learn.microsoft.com/azure/ai-services/openai/overview>, 2025.
- [85] Microsoft. What is azure private link? <https://learn.microsoft.com/azure/private-link/private-link-overview>, 2025.
- [86] Microsoft. What is azure role-based access control (azure rbac)? <https://learn.microsoft.com/azure/role-based-access-control/overview>, 2025.
- [87] Microsoft. What is azure sql database? <https://learn.microsoft.com/azure/azure-sql/database/sql-database-paas-overview>, 2025.
- [88] Microsoft. What is azure sql managed instance? <https://learn.microsoft.com/azure/azure-sql/managed-instance/sql-managed-instance-paas-overview>, 2025.
- [89] Microsoft. What is azure synapse analytics? <https://learn.microsoft.com/azure/synapse-analytics/overview-what-is>, 2025.
- [90] Microsoft. What is bicep? <https://learn.microsoft.com/azure/azure-resource-manager/bicep/overview>, 2025.
- [91] Microsoft. What is microsoft defender for cloud? <https://learn.microsoft.com/azure/defender-for-cloud/defender-for-cloud-introduction>, 2025.
- [92] Microsoft. What is microsoft entra id? <https://learn.microsoft.com/entra/fundamentals/whatis>, 2025.
- [93] Microsoft. Windowing functions in azure stream analytics queries. <https://learn.microsoft.com/stream-analytics-query/windowing-azure-stream-analytics>, 2025.
- [94] The Apache Software Foundation. Apache airflow: programmatically author, schedule, and monitor workflows. <https://github.com/apache/airflow>, 2025.

- [95] The Apache Software Foundation. Apache kafka: a distributed event streaming platform. <https://github.com/apache/kafka>, 2025.
- [96] The Apache Software Foundation. Apache spark: unified engine for large-scale data analytics. <https://github.com/apache/spark>, 2025.
- [97] The Apache Software Foundation. Spark structured streaming programming guide. <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html>, 2025.

Index

- 429 errors, 66
- ABFS, 20
- access tiers, 21
- ACID
 - data lake, 117
- active geo-replication, 43
- activity, 145
- ADLS Gen2, 17, 85
- Airflow
 - equivalence, 296
- Always Encrypted, 262, 263
- AML pipeline, 219, 221
 - capstone, 225
 - project, 222
- anonymization, 193
- Apache Airflow, 162
- Apache Spark, 135
 - equivalence, 291
- API for NoSQL, 61
- API gateway, 75
- APIM, 79
- architectural decision record, 29, 48, 66, 83, 99,
109, 120, 129, 140, 150, 158, 166, 177,
185, 194, 203, 214, 223, 231, 239, 249,
258, 266
- Athena
 - equivalence, 294
- audit
 - security, 249
- audit evidence, 29
- audit table, 150
- auto-failover groups, 39, 43
- AutoML, 219, 220
- AWS Glue
 - equivalence, 295
- Az module, 5
- AzCopy, 23
- Azure AI Language, 237
- Azure AI Search, 237
 - at scale, 238
- Azure API Management, 74, 79
- Azure Bastion, 247
- Azure Cache for Redis, 74, 76
- Azure CLI, 1, 24, 45, 62, 80, 279
 - account show, 9
 - ADF, 280
 - Databricks, 280
 - Event Hubs, 280
 - identity and access, 279
 - Key Vault, 281
 - monitoring, 281
 - networking, 281
 - setup, 304
 - storage, 279
 - Synapse, 280
 - Windows, 5
- Azure Cloud Shell, 1, 5
- Azure Cognitive Search, 235
- Azure Cosmos DB, 57
- Azure Data Box, 23
- Azure Data Factory, 144
 - capstone, 152
 - object model, 145
- Azure Data Lake Storage Gen2, 17
- Azure Data Share, 125, 127
- Azure Database for PostgreSQL, 39, 42
- Azure Database Migration Service, 47
- Azure Databricks, 135
 - capstone, 142
 - workspace, 136
- Azure DMS, 154, 155, 157
- Azure free account, 1
 - credit, 2
- Azure Functions, 235, 237
- Azure Key Vault, 245, 247
- Azure Logic Apps, 162, 165
- Azure Machine Learning, 219
 - workspace, 220
- Azure ML Designer, 220
- Azure Monitor, 66, 83, 256
 - ADF, 167
- Azure OpenAI, 235
- Azure Policy, 264
- Azure Portal, 1

- sign-up, 2
- Azure PowerShell, 1, 5
 - setup, 305
- Azure RBAC, 1, 19
 - roles, 4
- Azure region, 8
- Azure Resource Manager, 19
- Azure SQL Database, 39, 80
 - purchasing models, 40
- Azure SQL Database Serverless, 85
- Azure SQL Managed Instance, 39, 47
- Azure SQL platform, 52
- Azure Storage, 17
- Azure Storage Explorer, 23
- Azure Stream Analytics, 181
 - architecture, 182
 - capstone, 187
- Azure subscription, 2, 304
- Azure Synapse Analytics, 92

- batch ELT, 169
- batch endpoint, 227, 228
- BI
 - equivalence, 298
- Bicep, 271, 272, 305
- Bigtable
 - equivalence, 293
- Blob Storage, 17
- blob versioning, 21
- blue-green deployment, 229
- Bounded Staleness, 60
- budget, 1
 - alerts, 7
 - setup, 306

- cache hit ratio, 76
- cache stampede, 83
- cache-aside, 74
- caching, 74
 - equivalence, 292
- CALCULATE, 201
- capstone project, 32, 52, 69, 102, 112, 122, 131, 142, 152, 160, 169, 179, 187, 196, 206, 216, 225, 233, 241, 251, 260, 269, 275
- case studies, 13
- Cassandra API, 61
- catalog
 - equivalence, 295
- CDC
 - equivalence, 296
- certification, 315
- CETAS, 114, 118
- champion-challenger, 227
- change data capture, 40, 154
 - capstone, 160
 - mechanisms, 155
- Change Feed, 57, 61, 85
- Chapter 1 preparation, 11
- checklist, 11, 285
- checkpointing, 189, 190
- churn model, 213
 - capstone, 216
- CI/CD, 271, 273
 - equivalence, 302
- citizen developer, 149
- cloning
 - equivalence, 293
- cluster
 - all-purpose vs job, 136
- clustered columnstore, 105, 106
- collection, 126
- columnstore
 - equivalence, 294
- command reference, 279
- community resources, 13
- component, 221
- compute cluster, 219, 220
- compute node, 94
- concurrency, 99
- connection pooling, 49
- consistency levels, 60
- Consistent Prefix, 60
- consumer group, 172, 174
- continuous processing, 191
- control flow, 162, 164
- control node, 94
- control plane, 19
- COPY statement, 96, 97
- Cosmos DB
 - APIs, 61
 - capstone, 69
- Cosmos DB Integrated Cache, 74, 78
- cost analysis, 7
- cost guardrails, 306
- Cost Management, 1
- cost optimization
 - Synapse, 98
- coverage, 129
- cross-cloud equivalence, 288
- cross-tenant sharing, 127
- crypto-shredding, 262
 - drill, 264
- customer-managed keys, 32, 245, 247

- data catalog, 125
- data classification, 125, 127
- data clean room, 128
- data contract, 154, 156
- data drift, 227, 229
- data exfiltration protection, 249
- data lake, 19
- data lineage, 125, 127, 254, 256
- data map, 125
- data mesh, 254, 257
 - capstone, 275
- data migration
 - equivalence, 296
- data movement, 95
- Data Movement Service, 94
- data plane, 19
- data privacy
 - equivalence, 302
- data product
 - go-live, 285
- data quality, 254
 - serving layer, 120
- data skew, 99
- Data Warehouse Units, 92
- data zones, 29
- database acceleration, 75, 76
- dataflow, 199
- DataFrame, 135
- dataset, 144
- Dataverse, 201
- DAX, 198, 201
- DBFS, 135
- dbt, 271, 273
- dbt tests, 254, 256
- dedicated gateway, 78
- dedicated SQL pool, 92
- Delta Lake, 114, 122, 162
 - operations, 283
 - reference, 282
 - transaction log, 117
- Delta Live Tables, 135, 137
- development environment, 305
- dimensional modeling, 92
- directory, 20
- DirectQuery, 198, 200, 202
- disaster recovery, 27, 43
- distributed databases
 - equivalence, 291
- distribution
 - benchmark, 108
 - hash, 95
 - replicated, 95
 - round-robin, 95
- DMV, 284
- document database, 57
- Document Intelligence, 235, 236
 - project, 238
- document modeling
 - equivalence, 291
- DP-203, 262, 265, 315
 - domains, 315
- DP-700, 262, 265, 315
 - domains, 316
 - revision, 275
- drift detection
 - project, 230
- drift monitoring
 - equivalence, 300
- drills, 287
- DTU, 40
- Dual mode, 200
- Dynamic Data Masking, 262, 263
- e-commerce, 85
- eastus, 8
- elastic compute
 - equivalence, 290
- environment, 221
- environment setup, 304
 - equivalence, 288
- ETL service
 - equivalence, 295
- evaluation
 - generative AI, 239
- Event Grid, 154, 156
 - capstone, 169
 - retraining, 229
 - trigger, 165
- Event Hubs, 172
 - at scale, 176
 - capstone, 179
 - partitions, 174
 - producer, 175
- Event Hubs Capture, 172, 174
- event streaming
 - equivalence, 297
- Eventual consistency, 60
- eviction, 74
- exactly-once, 189, 191
- exam mapping, 315
- exam objectives
 - chapter map, 316
- exam preparation, 265

- expand–contract, 271, 273
 - checklist, 286
- expectations, 137
- external table, 117
- feature engineering, 210
 - discipline, 211
- feature store
 - equivalence, 299
- FinOps, 254, 256
 - equivalence, 301
- Flexible Server, 42
- ForEach, 164
- forecasting, 214
- fraud detection, 206
- free tier
 - equivalence, 288
- gaming leaderboard, 65, 69
- GDPR, 262
 - capstone, 269
- generative AI
 - equivalence, 300
- geo-replica
 - Azure SQL, 45
- geo-replicated platform, 52
- geo-replication, 39
- GitHub Actions, 271, 273
- Global Administrator, 4
- global distribution, 57, 59
- glossary, 329
- governance
 - checklist, 285
 - Power BI, 203
- GPU, 222
- Great Expectations, 256
- GRS, 20
- GZRS, 20, 27
- Hadoop migration, 139
- hands-on lab, 24, 45, 62, 80, 97, 108, 118, 128, 138, 148, 157, 165, 175, 184, 192, 202, 213, 222, 230, 238, 248, 257, 264, 274
 - solutions, 308
- health check, 109
- healthcare, 27
- heap, 106
- hierarchical namespace, 17
- high availability, 52
- HIPAA, 27
- HITRUST, 27
- HL7, 193
- Hybrid ingestion, 74
- Hyperscale, 39, 40
- idempotency, 150
- identity resolution, 201
- immutable storage, 32
- Import mode, 198, 200
- in-database ML
 - equivalence, 299
- in-database scoring, 210
- in-memory cache
 - equivalence, 292
- incremental models, 273
- infrastructure as code, 271
 - equivalence, 302
- ingestion gateway, 79
- Integration Runtime, 147
 - types, 147
- interview questions, 320
- IoT Hub, 172, 177
- job scheduling, 140
- Kafka
 - equivalence, 297
 - Event Hubs endpoint, 174
 - Structured Streaming, 191
- Kimball, 94
- lab solutions, 308
- lakehouse, 19, 114
 - architecture, 119
 - capstone, 131
- late arrival, 181, 183
 - case study, 185
- latency, 83
- least privilege, 4
- legal hold, 264
- lifecycle management, 21
- lifecycle policy
 - JSON, 24
- linked service, 144
- live sports, 82
- logical data warehouse, 114, 117, 122
- low latency, 82
- LRS, 20
- Managed Airflow, 165
- managed endpoint, 227
- managed feature store, 219
- managed identity, 30, 49, 66, 83, 245, 246
 - equivalence, 301
 - ingestion, 96

- Managed Instance cutover, 52
- managed online endpoint, 228
- management plane, 19
- Mapping Data Flows, 144, 146
 - project, 148
- masking
 - equivalence, 302
- materialized view, 105, 107
- materialized views
 - equivalence, 292
- medallion architecture, 162
 - equivalence, 290
 - zones, 163
- medical imaging, 27
- medical imaging archive, 32
- MFA, 4
- micro-batch, 191
- Microsoft account, 1, 2
- Microsoft Cost Management, 7
- Microsoft Defender for Cloud, 264
- Microsoft Entra ID, 1, 4, 245, 246, 304
- Microsoft Purview, 125, 131, 254
 - classifications, 255
 - data map, 126
 - project, 128, 257
- migration, 23
 - online, 155
- Milestone 1 capstone, 85
- ML platform
 - equivalence, 299
- MLOps, 219, 227
 - capstone, 233, 241
- mobile application, 82
- model inventory, 214
- model monitoring, 227, 229
- model promotion
 - checklist, 286
- model serving
 - equivalence, 300
- MongoDB API, 61
- MPP, 92
 - architecture, 94
- MPP warehouse
 - equivalence, 293
- multi-cloud, 288
- multi-region writes, 62, 65, 69
- multi-tier data backend, 85
- naming convention, 8, 29
- NoSQL, 57
 - equivalence, 291
- object replication, 32
- object storage, 20
 - equivalence, 289
- observability, 49, 66, 83, 254
 - capstone, 260
 - equivalence, 301
- OCR, 235, 236
- ONNX, 210, 212
- OPENROWSET, 114, 116
- operational analytics, 58
- Oracle
 - migration, 158
- orchestration, 162
 - equivalence, 296
- out-of-order, 183
- parameterization, 147
- partition, 137
 - Event Hubs, 172
- partition key, 57
- partitioning, 59, 105
 - Synapse, 107
- performance, 66
- Photon, 140
- pipeline, 144
- pipeline JSON, 147
- pipeline schedule, 223
- platform review, 258
 - agenda, 287
- point-in-time recovery
 - equivalence, 293
- policy, 79
- PolyBase, 96
- portfolio projects, 13
- POSIX ACL, 20
- PostgreSQL, 39
 - high availability, 42
- Power BI, 198
 - architecture, 199
- PowerShell, 5
- PREDICT, 210, 212
- privacy operations, 266
- Private Endpoint, 245, 247
 - project, 248
- private endpoint, 30
- production readiness, 285
- promotion gate, 829
- Purview
 - equivalence, 295
- PySpark, 138
 - reference, 282
- Python SDK, 24, 45, 62

- setup, 305
- PyTorch, 222
- quarantine, 156, 157
- query acceleration
 - equivalence, 292
- query optimization, 105
 - capstone, 112
 - case study, 109
- query performance, 49
- question bank, 320
- queue, 173
- RA-GRS, 20
- rate limiting, 79
- read replica, 43
- recommendation engine, 231
- Redis, 76
- redis-py, 80
- Redshift
 - equivalence, 293
- redundancy, 20
- refresh, 203
- relational database, 40
- relational databases
 - equivalence, 289
- replica lag, 49
- replication lag, 154
- Request Units, 57, 59, 78
- resource group, 1
 - delete, 9
- result-set caching, 105
- retrieval-augmented generation, 235
- reverse ETL, 201
- RocksDB, 190
- row-level security, 198, 201
 - at scale, 203
- rowgroup, 99, 106
- RPO, 19, 43
- RTO, 19, 43
- RU/s, 59
- rubric, 30, 49, 67, 83, 100, 110, 121, 130, 140, 150, 159, 167, 177, 186, 194, 204, 215, 223, 232, 239, 250, 259, 267
- SageMaker
 - equivalence, 299
- scan cadence, 129
- scan cost, 120
- SCD Type 2, 102
- secret rotation, 247
- self-assessment, 317
- self-healing, 166
- Self-Hosted Integration Runtime
 - operations, 158
- semantic model, 198
 - equivalence, 298
- sensitivity labels, 255
- serverless, 40
- serverless database, 39
- serverless SQL
 - equivalence, 294
- serverless SQL pool, 114
 - billing, 116
- Service Bus, 172, 173
- service endpoint, 247
- service mapping, 288
- service principal, 4, 246
- serving layer, 58, 65, 75
- Session consistency, 57, 60, 65, 69
- shuffle, 135, 137
 - reference, 283
- SLI
 - endpoint, 231
- SLO, 256
- soft delete, 21
- Spark
 - behind data flows, 146
 - execution model, 137
- Spark SQL, 282
 - read/write, 282
- spending limit, 7
- SQL Server migration, 47
- staleness, 78
- star schema, 92, 97, 102
- state backend, 272
- state store, 189, 190
- statistics, 109
- storage account, 17
 - architecture, 19
 - create, 9
- storage classes
 - equivalence, 289
- stream SQL
 - equivalence, 297
- stream-stream join, 189, 191
- streaming
 - checklist, 286
 - operations, 194
- streaming units, 182
 - sizing, 185
- Strong consistency, 60
- Structured Streaming, 189

- capstone, 196
- equivalence, 298
- project, 192
- reference, 283
- study strategy, 317
- Synapse
 - T-SQL reference, 284
- Synapse Spark, 210, 212
- SynapseML, 212
- tags, 8
 - cost allocation, 7
- Terraform, 271, 272
 - project, 274
 - setup, 305
- throttling, 66, 83
- throughput units, 174
- time travel, 117
 - equivalence, 293
- TIMESTAMP BY, 181
- tokenization, 265
- tool requirements by chapter, 306
- tooling reference, 304
- topic, 173
- traffic spikes, 82
- training-serving skew, 210
- transactional consistency, 40
- Translator, 237
- Transparent Data Encryption, 262, 263
- troubleshooting, 30, 49, 66, 83
 - ADF, 150
 - AI services, 239
 - AML, 223
 - ASA, 185
 - compliance, 267
 - endpoints, 231
 - Event Hubs, 177
 - governance, 258
 - integration, 158
 - networking, 249
 - orchestration, 167
 - Power BI, 204
 - PREDICT, 215
 - Purview, 129
 - serverless SQL, 120
 - Spark, 140
 - Structured Streaming, 194
 - Synapse dedicated pool, 100
 - Synapse performance, 110
- TTL, 74
- tumbling window, 181
 - project, 184
- Until, 164
- use case, 98, 109, 119, 128, 139, 149, 158, 166, 176, 185, 193, 203, 214, 222, 231, 238, 249, 257, 265
- vCore, 40
- vector search
 - equivalence, 300
- verification, 9
- virtual warehouse
 - equivalence, 290
- visual ETL, 144
- watermark, 181, 183
 - Spark, 189
- wide-column
 - equivalence, 293
- windowing, 181
 - equivalence, 297
 - types, 183
- workload classifier, 107
- workload isolation
 - equivalence, 294
- workload management, 105, 107
- zero trust, 245
 - capstone, 251, 275
 - equivalence, 301
- zero-downtime migration, 47
- zone-redundant HA, 42
- ZRS, 20